Thomas Hanne
Rolf Dornberger

Computational Intelligence in Logistik und Supply Chain Management

Springer Gabler

Computational Intelligence in Logistik und Supply Chain Management

Thomas Hanne • Rolf Dornberger

Computational Intelligence in Logistik und Supply Chain Management

Springer Gabler

Thomas Hanne
Institut für Wirtschaftsinformatik
Fachhochschule Nordwestschweiz
Olten, Schweiz

Rolf Dornberger
Institut für Wirtschaftsinformatik
Fachhochschule Nordwestschweiz
Basel, Schweiz

This book is a translation of the original English edition "Computational Intelligence in Logistics and Supply Chain Management" by Hanne, Thomas, published by Springer Nature Switzerland AG in 2017. The translation was done with the help of artificial intelligence (machine translation by the service DeepL.com). A subsequent human revision was done primarily in terms of content, so that the book will read stylistically differently from a conventional translation. Springer Nature works continuously to further the development of tools for the production of books and on the related technologies to support the authors.

ISBN 978-3-031-21451-6 ISBN 978-3-031-21452-3 (eBook)
https://doi.org/10.1007/978-3-031-21452-3

Die Deutsche Nationalbibliothek verzeichnet diese Publikation in der Deutschen Nationalbibliografie; detaillierte bibliografische Daten sind im Internet über http://dnb.d-nb.de abrufbar.

Springer Gabler
© Der/die Herausgeber bzw. der/die Autor(en), exklusiv lizenziert an Springer Nature Switzerland AG 2023
Das Werk einschließlich aller seiner Teile ist urheberrechtlich geschützt. Jede Verwertung, die nicht ausdrücklich vom Urheberrechtsgesetz zugelassen ist, bedarf der vorherigen Zustimmung des Verlags. Das gilt insbesondere für Vervielfältigungen, Bearbeitungen, Übersetzungen, Mikroverfilmungen und die Einspeicherung und Verarbeitung in elektronischen Systemen.
Die Wiedergabe von allgemein beschreibenden Bezeichnungen, Marken, Unternehmensnamen etc. in diesem Werk bedeutet nicht, dass diese frei durch jedermann benutzt werden dürfen. Die Berechtigung zur Benutzung unterliegt, auch ohne gesonderten Hinweis hierzu, den Regeln des Markenrechts. Die Rechte des jeweiligen Zeicheninhabers sind zu beachten.
Der Verlag, die Autoren und die Herausgeber gehen davon aus, dass die Angaben und Informationen in diesem Werk zum Zeitpunkt der Veröffentlichung vollständig und korrekt sind. Weder der Verlag, noch die Autoren oder die Herausgeber übernehmen, ausdrücklich oder implizit, Gewähr für den Inhalt des Werkes, etwaige Fehler oder Äußerungen. Der Verlag bleibt im Hinblick auf geografische Zuordnungen und Gebietsbezeichnungen in veröffentlichten Karten und Institutionsadressen neutral.

Planung/Lektorat: Susanne Kramer
Springer Gabler ist ein Imprint der eingetragenen Gesellschaft Springer Nature Switzerland AG und ist ein Teil von Springer Nature.
Die Anschrift der Gesellschaft ist: Gewerbestrasse 11, 6330 Cham, Switzerland

Vorwort

In den letzten Jahrzehnten haben sich Logistik und Supply Chain Management (SCM) zu einem der am häufigsten und intensivsten diskutierten Bereiche in Management und Wirtschaft entwickelt. Obwohl viele der in der Logistik und im SCM verwendeten Ideen und Konzepte schon recht alt sind, wurden große Anstrengungen unternommen, um sie in die Praxis zu übertragen und weiter zu verbessern. Es sind zahlreiche Veröffentlichungen sowohl in der akademischen als auch in der anwendungsorientierten Literatur erschienen. Logistik und SCM haben sich zu Bereichen entwickelt, die reich an Innovationen und Fortschritten sind.

Trotz dieser vielversprechenden Entwicklungen gibt es immer noch Hindernisse, um fortschrittliche Visionen einer verbesserten Planung und Zusammenarbeit entlang logistischer Prozesse und Lieferketten in die Realität umzusetzen. Einerseits gibt es viele praktische Probleme, wie z. B. die Verfügbarkeit und transparente Verarbeitung von Informationen, Schwierigkeiten beim Aufbau von Kooperationen oder aufgrund eines zunehmend unsicheren oder sich schnell verändernden Planungsumfelds. Zum anderen zeigt sich immer deutlicher, dass die zugrundeliegenden Planungsprobleme selbst dann sehr komplex und schwer zu lösen sind, wenn die entsprechenden Daten vollständig verfügbar und sicher sind.

Aus rechnerischer Sicht können viele dieser Probleme als NP-schwer bezeichnet werden, was bedeutet, dass die Anzahl der möglichen Lösungen exponentiell mit der Problemgröße zunimmt und dass vermutlich keine Algorithmen existieren, die sie innerhalb akzeptabler Zeitgrenzen exakt lösen können – zumindest wenn die Probleme „ziemlich groß“ sind. Leider können die meisten Probleme der realen Welt als ziemlich groß angesehen werden.

Vor allem in den letzten 20 Jahren wurden diese Probleme in der akademischen Literatur intensiv untersucht, und es wurden viele geeignete Lösungsansätze vorgeschlagen. Da die Probleme in der Regel nicht innerhalb einer akzeptablen Zeit exakt gelöst werden können, erlauben diese Methoden, ausreichend gute, wenn auch nicht unbedingt optimale Lösungen zu finden.

Eine der immer noch wachsenden Strömungen von Methoden gehört zum Bereich der Computational Intelligence (CI), der hauptsächlich Ansätze umfasst, die von Konzepten aus der Natur inspiriert sind, z. B. der natürlichen Evolution oder

dem Verhalten von Schwärmen. Diese Methoden basieren auf allgemeinen heuristischen Ideen und Konzepten zur Problemlösung, die – mit einigen Anpassungen – auf ein breites Spektrum von Problemen angewendet werden können. Um diese Methoden von einfachen Heuristiken zu unterscheiden, die oft sehr spezifisch für einen einzigen Problemtyp sind, werden sie auch als Metaheuristiken bezeichnet.

Obwohl die entsprechenden Computational-Intelligence-Methoden in zahlreichen Anwendungen im Zusammenhang mit Logistik und Supply Chain Management untersucht wurden, werden sie in allgemeinen Lehrbüchern in diesen Bereichen kaum behandelt. Die Behandlung formaler Planungsprobleme in diesen Büchern geht oft nicht über einige recht einfache und allgemeine Ergebnisse hinaus, die in der Praxis oft nicht anwendbar sind, wie z. B. die mehr als 100 Jahre alte Gleichung zur Berechnung wirtschaftlicher Bestellmengen.

Das Buch soll diese Lücke zwischen allgemeinen Lehrbüchern in Logistik und Supply Chain Management und neueren Forschungen zu formalen Planungsproblemen und entsprechenden Algorithmen schließen. Es konzentriert sich auf Ansätze aus dem Bereich der Computational Intelligence und anderer Metaheuristiken zur Lösung der komplexen operativen und strategischen Probleme in diesen Bereichen.

Das Buch richtet sich somit an Leser, die von einführenden Texten über Logistik und Supply Chain Management zur wissenschaftlichen Literatur übergehen wollen, die sich mit der Anwendung fortgeschrittener Methoden beschäftigt. Dazu werden aktuelle Beschreibungen der entsprechenden Probleme und geeignete Methoden zu deren Lösung bereitgestellt. Das Buch richtet sich in erster Linie an Studierende und Praktiker als potentielle Leser. Es kann als zusätzliches Nachschlagewerk für Grundkurse in Logistik, Supply Chain Management, Operations Research oder Computational Intelligence oder als Hauptlehrmittel für einen entsprechenden Postgraduiertenkurs verwendet werden. Praktiker können das Buch lesen, um sich mit fortgeschrittenen Methoden vertraut zu machen, die in ihrem Arbeitsbereich zum Einsatz kommen können. Für den Leser wird ein grundlegendes Verständnis der mathematischen Notation und Algebra sowie einführendes Wissen über Operations Research (z. B. über den Simplex-Algorithmus oder Graphen) empfohlen.

Das Buch ist wie folgt gegliedert: Die ersten beiden Kapitel bieten allgemeine Einführungen in Logistik und Supply Chain Management einerseits und in Computational Intelligence andererseits. In den folgenden Kapiteln werden spezifische Bereiche der Logistik und des Supply Chain Management behandelt, die wichtigsten Probleme dieser Bereiche herausgearbeitet und Lösungsansätze diskutiert. In Kap. 3 werden Probleme der Transportplanung, wie z. B. verschiedene Arten von Vehicle-Routing-Problemen, betrachtet. In Kap. 4 werden Probleme aus dem Bereich der Produktions- und Lagerverwaltung erörtert. Kap. 5 befasst sich mit Planungsaktivitäten auf einer feineren Granularitätsebene, die üblicherweise als Scheduling bezeichnet wird. Während die Kap. 3 bis 5 eher Planungsprobleme behandeln, die auf der operativen Ebene auftreten, werden in Kap. 6 die strategischen Probleme bei der Gestaltung einer Lieferkette oder eines Netzwerks erörtert. Das letzte Kapitel gibt einen Überblick über akademische und kommerzielle Software und Informationssysteme für die diskutierten Anwendungen.

Wir hoffen, den Lesern einen umfassenden Überblick mit spezifischen Details über den Einsatz von Computational Intelligence in Logistik und Supply Chain Management zu geben.

Olten, Schweiz — Thomas Hanne
Basel, Schweiz — Rolf Dornberger

Inhaltsverzeichnis

1 Einführung in Logistik und Supply Chain Management 1
1.1 Das Konzept der Logistik und des Lieferkettenmanagements. 1
1.2 Eine kurze Geschichte der Logistik 4
1.3 Jüngste Trends und die moderne Bedeutung der Logistik 5
1.4 Die Notwendigkeit einer besseren Planung 10
Literatur. 13

2 Computational Intelligence 15
2.1 Grundlagen von Computational Intelligence 15
2.1.1 Künstliche Intelligenz, Computational Intelligence und verwandte Techniken 15
2.1.2 Eigenschaften von Computational Intelligence 18
2.1.3 Das große Bild von Computational Intelligence 20
2.1.4 Anwendungsbereiche von Computational Intelligence 22
2.2 Methoden der Computational Intelligence. 25
2.2.1 Evolutionäre Berechnungen – Evolutionary Computation . . . 25
2.2.2 Evolutionäre Algorithmen – Evolutionary Algorithms 26
2.2.3 Schwarmintelligenz – Swarm Intelligence 36
2.2.4 Neuronale Netze – Neural Networks 39
2.2.5 Fuzzy-Logik – Fuzzy Logic. 40
2.2.6 Künstliches Immunsystem – Artificial Immune Systeme . . . 41
2.2.7 Weitere verwandte Methoden 41
Literatur. 44

3 Transportprobleme 47
3.1 Zuordnungsprobleme 48
3.2 Kürzeste Wege 49
3.3 Das Travelling-Salesman-Problem 51
3.4 Methoden zur Lösung des Travelling-Salesman-Problems 54
3.4.1 Heuristiken für das Travelling-Salesman-Problem 55
3.4.2 Evolutionäre Algorithmen für das Travelling-Salesman-Problem 56

3.4.3 Andere Metaheuristiken und neuronale Netze für das Travelling-Salesman-Problem 60
3.4.4 Über die Leistungsfähigkeit von Lösungsansätzen 62
3.5 Das Vehicle-Routing-Problem 63
3.5.1 Das Vehicle-Routing-Problem mit Zeitfenstern. 64
3.5.2 Das Vehicle-Routing-Problem mit mehreren Fahrzeugen . . . 65
3.5.3 Das Vehicle-Routing-Problem mit mehreren Depots. 66
3.5.4 Weitere Problemvarianten 67
3.6 Lösungsansätze für Vehicle-Routing-Probleme. 68
3.7 Abhol- und Zustellprobleme 71
3.8 Netzwerk-Flussprobleme 73
Literatur. 74

4 Bestandsplanung und Losgrößenbestimmung 79
4.1 Die Notwendigkeit einer Bestandsplanung 79
4.2 Wirtschaftliche Bestellmengen und Sicherheitsbestände 81
4.3 Losgrößenprobleme mit beschränkten Kapazitäten. 86
4.4 Lösungsansätze für kapazitätsbezogene Losgrößenprobleme 89
4.5 Planung des Lagerbetriebs 92
4.6 Lagerorte 94
4.7 Inventory Routing 95
Literatur. 102

5 Scheduling 107
5.1 Einleitung. 107
5.2 Einfache Regeln und Heuristiken 108
5.3 Standard-Scheduling-Probleme 112
5.3.1 Job-Shop-Scheduling 113
5.3.2 Flow-Shop-Scheduling 115
5.3.3 Open Shop Scheduling. 116
5.4 Spezifische Scheduling-Probleme in der Logistik 117
5.5 Lösung von Scheduling-Problemen mit Techniken der Computational Intelligence 119
5.5.1 Aspekte der Kodierung 120
5.5.2 Verwendung von Metaheuristiken im Scheduling 125
Literatur. 126

6 Standortplanung und Netzwerkgestaltung. 129
6.1 Standortplanung als multikriterielles Entscheidungsproblem 129
6.2 Diskrete Standortprobleme 132
6.2.1 Das p-Median-Problem 132
6.2.2 Das p-Center-Problem 137
6.2.3 Das Unkapazitierte Facility Location-Problem (UFLP) 139
6.2.4 Das Capacitated Facility Location Problem (CFLP) 142
6.3 Kontinuierliche Standortprobleme. 144

6.3.1 Das „Uncapacitated Multi-facility Weber Problem“ (UMWP) . . . 145
6.3.2 Das Capacitated Multi-facility Weber Problem (CMWP). . . . 148
6.4 Location-Routing-Probleme . . . 151
6.5 Probleme mit Hub-Standorten . . . 154
6.6 Multi-Echelon-Netzwerkgestaltung . . . 156
6.7 Schlussfolgerungen . . . 157
Literatur . . . 158

7 Intelligente Software für die Logistik . . . 163
7.1 Allgemeine Optimierungssoftware . . . 163
7.1.1 Erstellen eines geeigneten Modells für die Optimierungssoftware . . . 165
7.1.2 Integration von Optimierungssoftware mit Logistikanwendungen . . . 166
7.1.3 Anpassung der Methode an das zu betrachtende Problem . . . 167
7.2 Softwaremit spezifischen Optimierungsalgorithmen oder für spezifische Optimierungsprobleme . . . 168
7.3 Allgemeine Business-Software . . . 171
7.4 Logistik-Software . . . 174
7.4.1 Lagerverwaltungssysteme . . . 174
7.4.2 Software für die Transportplanung . . . 176
7.4.3 Software für das Packen und Laden . . . 177
7.5 Schlussfolgerungen . . . 178
Literatur . . . 179

Anhang A. Kurzbiographien der Autoren . . . 181

Danksagungen

Die Autoren danken ihrem Arbeitgeber, der Fachhochschule Nordwestschweiz, insbesondere dem Fachbereich Wirtschaft, für die Unterstützung beim Lektorat des Buches. Unser besonderer Dank gilt Christine Lorgé, Assistentin an unserem Institut für Wirtschaftsinformatik, die dieses Fachbuch mit großer Leidenschaft gelesen hat, obwohl sie nicht aus diesen Disziplinen kommt.

Unsere tiefe Dankbarkeit gilt unseren geliebten Familien, d. h. unseren Frauen und Kindern. Als Professoren, die in Forschung und Lehre tätig sind, Rolf zusätzlich als Institutsleiter und Thomas als Leiter eines der Kompetenzzentren, verbringen wir so viel Zeit mit der Arbeit, dass wir immer das Gefühl haben, dass unsere Familien zu kurz kommen. Deshalb möchten wir ihnen unseren größten Dank für ihr großes Verständnis und ihre unermüdliche Unterstützung aussprechen! Thomas dankt außerdem seiner Frau Doris für das Korrekturlesen einiger Kapitel, für die Diskussion einiger Inhalte und für die Unterstützung bei den Listen der Symbole und Akronyme.

Liste von Symbolen

λ	Anzahl der Nachkommen (Parameter einer Evolutionsstrategie)
π	Permutation
$\mathfrak{R}$	Menge der reellen Zahlen
ρ	Anzahl der rekombinierten Eltern (Parameter einer Evolutionsstrategie)
σ	Standardabweichung, Vorhersagefehler
σ_d	Standardabweichung für die Nachfrage
σ_{LT}	Standardabweichung für die Vorlaufzeit
μ	Anzahl der Eltern (Parameter einer Evolutionsstrategie)
A	(zulässige) Menge von Alternativen, Suchraum
A, B	Start- und Zielort eines Transports
a	Kapazitätsbedarf pro Einheit
a_i	Kapazitätsbedarf am Standort i
(a_{i1}, a_{i2})	Koordinaten des Kunden i
a_{ij}	Einflussfaktoren (in der Particle Swarm Optimization)
a_p	Kapazitätsbedarf pro Einheit von Produkt p
at_i	Ankunftszeit von Auftrag i
b	Kapazitätsbedarf pro Rüstvorgang/Setup
C, C_i	Erledigungszeit (von Auftrag i)
c, c_t	Kosten, Stückkosten
c_i^{best}, c_i^{global}	Beschleunigungsfaktoren (bei der Particle Swarm Optimization)
c_{ij}	Transportkosten (zwischen i und j)
C_{max}	Gesamtdurchlaufzeit
D	Gesamtbedarfsmenge (im Zeithorizont)
$d(.,.)$	Abstandsfunktion
dd_i	Fälligkeitsdatum des Auftrags i
d_{ij}	Abstand zwischen i und j
dl_i	Deadline für Auftrag i
d_p	Nachfrage nach Produkt p
d_t	Nachfrage in t

$\hat{d}_t$	Voraussichtliche Nachfrage in *t*
$\overline{d}$	Durchschnittliche Nachfrage
E	Menge der Kanten eines Graphen
ea	Verfrühung
e_i	Kante eines Graphen
$f(.)$	Zielfunktion
f_i	Fertigstellungszeit
f_j	Einrichtungskosten des Standorts j
$G = (V, E)$	Graph
$G = (V, E, c)$	Graph mit Gewichten (die den Kosten entsprechen)
h, h_t, h_p	(Stück-) Lagerkosten
h^r	Lagerkostensatz
i_{pt}	Bestand des Produkts *p* in *t*
i_{pt}^A, i_{pt}^B	Bestand des Produkts am Ort A(*B*) in *t*
i_t	Bestand in *t*
L	Menge von Standorten
L^C	Menge der Kundenstandorte
L^D	Menge der Depotstandorte
LT	Vorlaufzeit (Lead Time)
$\overline{LT}$	Durchschnittliche Vorlaufzeit
la	Verspätung
l_i	Standort
l_j^C	Standort eines Kunden
l_i^D	Standort eines Depots
M	Große konstante Zahl
m	Anzahl der Neuronen
N(0, σ)	Normalverteilung mit Erwartungswert 0 und Standardabweichung σ
n	Anzahl der Variablen (z. B. Orte in einer Tour)
$n!$	Fakultät von n
$O(.)$	Laufzeitkomplexität eines Algorithmus (Groß-O-Schreibweise)
P	Teilmenge der Standorte
p	Preis pro Einheit
p	Anzahl der zu eröffnenden Standorte bei einem *p*-Median-Problem
p^{global}	Global beste Position eines Partikels (in der Particle Swarm Optimization)
p_i	Partikel i (in der Particle Swarm Optimization)
p_i^{best}	Beste vorherige Position von Partikel i (in der Particle Swarm Optimization)
p_{ij}	Bearbeitungszeit von Auftrag *i* (auf Maschine *j*)
q, q_j	Kapazität (z. B. eines Fahrzeugs oder eines Standorts)
q_{ij}	Transportierte Menge zwischen zwei Orten
RP	Bestellpunkt (Reorder Point)

r_i^{best}, r_i^{global}	Zufällige Koeffizienten (in der Particle Swarm Optimization)
S	Menge von Standorten der Einrichtungen
SS	Sicherheitsbestand (Safety Stock)
s_i	Startzeit des Auftrags i
s_{ik}	Ankunftszeit des Fahrzeugs k am Ort i
s_i^{max}	Letzte Ankunftszeit am Ort i (mit angegebenem Zeitfenster)
s_i^{min}	Früheste Ankunftszeit am Ort i (mit angegebenem Zeitfenster)
T	Planungshorizont, Zeithorizont, Gesamtzahl der Perioden
t_{ij}	Fahrzeit von i nach j
u, u_t, u_p	Fixkosten pro Auftrag, Rüstkosten eines Produktionsprozesses
U_i	Bestellung bis zu einer bestimmten Menge eines Produkts i (Order-up-to level)
V	Menge der Knoten eines Graphen, Menge der Fahrzeuge
v_i	Knoten eines Graphen, Geschwindigkeitsvektor eines Partikels (in der Particle Swarm Optimization)
w_i	Trägheitsgewicht (in der Particle Swarm Optimization)
w_L, w_E	Gewichte (für Verspätung und Verfrühung)
w_t	Produktionskapazität in Periode t
x, x_{ij}, x_{ijk}, x_{pt}	Entscheidungsvariablen
x_o	Wirtschaftliche Bestellmenge (Economic Order Quantity)
x_t	Produktionsmengen in t
y_t, y_{pt}	Binäre Entscheidungsvariablen
Z	Service Level
z	Hilfsvariable für Zielfunktionswerte
z_{it}	Binäre Variablen (die angeben, ob der Knoten i zum Zeitpunkt t besucht wird)

Liste der Abkürzungen und Akronyme

2E-VRP	Zwei-Echelon Vehicle Routing Problem
ABC	Artificial Bee Colony
ACO	Ant Colony Optimization
AGV	Fahrerlose Transportfahrzeuge (Automated Guided Vehicles)
AI	Künstliche Intelligenz (Artificial Intelligence)
AIMMS	Advanced Interactive Multidimensional Modeling System
AIS	Künstliches Immunsystem (Artificial Immune System)
AMPL	A Mathematical Programming Language
ANN	Künstliches neuronales Netz (Artificial Neural Network)
AP	Alternating Position Crossover
APS	Advanced Planning System oder Advanced Planning and Scheduling
AS/RS	Regalbediengerät (Automated Storage/Retrieval System)
ATP	Available-to-promise (Funktionalität, die in APS zur Unterstützung der Auftragszusage und -erfüllung verwendet wird)
BA	Bienen-Algorithmus
BE	Bionic Engineering
BOM	Stückliste (Bill Of Materials)
CCLSP	Koordiniertes Losgrößenproblem mit Kapazitäten (Coordinated Capacitated Lot-Sizing Problem)
CFLP	Problem des Standortplanung mit Kapazitäten Einrichtung (Capacitated Facility Location Problem)
CI	Computational Intelligence
CMA	Anpassung der Kovarianzmatrix
CMA-ES	Evolutionsstrategie mit Anpassung der Kovarianzmatrix
CMWP	Weber-Problem bei mehreren kapazitätsbeschränkten Einrichtungen (Capacitated Multi-facility Weber Problem)
COI	Cube-per-Order-Index
CS	Cuckoo Search
CSCMP	Council of Supply Chain Management Professionals

CULSP	Koordiniertes Losgrößenproblem ohne Kapazitäten (Coordinated Uncapacitated Lot-Sizing Problem)
CVRP	Problem der Tourenplanung mit kapazitötsbeschränkten Fahrzeugen (Capacitated Vehicle Routing Problem)
CX	Cycle – Crossover
DE	Differential Evolution
DLL	Dynamisch verknüpfte Bibliothek (Dynamic Linked Library)
DPSO	Diskrete Partikelschwarmoptimierung (Discrete Particle Swarm Optimization)
DPX	Abstandserhaltendes Crossover
EA	Evolutionärer Algorithmus
EC	Evolutionäre Berechnungen (Evolutionary Computation)
ELS	Wirtschaftliche Losgröße (Economic Lot Size)
EOQ	Wirtschaftliche Bestellmenge (Economic Order Quantity)
EP	Evolutionäre Programmierung
ER	Genetisches Kantenrekombination-Crossover (Genetic Edge Recombination Crossover)
ERP	Enterprise Resource Planning
ERX	Kantenrekombination-Crossover (Edge Recombination Crossover)
ES	Evolutionsstrategie
FA	Firefly-Algorithmus
FIFO	First-in-first-out
FL	Unscharfe Logik (Fuzzy Logic)
FNN	Feedforward Neural Network
GA	Genetischer Algorithmus
GAMS	General Algebraic Modeling System
GCLSP	Allgemeines Losgrößenproblem mit Kapazitäten (General Capacitated Lot-Sizing Problem)
BIP	Bruttoinlandsprodukt
GE	Grammatical Evolution
GEP	Gene Expression Programming
GIS	Geografische Informationssysteme
GNX	Verallgemeinerter n-Punkt-Crossover (Generalized n-Point Crossover)
GP	Genetische Programmierung
GPS	Global Positioning System
GRASP	Greedy Randomized Adaptive Search Procedure
GSO	Glowworm Swarm Optimization (Glühwürmchen-Schwarmoptimierung)
HNN	Neuronale Hopfield-Netze
HS	Harmony Search
IEEE	Institute of Electrical and Electronics Engineers
ILS	Iterierte lokale Suche
ILS-FDD	Iterierte lokale Suche mit fitnessdistanzbasierter Diversifizierung

JiT	Just-in-time
LCS	Lernende Klassifizierungssysteme (Learning Classifier Systems)
LGP	Lineare genetische Programmierung
LOX	Linear Order Crossover
LSAP	Linear Sum Assignment-Problem
MA	Memetische Algorithmen
MACS	System mit mehreren Ameisenvölkern (Multiple Ant Colony System)
MCFP	Multicommodity-Flussproblem
MCLC	Kriterium der maximalen Abdeckung des Standorts (Maximal Covering Location Criterion)
MGP	Metagenetische Programmierung
MICLSP	Kapazitätsbeschränktes Losgrößenproblem für mehrere Produkte (Multi-Item Capacitated Lot-Sizing Problem)
MIP	Gemischt-ganzzahlige Optimierungsprobleme
ML	Maschinelles Lernen
MLLSP	Mehrstufiges Problem der Losgrößenbestimmung (Multilevel Lot-Sizing Problem)
MLULSP	Mehrstufiges Losgrößenproblem ohne Kapazitäten (Multilevel Uncapacitated Lot-Sizing Problem)
MNS	Suche in mehreren Nachbarschaften (Multi-Neighborhood Search)
MOO	Mehrzieloptimierung (Multiobjective Optimization)
MPX	Maximal Preservative Crossover
MRP	Materialbedarfsplanung (Material Requirement Planning)
MRP II	Manufacturing Resource Planning
NC	Natürliches Rechnen (Natural Computing)
NN	Neuronale Netze
NP	Nichtdeterministisch Polynomiell (Komplexitätsklasse)
NP-schwer	Komplexitätsklasse
NSGA, NSGA-II	Non-dominated Sorting Genetic Algorithms
OOP	Objektorientierte Programmierung
OpenOPAL	Software-Toolbox für Optimierung und Lernen
OPL	Optimierungsprogrammiersprache
OptimJ	Java-basierte Modellierungssprache für die Optimierung
OX, OX1, OX2	Order (oder Order-based) Crossover
PACO	Populationsbasierte Ameisenkolonie-Optimierung (Population-based Ant Colony Optimization)
PMX	Partially Mapped Crossover
PNN	Neuronale Perceptron-Netze
POS	Positionsbasiertes Crossover
POX	Precedence Preserving Order-based Crossover
PPS	Präzedenzerhaltende Verschiebungsmutation (Precedence Preserving Shift Mutation)
PSO	Partikelschwarm-Optimierung (Particle Swarm Optimization)

RFID	Radio-Frequency Identification
RIL	Verstärkungslernen (Reinforcement Learning)
ROI	Kapitalrendite (Return-on-investment)
RVNS-VNS	Reduzierte und Standard-Variable-Nachbarschaftssuche (Reduced and Standard Variable Neighborhood Search)
SA	Schwarm-Algorithmen
SC	Soft Computing
SCM	Supply Chain Management
SCX	Sequentiell-konstruktives Crossover (Sequential Constructive Crossover)
SI	Schwarmintelligenz
SICLSP	kapazitätsbeschränktes Losgrößenproblem bei einem Produkt (Single-Item Capacitated Lot-Sizing Problem)
SLAP	Problem der Lagerortzuweisung (Stock Location Assignment Problem)
SLSP	Single Link Shipping Problem
SOFM	Selbstorganisierende Merkmalskarten (Self-Organizing Feature Maps)
SOM	Selbstorganisierende Karten (Self-Organizing Maps)
SPEA, SPEA2	Strength Pareto Evolutionary Algorithms
SPT	Kürzeste Bearbeitungszeit (Shortest Processing Time)
SSCFLP	kapazitätsbeschränktes Standortproblem mit einer Quelle (Single Source Capacitated Facility Location Problem)
TMS	Transport-Management-System
TSP	Handlungsreisenden-Problem (Traveling Salesman Problem)
TSPLIB	Bibliothek für Traveling Salesman-Probleme
UFLP	Standortproblem ohne Kapazitäten (Uncapacitated Facility Location Problem)
UMWP	Weber-Problem mit mehreren Standorten ohne Kapazitätsbeschränkungen (Uncapacitated Multi-facility Weber Problem)
USILSP	Losgrößenproblem bei einem Produkt ohne Kapazitätsbeschränkung (Uncapacitated Single Item Lot-Sizing Problem)
VLSN	Sehr groß angelegte Nachbarschaftssuche (Very Large-scale Neighborhood Search)
VND	Variable Neighborhood Descent
VNDS	Variable Neighborhood Decomposition Search
VNS	Variable Nachbarschaftssuche (Variable Neighborhood Search)
VR	Voting Recombination Crossover
VRP	Tourenplanungsproblem (Vehicle Routing Problem)
VRPTW	Vehicle Routing-Problem mit Zeitfenstern (time windows)
WES, WMS	Lagerverwaltungssystem (Warehouse Execution/ Management System)
LVS	Lagerverwaltungssystem
ZIO	Bestellung ohne Lagerbestand (Zero Inventory Ordering)

Kapitel 1
Einführung in Logistik und Supply Chain Management



1.1 Das Konzept der Logistik und des Lieferkettenmanagements

Grob vereinfacht kann ein Unternehmen als ein System betrachtet werden, das einen bestimmten Input erhält, daraus Waren oder Dienstleistungen herstellt und den Output an seine Kunden liefert. Diese drei Hauptaktivitäten können als Beschaffung (oder Einkauf), Produktion und Vertrieb (oder Verkauf) bezeichnet werden. Bei Input und Output handelt es sich häufig um physische Objekte oder Materialien, die eine bestimmte Handhabung wie Transport oder Lagerung erfordern, um die Grundfunktionen eines Unternehmens zu erfüllen. Diese Tätigkeiten werden als die Kernaufgaben der Logistik betrachtet (Abb. 1.1).

Genauer gesagt kann die Inputs eines Unternehmens, die oft als Produktionsfaktoren bezeichnet werden, in einen Input, der während der Produktion verbraucht wird, und einen Input, der über einen längeren Zeitraum zur Verfügung steht, unterschieden werden. Die erste Gruppe der Produktionsfaktoren wird oft nur als Material oder Verbrauchsfaktoren bezeichnet, während die zweite Gruppe die klassischen Produktionsfaktoren Kapital und Arbeit (Personal) umfasst. Diese Faktoren werden oft auch als Ressourcen bezeichnet und umfassen Maschinen, Anlagen und Gebäude. Sie werden während der Produktion nicht verbraucht, aber häufig muss ihre Kapazität (verfügbare Zeit in einem bestimmten Zeitraum) mit der für die Produktion benötigten Zeit in Einklang gebracht werden. Auch „moderne“ Produktionsfaktoren wie Informationen, Humankapital oder Management gehören zu der letztgenannten Gruppe.

In einem eher traditionellen Verständnis bezieht sich die Logistik nur auf die Materialien und könnte daher auch als Materiallogistik bezeichnet werden. Natürlich lassen sich viele Konzepte der Logistik auch auf andere „Objekte“ wie Menschen anwenden. Auch Menschen (z. B. Touristen) müssen transportiert und untergebracht werden.

© Der/die Autor(en), exklusiv lizenziert an Springer Nature Switzerland AG 2023

T. Hanne, R. Dornberger, *Computational Intelligence in Logistik und Supply Chain Management*, https://doi.org/10.1007/978-3-031-21452-3_1

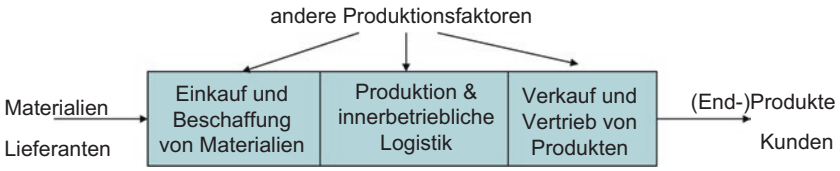


Abb. 1.1 Das Unternehmen und seine Hauptfunktionen

Abstrakter ausgedrückt befasst sich die Logistik mit dem räumlichen, zeitlichen und mengenmäßigen Transfer von Materialien von der Beschaffung der für die Produktion benötigten Materialien über die Lagerung von Materialien, Zwischenprodukten und Fertigprodukten bis hin zur physischen Verteilung an die Kunden. Die Logistik befasst sich also mit der Planung und Durchführung von räumlichen, zeitlichen und quantitativen Transfers.

Räumliche Transfers können einfach als Transport bezeichnet werden und lassen sich in Lang- und Kurzstreckentransporte unterteilen. Langstreckentransporte sind Transporte zwischen verschiedenen Orten wie Lagern, Fabriken und verschiedenen Unternehmen, die in erster Linie mit Lastwagen, Zügen, Schiffen und Flugzeugen durchgeführt werden. Unter Kurzstreckentransport versteht man Transporte innerhalb eines Ortes (Werk, Lager). Diese Art des räumlichen Transfers wird gelegentlich auch als Materialfluss bezeichnet. Im Kurzstreckentransport werden in der Regel andere Geräte als im Langstreckentransport eingesetzt, z. B. Gabelstapler, Förderbänder oder fahrerlose Transportsysteme.

Zeitlicher Transfer bedeutet „Transport“ über die Zeit, d. h. von heute, wo ein Material verfügbar ist, in die Zukunft, wenn das Material benötigt wird. Dies ist der Zweck dessen, was wir einfacher als Lagerung oder Lagerhaltung bezeichnen.

Mengenmäßige Transfers finden statt, wenn z. B. große Mengen bestimmter Waren in kleineren Mengen aufgegliedert werden. Dies ist eine der üblichen Tätigkeiten von Einzelhändlern, die Waren in größeren Mengen von den Herstellern oder Großhändlern kaufen und sie in der Regel in kleineren Mengen an die Endkunden verkaufen. Mengenänderungen finden auch statt, wenn Kundenaufträge erfüllt werden. Die bestellten Artikel werden aus dem Lager geholt (wo sie in der Regel in größeren Mengen vorhanden sind), zusammengestellt, verpackt und an die Kunden versandt.

Unter Berücksichtigung dieser drei Aspekte können wir die Hauptaufgaben der Logistik als Prozesse zum Ausgleich von räumlichen, zeitlichen und mengenmäßigen Unterschieden bei Gütern definieren. Es sei darauf hingewiesen, dass die Produktion selbst nicht als eine logistische Tätigkeit angesehen wird. Logistik ist vielmehr alles, was „um“ die Produktion herum im Hinblick auf die physischen Produkte benötigt wird, die schließlich an die Kunden geliefert werden.

Eine andere häufig genutzte Möglichkeit, Logistik zu definieren, besteht darin, die Aufgaben der Logistik durch verschiedene Aspekte auszudrücken, die richtig gemacht werden müssen. Heute wird dies durch die sechs (oder sieben) Rs der Logistik spezifiziert: Die richtigen Artikel (Material), in der richtigen Menge, zur rich-

tigen Zeit, am richtigen Ort, in der richtigen Qualität (Zustand), mit den richtigen Kosten (und den richtigen Informationen).

Betrachtet man neuere Definitionen der Logistik, so stellt man fest, dass sie ausführlicher formuliert sind. So definiert Christopher (2010, S. 4) Logistik als „den Prozess des strategischen Managements der Beschaffung, Bewegung und Lagerung von Materialien, Teilen und Fertigbeständen (und der damit verbundenen Informationsflüsse) durch das Unternehmen und seine Marketingkanäle in einer Weise, dass die aktuelle und zukünftige Rentabilität durch die kosteneffiziente Erfüllung von Aufträgen maximiert wird".

Eine andere, eher allgemein gehaltene Definition des Council of Logistics Management beschreibt Logistik als „den Prozess der Planung, Umsetzung und Kontrolle des effizienten, effektiven Flusses und der Lagerung von Waren, Dienstleistungen und damit zusammenhängenden Informationen vom Ursprungsort bis zum Verbrauchsort mit dem Ziel, die Kundenanforderungen zu erfüllen" (Lambert, 2008).

Die Logistik betrifft also nicht nur die Produktion von physischen Gütern, sondern von Waren und Dienstleistungen im Allgemeinen. Sie ist daher auch für die öffentliche Verwaltung und Institutionen wie Krankenhäuser, Schulen und Dienstleistungsunternehmen wie Händler, Banken und andere Finanzdienstleister oder Versicherungen relevant. Ein weiterer interessanter Aspekt in solchen Definitionen ist, dass sich Logistik nicht nur auf unternehmensinterne Prozesse und Prozesse mit direkten Marktpartnern bezieht, sondern auch darüber hinausgehende Prozesse umfasst. Streng genommen umfasst „from point of origin to point of consumption" alles von der Urproduktion landwirtschaftlicher Produkte oder der Gewinnung von Rohstoffen in Bergwerken bis hin zu fertigen Produkten, die von Kunden (oder Unternehmen) genutzt werden. In der Regel sind an den entsprechenden Prozessen eine Vielzahl von Unternehmen beteiligt, und es ist oft weder sinnvoll noch möglich, jeden Schritt dieser Prozesse zu berücksichtigen. Realistischerweise sollten nur diejenigen Teile der Gesamtprozesse berücksichtigt werden, bei denen eine gemeinsame Planung der Aktivitäten möglich und für die beteiligten Partner sinnvoll ist.

Zusammenfassend lässt sich sagen, dass es eine eher enge Sichtweise gibt, die die Logistik auf den Transport und die Lagerung physischer Güter (materielle Logistik) beschränkt, und eine umfassendere Sichtweise, die immaterielle Güter (Dienstleistungen) einbezieht, benachbarte Prozesse berücksichtigt und den Anwendungsbereich auf andere Unternehmen im Beschaffungs- und Distributionsnetz ausdehnt. Wir haben uns für eine umfassendere Sichtweise entschieden, betrachten aber hauptsächlich Situationen in der Materiallogistik, da sie anschaulicher sind.

Der Trend zu einer umfassenderen Definition der Logistik hat sich in der Idee dessen niedergeschlagen, was heute als Supply Chain Management bezeichnet wird. Dieses Konzept wurde insbesondere durch die Erkenntnis motiviert, dass viele logistische Prozesse nicht nur in einem betrachteten Unternehmen relevant sind, sondern dass solche Prozesse auch bei Lieferanten und Kunden berücksichtigt werden sollten, um den Endkunden ein gutes Produkt oder eine gute Dienstleistung zu liefern. Darüber hinaus ist es aus Sicht der beteiligten Unternehmen oft sinnvoll,

diese Aktivitäten integriert zu planen, um den Beitrag der Partner in der Supply Chain zu nutzen und ihren Mehrwert zu maximieren.

Werfen wir einen Blick auf einige neuere Definitionen des Supply Chain Managements. Cooper und Ellram (1993) definieren es als „eine integrative Philosophie zur Verwaltung des gesamten Flusses eines Vertriebskanals vom Lieferanten bis zum Endverbraucher". Harland (1996) definiert Supply Chain Management als „das Management eines Netzes von miteinander verbundenen Unternehmen, die an der letztendlichen Bereitstellung von Produkt- und Dienstleistungspaketen für den Endkunden beteiligt sind". Eine offiziellere Definition stammt vom Council of Supply Chain Management Professionals (CSCMP): „Supply Chain Management umfasst die Planung und das Management aller Aktivitäten, die mit der Beschaffung, der Umwandlung und dem gesamten Logistikmanagement verbunden sind. Wichtig ist, dass es auch die Koordinierung und Zusammenarbeit mit den Vertriebspartnern umfasst, bei denen es sich um Lieferanten, Zwischenhändler, Dienstleister und Kunden handeln kann. Im Wesentlichen integriert das Supply Chain Management das Management von Angebot und Nachfrage innerhalb und zwischen Unternehmen".

1.2 Eine kurze Geschichte der Logistik

Der Ursprung des Wortes „Logistik" ist recht alt. Es leitet sich von dem altgriechischen Wort „logos" ab, was so viel wie (geschriebene) Rede, Sprache, Wort, Vernunft und Berechnung bedeutet. Abgesehen vielleicht von „Vernunft" gibt es wenig offensichtliche Verbindung zu unserem heutigen Verständnis von Logistik.

Der Aspekt der Berechnung wurde jedoch zur Bezeichnung einer Verwaltungsperson im antiken Rom oder Griechenland als „Logistika" herangezogen, die für Finanzen, Beschaffung und Vertrieb zuständig war, wobei der Schwerpunkt auf militärischen Aktivitäten lag (Tudor, 2012). Ein solcher Fokus auf militärische Aktivitäten wurde in der Neuzeit aufgegriffen. Im neunzehnten Jahrhundert wurden die Begriffe Taktik, Strategie und Logistik allgemein verwendet, um wesentliche Aktivitäten für eine erfolgreiche Kriegsführung zu unterscheiden. Dem Oxford English Dictionary zufolge ist Logistik „der Zweig der Militärwissenschaft, der sich mit der Beschaffung, der Instandhaltung und dem Transport von Material, Personal und Einrichtungen befasst". Es handelte sich also nicht um militärische Aktivitäten im engeren Sinne, sondern um alles, was für die Beherrschung solcher Aktivitäten als notwendig erachtet wurde.

Genau wie Taktik und Strategie fand die Logistik ihren Weg von der militärischen Planung in den zivilen Bereich. Die erste breitere Anwendung in der Wirtschaft fand in den 1960er-Jahren in den USA statt, wobei der Schwerpunkt auf der Planung und Organisation von Vertriebsaktivitäten lag.

Von da an wurde die Logistik immer populärer, ihre Bedeutung und Anwendungsbereiche wurden erheblich erweitert. Die Logistik wurde durch weitere Konzepte ergänzt, insbesondere durch das „Supply Chain Management", das in den

1980er-Jahren aufkam und sich auf ein noch breiteres Feld konzentrierte. Die Einsicht, dass sich die Logistik nicht nur auf isolierte Aktivitäten (wie einen einzelnen Transport oder die Lagerung einer Ware an einem bestimmten Ort) konzentrieren sollte, war der Ausgangspunkt für ein prozessorientiertes Denken und die Betrachtung logistischer Netzwerke (oder „Lieferketten"). Es wurde deutlich, dass logistische Aktivitäten in der Regel eng mit anderen (logistischen oder nicht-logistischen) Aktivitäten verbunden sind, so dass eine integrierte Planung einen höheren Nutzen für die Kunden bzw. eine größere Wertschöpfung für die beteiligten Unternehmen erzeugen kann.

Obwohl es scheint, dass Logistik und Supply Chain Management eher moderne Ideen sind, gibt es viele Hinweise dafür, dass selbst komplexe Logistikprobleme schon in der fernen Vergangenheit erfolgreich bewältigt wurden. Wenn wir in die prähistorische Zeit zurückgehen, wurden die frühesten menschlichen Zivilisationen als Jäger und Sammler bezeichnet. Diese Zivilisationen wurden also nach einer logistischen Tätigkeit, der Beschaffung von Nahrung, benannt. Abgesehen davon können wir davon ausgehen, dass auch andere logistische Tätigkeiten eine große Rolle spielten: Aufgrund des wechselhaften Nahrungsangebots – oft unsicher und schwankend im Laufe eines Jahres – waren Maßnahmen zur Lagerung von Nahrungsmitteln für das Überleben eines Stammes unerlässlich. Ein weiterer Aspekt war die Verteilung der Nahrung: Da nicht alle Mitglieder eines Stammes auf die Jagd gingen oder Nahrung sammelten – und da die Jäger und Sammler nicht alle gleich erfolgreich waren – war es sinnvoll, die Beute und den Ertrag zu teilen, um eine bessere Arbeitsteilung zu erreichen und die Risiken zu verringern.

Ein weiteres prähistorisches Beispiel für Logistik ist der Transport und Handel von Waren wie Steinen, Keramik, Metallen oder anderen Rohstoffen, manchmal sogar über große Entfernungen. So gibt es gute Belege dafür, dass Obsidian, ein vulkanisches Glas, das weltweit nur begrenzt vorkommt, über mehrere hundert Kilometer transportiert wurde (Dogan & Michailidou, 2008).

Antike Beispiele für große Logistikoperationen sind die antiken Eroberungskriege. Beeindruckende Beispiele im zivilen Bereich sind z. B. Bewässerungssysteme, die manchmal auf Netzen von Kanälen, Tunneln oder Rohren von mehreren hundert Kilometern Länge beruhen, z. B. in der Maya-, der persischen oder der römischen Zivilisation.

1.3 Jüngste Trends und die moderne Bedeutung der Logistik

Obwohl viele Probleme und Konzepte in der Logistik nicht so neu sind, gibt es gute Gründe, warum die Logistik in den letzten Jahrzehnten an Bedeutung gewonnen hat und warum die zunehmende öffentliche (und akademische) Aufmerksamkeit in diesem Bereich nicht nur ein moderner Hype ist.

Einerseits gibt es eine Reihe allgemeiner wirtschaftlicher Trends, die ein verstärktes Engagement in Logistikfragen begünstigen. Andererseits war der technologische Fortschritt in der Logistik und verwandten Bereichen erheblich.

Beginnen wir mit allgemeinen wirtschaftlichen Aspekten. Eines der langfristigen Konzepte zur Darstellung der wirtschaftlichen Entwicklung ist die Drei-Sektoren-Hypothese. Grob gesagt basiert diese Hypothese auf einer Klassifizierung der wirtschaftlichen Aktivitäten in drei Sektoren: Dem primären Sektor, der sich mit der Produktion von Rohstoffen (insbesondere die Landwirtschaft) befasst, dem sekundären Sektor, der das verarbeitende Gewerbe und die Industrie umfasst, und dem dritten oder tertiären Sektor, der sich mit den Dienstleistungen befasst. Während der Reifung von Volkswirtschaften findet in der Regel die folgende Entwicklung statt: Zunächst (d. h. vor einigen hundert Jahren) dominiert der Primärsektor stark. Dann wächst der sekundäre Sektor und wird zum wichtigsten Sektor, z. B. während der industriellen Revolution in Europa und den USA im 19. Jahrhundert. Später, insbesondere in den Industrieländern in der zweiten Hälfte des 20. Jahrhunderts, beginnt der zweite Sektor zu schrumpfen, während der dritte Sektor wächst und schließlich dominiert. Diese langfristigen Entwicklungen zeigen die Bedeutung der Logistik, denn logistische Tätigkeiten können als Dienstleistungen eingestuft werden und gehören daher zum wachsenden dritten Sektor.

Auch wenn dies wenig über die wachsende Bedeutung bestimmter Logistikaktivitäten aussagt, gibt es weitere Anhaltspunkte: Im Zuge der langfristigen wirtschaftlichen Entwicklung sind Wachstum und ein höherer Lebensstandard zu beobachten. Dieser gestiegene Lebensstandard führt zu komplexeren und anspruchsvolleren Kundenbedürfnissen. So verlangen die Kunden beispielsweise eine höhere Qualität von Waren und Dienstleistungen, die Schnelligkeit und Sicherheit von Lieferungen sind wichtiger geworden, und auch Sonderwünsche wie individualisierte Waren oder Dienstleistungen sind oft zu berücksichtigen. Einige dieser Aspekte haben klare logistische Auswirkungen: Für eine schnelle und sichere Versorgung sind gute Bedarfsprognosen, eine angemessene Lagerhaltung oder schnelle Transportsysteme sehr wichtig. Andere Aspekte wie die Qualität der Produkte sind teilweise von einer guten Beherrschung der Logistik abhängig. Die Auswahl der Lieferanten, ein effizienter Umgang mit den Materialien, die Vermeidung mangelhafter Produkte, ein angemessener Umgang mit Kundenreklamationen usw. können zu der von den Kunden wahrgenommenen Qualität beitragen.

Neben diesen kundenorientierten Aspekten sind verschiedene andere globale Entwicklungen in der Wirtschaft für die Entwicklung von Logistik und Supply Chain Management von Bedeutung: Eine davon ist die wirtschaftliche Liberalisierung. Über einen längeren Zeitraum hinweg haben wir eine Stärkung des Freihandels beobachtet. Seitdem Ricardo im 19. Jahrhundert die Vorteile des Freihandels nachwies, wurden in vielen Teilen der Welt Maut- und Zollgebühren gesenkt oder abgeschafft. Auch andere Beschränkungen des internationalen Handels, wie Einfuhr- oder Ausfuhrkontingente, wurden in vielen Fällen abgebaut. Die Niederlassungsfreiheit wurde in vielen Teilen der Welt verwirklicht und es ist heute viel einfacher geworden, in anderen Ländern ein Unternehmen zu gründen. Der Zugang zu bestimmten Branchen und Märkten (z. B. in der Schifffahrt) wurde erleichtert. All dies führt zu besonderen Chancen, aber auch zu neuen Herausforderungen durch einen erhöhten Wettbewerbsdruck.

Seit ihrem Zusammenbruch Ende der 1980er-Jahre haben die ehemals kommunistischen Länder eine besondere Rolle gespielt. Heute haben sich die meisten von ihnen in Marktwirtschaften verwandelt. Andere Nationen, die noch vor wenigen Jahrzehnten als Entwicklungsländer bezeichnet wurden, haben ein starkes Wirtschaftswachstum gezeigt, und einige von ihnen werden heute eher als Schwellenländer bezeichnet. Zu den bekanntesten gehören die BRIC-Länder Brasilien, Russland, Indien und China. Diese Ländergruppe wurde erstmals von O'Neill (2001) analysiert, und es wird erwartet, dass sie (gemessen am BIP) bis 2050 größer wird als die G6, die ehemals größten Industrieländer (USA, Japan, Deutschland, Frankreich, Großbritannien und Italien) (Wilson & Purushothaman, 2003).

Besonders seit Anfang des 21. Jahrhunderts hat der Handel zwischen weit entfernten Teilen der Welt (insbesondere zwischen Ostasien und Europa/USA) erheblich zugenommen. Während Länder wie China vor 10 oder 20 Jahren allgemein als Hersteller billiger, minderwertiger Waren eingeschätzt wurden, werden dort heute viele qualitativ hochwertige Produkte hergestellt. Die Herausforderungen des Wettbewerbs sind offensichtlich. Nicht nur die Produktionsbedingungen in den verschiedenen Teilen der Welt werden immer ähnlicher, sondern auch die Kundenwünsche passen sich an. So ist es zum Beispiel schon vor einigen Jahrzehnten üblich geworden, in westlichen Ländern chinesisches Essen zu essen. Aber auch ein umgekehrter Trend ist inzwischen zu beobachten: Während ein Produkt wie Schokolade vor 20 Jahren in China eine verschwindend geringe Rolle spielte, ist es heute sehr beliebt. Es gibt also auch klare Chancen für Industrien in westlichen Ländern, die oft auf eher teure Qualitätsprodukte setzen. Steigende Einkommen in Schwellenländern machen solche Produkte für die Bevölkerung erschwinglicher.

Wie bereits erwähnt, besteht eine gewisse Folge dieser Globalisierung darin, dass immer mehr Waren über größere Entfernungen transportiert werden. Infolgedessen sind höhere Transportkosten in den Volkswirtschaften zu erwarten. Betrachtet man die Transportkosten (z. B. für die USA, siehe unten) in Prozent des BIP, so stellt man fest, dass sie in den letzten Jahrzehnten eher gesunken sind. Dies ist umso bemerkenswerter, als die Treibstoffpreise in dieser Zeit tendenziell gestiegen sind (mit starken Schwankungen). Die Hauptgründe hierfür sind in Aspekten des technischen Fortschritts zu suchen. Zunächst einmal sind viele Fahrzeugtypen (Lkw, Flugzeuge, Schiffe) sparsamer als früher. Oft sind die Fahrzeuge größer geworden (vor allem Schiffe), was ebenfalls zu niedrigeren Kosten pro transportierter Tonne Fracht führt. Größenvorteile ergeben sich nicht nur aus technischen Gründen. So kann zum Beispiel die Besatzung eines größeren Schiffes in der Größe konstant gehalten werden.

Neben diesen Aspekten der einzelnen Transportaktivitäten wurde auch die gesamte Infrastruktur für Transporte verbessert. Zum Beispiel können Umschlagaktivitäten in einem Hafen heute reibungsloser durchgeführt werden. Der Container als standardisiertes Transportmittel spielt dabei eine große Rolle. Eine Transportkette aus Lkw-, Bahn- oder Schiffstransport (intermodaler Transport) funktioniert effizienter als in der Vor-Containerzeit. Der Transport wird heute besser durch Informations- und Kommunikationstechnologie unterstützt als früher. GPS-basierte Navigationssysteme erleichtern die Planung und Durchführung von Transporten. Bessere

Planungsalgorithmen ermöglichen einen effizienteren Transport. Dieser Aspekt wird in Kap. 3 ausführlicher behandelt.

Abgesehen von diesen transportbezogenen Aspekten unterstützt der technische Fortschritt im Bereich der Informations- und Kommunikationstechnologien auch andere logistische Aktivitäten (und natürlich Geschäftsaktivitäten im Allgemeinen). Einige der offensichtlichsten Aspekte sind die verbesserte Leistung von Computern und die Senkung der entsprechenden Kosten in den letzten Jahrzehnten. Eine weitere offensichtliche Entwicklung ist der Aufstieg des Internets und das Aufkommen des E-Business. Betrachten wir nur ein Beispiel dafür, wie die Logistik direkt von diesen Trends beeinflusst wird: Aufgrund des E-Business werden Waren heute immer häufiger direkt an die Kunden versandt, anstatt über den Einzelhandel vertrieben zu werden. Im Vergleich zu Sendungen an Einzelhändler erfordert dies eine wesentlich größere Anzahl von meist kleineren Sendungen.

Eine weitere, weniger sichtbare Auswirkung des technologischen Fortschritts ist der verstärkte Einsatz von Automatisierungstechnik in Unternehmen. Unternehmen bewältigen ihre innerbetrieblichen Logistikaufgaben heute oft mit einem hohen Anteil an Materialflusstechnik oder Robotik. Beispiele für solche Technologien sind Förderanlagen für den innerbetrieblichen Transport, automatische Regalbediengeräte, fahrerlose Transportsysteme (FTS) oder die verstärkte Nutzung mobiler Geräte durch den Menschen. Eine spezielle Technologie, die in den letzten 10 Jahren intensiv diskutiert wurde, ist RFID (Radiofrequenz-Identifikation). Kleine Chips (oder Smart Tags), die aus einer elektronischen Schaltung (einschließlich eines Speichers und möglicherweise Sensoren) und einer Antenne für die Kommunikation bestehen, können an Waren (oder Paketen) angebracht werden. Diese Technologie kann zur besseren Identifizierung von Gegenständen, zur besseren Überwachung und Verfolgung während des Transports oder für intelligentere Zwecke wie die Überwachung einer Kühlkette für Tiefkühlkost eingesetzt werden.

Zusammen mit neuen Planungstechnologien und entsprechender Software kann dieser technische Fortschritt zu innovativen Logistiklösungen und/oder zur Senkung der Kosten führen.

Wenn wir die gegenwärtige Bedeutung der Logistik verstehen wollen, ist es sinnvoll, die quantitativen Daten dieses Wirtschaftssektors zu betrachten. Für die USA gibt es eine regelmäßige Statistik über die Logistik in den jährlichen „US State of Logistics Reports“, die vom Council of Supply Chain Management Professionals (CSCMP) in Auftrag gegeben werden. Für das Jahr 2014 werden in dem jeweiligen Bericht die folgenden Daten zu den Logistikkosten ausgewiesen (Wilson, 2013; Robinson, 2015):

- 917 Mrd. US$ für den Transport (702 Mrd. US$ für Strassentransporte)
- 476 Mrd. US$ für Lagerhaltung einschließlich Zinsen und Steuern usw.
- 56 Mrd. US$ für Logistikverwaltung
- 1449 Mrd. US$ Gesamtlogistikkosten

Bei diesen beeindruckenden Zahlen sollte man bedenken, dass viele Kosten für Logistikdienstleistungen (insbesondere für intern erbrachte) in den offiziellen Statistiken gar nicht berücksichtigt werden. Sehr oft kennen die Unternehmen nicht einmal

Tab. 1.1 Logistikkosten in den USA

	Logistikkosten insgesamt (%)	Lagerkosten (%)	Transportkosten (%)
1981	16,2	8,3	7,3
1986	11,6	4,9	6,3
1991	10,6	4,3	5,9
1996	10,3	3,9	6
2001	9,5	3,4	5,8
2003	8,6	2,8	5,4
2006	9,9	3,2	5,8
2007	10,1	3,5	6,2
2008	9,4	2,9	6,1
2009	7,7	2,5	4,9
2010	8,3	2,7	5,2
2011	8,5	2,8	5,3
2012	8,5	2,8	5,3
2013	8,5	2,9	5,3
2014	8,0	2,7	5,0

Quelle: Eigene Berechnungen und Wilson (2013)

ihre eigenen Logistikkosten, da ein Teil davon in ihrer Finanzbuchhaltung nicht genau zugeordnet wird (z. B. Kosten für innerbetriebliche Transporte).

Tab. 1.1 zeigt die Logistikkosten für die USA, ausgedrückt in Prozent des Bruttoinlandsprodukts (BIP) dieses Landes. Dargestellt sind die gesamten Logistikkosten und die beiden wichtigsten Unterkategorien, die lagerhaltungsbezogenen Kosten und die transportbezogenen Kosten (die Kategorie Verwaltungskosten wurde weggelassen, so dass die beiden Prozentsätze der Unterkategorien etwas geringer sind als der Prozentsatz für die gesamten Logistikkosten). Ferner wird die Entwicklung dieser Kosten von den frühen 1980er-Jahren bis 2012 dargestellt.

Bei der Betrachtung der zeitlichen Entwicklung ist es interessant zu sehen, dass der Prozentsatz in den letzten 30 Jahren zurückgegangen ist (wobei in einigen Jahren ein erneuter Anstieg der Kosten zu verzeichnen war). Die größten Kostensenkungen wurden in der ersten Hälfte der 1980er-Jahre, um das Jahr 2000 und dann wieder nach 2007 erzielt.

Die Prozentsätze für die Unterkategorien zeigen, dass die größten Rückgänge bei den Lagerkosten zu verzeichnen waren, nämlich von 8,3 % im Jahr 1981 auf 2,8 % im Jahr 2012. Einerseits kann dies auf einige allgemeine wirtschaftliche Gründe zurückgeführt werden, wie z. B. eine schlechte Wirtschaftslage zu Beginn der 1980er-Jahre, die häufig mit hohen Lagerbeständen einhergeht. Andererseits ist davon auszugehen, dass Fortschritte durch ein erhöhtes Bewusstsein für diese Kostenkomponente in den Unternehmen, durch eine stärkere Fokussierung auf Supply-Chain-Management-Konzepte und durch den Einsatz entsprechender Planungstechniken erzielt wurden. Außerdem können wir davon ausgehen, dass die Kosteneinsparungen nicht auf Kosten häufigerer Stock-Out-Situationen erzielt wurden. Stock-outs wurden in den letzten 40 Jahren reduziert, schienen dann aber auf einem durchschnittlichen Niveau von etwa 7–8 % im Einzelhandel zu verharren

Tab. 1.2 Logistikkosten in Europa

	Umsatz in der Logistik (Bio EUR im Jahr 2007)	Logistikkosten (in % des BIP 2007)
Deutschland	205	7,8
Frankreich	111,9	8,5
UK	107,3	8,3
Italien	81,3	5,3
Spanien	81	8,3
Niederlande	45,9	8,4
Belgien	31,8	7,2
Schweden	28	8,6
Polen	26,1	(fehlt)
Finnland	22,4	14,7

(Fernie & Grant, 2008). In ihrer umfassenden Studie stellen Gruen et al. (2002) ein weltweites Niveau von etwa 8,3 % an Stock-Outs fest, das in Europa etwas höher und in den USA niedriger ist, und spekulieren über ein „natürliches Niveau" für Stock-Outs.

Nicht nur die lagerhaltungsbezogenen Logistikkosten wurden gesenkt, auch bei den Transportkosten ist ein gewisser Rückgang zu beobachten, insbesondere in den 1980er-Jahren und nach 2008. Dies ist erstaunlich, da die Kraftstoffpreise, ein wesentlicher Bestandteil der Transportkosten, tendenziell stiegen – wenn auch mit starken Schwankungen. Diese Senkung der Transportkosten ist umso bemerkenswerter, als wir in den letzten Jahrzehnten einen Anstieg des Transportvolumens und eine Vergrößerung der Transportentfernungen feststellen. Wir gehen daher davon aus, dass die überwiegenden Gründe für den Rückgang der Transportkosten (in Prozent des BIP) der technologische Fortschritt und die Nutzung besserer Planungstechniken sind.

Die Logistikkostendaten für die USA scheinen mit denen aus anderen Teilen der Welt vergleichbar zu sein. Tab. 1.2 zeigt die absoluten Logistikkosten (gemessen am Umsatz von Logistikdienstleistern) und die Logistikkosten in Prozent des BIP für mehrere europäische Länder (Klaus & Kille, 2009; DHL, o. J.). Wir sehen, dass die Kosten meist im Bereich von 7–9 % liegen, mit einigen Ausnahmen, wie Italien mit einem deutlich niedrigeren und Finnland mit einem viel höheren Prozentsatz.

1.4 Die Notwendigkeit einer besseren Planung

Wie wir gesehen haben, hat die Bedeutung der Logistik zugenommen, aber gleichzeitig waren der Wettbewerb und der Kostendruck enorm und haben dazu geführt, dass weniger Ausgaben für Logistikaktivitäten erforderlich waren. Es besteht Grund zu der Annahme, dass dieser Trend auch in Zukunft anhalten wird. Eine große He-

rausforderung ist daher, wie Unternehmen strategische Wettbewerbsvorteile im Bereich der Logistik erzielen können.

Solche Wettbewerbsvorteile können vielfältig sein, aber Kostenaspekte werden wahrscheinlich eine dominierende Rolle spielen. Weitere Wettbewerbsvorteile oder spezifische Ziele im Zusammenhang mit der Logistik können insbesondere zeitliche Aspekte, Qualitätsaspekte und Aspekte der Flexibilität sein.

Im Hinblick auf die Zeit werden häufig kürzere Lieferzeiten und schnellere Reaktionen auf Nachfrageänderungen diskutiert. Aufgrund des technologischen Fortschritts oder schwankender Kundenpräferenzen wird auch die Markteinführungszeit neuer Produkte und Dienstleistungen immer wichtiger.

Qualität bezieht sich auf viele Aspekte, die zur Erfüllung der Kundenanforderungen beitragen. Sie beziehen sich auf die physischen Produkte, aber auch auf begleitende Dienstleistungen, z. B. Lieferzeiten, Vermeidung von Ausschuss oder angemessene Reaktionen auf Kundenreklamationen und eine schnelle und zufriedenstellende Reparatur oder Überholung von Produkten. Flexibilität ist schwieriger zu spezifizieren, konzentriert sich aber auf das Potenzial eines Unternehmens, schnell und angemessen auf Veränderungen der Kundenpräferenzen und der Nachfrage, des Wettbewerbs und anderer Umweltfaktoren zu reagieren.

Viele verschiedene Managementaktivitäten – abhängig von den spezifischen Branchen und Märkten – können zur Erreichung dieser Ziele beitragen. In jedem Fall unterstützt eine bessere Planung das Erreichen solcher Ziele. Eindrucksvolle Beispiele für die Auswirkungen einer besseren Planung, insbesondere einer besseren Planung auf der Grundlage fortgeschrittener quantitativer Methoden, finden sich auf der INFORMS-Website „Getting Started with Analytics“ (INFORMS, 2016). Die Website ist mit einer Datenbank verbunden, die Dutzende von kurzen Fallstudien enthält, die die Potenziale von Analytics in einer Vielzahl von Branchen, Funktionsbereichen oder in Bezug auf verschiedene Arten von Nutzen zeigen. Ein beträchtlicher Teil der enthaltenen Fallstudien bezieht sich auf logistische Aspekte wie Bestandsmanagement, Transport, Routenplanung oder Scheduling oder befasst sich mit dem Supply Chain Management im allgemeineren Sinne. Bei den berichteten Vorteilen handelt es sich häufig um Kosteneinsparungen (oder Umsatzsteigerungen) in einer Größenordnung von mehreren Dutzend Millionen US-Dollar bis zu mehr als 100 Millionen US-Dollar. In seiner Übersicht über mehrere Erfolgsgeschichten berichtet Lustig (2013) von Kosteneinsparungen, die häufig im Bereich von 5 % oder mehr liegen, und von Einsparungen bei bestimmten Ressourcen von 30 % oder mehr, wenn entsprechende Planungstechniken eingesetzt wurden.

Aus diesen Gründen besteht ein dringender Bedarf an einer besseren Planung der Logistikaktivitäten, um Kosten zu senken und andere Ziele zu erreichen. Ein Teil des Fortschritts in der Logistik während der letzten Jahrzehnte wurde bereits durch eine bessere Planung der jeweiligen Prozesse erzielt. Für einige Prozesse ist der Einsatz modernster Planungstechnologien bereits Realität geworden. Wenn beispielsweise ein einzelner Transport geplant werden soll, ist der Einsatz von Navigationssoftware gängige Praxis. Solche Tools ermöglichen eine einfache und präzise Lokalisierung der aktuellen Position einer Person, einer Ware oder eines Fahrzeugs. Sie liefern aktuelle Informationen über Karten und Verkehrsnetze. Und sie ermögli-

chen die Berechnung eines recht präzisen kürzesten (oder schnellsten) Weges von A nach B. Andere Planungsbereiche, in denen große Fortschritte erzielt wurden, sind zum Beispiel die Vorhersage der Nachfrage oder die Planung von Lagerbeständen.

Dennoch werden schwierigere Planungsaufgaben in der Praxis noch nicht sehr gut unterstützt. Dies hat damit zu tun, dass viele der damit verbundenen Optimierungsprobleme rechnerisch schwierig sind, so dass es keine effizienten Algorithmen gibt, um sie optimal zu lösen. Darüber hinaus sind einige der damit zusammenhängenden formalen Probleme recht neu, so dass noch nicht viel Aufwand betrieben wurde, um praktische Lösungen und Software bereitzustellen.

Eine weitere Schwierigkeit besteht darin, dass die logistischen Probleme eines Unternehmens häufig schwer zu standardisieren sind. Eine allgemeine Formulierung des Planungsproblems und eine Konzeption und Implementierung der entsprechenden Software ist für die spezifischen Aufgaben unzureichend. Dieses Fehlen eines „allgemeinen Problemlösers“ erschwert und verteuert die Entwicklung und Bereitstellung von Lösungen, die den tatsächlichen Planungsproblemen eines Unternehmens gerecht werden. Von einem eher theoretischen Standpunkt aus betrachtet, wissen wir, dass der Teufel oft im Detail steckt. Geringfügige Änderungen in einer Problemformulierung können große Auswirkungen auf ihre Komplexität oder die geeigneten Algorithmen zu ihrer Lösung haben.

In diesem Buch wird versucht, typische Planungsprobleme der Logistik zu formulieren und Algorithmen vorzustellen, die grundsätzlich geeignet sind, diese Probleme zu lösen. Da ein spezifisches Problem in der Regel mehr oder weniger einzigartig ist, scheinen besondere Anpassungen der Problemformulierungen oder Spezifikationen der Algorithmen notwendig zu sein. Erfreulicherweise ist die betrachtete Klasse von Methoden, d. h. Ansätze aus der Computational Intelligence und insbesondere Metaheuristiken, anpassungsfähig. Sie sind auf eine problemspezifische Anpassung ausgelegt. Obwohl viele von ihnen durch generische Formulierungen spezifiziert werden können, erfordert ihre Leistungsfähigkeit und Überlegenheit gegenüber klassischen Optimierungsansätzen in der Regel einen problemspezifischen Entwurf. Dies betrifft z. B. die Frage, wie ein Problem bzw. seine Lösungen kodiert werden sollten, um es für eine entsprechende Methode behandelbar zu machen. Darüber hinaus bestehen die Verfahren in der Regel aus grundlegenden Schritten oder Phasen (oft als Operatoren bezeichnet), die auch problemspezifische Anpassungen ermöglichen. Ein evolutionärer Algorithmus basiert beispielsweise auf mehreren evolutionären Operatoren wie Mutation, Rekombination oder Selektion, die auf vielfältige Weise implementiert werden können. Jeder dieser Operatoren kann oder sollte spezifisch sein, um für das betrachtete Optimierungsproblem am effektivsten zu sein. Im weiteren Verlauf dieses Buches werden wir einige dieser Anpassungen betrachten.

Literatur

Christopher, M. (2010). *Logistics and supply chain management* (4. Aufl.). Financial Times/Prentice Hall.

Cooper, M. C., & Ellram, L. M. (1993). Characteristics of supply chain management and the implications for purchasing and logistics strategy. *International Journal of Logistics Management, 4*(2), 13–24.

DHL. (o. J.). *The macroeconomic significance of logistics*. http://www.dhl-discoverlogistics.com/cms/en/course/trends/macroeconomics.jsp. Zugegriffen am 12.03.2016.

Dogan, I. B., & Michailidou, A. (2008). Trading in prehistory and protohistory: Perspectives from the eastern Aegean and beyond. In C. Papageorgiadou & A. Giannikouri (Hrsg.), *MELETIMATA 53: Sailing in the Aegean, readings on the economy and trade routes* (S. 17–53). Institute of Greek and Roman Antiquity (IGRA), National Hellenic Research Foundation.

Fernie, J., & Grant, D. B. (2008). On-shelf availability: The case of a UK grocery retailer. *International Journal of Logistics Management, 19*(3), 293–308.

Gruen, T. W., Corsten, D. S., & Bharadwaj, S. (2002). *Retail out-of-stocks: A worldwide examination of extent. causes and consumer responses*. Grocery Manufacturers of America.

Harland, C. M. (1996). Supply chain management, purchasing and supply management, logistics, vertical integration, materials management and supply chain dynamics. In N. Slack (Hrsg.), *Blackwell encyclopedic dictionary of operations management*. Blackwell.

INFORMS. (2016). *Getting started with analytics*. https://www.informs.org/Sites/Getting-Started-With-Analytics. Zugegriffen am 12.03.2016.

Klaus, P., & Kille, C. (2009). *Der deutsche Logistikmarkt bleibt weiterhin ein Wachstumsmarkt! Deutliche Zuwächse bei Mengen und Beschäftigung*. Fraunhofer Institut Integrierte Schaltungen, Deutscher Verkehrs-Verlag.

Lambert, D. M. (2008). *Supply chain management: Processes, partnerships, performance* (3. Aufl.). Supply Chain Management Institute.

Lustig, I. (2013). *Optimization-based solutions: Smarter decisions for a smarter planet*. IBM Research – Business Analytics and Mathematical Sciences. https://irinadumitrescublog.files.wordpress.com/2013/04/optimization-smarter-planet-irv-public.pdf. Zugegriffen am 12.03.2016.

O'Neill, J. (2001). *Building better global economic BRICs*. Global economics paper no. 66. Goldman Sachs.

Robinson, A. (2015). *A complete breakdown of the 26th State of Logistics report*. http://cerasis.com/2015/07/10/state-of-logistics-report/. Zugegriffen am 12.03.2016.

Tudor, F. (2012). Historical evolution of logistics. *Revista de Ştiinţe Politice/Revue des Sciences Politiques, 36*, 22–32.

Wilson, D., & Purushothaman, R. (2003). *Dreaming with BRICs: The path to 2050*. Global economics paper, 99. Goldman Sachs.

Wilson, R. (2013). *24th annual State of Logistics report*. Council of Supply Chain Management Professionals (CSCMP).

Kapitel 2
Computational Intelligence

2.1 Grundlagen von Computational Intelligence

Seit Hunderten von Millionen Jahren entwickeln sich in der Natur kontinuierlich neue Strategien, wie Pflanzen, Tieren und Menschen Hindernisse im täglichen Leben überwinden. Diese Strategien sind ein Fundus für die Lösung von Problemen in der unserer Welt.

2.1.1 Künstliche Intelligenz, Computational Intelligence und verwandte Techniken

2.1.1.1 Künstliche Intelligenz

Künstliche Intelligenz (McCarthy, 2007), abgekürzt KI (Artificial Intelligence, AI), wurde in der Mitte des letzten Jahrhunderts „wissenschaftlich geboren". John McCarthy (McCarthy et al., 1955) war der erste Wissenschaftler, der KI als Technologie um das Jahr 1955 in die Informatik einführte und erklärte, dass KI die Grundlage für eine „wirklich intelligente Maschine" sei. Das Interesse an der KI als Teilgebiet der Informatik hat seitdem rasch zugenommen. Heute sind viele andere Disziplinen wie Technik, Biologie, Psychologie, Linguistik usw. an der Entwicklung und Anwendung von KI beteiligt und beschäftigen sich zunehmend interdisziplinär mit ihr.

Während die Forschung im Bereich der KI weiter zunimmt, verschwimmen Definitionen, Klassifizierungen und Formulierungen (Neapolitan & Jiang, 2012): Ursprünglich wurde zwischen *starker KI* und Standard-KI (auch normale KI oder *schwache KI* genannt) unterschieden. Starke KI bedeutet allgemeine Intelligenz im Sinne der Schaffung einer menschenähnlichen Intelligenz von Maschinen. Starke KI konzentriert sich also auf die Entwicklung intelligenter Maschinen

© Der/die Autor(en), exklusiv lizenziert an Springer Nature Switzerland AG 2023

T. Hanne, R. Dornberger, *Computational Intelligence in Logistik und Supply Chain Management*, https://doi.org/10.1007/978-3-031-21452-3_2

(d. h. Software- oder Hardwareroboter) und die Gestaltung der menschlichen Kognition, die Kreativität, Bewusstsein und Emotionen umfasst, um die Roboter in gewisser Weise menschenähnlich zu machen. Ziel ist es hier, eine der menschlichen Intelligenz vergleichbare technische Intelligenz zu erreichen oder diese sogar zu übertreffen. Im Gegensatz dazu widmet sich die schwache KI eher der Lösung bestimmter Probleme unter Nachahmung menschlichen intelligenten Verhaltens: Vernunft, Anpassung, Lernen und soziale Intelligenz sind Schlüsselkomponenten, die in Computerprogrammen eingesetzt werden, um bestimmte Probleme zu lösen (z. B. Text- oder Spracherkennung).

Mit anderen Worten: Schwache KI zielt darauf ab, intelligente, smarte Algorithmen bereitzustellen, die in Software-Lösungen implementiert sind, während starke KI eher eine Verkörperung von Intelligenz in roboterähnlichen Maschinen hervorbringt und diese mehr oder weniger intelligent interagieren lässt. Heute vermischen sich die beiden Richtungen der KI wieder: Unter KI wird verstanden, dass sie sowohl Maschinen (bzw. Software) intelligent macht als auch intelligente Maschinen schafft.

Die KI umfasst verschiedene Methoden zur intelligenten Verarbeitung von Informationen (z. B. Wahrnehmung, Lernen, Planung, Wissen, Argumentation und Kommunikation) und verleiht – insbesondere in der Robotik – Maschinen und Computern die Fähigkeit, sich in der realen Welt zu bewegen, mit ihr und insbesondere den Menschen zu interagieren und am Arbeits- und Privatleben teilzunehmen. Als Ausgangspunkte verwendet die KI ursprünglich symbolische Techniken wie logik- und wissensbasierte Methoden, Entscheidungsbäume, fallbasiertes Schließen und stochastische Automaten. Mit Hilfe dieser Techniken werden logische Schlussfolgerungen oder Automatenzustände abgeleitet.

In den letzten 60 Jahren haben sich viele Forschungsrichtungen in der KI weiterentwickelt: Probabilistische Methoden werden für Schlussfolgerungen in Problemsituationen unter Unsicherheit verwendet. Klassifikatoren und statistische Lernmethoden unterstützen das Lernen. Such- und Optimierungsmethoden führen mathematische Optimierungen durch. Neuronale Netze, Logik und Kontrolltheorie haben sich entwickelt. Programmiersprachen für künstliche Intelligenz (z. B. Lisp und PROLOG) wurden weiterentwickelt. Und in jüngster Zeit hat die Forschung im Bereich der intelligenten Maschinen und der Robotik große Sprünge gemacht.

Doch trotz all ihrer Erfolge wird der KI oftmals ein Nachteil nachgesagt, nämlich das Fehlen allgemeinerer fehlertoleranter Mechanismen: KI ist oft „zu logisch“, um Probleme robust zu lösen, die nicht klar formuliert sind, wie es bei den meisten realen Problemen der Fall ist. Aber die Realität ist das perfekte Testfeld. Die Realität ist unvollkommen, nicht perfekt, oft unsicher, verrauscht und unbeständig – die Realität ist real. So sieht unsere Welt aus und so verhält sie sich. Hier spielt die Computational Intelligence (CI) eine wichtige Rolle.

2.1.1.2 Computational Intelligence

Computational Intelligence (Kruse et al., 2013) (abgekürzt CI) wird oft im Zusammenhang mit KI genannt. Viele Forscher erklären (Fulcher & Jain, 2008), dass sie CI nicht als einen Teilbereich der KI bezeichnen, sondern dass CI und KI teilweise

nur die gleichen oder zumindest ähnliche Methoden umfassen. Allerdings verwenden CI und KI unterschiedliche Methodologien und Ansätze, um die Grundsätze ihrer Methoden zu beschreiben. Der Name CI tauchte in den 1990er-Jahren auf (Poole et al., 1998) und konzentrierte sich zunächst auf intelligente Roboter. Heute scheint CI interessanterweise klarer als KI definiert zu sein: CI befasst sich hauptsächlich mit „von der Natur inspirierten" Berechnungsmethoden zur Simulation eines intelligenten Verhaltens in komplexen und sich verändernden Umgebungen (Engelbrecht, 2007).

Darüber hinaus unterscheidet sich CI von KI in dem Sinne, dass CI (meist) subsymbolische Ansätze verwendet, d. h. Ansätze, die eine bestimmte symbolische Ebene nicht erreichen bzw. erst gar nicht benötigen. Ein Zustand oder eine Probleminstanz wird numerisch durch Zahlen ausgedrückt oder dargestellt anstatt durch eine symbolische Einheit, wie es in der KI im Allgemeinen geschieht. Je nach Anwendungsbereich und Problemstellung können die Vorteile von CI-Methoden der KI überlegen sein: CI-Methoden führen effiziente Approximationen in angemessener Rechenzeit durch, da sie häufig nichtdeterministische, stochastische Komponenten, Heuristiken und Metaheuristiken verwenden, die auch bei Rauschen, Unsicherheit, sich ändernden Zielen und sich verändernden Suchräumen schnell und robust zu einer oder mehreren Lösungen konvergieren.

Weitere Vorteile von CI-Methoden sind, dass sie oft effizient arbeiten und nur ein grobes Modell des Problems benötigen, dass sie fehlertolerant sind und dass sie sich meist gut zur Parallelisierung eignen. Andererseits besteht der größte Nachteil von CI im Vergleich zu deterministischen Methoden darin, dass die optimale(n) Lösung(en) in der Regel weder bewiesen noch garantiert werden kann (können).

2.1.1.3 Techniken im Zusammenhang mit Künstliche Intelligenz und Computational Intelligence

Weitere verwandte Forschungsbereiche sind das maschinelle Lernen (Mitchell, 1997) (Machine Learning, ML), Soft Computing (SC), Natural Computing (Rozenberg et al., 2012) (oder Natural Computation, NC), Bionik (oder Bionic Engineering, BE), und einige mehr.

ML wird als Unterkategorie der KI gezählt und beschäftigt sich mit dem Lernen aus Daten. Der Bezug zur CI ist nur über die neuronalen Netze gegeben. Außerdem konzentrieren sich die Paradigmen des ML nicht unbedingt auf „von der Natur inspirierte" Verfahren.

SC (Soft Computing) ist ein Forschungsgebiet, das sich mit Methoden zur effizienten (aber ungenauen) Lösung rechenintensiver und schwieriger Probleme in riesigen Suchräumen beschäftigt. NC (Natural Computing) kombiniert das Paradigma „von der Natur inspiriert" mit den Kategorien „Berechnung von Naturphänomenen" und oft „Verwendung von Naturmaterialien". NC konzentriert sich also auf die Synthese der Berechnung der Natur, auf die Entwicklung neuartiger naturinspirierter Hardware und auf das Verständnis von Naturphänomenen als Informationsverarbeitung.

CI, SC und NC umfassen zum Teil dieselben Methoden, wie z. B. neuronale Netze, evolutionäre Berechnungen, Schwarmintelligenz und Fuzzy-Logik. Der Anwendungsschwerpunkt von CI liegt jedoch auf der Lösung verschiedener Probleme der realen Welt, wohingegen der Anwendungsschwerpunkt von SC in der Berechnung schwieriger Probleme in riesigen Suchräumen liegt und der Anwendungsschwerpunkt von NC im Verständnis der Natur durch Berechnung und Nachbildung derselben.

Eine weitere verwandte Disziplin ist die Bionik (Bionics Engineering). Im Gegensatz zu den anderen genannten Forschungsbereichen, die sich auf Informatikaspekte konzentrieren, spielt die Bionik eine Rolle in der physikalischen oder technischen Ebene. In der Bionik geht es darum, die biologischen Strukturen oder physikalischen Fähigkeiten der Tier- und Pflanzenwelt nachzuahmen und in technische Produkte zu übertragen. Dennoch hat die Bionik durch die Nachahmung von in der Natur vorkommenden Mechanismen oder Prozessen einen Beitrag zur Erforschung von evolutionären Berechnungen, neuronalen Netzen und Schwarmintelligenz geleistet. Beispiele für bionisch hergestellte Produkte sind besondere Materialoberflächen zur Verringerung des Strömungswiderstands (z. B. Haifischhaut, Vogelflügel) oder zur Vermeidung von Schmutzanhaftungen (z. B. Lotuseffekt).

2.1.1.4 Interesse an Computational Intelligence

Das Interesse an CI und das Potential von CI sind groß und wachsen stetig weiter. Immer mehr Konferenzen beschäftigen sich mit CI. Die wichtigsten sind wohl der alle zwei Jahre stattfindende IEEE World Congress on Computational Intelligence (IEEE WCCI) mit den jährlichen Konferenzen IEEE Congress on Evolutionary Computation (IEEE CEC), die International Joint Conference on Neural Network (IJCNN) und die IEEE International Conference on Fuzzy Systems (FUZZ-IEEE) sowie die jährliche IEEE Symposium Series on Computational Intelligence (SSCI). Die weltgrößte CI-Organisation ist die IEEE Computational Intelligence Society, die diese Konferenzen initiiert.

2.1.2 Eigenschaften von Computational Intelligence

Das Hauptparadigma der CI ist „naturinspiriert“. Dies umfasst verschiedene Facetten: Die Biologie und insbesondere Flora und Fauna liefern verschiedene Mechanismen in der Natur, die erforscht und auf Algorithmen übertragen werden, welche diese Mechanismen auf einer rechnerischen Ebene nachahmen. Solche Mechanismen sind zum Beispiel die Evolution von Lebewesen, denen es über viele Generationen gelungen ist, sich besser an ihre Umweltbedingungen anzupassen und den anderen überlegen zu sein. Weitere Mechanismen sind die Simulation der Funktionsweise von Gehirnen zum Lernen und Erinnern oder die Erkennung des Eindringens von Krankheitserregern in den Körper oder die Nachahmung des Verhaltens von Schwärmen wie bei Vögeln, Fischen oder Ameisen.

Die meisten Methoden der Informationsgesellschaft sind mehr oder weniger von der Natur abgeleitet. Die Natur liefert bestimmte Problemlösungsstrategien in der realen Welt, insbesondere wie man in bestimmten Umgebungen am besten lebt, überlebt und sich fortpflanzt. Während CI ausschließlich solche Methoden umfasst, befasst sich KI intensiver nur mit einigen der von der Natur inspirierten Methoden (z. B. neuronale Netze als häufigste Methode, die sowohl in der KI als auch in der CI eingesetzt wird).

Die Methoden von CI sind an biologische Modelle oder Prozesse in der Natur angelehnt bzw. von ihnen abgeschaut. Sie werden in theoretische Modelle gegossen und dann in Algorithmen kodiert, oft ohne sie speziell an den Anwendungsbereich anpassen zu müssen. Typischerweise lassen sich die Methoden der CI durch folgende drei Aspekte charakterisieren:

- Es handelt sich um von der Natur inspirierte Berechnungsmethoden.
- Sie lösen komplexe Probleme ohne (viel) problemspezifisches Wissen.
- Sie sind gut geeignet, um reale Probleme zu lösen.

Heute wird im Allgemeinen die Computational Intelligence (CI) in die nachfolgenden fünf Hauptbereiche eingeteilt. Wir versuchen, jeden CI-Zweig mit drei individuellen Eigenschaften zu charakterisieren:

- Evolutionäre Berechnungen (Evolutionary Computation, EC):
  - EC wendet die biologischen Mechanismen der Evolution an, indem es das Prinzip des Überlebens des Stärkeren befolgt.
  - Eine Population, die aus mehreren Individuen besteht, verbessert sich im Laufe der Generationen durch Selektion, Kreuzung und Mutation.
  - EC ist sehr leistungsfähig bei der Lösung von Optimierungs- und Suchproblemen in komplexen Suchräumen.
- Schwarmintelligenz (SI):
  - SI ist das kollektive Verhalten von selbstorganisierenden Multiagentensystemen.
  - Die Gesamtheit einer Population, die aus einfachen Agenten besteht, die nur lokal interagieren, führt zu einem intelligent-ähnlichen globalen Verhalten.
  - SI hilft meist bei der Lösung von Optimierungs- und Suchproblemen, die oft zu einer Fragestellung der Distanzminimierung gehören.
- *Neuronales Netz* (NN; bzw. künstliches Neuronales Netz, KNN; Artificial Neural Network, ANN)
  - NN sind von echten biologischen neuronalen Netzen in Gehirnen von Tieren oder Menschen inspiriert.
  - Die Daten werden über in einem Netz miteinander verbundener künstlicher Neuronen verarbeitet.
  - NN erkennen Beziehungen zwischen Eingabe- und Ausgabedaten, finden Muster und generieren Informationen.

- Fuzzy-Logik (Fuzzy Logic, FL):
  - FL ist eine vielschichtige Logik, ähnlich dem menschlichen Denken.
  - Sie ermöglicht die Verwendung von Näherungswerten und unvollständigen oder mehrdeutigen Daten.
  - FL liefert ungefähre Lösungen oder Schlussfolgerungen.
- Künstliches Immunsystem (Artificial Immune Systeme, AIS):
  - AIS ist vom Immunsystem von Menschen, Tieren und Pflanzen inspiriert.
  - AIS übernimmt die Eigenschaften und Prozesse des realen Immunsystems für Lernen und (Wieder-)Erkennen.
  - AIS wird in adaptiven Systemen zur Problemlösung eingesetzt.

Bisweilen werden weitere Methodenbereiche zu CI gezählt, manchmal nicht konsequent, wie *Reinforcement Learning* (Kramer, 2009) und *Simulated Annealing*. Reinforcement Learning passt das Verhalten eines künstlichen Agenten (z. B. eines Software-Agenten oder eines Roboters) an, indem erwünschtes Verhalten belohnt und unerwünschtes Verhalten bestraft wird. Simulated Annealing nutzt ein thermodynamisches Prinzip aus der Physik: Bei Schmelzvorgängen (im Speziellen bei der Abkühlung von Metallschmelzen) versucht das Material, in einen Zustand minimaler Energie auszuhärten. Simulated Annealing simuliert diesen Effekt auf einer rechnerischen Ebene und wird hauptsächlich für Minimierungsprobleme verwendet.

Die Methoden der CI ermöglichen eine andere Art der Informationsverarbeitung, wie sie klassisch aus der KI bekannt ist. CI-Methoden machen sich einige einfache Prinzipien der Natur zu eigen, die es ihnen ermöglichen, entweder nach alternativen Lösungen (z. B. einem oder mehreren lokalen Minima oder Maxima) oder nach globalen Optima zu suchen. Einige dieser Methoden eignen sich gut zur Anpassung an sich ändernde Suchräume oder Bedingungen und werden daher auch zur Lösung von Steuerungsproblemen eingesetzt.

Einige der CI-Methoden sind in der Lage, zu lernen und sich zu erinnern – sie werden daher bei der Mustererkennung oder Klassifizierung eingesetzt. CI-Methoden sind mehr oder weniger fehlertolerant gegenüber unvollständigen, verrauschten oder unbestimmten Eingaben. Viele CI-Methoden erlauben eine parallele Verarbeitung (z. B. erlauben EC und SI die parallele Berechnung von Lösungskandidaten). Oft reicht es aus, die CI-Methoden auf ein einfaches Modell anzuwenden. Es wird also (fast) kein problemspezifisches Wissen benötigt. Insgesamt lassen sich mit CI je nach Problemstellung oft anspruchsvollere, von der Natur inspirierte Lösungsstrategien anwenden als mit KI.

2.1.3 Das große Bild von Computational Intelligence

Abb. 2.1 bietet einen systematischen Überblick über CI – genauer gesagt ein Gesamtbild von CI – mit Schwerpunkt auf Evolutionary Computation (EC) und Schwarmintelligenz (Swarm Intelligence, SI). Im Allgemeinen werden bei EC evo-

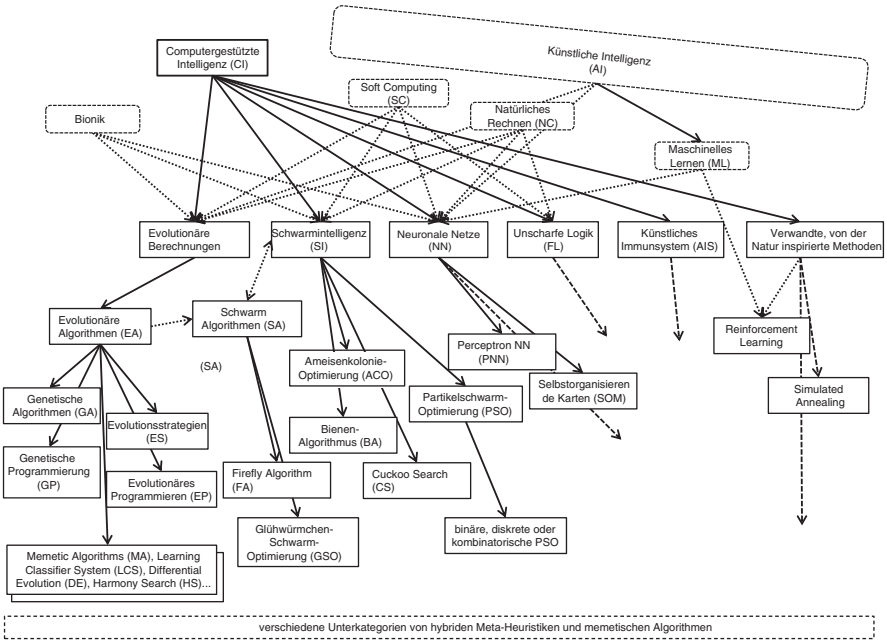


Abb. 2.1 Methoden der Computational Intelligence – mit Schwerpunkt auf den Unterzweigen Evolutionary Computation und Swarm Intelligence

lutionäre Algorithmen (EA) eingesetzt. Die wichtigsten Vertreter dieser Methoden sind Genetische Algorithmen (GA), Evolutionsstrategien (ES), Genetische Programmierung (GP) und Evolutionäre Programmierung (EP). Weitere Methoden, die zu EC und EA gehören, sind Memetic Algorithms (MA), Learning Classifier Systems (LCS), Differential Evolution (DE), Harmony Search (HS) und einige mehr.

In der Vergangenheit wurden Schwarmalgorithmen (SA) oft auch als Unterkategorie von EC erwähnt. Heutzutage werden SA zu den repräsentativen Methoden der SI gezählt, zu denen z. B. Ant Colony Optimization (ACO), Particle Swarm Optimization (PSO), Bees Algorithm (BA) und Cuckoo Search (CS) gehören. PSO kennt weitere Untermethoden wie binäres, diskretes oder kombinatorisches PSO.

Heute ist die Liste solcher von der Natur inspirierter Algorithmen recht lang. In der Übersicht von Fister Jr. et al. (2013) werden mehr als 70 Methoden klassifiziert, von denen die meisten aus dem Bereich der SI-Ansätze stammen. Die Such- und Optimierungsstrategien aus dem Bereich der EC und SI werden auch als Metaheuristiken bezeichnet. Einige von ihnen umfassen auch andere allgemeine Problemlösungsansätze, die sich nicht unbedingt auf das Paradigma „von der Natur inspiriert“ beziehen.

Die Welt der neuronalen Netze (NN) ist sogar noch größer, wobei Perceptron Neural Networks (PNN) (auch als Feedforward Neural Networks, FNN, bezeichnet) und Self-Organizing Maps (SOM, selbstorganisierende Karten) die altbekannten Methoden der NN sind. Fuzzy Logic (FL) und Artificial Immune Systems (AIS)

sind die anderen wichtigen Zweige von CI, welche hier nur kurz erwähnt werden. Denn das Hauptaugenmerk in unserem Buch liegt auf EC und SI, da die jeweiligen Methoden dieser Bereiche besonders erfolgreich in der Logistik eingesetzt werden. Am Ende dieses Kapitels gehen wir noch kurz auf einige weitere Metaheuristiken ein, die Ähnlichkeiten mit EC- und SI-Methoden aufweisen und ebenfalls zu den am häufigsten verwendeten Ansätzen für Logistikprobleme gehören.

2.1.4 Anwendungsbereiche von Computational Intelligence

2.1.4.1 Optimierung und Suche

Ein großer, vielleicht der größte Anwendungsbereich von CI ist die Optimierung in all ihren Facetten (Burke & Kendall, 2006). Bei der Optimierung bzw. der mathematischen Optimierung besteht das Problem – allgemein gesprochen – darin, Minima oder Maxima zu finden, die möglicherweise Beschränkungen bzw. Nebenbedingungen unterliegen.

Ein Optimierungsproblem ist mathematisch wie folgt definiert:

Gegeben: Eine Funktion $f : A \rightarrow \mathbf{R}$ einer Menge A aus reellen Zahlen
Gesucht: Ein Element x_0 in A, so dass gilt $f(x_0) \leq f(x)$ für alle x in A (sogenannte *Minimierung*) oder $f(x_0) \geq f(x)$ für alle x in A (sogenannte *Maximierung*).

Mathematische Definitionen sind gut geeignet, um einen mathematischen Aspekt klar und eindeutig zu erklären. Für Nicht-Mathematiker sind diese Definitionen jedoch oft nicht selbsterklärend. Daher erklären wir unseren Studentinnen und Studenten die Optimierung anhand der in Abb. 2.2 gezeigten Illustration wie folgt.

Ein offensichtlich identifizierbares kontinuierliches Optimierungsproblem besteht darin, den höchsten Gipfel einer Bergkette zu finden: Die Grenze zwischen

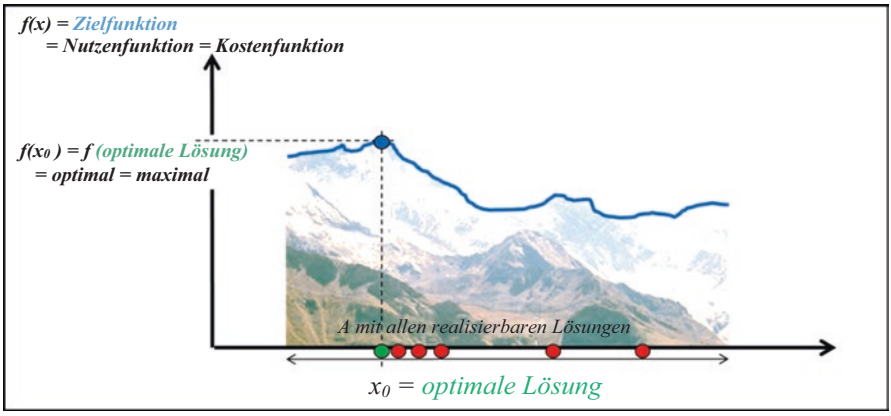


Abb. 2.2 Eine optimale Lösung x_0 und ihr Maximum $f(x_0)$ der Zielfunktion $f(x)$

den Bergen und dem Himmel ist vergleichbar mit der mathematischen Funktion $f(x)$, die die Höhe der Berge an jeder Position x angibt. Der Bereich A ist der Suchraum (auch Auswahlmenge genannt), in dem alle möglichen Positionen x die Menge der Kandidatenlösungen (zulässige Lösungen) bilden. Ein Lösungskandidat x ist somit ein Punkt des Suchraums A der möglichen Lösungen für ein bestimmtes Problem.

Die Funktion $f(x)$ ist die Zielfunktion, die manchmal auch Kostenfunktion (bei Minimierungsproblemen) oder Nutzenfunktion (bei Maximierungsproblemen) oder *Fitnessfunktion* genannt wird, wenn eine evolutionäre Berechnungsmethode angewandt wird.

Das Ziel der Optimierung besteht darin, eine bestimmte zulässige Lösung x_0 zu finden, die die Zielfunktion maximiert. Diese bestimmte zulässige Lösung x_0 mit $f(x_0) \geq f(x)$ für alle x in A wird dann als optimale Lösung bezeichnet. Bei Minimierungsproblemen besteht das Ziel darin, die zulässige Lösung x_0 für $f(x_0) \leq f(x)$ für alle x in A zu finden.

Außerdem enthält ein Optimierungsproblem oft Nebenbedingungen, die den Suchraum einschränken. Mathematisch gesehen müssen die Lösungsvorschläge dann zusätzliche Ungleichheits- und Gleichheitsbedingungen erfüllen, wie z. B. $g(x) \leq 0$ und $h(x) = 0$.

Optimierungsprobleme können Skalare oder Vektoren x mit kontinuierlichen, diskreten oder kombinierten Variablen im Suchraum umfassen. Die Kostenfunktion kann kontinuierlich oder diskret sein und möglicherweise nur implizit existieren. Bei einigen Optimierungsproblemen handelt es sich um kombinatorische Optimierungsprobleme, bei denen eine bestimmte Folge möglicher Werte gefunden werden muss. Verschiedene Möglichkeiten, mit solchen Optimierungsproblemen umzugehen, werden später gezeigt.

In der Physik oder im Ingenieurwesen bezieht sich die Optimierung häufig auf die Maximierung von Kraft, Leistung, Effizienz oder Stabilität oder auf die Minimierung von Energieverbrauch, Verlusten, Zeit oder Fehlern. In der Wirtschaft zielt die Optimierung oft auf die Maximierung des Gewinns, des Return-on-Investment (ROI) oder des wirtschaftlichen Gewinns ab, während die Minimierung eher den geringsten Aufwand an investierten Ressourcen sucht. In der Logistik führt die Optimierung z. B. oft zum kürzesten, schnellsten oder billigsten Transportweg.

Verwandte Anwendungsbereiche sind Suchprobleme, die manchmal nicht als Optimierungsprobleme bezeichnet werden. Bei Suchproblemen besteht das Ziel darin, entweder ein Element mit bestimmten Eigenschaften aus einer Menge unterschiedlichster Elemente zu finden – oder überhaupt eine zulässige Lösung oder eine größere Anzahl guter Alternativen (z. B. Mengen lokaler Minima oder Maxima). Meist können dazu die klassischen Optimierungsalgorithmen durch zusätzliche Erweiterungen der Algorithmen (z. B. Niching oder Sharing) erweitert werden.

Bei den CI-Methoden wird nicht wirklich zwischen Optimierung und Suche unterschieden. Je nach den Eigenschaften des Problems (z. B. NP-hart) und des Suchraums (z. B. diskret, kontinuierlich) sind einige CI-Methoden besser geeignet als andere.

Im Allgemeinen handelt es sich bei den CI-Methoden um Heuristiken oder Metaheuristiken: Ein Ansatz wird als *Heuristik* bezeichnet, wenn er dazu dient, qualitativ hochwertige Lösungen zu erhalten, obwohl die Optimalität nicht garantiert werden kann, ohne alle möglichen Lösungen zu prüfen. *Metaheuristiken* beruhen im Allgemeinen auf mehreren verknüpften heuristischen Konzepten. Sie können in der Regel auf ein breites Spektrum unterschiedlicher Such- oder Optimierungsprobleme angewandt werden, insbesondere in rechenintensiven Bereichen, in denen exakte Optimierungsansätze manchmal als unzureichend oder zu zeitaufwändig erscheinen. Obwohl für einige der bekannten Metaheuristiken Konvergenznachweise (d. h. Beweise für das Erreichen optimaler Lösungen unter geeigneten Bedingungen) vorliegen, wurde der Begriff „Metaheuristik“ beibehalten.

Einige Suchmethoden, also die entsprechenden Algorithmen, führen eine deterministische Suche durch. Das bedeutet, dass sie bei exakt gleichen Eingabe- und Startbedingungen immer die gleiche Antwort liefern. CI-Methoden hingegen sind (meist) nicht-deterministische Suchmethoden bzw. Algorithmen. Das bedeutet, dass sie bei genau demselben Problem unterschiedliche Optimierungspfade verfolgen und unterschiedliche Antworten liefern können. Die verschiedenen Antworten sollten (müssen aber nicht) im Laufe der Zeit zu denselben optimalen Lösungen konvergieren.

Außerdem können CI-Methoden mehr als nur Werte von Variablen mathematisch suchen und optimieren: Einige dieser Methoden, wie sie z. B. im Data Mining eingesetzt werden (z. B. insbesondere NN, AIS), bieten die Fähigkeit zu lernen und zu klassifizieren. Andere dienen der Erkennung von Mustern, sowohl in gedruckten oder handgeschriebenen Texten als auch in gesprochener Sprache (z. B. NN). Andere dienen der Anpassung an sich ändernde Suchräume oder Bedingungen, eine Eigenschaft, die für die Entwicklung von Steuerungen erforderlich ist (z. B. EC, FL). Wiederum andere helfen dabei, biologische Evolutionsprozesse, die Funktionsweise des Gehirns oder technische oder wirtschaftliche Prozesse zu modellieren und zu simulieren (z. B. EC, SI, NN). Weitere Methoden ermöglichen evolutionäres Design und Kunst oder die Komposition von Musik (z. B. EC). Einige werden in Computerspielen oder in agentenbasierten Systemen eingesetzt (z. B. SI, EC, NN), um Experimente zu modellieren und zu simulieren, die in der Realität so nicht durchgeführt werden können; andere um Big Data zu verarbeiten (z. B. NN) usw.

2.1.4.2 Mehrzieloptimierung

Wenn mehr als eine Zielfunktion gemeinsam optimiert werden muss, wird das Problem zu einem Problem der *Mehrzieloptimierung* (Multi-Objective Optimization, MOO). Bei dieser Mehrzieloptimierung, auch Multikriterien- oder Multiattribut-Optimierung genannt, stehen die zwei oder mehr zu optimierenden Ziele oftmals in Konkurrenz, wobei auch noch bestimmte Nebenbedingungen zu beachten sind.

Im Allgemeinen kann man bei solchen Mehrzielproblemen nicht eine einzige Lösung finden, die alle Ziele gleichzeitig optimiert. Bei der Suche nach Lösungen

wird der Versuch, ein Ziel weiter zu verbessern, zu einer Verschlechterung eines anderen Ziels führen. Das Ergebnis ist eine Menge von Lösungen, die als nicht-dominiert, Pareto-optimal oder Pareto-effizient bezeichnet werden. Sie zeichnen sich dadurch aus, dass sie nicht vernachlässigt werden können, indem sie durch andere Lösungen ersetzt werden, die eine Zielfunktion verbessern, ohne eine andere zu verschlechtern. Die Lösung eines Mehrzieloptimierungsproblems bedeutet, durch eine sogenannte Pareto-Optimierung solche nicht-dominierten Lösungen, also die Pareto-optimalen Lösungen, zu finden.

Später werden unter dem Stichwort *Multi-Objective Evolutionary Algorithms* zwei evolutionäre Algorithmen vorgestellt, die eine solche Pareto-Optimierung durchführen.

2.2 Methoden der Computational Intelligence

2.2.1 Evolutionäre Berechnungen – Evolutionary Computation

Evolutionary Computation (EC) geht auf die 1950er- und 1960er-Jahre zurück, als neue Optimierungsverfahren mit einer Art Selbstanpassung ans Problem untersucht wurden. Damals wurde EC als ein Teilgebiet von KI betrachtet. Heute wird EC auch als Teilgebiet von CI angesehen, da die Methoden, die zu EC gehören, biologische Evolutionsmechanismen adaptieren. Dies drückt sich in vielen übernommenen Ausdrücken der biologischen Evolution aus, um die Berechnungsalgorithmen der EC zu beschreiben:

- Im Allgemeinen arbeiten die EC-Methoden mit einer Menge von Lösungskandidaten. Eine solche Menge wird als Population bezeichnet, die Lösungskandidaten als Individuen. Die Individuen sind oft durch sogenannte Chromosomen kodiert, ähnlich wie in der Biologie mit der DNA.
- Die EC-Methoden sind iterative Methoden. Innerhalb einer Iteration wird die Menge der Individuen eine Generation genannt. Bei der EC-Methode wird die Lösung also mit Hilfe einer Population von Generationen von sich verändernden Individuen berechnet, von denen jedes einen Lösungskandidaten darstellt.
- Die EC-Methoden verbessern iterativ die „Qualität“ der Individuen. Diese Qualität wird als Fitness bezeichnet und durch die Zielfunktion, die sogenannte Fitnessfunktion, gemessen. Während Fitness oft bedeutet, dass die Fitness maximiert werden muss, kann die Zielfunktion bei Minimierungsproblemen auch als Kostenfunktion bezeichnet werden.
- Bei den iterativen Schritten handelt es sich um Generationen, in denen – von einer Generation zur nächsten – einige Elternindividuen ihre guten Eigenschaften und Merkmale durch Reproduktionsmechanismen wie Crossover (eine Art der Kreuzung und Rekombination), Mutation und Selektion an die Nachkommenindividuen weitergeben.

Diese Methoden der EC sind gut geeignet, um Optimierungs- und Suchprobleme zu lösen, so wie die Evolution die Lebewesen in ihren natürlichen Umgebungen kontinuierlich optimiert. Diese natürlichen Umgebungen entsprechen den Suchräumen in der mathematischen Optimierung. Wenn sich nun die Suchräume im Laufe der Zeit ändern, muss der Optimierungsprozess den beweglichen Zielen auch folgen. EC tut dies aufgrund seines iterativen Charakters automatisch. Dies kann bei der Entwicklung von Steuerungen genutzt werden (z. B. fahrerlose autonome Autos oder Steuerung des Verbrennungsprozesses in Brennern (Büche et al., 2002)). Versteht man den Optimierungsprozess der EC als Entwurfsprozess, so lassen sich viele Anwendungen im Design finden (z. B. Entwicklung von Gasturbinen (Dornberger et al., 2000) oder des Mars Rovers oder von Antennen und Computerschaltungen). Über die mathematische Optimierung sowie die Natur- und Ingenieurwissenschaften hinaus werden die EC-Methoden auch in den Wirtschafts- und Sozialwissenschaften sowie in der Kunst eingesetzt.

2.2.2 Evolutionäre Algorithmen – Evolutionary Algorithms

Evolutionäre Algorithmen (Evolutionary Algorithms, EA) werden entweder als eine Unterkategorie von EC oder als Synonym von EC betrachtet. Im Allgemeinen beschreiben EA eine Reihe von stochastischen, populationsbasierten, (meta)heuristischen Optimierungsalgorithmen. Alle Algorithmen, die zu EA gehören, sind durch Reproduktion unter Anwendung von Selektion, Rekombination und Mutation gekennzeichnet. Die Anwendung dieser Operatoren führt zu einem künstlichen Evolutionsprozess auf der rechnerischen Ebene, der die reale Evolution in bestimmten Aspekten nachahmt.

Die wichtigsten Methoden, die zu EA gehören, sind Genetische Algorithmen (GA), Evolutionsstrategien (ES), Genetische Programmierung (GP) und Evolutionäre Programmierung (EP). Teilweise werden auch Schwarmalgorithmen (SA) als zu EA gehörig betrachtet; aber wir verbinden die SA mit dem anderen unabhängigen großen Zweig der CI namens Schwarmintelligenz (SI). Zusätzlich bieten alle diese Methoden verschiedene Varianten, die zu vielen weiteren Metaheuristiken führen: Differential Evolution (DE), Learning Classifier Systems (LCS), Harmony Search (HS) usw.

Hybride Metaheuristiken entstehen durch die Verschmelzung zweier oder mehrerer Methoden. Memetische Algorithmen (MA) werden durch die Kombination von EA-Methoden mit lokalen Suchmethoden entwickelt. Metaheuristiken, die mit Ansätzen aus der mathematischen Programmierung (wie Branch-and-Bound, Simplex-Algorithmus oder Newton-Gradienten-Methoden) hybridisiert sind, werden oftmals als *Matheuristiken* bezeichnet.

Die Algorithmen unterscheiden sich durch die interne Repräsentation des Problems, d. h. die Kodierung der Lösungskandidaten, durch unterschiedliche Arbeitsweisen der EA-Operatoren und durch die Reihenfolge der Ausführung dieser Ope-

ratoren. Dabei muss nicht unbedingt eine Rekombination stattfinden, da ein durch Mutationen verändertes Elternteil ausreichen kann, und umgekehrt.

Die Prinzipien der EA-Methoden sind einfach. Die folgende Liste zeigt, wie eng die Beziehung zwischen bestimmten Aspekten in der Natur bzw. Biologie und den Berechnungsalgorithmen von EA ist:

- Das Paradigma, das der Evolution in der Natur entnommen ist, baut auf Darwins Prinzip „survival of the fittest".
- In der Natur kodiert die Biologie die Eigenschaften eines Organismus, den sogenannten Phänotyp, mit vier Aminosäuren in der DNA. Auf der Ebene der Datenverarbeitung gibt es diese Beschränkung auf die Zahl vier nicht; in den Individuen sind verschiedene, völlig unterschiedliche Kodierungen der Lösungsvorschläge möglich. Im Allgemeinen erfolgt die Kodierung mit binären, ganzzahligen oder reellen Werten oder mit einer Menge vorgegebener Objekte, die die Erbinformation tragen, genannt Genotyp.
- Jedes Individuum, in EA also jeder kodierte Lösungskandidat, erhält eine bestimmte Qualität (Fitness genannt), die angibt, wie gut das Individuum das Problem im Vergleich zu den anderen Lösungskandidaten lösen kann.
- In Anlehnung an den natürlichen Selektionsprozess werden die besseren Individuen mit höherer Wahrscheinlichkeit ausgewählt, um die Nachkommen (neue Lösungskandidaten) für die nächste Generation (nächste Iteration) zu erzeugen.
- Die ausgewählten Individuen werden rekombiniert, wobei durch Mutation zusätzliche genetische Variationen hinzugefügt werden.

In der Informatik werden diese Prinzipien in einem Algorithmus umgesetzt. Die Reihenfolge und Bedeutung der einzelnen Prinzipien können von Fall zu Fall unterschiedlich sein. Das allgemeine Flussdiagramm eines EA kann jedoch wie folgt notiert werden und ist in Abb. 2.3 (links) dargestellt:

1. *Es wird ein zufälliges Set von Individuen als Null-(Anfangs-)Generation erzeugt.*
2. *Für jedes Individuum wird die Fitness berechnet.*
3. *Solange kein Abbruchkriterium erfüllt ist, wird eine weitere neue Generation erzeugt.*
4. *Dann werden bestimmte Individuen je nach ihrer Fitness ausgewählt, um die neue Gruppe von Individuen der neuen Generation zu bilden.*
5. *Gemäß einer vorgegebenen bestimmten Wahrscheinlichkeit werden sie durch Kreuzung miteinander rekombiniert.*
6. *Sie erhalten mit einer gewissen Wahrscheinlichkeit eine Mutation.*
7. *Ihre neue Fitness wird berechnet. (Und mit Schritt 3 fortgefahren.)*

Das Klassendiagramm für die objektorientierte Programmierung (OOP) einer EA-Implementierung ist in Abb. 2.3 (rechts) dargestellt:

- Die Klasse „Population" enthält *1-to-many* „Generation(s)". Alle „Generation(s)" gehören zu *dieser* „Population".
- Jede „Generation" besteht aus *1-to-many* „Individual(s)", wobei diese „Individual(s)" zu *dieser* „Generation" gehören.
- Jedes „Individual" hat genau *1* „Fitness"-Wert.

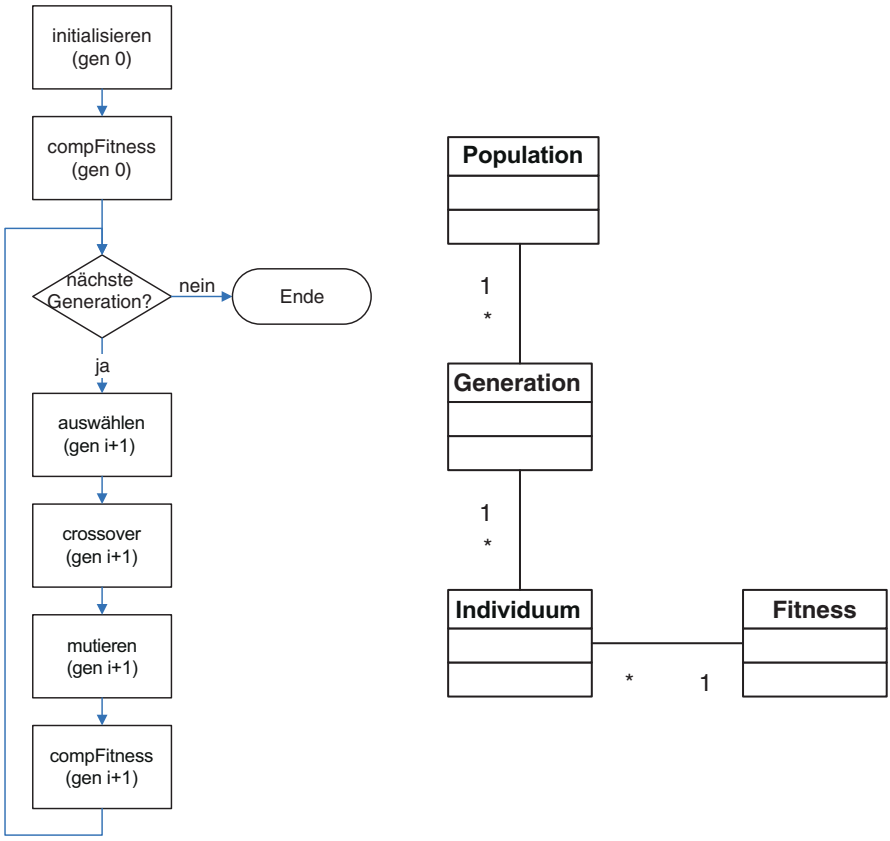


Abb. 2.3 Allgemeines Flussdiagramm eines EA (*links*) und das Klassendiagramm für die objektorientierte Programmierung (OOP) eines EA (*rechts*)

Die Auswahl bestimmt, welche Individuen die Population der nächsten Generation bilden. Sie kann auf unterschiedliche Weise erfolgen: Entweder werden diejenigen Elternindividuen ausgewählt, die für Rekombination und Mutation weiterverwendet werden. Oder eine Selektion wird verwendet, um die besten Individuen aus allen neu erzeugten Individuen und den vorhandenen alten Individuen herauszunehmen, um die nächste Generation zu bilden. Oder eine Selektion legt fest, welche Individuen eliminiert werden und nicht für die Nachkommenschaft verwendet werden sollen. Es sind also verschiedene Selektionsoperatoren möglich, wie hier mit den wichtigsten gezeigt:

- Bei der Standardauswahl werden die n fittesten, ranghöchsten Individuen als Eltern ausgewählt, wobei n die Anzahl der Individuen pro Generation ist.
- Die Roulette-Rad-Auswahl (Roulette-Wheel Selection) ermöglicht eine fitnessproportionale Auswahl, bei der die Möglichkeit der Auswahl eines Individuums von der Größe seines Fitnesswertes im Verhältnis zu den Fitnesswerten der anderen Individuen abhängt.

- Bei der Turnierauswahl (Tournament Selection) wird das beste Individuum von zwei zufällig ausgewählten Elternindividuen ausgewählt.
- Bei der Truncation Selection wird ein bestimmter Teil der besten Individuen ausgewählt.
- Bei der Eliteselektion (kurz Elitismus bzw. Elitism genannt) wird immer das beste Individuum aus der alten Generation in die neue Generation kopiert, um zu gewährleisten, dass kein Rückschritt im Evolutionsprozess erfolgt.

Verschiedene Auswahloperatoren können kombiniert werden. Insbesondere der Elitismus wird häufig eingesetzt, um immer die letzte beste Lösung zu erhalten.

Die Vorteile der EA-Methoden sind, dass sie (fast) problemunabhängig und sehr robust das Problem lösen. Somit können EA als Black-Box-Optimierungs- und Suchalgorithmen eingesetzt werden. Es sind (fast) keine Kenntnisse über das zu lösende Problem und nur geringe Kenntnisse über den Suchraum notwendig. Es können verschiedene Klassen von Optimierungsproblemen gelöst werden, da die Probleme weder stetig noch differenzierbar sein müssen. Die EA ist eine stochastische Erkundung des Suchraums für lokale und globale Optimierung und ist im Idealfall sehr viel schneller als eine vollständige Suche nach allen Lösungen in einem endlichen, aber großen Suchraum.

2.2.2.1 Evolutionsstrategie – Evolution Strategy

In den 1960er- und 1970er-Jahren führte Ingo Rechenberg (1994) neue Methoden zur Optimierung von Konstruktionen wie Tragflächen oder Flugzeugstrukturen ein. Er nannte diesen Ansatz *Evolutionsstrategie* (Rechenberg, 1973) (Evolution Strategy, ES), da eine Art künstliche Evolution durchgeführt wird: Die Merkmale der besseren Entwürfe werden selektiert, kombiniert, stochastisch leicht verändert und an eine nächste Generation mit modifizierten Entwürfen weitergegeben. Obwohl dieser Ansatz an Hardware-Designs durchgeführt wurde, ist es offensichtlich, dass dabei Prinzipien der evolutionären Berechnung angewendet werden. Rechenberg kombinierte diesen Ansatz mit der Bionik und übertrug Mechanismen aus der Natur auf technische bzw. ingenieurtechnische Produkte.

Während Rechenberg die ES anfangs eher in der experimentellen Optimierung von Hardware-Entwürfen anwandte, erweiterte Hans-Paul Schwefel (1977) die Ideen mehr auf die rechnerische Optimierung und kodierte die technischen Entwürfe (Phänotypen) durch Vektoren mit den reellen Zahlen aus R^n (Genotypen).

Die Grundsätze der ES für die rechnerische Optimierung mit ES sind die folgenden: Eine Menge von Vektoren, die aus n reellwertigen Komponenten bestehen, stellt eine Menge von verschiedenen Lösungskandidaten dar. Jede Vektorkomponente beschreibt ein bestimmtes Merkmal und kann durch Hinzufügen eines normalverteilten Zufallswertes mutiert werden. Die Größe dieses Zufallswertes wird als Mutationsstärke bezeichnet und durch bestimmte Anpassungsmechanismen bestimmt. Jeder Vektor wird getestet, also evaluiert, um das Problem zu lösen, und je nach seinem Fitnesswert eingestuft.

Es sind verschiedene Haupttypen der ES mit einer dazumal definierten *(μ, λ)*-Notation oder *(μ+λ)*-Notation möglich. μ steht für die Anzahl der jüngsten Lösungsvorschläge in der Grundgesamtheit und λ für die Anzahl der zusätzlichen Lösungsvorschläge, die pro Iteration generiert werden:

- Eine *(1+1)-ES* ist die einfachste ES. Diese Konfiguration besteht aus einem übergeordneten Vektor (Kandidatenlösung) und einem modifizierten Vektor. Die bessere Kandidatenlösung wird als Elternteil für die nächste Generation genommen. Die einfachste ES entspricht dem Greedy Hill Climbing-Algorithmus: In jeder Iteration wählt der Algorithmus die tatsächlich beste Kandidatenlösung aus einer Menge von mindestens zwei Alternativen.
- Häufiger ist die *(1+λ)-ES*, die aus einem übergeordneten Vektor und λ modifizierten Vektoren besteht. Hier konkurrieren die λ modifizierten Kandidatenlösungen und die jüngste Stammlösung um die Auswahl für die nächste Generation.
- Bei der *(1, λ)-ES* konkurrieren nur die λ modifizierten Vektoren um die Auswahl für die nächste Generation.
- Um lokale Optima zu überwinden, schlägt die *(μ, λ)-ES* eine Generation vor, die aus μ übergeordneten Vektoren und λ modifizierten Vektoren besteht, wobei (falls $\lambda > \mu$gilt) die μ besten Lösungsvorschläge in das neue λ für die nächste Generation übernommen werden.
- Bei der *(μ+λ)-ES* konkurrieren alle μ Eltern- und λ modifizierten Vektoren im Auswahlverfahren für die nächste Generation.
- Häufig wird zusätzlich eine Rekombination zwischen den Elternvektoren durchgeführt, die mit ρ und *(μ/ρ, λ)-ES* bezeichnet wird. ρ ist die Anzahl der Eltern, die unter Verwendung bestimmter Rekombinationsoperatoren zur nächsten Generation beitragen.

In der ES ist die Mutation der treibende Operator zur Veränderung der Vektoren, die die Lösungsvorschläge darstellen. Das optimale Maß der Mutation, die so genannte Mutationsstärke, zu finden, ist jedoch ein eigenes Optimierungsproblem. Rechenberg führte die so genannte 1/5-Regel ein, die vorschlägt, die Mutationsstärke so anzupassen, dass in jeder Generation etwa 20 % aller getesteten Mutationen zu besser generierten Kandidatenlösungen als die ursprünglichen Elternlösungen führen.

Eine andere Methode, eine geeignete Mutationsstärke zu finden, ist die Selbstanpassung, bei der eine zusätzliche Komponente in den Vektor integriert wird, der den Wert der Mutationsstärke enthält. Auf diese Weise optimiert sich die Mutationsstärke selbst.

Eine weitere Erweiterung der ursprünglichen ES ist die kontrollierte Mutation von Vektorkomponenten durch eine sogenannte Kovarianzmatrixanpassung (Covariance Matrix Adaptation, CMA). Diese Art von ES wird als CMA-ES (Hansen & Ostermeier, 2001) bezeichnet, welche die Mutationsstärke in die Richtung der möglichst höchsten Steigerung des Fitnesswertes lenkt. Obwohl die ES stochastische, ableitungsfreie Methoden sind, funktioniert die CMA-ES ähnlich einer Verwendung von Ableitungsinformationen in Gradientenmethoden.

2.2.2.2 Genetischer Algorithmus – Genetic Algorithm

In den 1970er-Jahren entwickelte John H. Holland (1975) eine Methode zur Lösung von Such- und Optimierungsproblemen unter Anwendung der Grundsätze von EA. Er nannte die Methode *Genetischer Algorithmus* (Genetic Algorithm, GA) und die Lösungsvorschläge *Individuen*. Diese Individuen werden durch *Chromosomen* kodiert, die üblicherweise als Vektoren oder Strings implementiert werden. Jede Komponente des Strings (oder Vektors) wird als Gen bezeichnet. Der Hauptunterschied zur reellwertigen ES besteht darin, dass Holland die einfache binäre Kodierung $\{0, 1\}^N$ zur Kodierung einer Lösungsinstanz eingeführt hat.

Während alle Varianten von ES eher auf der Ebene des Phänotyps arbeiten, optimieren die Varianten von GA vollständig auf der Ebene des Genotyps, der von den realen Probleminstanzen auf binäre Ziffern abgebildet wird. Um auf diese Ebene der kodierten Individuen zu gelangen, ist eine eindeutige Abbildung der Repräsentation des Suchraums (z. B. reellwertig oder ganzzahlig diskret) erforderlich. Dies wird als Genotyp-Phänotyp-Abbildung bezeichnet. Die Grundidee von Holland war, dass eine Darstellung mit binären Ziffern für einen Computer am einfachsten zu handhaben ist.

Außerdem ähneln die beiden Werte 0 und 1 im GA der Anzahl an vier Nukleinsäuren in der DNA. Die Vorteile des ursprünglichen GA mit der binären Kodierung nahmen jedoch mit zunehmender Rechenleistung ab. Heutzutage werden verschiedene Kodierungsschemata in GA verwendet. Auch eine variable Längenkodierung der Chromosomen ist möglich.

In einem GA funktionieren die genetischen Operatoren und die Berechnung der Fitness folgendermaßen:

- *Rekombination/Kreuzung (Crossover)*: Die Rekombination wurde ursprünglich als einfache Kreuzung eingeführt, bei der zwei Elternindividuen ihre Gene an einer bestimmten Kreuzungsposition austauschen, wie in Abb. 2.4 dargestellt. Neben einer *Ein-Punkt-Kreuzung* tauscht eine *n-Punkt-Kreuzung* Gensequenzen zwischen zwei Individuen aus. Bei kombinatorischen Problemen ermöglichen spezielle Arten von Crossover, wie z. B. Partial Mapped Crossover (PMX), die

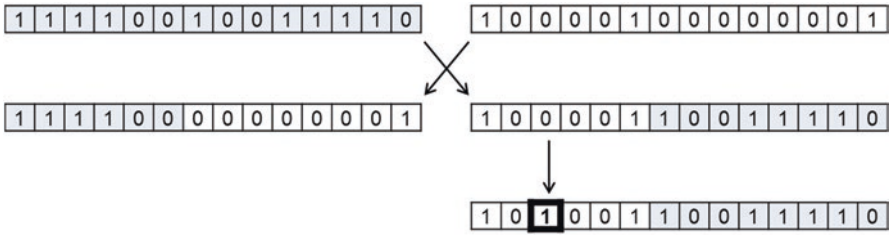


Abb. 2.4 Einfaches Crossover zwischen dem 6. und 7. Gen, angewandt auf die beiden Elternindividuen (*oben*), führt zu den beiden Nachkommenindividuen (*Mitte*). Zusätzlich ist das rechte Nachkommenindividuum auf dem 3. Gen mutiert (*unten rechts*)

Rekombination der Informationen von Individuen, ohne (zu viele) Nachkommenindividuen mit ungültiger Kodierung zu erzeugen.
- *Mutation*: Bei der binären Kodierung ist eine Mutation offensichtlich, da das Bit des jeweiligen Gens nur invertiert werden muss (Abb. 2.4). Bei den anderen Kodierungsschemata muss eine Mutation definiert werden, wie sich der Wert eines Gens in einen anderen Wert ändert.
- *Fitness*: Die Berechnung des Fitnesswerts erfordert eine Recodierung von der Genotypendarstellung auf die Phänotypenebene, da das zu lösende Problem häufig nicht in einem binären Suchraum dargestellt wird. Darüber hinaus können GA in dem Sinne interaktiv sein, dass die Berechnung des Fitnesswerts außerhalb des GA erfolgt – vielleicht interaktiv durch den Benutzer oder innerhalb einer technischen Software.

Holland und Goldberg (Goldberg, 1989) beschreiben die Fähigkeit von GA, zu Optima zu konvergieren, mit der Bausteinhypothese. Sie besagt, dass ein GA in der Lage ist, positive Eigenschaften zu sammeln, die in kurzen Sequenzen von Genen, den so genannten Bausteinen, gespeichert sind, die vererbt und über Generationen von sich verbessernden Individuen summiert werden.

2.2.2.3 Genetische Programmierung – Genetic Programming

Genetische Programmierung (Genetic Programming, GP) wurde von John R. Koza (1992) in den 1980er-Jahren entwickelt, um zur Funktionsapproximation und zur automatischen Generierung von Computercode und Programmen eingesetzt zu werden. GP basiert auf GA und verwendet eine genotypische Darstellung des Problems, z. B. der mathematischen Formeln oder des gesamten Quellcodes. Während ein GA meist Individuen mit fester Länge verwendet, lässt GP Individuen mit variabler Länge zu.

In der Standard-GP sind die Chromosomen baumartige Darstellungen der mathematischen Funktionen oder Code-Ausdrücke, wie in Abb. 2.5 gezeigt. Bei der Verwendung solcher Bäume zielt GP darauf ab, die gesamte Struktur des Baumes zu optimieren und nicht nur die Werte oder Operatoren bestimmter Knoten zu optimieren.

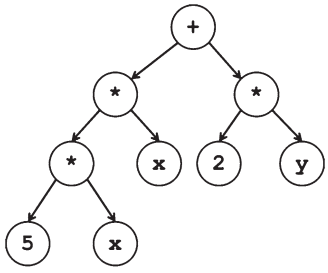


Abb. 2.5 Baumstrukturdarstellung der Funktion $5x^2 + 2y$

Die ursprüngliche Idee von GP bevorzugt Programmiersprachen, die von Natur aus baumbasierte interne Datenstrukturen verwenden, wie z. B. Lisp. Jedes Chromosom ist ein Codekandidat, der einen Baum darstellt. Die Knoten – und damit alle von diesem Knoten abhängigen Zweige – können frei ausgetauscht, neu kombiniert und mutiert werden. Dies geschieht nicht nur zwischen verschiedenen Chromosomen, sondern auch innerhalb jedes einzelnen Chromosoms. Ein Nebeneffekt ist, dass viele Code-Kandidaten mit einer ungültigen Syntax erzeugt werden, die aussortiert werden müssen. Die Fitness wird anhand der Erfüllung der Syntaxregeln, der Anforderungen an den zu entwickelnden Code und seiner „Eleganz" bzw. seiner Eignung, von einem Parser verarbeitet werden zu können, bewertet.

Es gibt viele Erweiterungen der ursprünglichen GP: Um auch andere Programmiersprachen (jenseits der baumbasierten Sprachen) wie Assembler oder imperative Sprachen zu ermöglichen, wurde GP zur *linearen genetischen Programmierung* (Linear Genetic Programming, LGP) erweitert. LGP kodiert die in den Programmiersprachen verwendeten Anweisungen in den Chromosomen als eine Folge von Anweisungen. Die Standard-GA-Operatoren werden dann angewendet, um den Code im Laufe der Zeit zu verbessern. Bei der *Grammatical Evolution* (GE) wird das gesamte Computerprogramm mit Hilfe einer bestimmten „Grammatik" in einer ganzzahligen Zeichenkette kodiert: Jede mögliche Codeeinheit ist ein bestimmter ganzzahliger Wert, und GE wendet die GA-Operatoren auf die Zeichenkette (Chromosom) an. GE ermöglicht die Optimierung auf dem abstrakten Genotyp des Problems, während die Standard-GP eher auf der Ebene des Phänotyps auf dem direkten Code arbeitet. *Gene Expression Programming* (GEP) verwendet Chromosomen fester Länge, die als Bäume kodiert werden. Durch modifizierte genetische Operatoren wird die Erstellung eines ungültigen Codes vermieden. *Meta-Genetic Programming* (MGP) bietet mehr Freiheit bei der Entwicklung, wie und wo die genetischen Operatoren (Rekombination und Mutation) die Chromosomen verändern können. MGP ermöglicht nicht nur die Optimierung des Codes, sondern auch die Optimierung des genetischen Optimierungsprozesses an sich.

2.2.2.4 Evolutionäre Programmierung – Evolutionary Programming

Evolutionäre Programmierung (Evolutionary Programming, EP) wurde in den 1960er-Jahren von Lawrence J. Fogel, Alvin J. Owens und M. J. Walsh eingeführt (Fogel, 1999). EP ist eine spezielle Art von ES, was ursprünglich zur Optimierung der Parameter von endlichen Automaten zur Implementierung in Computerprogrammen oder logischen Schaltkreisen verwendet wurde. EP arbeitet vollständig auf der Ebene des Phänotyps und benötigt keine Darstellung des Genotyps. Im Gegensatz zu GP zielt EP also direkt auf die Optimierung der numerischen Parameter eines gegebenen Problems (z. B. eines Computerprogramms) ab, nicht aber auf die Struktur des Quellcodes selbst.

EP wendet hauptsächlich die selbstanpassende Mutation auf eine Gruppe (d. h. die Population) bestimmter Individuen an. Im Allgemeinen wird jedes Elternindividuum genommen, um ein Nachkommenindividuum zu erzeugen. Es wird kein klas-

sisches Crossover eingesetzt, sondern ein Variationsoperator ermöglicht die Kombination der Informationen verschiedener Eltern für die Nachkommen. Die Selektion erfolgt in umgekehrter Weise wie bei EA: Bei dieser EP-Selektion, der so genannten Überlebenden-Selektion, werden die schlechten Individuen (Zwischenlösungen) entfernt, die nicht in die nächste Generation übernommen werden sollen.

2.2.2.5 Multi-Objektive Evolutionäre Algorithmen

Non-dominated Sorting Genetic Algorithms (NSGA) und *Strength Pareto Evolutionary Algorithms* (SPEA) sind Varianten von EA (bzw. GA), die in der Mehrzieloptimierung (auch als Multikriterien- oder Pareto-Optimierung bezeichnet) eingesetzt werden, um Optimierungsprobleme mit mehr als einer gleichzeitig zu optimierenden Zielfunktion zu lösen. Diese Form der Optimierung ist eine Art der mehrkriteriellen Entscheidungsfindung, bei der das Ergebnis des Optimierungsprozesses eine mehrdimensionale Hyperfläche aus nicht-dominierten (Pareto-optimalen) und dominierten Lösungen ist.

Zwei wichtige Beispielmethoden von EA, die sich zur Lösung solcher Mehrziel-Optimierungsprobleme eignen, sind *Non-dominated Sorting Genetic Algorithm* (NSGA und der verbesserte NSGA-II) (Deb et al., 2002) und *Strength Pareto Evolutionary Algorithm* (SPEA und der verbesserte SPEA2) (Zitzler et al., 2001). Sie nutzen den Vorteil, dass ein GA ein populationsbasiertes Verfahren ist. Bei der Anwendung auf Mehrziel-Optimierungsprobleme konvergieren die Individuen zur Pareto-Front, während die Verteilungsmechanismen dieser Methoden dafür sorgen, dass die einzelnen Individuen (Lösungskandidaten) breit entlang der Pareto-Front verteilt sind und sich nicht an einem Punkt kumulieren.

2.2.2.6 Memetische Algorithmen

Memetische Algorithmen (Chen et al., 2011) (Memetic Algorithm, MA) sind eine Sammlung verschiedener hybrider Algorithmen, die EA (oder zumindest einen populationsbasierten Ansatz) mit lokalen, oft deterministischen Such- oder Lernstrategien kombinieren. Beispiele für MA werden auch als Baldwinsche oder Lamarcksche EA, kulturelle Algorithmen oder genetische lokale Suche bezeichnet. Die Grundidee ist, dass der Evolutionsprozess eines EA durch selektive Veränderung der Individuen in der Population in jedem Iterationsschritt beeinflusst, kontrolliert oder angepasst wird. Ein Mem ist ein kulturelles Merkmal (z. B. eine Eigenschaft oder ein Verhalten), das von Mensch zu Mensch weitergegeben wird. Grob gesagt, sind MA erweiterte EA, bei denen zusätzliche Meme zusätzliche Strategien oder Wissen über das Problem liefern, um den Optimierungsprozess zu verbessern.

Der Standard-MA (manchmal auch als MA der ersten Generation bezeichnet) integriert eine zusätzliche verbesserte lokale Suche während des Evolutionsprozesses des EA, da ein EA in seiner populationsbasierten globalen Suche zwar effektiv ist, aber oft nicht ausreichend effizient zum Optimum konvergiert. Annahme ist,

dass in der Nähe des Optimums eine lokale Verfeinerungsstrategie effektiver ist. Ein Standard-MA kombiniert diese beiden Strategien auf der Ebene der individuellen Evolution der Individuen (d. h. der Lösungskandidaten) durch die selektive Anwendung lokaler Suchstrategien.

Die erweiterten MA, der oft als Multi-Meme-MA (oder MA der zweiten Generation) bezeichnet wird, verändert die Individuen direkt, indem er mögliche Meme-Merkmale, d. h. zusätzliches Wissen über das Problem, auf die Lösungskandidaten anwendet. Innerhalb der Iterationsschritte verbessern sich nicht nur die Lösungskandidaten, sondern auch die Art und Weise, wie die Meme angewendet werden. Es handelt sich um eine Art selbsterfüllende Prophezeiung: Die Individuen, welche die besseren Meme-Merkmale tragen, sind überlegen, was dazu führt, dass diese Meme ausgewählt werden, um die Lösungskandidaten weiter zu verbessern. Während bei dieser Art von MA die Menge der möglichen Meme vordefiniert ist, optimiert sich bei einem selbstgenerierenden MA (oder MA der dritten Generation) der Einsatz der möglichen Meme im Rahmen eines (Meta-) Optimierungsprozesses selbst.

2.2.2.7 Andere Techniken im Evolutionary Computation

In einem *Learning Classifier System* (LCS) werden nicht nur Zustände oder Eigenschaften entwickelt, sondern auch Regeln. Und neben diesen Regeln wird auch die Reihenfolge ihrer Anwendung optimiert, um bestimmte Aktionen in einer bestimmten Umgebung auszuführen. Diese Regeln sind in den Chromosomen kodiert, die mit Hilfe der GA-Operatoren verändert werden. Nach dem Prinzip des Reinforcement Learning wird der Fitnesswert berechnet, der die Qualität dieser Regeln und der damit verbundenen Aktionen in der jeweiligen Umgebung bemisst.

Differential Evolution (DE) ist mit EA verwandt und arbeitet mit reellwertigen Individuen in den Populationen und Operatoren, die in ES und GA verwendet werden. DE wird hauptsächlich für Optimierungsprobleme mit reellwertigen mehrdimensionalen, oft verrauschten und zeitlich veränderlichen Problemen verwendet, bei denen keine Gradienteninformationen verfügbar sind. Die Haupterweiterung von DE liegt in der Generierung von Nachkommenindividuen (Kandidatenlösungen) unter Verwendung der Informationen von drei übergeordneten Kandidatenlösungen. Die Kombination der in diesen drei Elternlösungen gespeicherten Informationen ermöglicht es, den Optimierungsprozess gezielter in Richtung einer schnelleren Konvergenz zu lenken.

Harmony Search (Geem et al., 2001) (HS) modifiziert iterativ die Vektorkomponenten in einer Menge von Lösungskandidaten in Analogie zur Improvisation melodischer, harmonischer Musik. Die Besonderheit von HS liegt in der Kombination aller Lösungskandidatenvektoren und der entsprechenden Fitnesswerten zu einer Matrix, dem Harmoniespeicher. Die Einträge der Matrix werden selektiv durch Methoden des Komponierens oder Arrangierens von Musik verändert und sollen so ermöglichen, dass die Lösungskandidaten zu einem globalen Optimum konvergieren.

Weitere neuere Techniken der evolutionären Berechnung sind die *Neuroevolution* (Entwicklung neuronaler Netze unter Verwendung von GP-Prinzipien), die

evolutionäre Robotik (Entwicklung von Steuerungen für autonome Roboter) und das *Learnable Evolution Model* (die Evolution wird durch Hypothesen gesteuert und nicht durch die zufällig angewandten EA-Operatoren zur Erzeugung neuer Individuen).

2.2.3 Schwarmintelligenz – Swarm Intelligence

Das Interesse an der Erforschung der *Schwarmintelligenz* (Swarm Intelligence, SI) begann in den frühen 1990er-Jahren (Meystell et al., 1990). Die zur SI gehörenden Methoden folgen dem Prinzip der Schwärme, das gemeinhin als „*Optimierung durch Emergenz*“ oder „*das Ganze ist mehr als seine Einzelteile*“ bezeichnet wird. Emergenz bedeutet hier das Auftreten von Mustern, größeren Einheiten oder Regelmäßigkeiten. Dabei kopiert SI das kollektive Verhalten von selbstorganisierenden Multiagentensystemen, wie sie in der Natur oder in natürlichen Prozessen vorkommen: Eine Population einfacher Agenten (Individuen, Lösungskandidaten) interagiert lokal, meist nach einfachen Regeln, was zu einem intelligenten Gesamtverhalten führt. Das einzelne Individuum hat dabei meist nur eine geringe Bedeutung.

Ein intelligentes Verhalten tritt erst nach der Interaktion mit anderen, meist ähnlichen agierenden Individuen auf. Stochastische Mechanismen sorgen für eine Variation innerhalb der Population, um die Lebensumgebung (d. h. den Suchraum) und die Lösungswege ausreichend zu erkunden, ohne (fast) jedes zusätzliche Wissen über das Problem. Dieses Schwarmverhalten wird meist zur Lösung von Optimierungs- und Suchproblemen eingesetzt. Ein großer Anwendungsbereich ist die Logistik, wo oft kombinatorische Aspekte zu sehr großen Suchräumen führen.

Schwarmalgorithmen (Swarm Algorithm, SA) werden oft als Unterkategorie von SI und EC genannt, weil sie den Prinzipien von Schwärmen und der Evolution folgen. In letzter Zeit werden SA mehr und mehr als die repräsentative Methode der SI betrachtet. Manchmal wird SA sogar als Synonym für SI verwendet.

Die wichtigsten Methoden, die zur SI bzw. SA gehören, sind die Partikelschwarmoptimierung (Particle Swarm Optimization, PSO), die Ameisenkolonieoptimierung (Ant Colony Optimization, ACO), der Bienenalgorithmus (Bee Algorithm, BA, in letzter Zeit häufiger als Artificial Bee Colony (ABC)-Optimierung bezeichnet) und die Kuckuckssuche (Cuckoo Search, CS). Viele weitere Methoden finden sich z. B. in der Klassifikation von Fister Jr. et al. (2013). PSO kennt weitere Untermethoden wie binäres, diskretes oder kombinatorisches PSO. Auch die Firefly Algorithmen (FA) und die sehr ähnliche Glowworm Swarm Optimization (GSO) werden als Unterverfahren von SA betrachtet. Und es entstehen ständig weitere Untermethoden der SI bzw. SA.

Die Anwendungsgebiete der SI-Methoden umfassen vor allem logistische oder Netzwerkprobleme, wie Routing-, Transport- und Netzwerkkonfigurationsprobleme, aber auch die Simulation von Menschenmassen, Partikelhaufen und Ansammlungen von bestimmten Gegenständen. Weitere Anwendungsgebiete sind die

Entwicklung von Software-Controllern und der Bereich der Kollaboration bzw. Interaktion mehrerer Roboter, folgerichtig als Schwarmrobotik bezeichnet.

2.2.3.1 Partikelschwarmoptimierung – Particle Swarm Optimization

Die Partikelschwarmoptimierung (Particle Swarm Optimization, PSO) wird – in seiner Standardform – verwendet, um das globale Optimum in einem kontinuierlichen *n-dimensionalen* Suchraum zu finden. Bei PSO werden die Lösungskandidaten als *Partikel* bezeichnet und nicht als Individuen wie bei EC. Diese Partikel bilden eine Population, den Schwarm, der sich durch den Suchraum bewegt. Iterativ ziehen diejenigen Partikel des Schwarms mit der relativ besseren Fitness die anderen Partikel mit einer relativ schlechteren Fitness an, so dass sich der Schwarm auf ein lokales Optimum zubewegt (konvergiert), während einige Partikel immer den Raum nach Lösungen mit einer besseren Fitness absuchen.

In der Grundvariante von PSO bewegt sich jedes Partikel p_i des Schwarms der Größe n in Abhängigkeit von seiner aktuellen Position x_i und seinem jeweiligen Geschwindigkeitsvektor v_i im Suchraum. Der Geschwindigkeitsvektor jedes Partikels, also die Bewegung jedes Einzelnen, wird zusätzlich von den Positionen und Geschwindigkeiten der anderen Partikel im Schwarm beeinflusst, vorzugsweise am meisten von dem oder den zuletzt bekannten besten Partikel(n) des Schwarms. Nach einer Iteration bewegen sich alle Partikel p_i in Abhängigkeit von ihrer früheren Position und ihren angepassten Geschwindigkeitsvektoren an eine neue Position. Ein anderes Partikel könnte nun das beste Teilchen des Schwarms in dieser Iteration sein (und die anderen entsprechend stark anziehen).

Zustand des Partikels p_i mit seiner Anfangsposition x_i und Geschwindigkeit v_i:

$$p_i = \left(x_i, v_i\right)$$

v_i^{new} ist die angepasste Geschwindigkeit von Partikel p_i in Abhängigkeit von der Geschwindigkeit aller anderen Partikel p_j, die mit den Einflussfaktoren a_{ij} berechnet wird. Diese Einflussfaktoren hängen hauptsächlich von der Entfernung zwischen den beiden Partikeln und der relativen Fitness aller Partikel p_j zu Partikel p_i ab:

$$v_i^{new} = v_i + \sum_{j=1}^{n} a_{ij}\left(v_j - v_i\right)$$

Die neue Position x_i^{new} des Partikels p_i ergibt sich nach dem Iterationsschritt Δt direkt aus v_i^{new} und der alten Position x_i:

$$x_i^{new} = x_i + v_i^{new} \cdot \Delta t$$

Eine Verbesserung von PSO wurde von James Kennedy und Russell Eberhart eingeführt (Kennedy & Eberhart, 1995). Sie definierten die neue Geschwindigkeit v_i^{new}

als eine Funktion von zwei Summanden, die zur aktuellen Geschwindigkeit addiert werden. Der erste Summand ist rückbezogen im Sinne der Position p^{best}, an der das Partikel p_i seine beste Fitness im Suchraum gefunden hatte. Der zweite Summand ist definiert als die Position p^{global}, an welcher der Schwarm im Gesamten bisher die beste Fitness gefunden hatte:

$$v_i^{new} = w_i v_i + c_i^{best} r_i^{best} \left(p_i^{best} - x_i \right) + c_i^{global} r_i^{global} \left(p^{global} - x_i \right)$$

Dabei sind die Koeffizienten c^{best} und c^{global} Beschleunigungsfaktoren, die es dem Partikel ermöglichen, sich mehr oder weniger dem Verhalten des gesamten Schwarms anzupassen oder individueller zu sein. Die Zufallskoeffizienten r^{best} und r^{global} folgen einer gleichmäßigen Zufallsverteilung im Intervall [0, 1], während w ein Trägheitskoeffizient ist, der normalerweise während des Optimierungsprozesses reduziert wird, um die Konvergenzfähigkeit zu erhöhen.

2.2.3.2 Diskrete Partikelschwarmoptimierung – Discrete Particle Swarm Optimization

Die *diskrete Partikelschwarmoptimierung* (Discrete Particle Swarm Optimization, DPSO), die auch als *kombinatorische PSO* oder im digitalen Fall als *binäre PSO* bezeichnet wird, ist eine Erweiterung der PSO auf diskrete Suchräume, wie sie bei kombinatorischen Problemen vorkommen. Dabei ist die Position x_i des Partikels p_i ein Lösungsvektor aus diskreten Zahlen in einem *n-dimensionalen* diskreten Suchraum. Bei dem kombinatorischen Problem, die fünf Zahlen {1, 2, 3, 4, 5} hinsichtlich ihres Wertes eindeutig absteigend anzuordnen, könnte ein Lösungskandidat beispielsweise $x_i = (3, 2, 4, 5, 1)$ sein, wobei die optimale Lösung $x_{opt} = (5, 4, 3, 2, 1)$ ist.

Im Gegensatz zur Hinzufügung einer geschwindigkeitsabhängigen Komponente zur aktuellen Position in jeder Iteration definiert der Geschwindigkeitsvektor v_i nun eine oder sogar eine Reihe von Austauschoperationen, die an der Kandidatenlösung x_i durchzuführen sind. $v_i = \{(1, 4)\}$ bedeutet z. B., dass der erste und der vierte Eintrag ausgetauscht werden:

$$x_i^{new} = v_i \left(x_i \right) = v_i^{(1,4)} \left(x_i \right) = v_i^{(1,4)} \left(3,\ 2,\ 4,\ 5,\ 1 \right) = \left(5,\ 2,\ 4,\ 3,\ 1 \right)$$

2.2.3.3 Ameisenkolonieoptimierung – Ant Colony Optimization

Die Ameisenkolonieoptimierung (Ant Colony Optimization, ACO) wurde Anfang der 1990er-Jahre von Marco Dorigo vorgeschlagen (Colorni et al., 1991). Bei ACO werden die Individuen als *Ameisen* (Ants) bezeichnet, die versuchen, den kürzesten Weg von ihrer Kolonie, dem Startpunkt, zu einer so genannten Nahrungsquelle durch einen Graphen möglicher Wege im Suchraum zu finden. ACO wird häufig bei

Planungs-, Routing- oder Zuordnungsproblemen eingesetzt, bei denen die Abfolge diskreter Schritte optimiert werden muss. Heute gibt es viele verschiedene Varianten von ACO.

Ursprünglich sind die Grundlagen des rechnergestützten Optimierungsalgorithmus ACO stark vom Verhalten echter Ameisen inspiriert: Ameisen markieren ihre Kriechspuren mit Pheromonen, einem Geruchsstoff, der kontinuierlich abgegeben wird, während sich die Ameisen bewegen. Die Pheromonspur verdunstet langsam mit der Zeit. Je öfter jedoch dieselben Ameisen oder weitere neue Ameisen auf derselben Spur krabbeln, desto intensiver wird der Pheromonlevel. Vorbeikommende Ameisen werden dann von der Pheromonintensität beeinflusst und wechseln mit höherer Wahrscheinlichkeit auf die attraktivere Spur mit dem höheren Pheromonlevel.

Jede Ameise repräsentiert einen Lösungskandidatenvektor x_i (die Spur) als eine Folge von diskreten Kanten, z. B. x_i = (A, D, B, C, E) vom Startpunkt A zum Endpunkt E. Zu Beginn werden alle Ameisen und Pheromonlevel auf den Kanten zufällig initialisiert. Die Fitness jeder Ameise wird berechnet. Dann fügen die Ameisen (vorzugsweise nur die mit den besseren Fitnesswerten) neues Pheromon ihren Pfaden hinzu, also auf allen Kanten ihres Lösungswegs. Anschließend aktualisiert jede Ameise ihren Lösungskandidatenvektor x_i, wobei einige Kanten aufgrund eines Kompromisses zwischen der Teilfitness der Kante und des darauf gesetzten bzw. wieder verdunsteten Pheromonlevels geändert werden können. Von Iteration zu Iteration neigen die Ameisen dazu, den Teilwegen mit den stärkeren Pheromonleveln zu folgen. Im Laufe der Zeit wird die kürzeste Route gefunden, da sie mit dem stärksten Pheromonlevel markiert ist.

2.2.4 Neuronale Netze – Neural Networks

Die Computermodelle, welche die Funktionsweise biologischer Gehirne oder zumindest kleiner Teile davon auf dem Computer simulieren, werden als *Neuronale Netze* (Neural Networks, NN) bezeichnet. In der Literatur wird manchmal auch die Bezeichnung *Künstliche Neuronale Netze* (KNN; Artificial Neural Networks, ANN) verwendet, um zwischen den neuronalen Netzen der biologischen Gehirne von Tieren und Menschen und dem Computermodell der NN zu unterscheiden. In diesem Buch verwenden wir nur die Abkürzung NN für die Computermodelle.

NN nutzen das Prinzip der natürlichen Kognition. Inspiriert von den realen biologischen neuronalen Netzen des Gehirns, verarbeiten NN Informationen durch ein Netz miteinander verbundener künstlicher Neuronen – eine abstrakte Sichtweise, die der Informationsverarbeitung in biologischen Gehirnen ähnelt. Es gibt verschiedene Arten von NN, von denen die am ältesten bekannten Feedforward Neural Networks (FNN) bzw. Perceptron Neural Networks (PNN) und Self-Organizing Maps (SOM) sind.

Die FNN sind die einfachsten NN. Sie verwenden einen Datenfluss von den Neuronen der Eingabeschicht über die Neuronen der verdeckten Schichten (Hid-

den Layers, falls es überhaupt welche gibt) zu den Neuronen der Ausgabeschicht. Während die jeweiligen Eingabewerte mit den Gewichten der entsprechenden Verbindungen zwischen den Neuronen multipliziert werden, summiert jedes Neuron alle seine Eingaben auf. Eine Aktivierungsfunktion legt einen Schwellenwert fest, bei dem das Neuron auf einen Ausgangswert im Allgemeinen größer als Null schaltet. Dieser einfache Algorithmus wird als Perzeptron bezeichnet. Während einer Trainingsphase des NN, also vor seiner Anwendung, werden die Gewichte und möglicherweise andere Netzparameter angepasst. Anhand von Trainingsdatensätzen werden alle Gewichte im NN so eingestellt, dass bei gegebenen Eingabedaten so gut wie möglich die generierten Ausgabedaten mit den bekannten Ausgabedaten übereinstimmen. Vereinfacht gesagt dient die Fehlerrückführung (Backpropagation) dazu, insgesamt das gesamte NN zu trainieren. Bei einfachen NN ist die Anzahl der Neuronen pro Schicht und die Anzahl der Schichten vorgegeben. High-Level-NN erlauben es, die Anzahl der Neuronen, Schichten und Verbindungen und damit Umfang des Netzes nach Bedarf während des Trainings anzupassen.

Im Gegensatz dazu werden SOM nicht explizit trainiert. SOM werden meist für die Zuordnung von hochdimensionalen zu niedrigdimensionalen Daten verwendet, z. B. bei Clustering-Problemen. SOM werden auch als Kohonen-Maps bezeichnet, benannt nach Teuvo Kohonen, der diese Art von NN in den 1980er-Jahren einführte (Kohonen, 1982). In Unterscheidung zu den klassischen NN verwenden SOM eine andere Art von Neuronen mit speziellen Nachbarschaftsfunktionen, um topologische Eigenschaften des Eingangsvektors zu identifizieren. SOM arbeiten hierbei mit Mapping: Die Vektoren der Trainingsdaten bilden eine Karte, die dann zur Klassifizierung eines unbekannten neuen Datenvektors verwendet wird.

Die Hauptanwendungsbereiche von NN (wie FNN, SOM u. v. m.) sind Klassifizierung und Optimierung. Hier bewerten NN implizit die Beziehungen zwischen Eingabe- und Ausgabedaten und finden Muster. NN sind fehler- und rauschtolerant und verwenden (meist) nichtlineare Schwellenwert-Aktivierungsfunktionen.

2.2.5 Fuzzy-Logik – Fuzzy Logic

Die *Fuzzy-Logik* (Fuzzy Logic, FL; Unschärfelogik) wendet das Prinzip der Unschärfe an, indem sie davon ausgeht, dass logische Werte nicht nur wahr oder falsch oder entweder 1 oder 0 sind, sondern jeden Wert aus dem Intervall [0, 1] annehmen können. Ähnlich wie die menschliche Kognition und das menschliche Denken ermöglicht FL eine Modellierung des Problems, um mit Unschärfe umgehen zu können: FL verwendet eine vielwertige Logik, die durch eine Menge logischer Werte dargestellt wird und Näherungswerte sowie unvollständige oder mehrdeutige Daten zulässt. FL ist in der Lage, Schlussfolgerungen und ungefähre Lösungen auf der Grundlage von Fuzzy-Informationen zu liefern. Die Anwendungsbereiche sind die Modellierung von Fuzzy-Ausdrücken, die Erstellung

von Schlussfolgerungen auf der Grundlage von Fuzzy-Informationen, die Generierung von Fuzzy-Regelsätzen und das Clustering von verrauschten Daten. FL wird gerne als sehr fehler- und rauschtolerant beschrieben.

2.2.6 Künstliches Immunsystem – Artificial Immune Systeme

Die Methoden, die zum *künstlichen Immunsystem* (Artificial Immune Systeme, AIS) gehören, folgen den Prinzipien der Bekämpfung von Krankheitserregern in biologischen Körpern. Das menschliche Immunsystem identifiziert bestimmte Krankheitserreger, deren Antigene – vereinfacht gesprochen – sich an bestimmte Antikörper binden, als potenzielle Krankheitsquelle. AIS orientiert sich ebenso am Immunsystem von Menschen und Tieren. Die AIS-Methoden verwenden eine Population von Agenten, die lernen, bestimmte Muster von Antigenen zu erkennen, die in ein System eindringen. Im Laufe der Generationen verbessert sich diese Erkennung durch positive stimulierte Selektion, klonale Vermehrung und Anpassung, um auch neue, möglicherweise mutierte Krankheitserreger zu erkennen. Diese Merkmale und Prozesse des biologischen Immunsystems für Lernen und Erkennen werden auf AIS übertragen. Anwendungsgebiete von AIS sind die Entwicklung adaptiver Systeme und optimaler Verhaltensweisen zur Problemlösung sowie die Mustererkennung.

2.2.7 Weitere verwandte Methoden

2.2.7.1 Reinforcement Learning

Reinforcement Learning (Kramer, 2009) (RIL) folgt dem Prinzip von „Belohnung und Bestrafung“. Erwünschtes Verhalten wird belohnt, während unerwünschtes oder nicht zielführendes Verhalten bestraft wird. Verschiedene Lernstrategien (z. B. dynamische Programmierung, Zeitdifferenzlernen) ermöglichen es dem System, iterativ ein besseres Verhalten zu erlernen. Anwendungsgebiete sind die Kontrolle von Eigenschaften und das Lernen von optimalem Verhalten.

2.2.7.2 Simulated Annealing

Simulated Annealing ist ein generischer, probabilistischer, metaheuristischer Algorithmus. Obwohl diese Methode nicht von der belebten Natur inspiriert ist, sondern von der Physik oder genauer gesagt von der Thermodynamik (dem Schmelz- bzw. Erstarrungsprozess in der Metallurgie), zählen wir Simulated Annealing zu den CI-Methoden. Simulated Annealing folgt dem thermodynamischen Prinzip einer

kontrollierten Abkühlung des geschmolzenen Metalls, um die Größe der entstehenden Kristallstruktur zu erhöhen und deren Defekte zu reduzieren. Während der inkrementellen Schritte einer so implementierten fiktiven sukzessiven Temperaturabsenkung werden lokale Minima oder Maxima übersprungen, um das globale Optimum zu finden, also eigentlich, um einen Zustand geringster Energie zu erreichen. Die Anwendungsgebiete von Simulated Annealing liegen in der globalen Optimierung, wo der Suchraum meist groß und diskret ist.

2.2.7.3 Weitere Metaheuristiken: Lokale Suche und Tabu-Suche

Im Folgenden wird kurz auf weitere Metaheuristiken eingegangen, die nicht von der Natur inspiriert sind, aber Ähnlichkeiten mit einigen der oben dargestellten naturinspirierten Ansätze aufweisen. Ein zusätzlicher Grund, sie in dieses Buch aufzunehmen, ist, dass diese Metaheuristiken zu den erfolgreichsten Methoden gehören, die auf verschiedene schwer zu lösende Probleme in der Logistik angewendet werden.

Lokale Suche – Neighborhood Search
Einer der am weitesten verbreiteten Ansätze in der Metaheuristik ist die Strategie der *lokalen Suche* – auch als *Neighborhood Search* bezeichnet. Das zugrundeliegende Konzept ist, dass es sich in der Regel erfolgversprechender erweist, verbesserte Lösungen in der Nachbarschaft bereits ermittelter Lösungen zu suchen als in beliebigen Regionen des gesamten Suchraums.

Ein weiterer Grund ist, dass die lokale Suche häufig für große Suchräume verwendet wird, die mit der Anzahl der Variablen exponentiell wachsen. Während das Durchsuchen der gesamten Menge zulässiger Lösungen im Hinblick auf die Rechenzeit meist nicht vorteilhaft ist, kann die Suche in einem kleinen Nachbarschaftsbereich sehr schnell durchgeführt werden. Verständlicherweise erfordert die lokale Suche die Definition von Nachbarschaften von Lösungen, die vom jeweiligen Problemtyp abhängen und als Abstand zwischen Lösungen oder Anzahl von Operationen zur Übertragung einer Lösung in eine andere bezeichnet werden können. Die lokale Suche ist – in ihrer Grundform – eine iterative Methode, die von einer Lösung zu einer anderen besseren Lösung in der Nachbarschaft „springt“, bis ein Abbruchkriterium (z. B. maximale Anzahl von Iterationen oder stagnierende Verbesserung) erfüllt ist. Ein offensichtlicher Nachteil der lokalen Suche ist, dass sie in einem lokalen, nicht-globalen Optimum stecken bleiben kann.

Häufig wird die lokale Suche als Hintergrundkonzept verwendet, auf dem andere, komplexere Metaheuristiken aufbauen oder die lokale Suche als eingebettete Methode zur Verbesserung von Zwischenlösungen nutzen. So verwendet beispielsweise Simulated Annealing die lokale Suche in einer impliziten Form oder manche memetischen Algorithmen die lokale Suche ergänzend zu den evolutionären Algorithmen. Auch die typischen Mutationen in evolutionären Algorithmen könnten als Begriffe der lokalen Suche interpretiert werden, da sie in der Regel die Bestimmung neuer Lösungen in der Nähe der Stammlösungen bevorzugen.

Die lokale Suche wird weiter verbessert, indem sie mit anderen Metaheuristiken kombiniert wird: In *Multistart Local Search* (Hart, 1998) wird das Problem, dass

man zwar in einem lokalen, aber nicht in einem globalen Optimum landet, dadurch gelöst, dass die lokale Suche von neuen (z. B. zufällig bestimmten) Startlösungen aus neu gestartet wird.

Bei der *Iterated Local Search* (ILS) (Den Besten et al., 2001) wird das Problem durch eine gezielte Ablenkung vom gefundenen lokalen Optimum gelöst, indem die Suche wiederholt fortgesetzt wird. Im Gegensatz zu einer mehrstufigen lokalen Suche sollte der Lösungspfad hinreichend von den lokalen optimalen Lösungen abweichen (um den lokalen Optima zu entkommen), aber nicht „völlig zufällig" sein.

Tabu-Suche – Tabu Search
Eine der bekanntesten Weiterentwicklungen der lokalen Suche ist *Tabu Search* (Glover, 1989, 1990), wobei der Name „Tabu" verkürzt für „verbotener Suchraumausschnitt" steht. Die zentrale Idee dieses Ansatzes besteht darin, eine Liste von zuvor besuchten Lösungen zu verwenden, die bei der nachfolgenden Suche vermieden werden sollen. Anstatt bereits evaluierte Lösungen zu speichern, können auch Eigenschaften von solchen Lösungen gespeichert werden, die bei der anschließenden Suche vermieden werden sollen. Die Umsetzung einer so genannten Tabu-Liste adressiert das Problem, dass insbesondere bei kombinatorischen Optimierungsproblemen oft identische Lösungen während der lokalen Suche immer wieder, also mehrfach, evaluiert werden. Abgesehen von der Vermeidung unnötiger Rechenzeit kann die Tabu-Liste auch dazu beitragen, lokalen Optima zu entgehen, und zwar in Verbindung mit dem Konzept, die Suche auch in Suchraumausschnitten mit vermeintlich schlechteren Lösungen fortzusetzen, die während der Iterationen gefunden werden. Dennoch bleibt die beste gefundene Lösung immer gespeichert und am Ende des Suchprozesses berücksichtigt. Es gibt viele Varianten der Tabu-Suche, z. B. hinsichtlich der Definition und Länge der Tabu-Liste und ihrer Verwendung innerhalb der Suche.

Variable Nachbarschaftssuche – Variable Neighborhood Search
Eine weitere erfolgreiche und weit verbreitete Variante der lokalen Suche ist die *Variable Neighborhood Search* (VNS) (Mladenović & Hansen, 1997). VNS geht davon aus, dass nicht nur eine, sondern mehrere unterschiedlich große Nachbarschaften existieren. In der Regel wird angenommen, dass es k Nachbarschaften gibt, wobei eine Nachbarschaft mit einem höheren Index die Nachbarschaften mit kleineren Indizes einschließt. VNS basiert auf der Idee, dass die lokale Suche zunächst in der kleineren, näheren Nachbarschaft stattfinden sollte und dann (wenn sie nicht erfolgreich ist) mit der nächstgrößeren und breiteren Nachbarschaft fortgesetzt wird. Wenn dann eine bessere Lösung gefunden ist, beginnt die Suche wieder in der nächsten kleinsten Nachbarschaft. Dies hilft zu vermeiden, dass man in einem lokalen Optimum einer sehr kleinen Nachbarschaft stecken bleibt, aber auch, dass man zu viel Rechenzeit für die Suche in großen Nachbarschaften aufwendet.

Es gibt mehrere Varianten der VNS: Bei der *Basic VNS* wird beispielsweise ein einzelner, zufällig bestimmter kleiner Bereich aus einer betrachteten Nachbarschaft durch lokale Suche verbessert, bevor die nächste Nachbarschaft untersucht wird. Bei der *Reduced VNS* entfällt der eingebettete Schritt der lokalen Suche, d. h. wenn ein zufällig ausgewählter Punkt nicht direkt besser ist, wird die Suche mit der

nächsten Nachbarschaft fortgesetzt. *Variable Neighborhood Descent* (VND) basiert auf der Idee, eine deterministische lokale Suche in einer betrachteten Nachbarschaft durchzuführen.

Greedy Randomized Adaptive Search Procedure (GRASP)
Als letztes Beispiel für häufig verwendete Metaheuristiken sei das Verfahren *Greedy Randomized Adaptive Search Procedure* (GRASP) (Feo & Resende, 1995) genannt, das auf Iterationen zur Konstruktion und anschließenden Verbesserung von Lösungen durch lokale Suche beruht. Die Konstruktionsphase ist problemspezifisch. Die allgemeine Idee besteht darin, nacheinander „Lösungselemente" hinzuzufügen, die ihre „Qualität" auf „gierige Weise" erhöhen – also Lösungsraumausschnitte suchen, in denen eine große Konvergenz zu besseren Lösungen vorliegt. Beim Hinzufügen dieser Lösungselemente wird eine gewisse Zufälligkeit eingebaut, um die Variabilität zu erhalten.

Literatur

Büche, D., Stoll, P., Dornberger, R., & Koumoutsakos, P. (2002). Multiobjective evolutionary algorithm for optimization of noisy combustion processes. *IEEE Transactions on Systems, Man, and Cybernetics Part C: Applications and Reviews, 32*(4), 460–473.

Burke, E. K., & Kendall, G. (Hrsg.). (2006). *Search methodologies – Introductory tutorials in optimization and decision support techniques*. Springer.

Chen, X., Ong, Y.-S., Lim, M.-H., & Tan, K. C. (2011). A multi-facet survey on memetic computation. *IEEE Transactions on Evolutionary Computation, 15*(5), 591–607.

Colorni, A., Dorigo, M., & Maniezzo, V. (1991). Distributed optimization by ant colonies. In *Proceedings of the First European Conference on Artificial Life* (Bd. 142, S. 134–142). Elsevier Publishing.

Deb, K., Pratap, A., Agarwal, S., & Meyarivan, T. (2002). A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation, 6*(2), 182–197.

Den Besten, M., Stützle, T., & Dorigo, M. (2001). Design of iterated local search algorithms. In *Applications of evolutionary computing* (S. 441–451). Springer.

Dornberger, R., Büche, D., & Stoll, P. (2000). Multidisciplinary optimization in turbomachinery design. In *European Congress on Computational Methods in Applied Sciences and Engineering (ECCOMAS 2000)*.

Engelbrecht, A. P. (2007). *Computational intelligence: An introduction*. Wiley.

Feo, T. A., & Resende, M. G. (1995). Greedy randomized adaptive search procedures. *Journal of Global Optimization, 6*(2), 109–133.

Fister, I., Jr., Yang, X. S., Fister, I., Brest, J., & Fister, D. (2013). A brief review of nature-inspired algorithms for optimization. *Elektrotehniski Vestnik/Electrotechnical Review, 80*(3), 116–122. arXiv preprint arXiv:1307.4186.

Fogel, L. J. (1999). *Intelligence through simulated evolution: Forty years of evolutionary programming*. Wiley Series on Intelligent Systems.

Fulcher, J., & Jain, L. C. (Hrsg.). (2008). *Computational intelligence: A compendium (Studies in computational intelligence* (Bd. 115). Springer.

Geem, Z. W., Kim, J. H., & Loganathan, G. V. (2001). A new heuristic optimization algorithm: Harmony search. *Simulation, 76*(2), 60–68.

Glover, F. (1989). Tabu search – Part I. *ORSA Journal on Computing, 1*(3), 190–206.

Glover, F. (1990). Tabu search – Part II. *ORSA Journal on Computing, 2*(1), 4–32.

Goldberg, D. E. (1989). *Genetic algorithms in search, optimization, and machine learning*. Addison-Wesley.
Hansen, N., & Ostermeier, A. (2001). Completely derandomized self-adaptation in evolution strategies. *Evolutionary Computation, 9*(2), 159–195.
Hart, W. E. (1998). Sequential stopping rules for random optimization methods with applications to multistart local search. *SIAM Journal on Optimization, 9*(1), 270–290.
Holland, J. H. (1975). *Adaptation in natural and artificial systems: An introductory analysis with applications to biology, control, and artificial intelligence*. University of Michigan Press.
Kennedy, J., & Eberhart, R. (1995). Particle swarm optimization. In *IEEE international conference on neural networks, 1995* (Bd. 4, S. 1942–1948). IEEE.
Kohonen, T. (1982). Self-organized formation of topologically correct feature maps. *Biological Cybernetics, 43*(1), 59–69.
Koza, J. R. (1992). *Genetic programming: On the programming of computers by means of natural selection*. MIT Press.
Kramer, O. (2009). *Computational intelligence – Eine Einführung*. Springer.
Kruse, R., Borgelt, C., Klawonn, F., Moewes, C., Steinbrecher, M., & Held, P. (2013). *Computational intelligence – A methodological introduction*. Springer.
McCarthy, J. (2007). *What is artificial intelligence*?. http://www-formal.stanford.edu/jmc/whatisai/whatisai.html. Zugegriffen am 17.04.2014.
McCarthy, J., Minsky, M., Rochester, N., & Shannon, C. (1955). *A proposal for the Dartmouth summer research project on artificial intelligence*. Archiviert auf http://www-formal.stanford.edu/jmc/history/dartmouth/dartmouth.html. Zugegriffen am 30.11.2013.
Meystell, A., Herath, J., & Gray, S. (Hrsg.). (1990). *Proceedings of 5th IEEE international symposium on intelligent control 1990*. IEEE.
Mitchell, T. M. (1997). *Machine learning*. McGraw-Hill.
Mladenović, N., & Hansen, P. (1997). Variable neighborhood search. *Computers & Operations Research, 24*(11), 1097–1100.
Neapolitan, R. E., & Jiang, X. (2012). *Contemporary artificial intelligence*. Chapman and Hall.
Poole, D. L., Mackworth, A. K., & Goebel, R. (1998). *Computational intelligence: A logical approach*. Oxford University Press.
Rechenberg, I. (1973). *Evolutionsstrategie – Optimierung technischer Systeme nach Prinzipien der biologischen Evolution*. Frommann-Holzboog.
Rechenberg, I. (1994). *Evolutionsstrategie '94*. Frommann-Holzboog.
Rozenberg, G., Bäck, T., & Kok, J. N. (Hrsg.). (2012). *Handbook of natural computing*. Springer.
Schwefel, H.-P. (1977). *Numerical optimization of computer models*. Birkhäuser.
Zitzler, E., Laumanns, M., & Thiele, L. (2001). *SPEA2: Improving the strength Pareto evolutionary algorithm*. TIK-report 103. Computer Engineering and Networks Laboratory (TIK), Swiss Federal Institute of Technology (ETH).

Kapitel 3
Transportprobleme



Der Bereich des Transports gehört zu den wichtigsten Bereichen der Logistik. Transportprobleme treten in verschiedenen Varianten und Komplexitäten auf und erfordern aufgrund ihrer Bedeutung und der Notwendigkeit, sie effizient, d. h. mit hoher Qualität und geringen Kosten zu lösen, eine sorgfältige Planung. Im Allgemeinen erfordert die Transportplanung die Bestimmung der Art und Menge der Güter, sowie von wo nach wo, zu welchem Zeitpunkt und mit welchen Ressourcen, insbesondere mit welchen Fahrzeugen, diese transportiert werden sollen. Auch die Festlegung eines bestimmten Weges oder einer Route gehört zum Bereich der Transportplanung.

In vielen Fällen sind einige dieser Aspekte von vornherein festgelegt, z. B. sind der Herkunfts- und der Bestimmungsort häufig vorgegeben. Wir gehen auch davon aus, dass Entscheidungen über Produktion und Lagerhaltung vorher getroffen werden und nicht in Kombination mit der Planung von Transporten betrachtet werden. (Einige solcher kombinierten Planungsprobleme werden in späteren Kapiteln des Buches behandelt).

Leider sind viele Varianten von Transportproblemen aus rechnerischer Sicht schwierig (NP-schwer). Das bedeutet, dass es keine Algorithmen gibt, die in der Lage sind, eine optimale Lösung mit einer akzeptablen Laufzeit zu finden, zumindest nicht für größere Instanzen von Transportproblemen.

Im Folgenden werden wir einige Hauptvarianten von Transportproblemen näher betrachten und Methoden, die in der Lage sind, sie zumindest mit ausreichender Qualität zu lösen. Zunächst werden Zuordnungsprobleme diskutiert, die eine Transportplanung auf einer eher groben Ebene ermöglichen, indem sie die Orte mit verfügbaren Gütern und den Orten mit Bedarf auf der Grundlage der Transportkosten zuordnen. Danach werden die vergleichsweise einfachen, aber weit verbreiteten Aufgaben der Planung kürzester Wege diskutiert. In den nächsten Abschnitten werden die wichtigsten Arten von komplexeren Tourenplanungsproblemen diskutiert: das Travelling-Salesman-Problem und das Vehicle-Routing-Problem. Insbesondere

© Der/die Autor(en), exklusiv lizenziert an Springer Nature Switzerland AG 2023

T. Hanne, R. Dornberger, *Computational Intelligence in Logistik und Supply Chain Management*, https://doi.org/10.1007/978-3-031-21452-3_3

für das Vehicle-Routing-Problem werden mehrere praktisch relevante Varianten betrachtet, darunter Abhol- und Zustellprobleme. Das Kapitel endet mit einer kurzen Diskussion von Netzwerkflussproblemen.

3.1 Zuordnungsprobleme

Eines der ersten Probleme im Bereich der Transportplanung besteht darin, zu bestimmen, von welchem Ursprungsort zu welchem Zielort Güter transportiert werden sollen, wobei davon ausgegangen wird, dass an bestimmten Orten eine Nachfrage nach Gütern besteht und die benötigten Güter an anderen Orten verfügbar sind. Außerdem wird davon ausgegangen, dass die Entfernungen zwischen den verschiedenen Standorten festgelegt sind, d. h. es besteht keine Notwendigkeit für eine detailliertere Festlegung der Transportwege. Auch der Einsatz von bestimmten Fahrzeugen oder Fahrzeugtypen wird nicht berücksichtigt.

Das Ziel dieser Art von Problem besteht lediglich darin, zu bestimmen, welche Menge eines bestimmten Gutes von welchen Orten (mit verfügbaren Gütern) zu welchen anderen Orten, an denen eine Nachfrage nach diesem Gut besteht, transportiert werden sollte, um die Nachfrage zu befriedigen und die Transportkosten zu minimieren. In der Regel wird davon ausgegangen, dass eine ausreichende Gesamtmenge an Gütern zur Verfügung steht, um die gesamte Nachfrage zu befriedigen.

Betrachten wir eine einfache Version dieses Typs eines transportbezogenen Zuordnungsproblems, das auch als Linear Sum Assignment Problem (LSAP) bezeichnet wird (Burkard & Cela, 1999). Nehmen wir an, dass wir n Standorte von Güterdepots haben, bezeichnet als Menge L^D, und n Standorte von Verbrauchern, bezeichnet als Menge L^C, die die Güter von einem der Depots benötigen. Wir nehmen an, dass eine Kostenfunktion gegeben ist, die die Kosten einer einzelnen Lieferung von einem Depot i zu einem Verbraucher j misst, d. h.,

$$c_{ij} := c\left(l_i^D, l_j^C\right) \text{ für } i \in \{1, \ldots, n\},\ \ j \in \{1, \ldots, n\}, l_i^D \in L^D, l_j^C \in L^C. \qquad (3.1)$$

Die Kosten könnten beispielsweise durch den euklidischen Abstand zwischen den Standorten von l_i^D und l_j^C gemessen werden.

Es gibt noch einige weitere Annahmen, die im allgemeinen Fall nicht besonders realistisch erscheinen: Wir gehen davon aus, dass jeder Verbraucher eine einzige Lieferung aus einem Depot benötigt und dass jedes Depot genau einen Kunden beliefern kann. Außerdem können Sendungen nicht aufgeteilt werden. Ansonsten gibt es keine weiteren Einschränkungen zu beachten.

Das Transportproblem besteht dann darin, geeignete Zuordnungen von Depots zu Kunden zu finden, die die Gesamtkostenfunktion minimieren. Die Zuordnung kann durch die Variablen x_{ij}, $i \in \{1, \ldots, n\}$, $j \in \{1, \ldots, n\}$ ausgedrückt werden, wobei $x_{ij} = 1$ ist, wenn der Kunde j vom Depot i beliefert wird, und $x_{ij} = 0$ andernfalls. Alle Variablen können kurz durch eine $n \times n$-Matrix X notiert werden.

Die zu minimierende Gesamtkostenfunktion kann dann wie folgt angegeben werden

$$\min c(X) = \sum_{i=1}^{n}\sum_{j=1}^{n} c_{ij} x_{ij} \tag{3.2}$$

mit den folgenden Nebenbedingungen für die Variablen:

$$\sum_{i=1}^{n} x_{ij} = 1, \; \textit{für alle} \; j \in \{1, \ldots, n\}, \tag{3.3}$$

$$\sum_{j=1}^{n} x_{ij} = 1, \; \textit{für alle} \; \mathrm{i} \in \{1, \ldots, n\}, \tag{3.4}$$

$$\textit{mit} \; x_{ij} \in \{0, 1\} \; \textit{für alle} \; i, j \in \{1, \ldots, n\}. \tag{3.5}$$

Dieses einfache Zuordnungsproblem (LSAP) kann mit verschiedenen Methoden gelöst werden. Methoden der linearen Programmierung gehören zu diesen Ansätzen, da das Problem zu einem linearen Problem relaxiert werden kann, indem (3.5) durch $x_{ij} \geq 0$ für alle $i,j \in \{1, \ldots, n\}$ ersetzt wird. Somit kann das LSAP mit den verfügbaren Ansätzen der linearen Programmierung wie dem Simplex-Algorithmus und Varianten gelöst werden. Obwohl diese Methoden im allgemeinen Fall nicht polynomiell sind, kann die LSAP aufgrund der Nutzung spezieller Eigenschaften dieses Problemtyps in polynomieller Zeit gelöst werden. Unter diesen Methoden, die mit polynomialer Rechenzeit auskommen, ist die bekannteste die sogenannte ungarische Methode, die eine Laufzeit von $O(n^4)$ benötigt. Diese Methode arbeitet mit sukzessiven Zeilen- und Spaltenreduktionen, die es erlauben zu bestimmen, welche Variablen auf Eins und welche auf Null gesetzt werden. Es gibt Verbesserungen dieser Methode mit einer kleineren Laufzeit von $O(n^3)$.

3.2 Kürzeste Wege

Die Suche nach einem kürzesten Weg ist eines der häufigsten Probleme im Transportwesen und verwandten Bereichen. Bei zwei Orten A (Startpunkt) und B (Zielpunkt), die durch ein Netz (oder einen Graphen) verbunden sind, besteht die Aufgabe darin, eine zusammenhängende Folge von Straßen (oder Kanten im Allgemeinen) mit der kürzesten Entfernung zu finden. Die Entfernung ist definiert durch die Summe der Längen der verbundenen Straßenabschnitte (bzw. der Kanten des Graphen). Anstatt die Entfernung zu minimieren, kann man auch die Zeit oder die Kosten der benutzten Verbindungen des Netzes minimieren.

$G = (V, E, c)$ sei ein gewichteter Graph, d. h.: V ist eine Menge von Knoten (auch als Ecken bezeichnet), d. h. die Kreuzungspunkte von Straßen, $E \subseteq V \times V$ ist eine Menge von Kanten zwischen den Knoten (d. h. Straßenabschnitte, die Knoten verbinden) und $c, c: E \rightarrow R$, ist eine Kostenfunktion für die Kanten (z. B. die Entfernung zwischen zwei

direkt verbundenen Knoten oder die Zeit, die für das Zurücklegen der Strecke zwischen den Knoten benötigt wird, oder eine andere Art von Kostenmaß). Normalerweise wird angenommen, dass die Kosten nicht negativ sind, d. h. $c(e) \geq 0$ für alle $e \in E$.

Die Aufgabe ist dann, eine Folge von Kanten zu finden, e_1, e_2, …, e_n mit $e_1 = (A, v_2)$, $e_2 = (v_2, v_3)$, …, $e_{n-1} = (v_{n-1}, v_n)$, $e_n = (v_n, B)$ (d. h. die Folge beginnt im Startpunkt A und endet im Zielpunkt *B*), und jede Kante endet im Startpunkt der nachfolgenden Kante, so dass die Kosten minimiert sind.

Glücklicherweise können derartige Probleme mit einem angemessenen Zeitaufwand gelöst werden. Einer der bekanntesten Algorithmen zur Lösung eines Problems des kürzesten Weges ist der Dijkstra-Algorithmus. Diese Methode funktioniert wie folgt:

Schritt 1: (Initialisierung)

a. Jedem Knoten werden zwei Kennzeichnungen zugewiesen, eine für die *vorläufige Entfernung*, die mit unendlich initialisiert wird, und eine boolesche Kennzeichnung „*besucht*“, die mit falsch initialisiert wird. Für den Knoten A wird die Kennzeichnung der *vorläufigen Entfernung* mit 0 initialisiert und die Kennzeichnung „*besucht*“ auf wahr gesetzt.
b. Die Menge „*unbesucht*“ wird durch alle Knoten mit besucht = falsch initialisiert.
c. Die Variable „aktueller Knoten“ wird mit A initialisiert.

Schritt 2:

Für den „aktuellen Knoten“ wird für alle nicht besuchten Nachbarn (also Knoten, die über eine Kante erreicht werden können) geprüft, ob der Abstand des aktuellen Knotens + der Abstand vom aktuellen Knoten zum betrachteten Knoten kleiner ist als der *vorläufige Abstand* des betrachteten Knotens. In diesem Fall wird die *vorläufige Entfernung* des betrachteten Nachbarknotens durch diesen kleineren Wert aktualisiert.

Schritt 3:

Für den aktuellen Knoten wird *besucht* auf wahr gesetzt und der Knoten wird aus der Menge *unbesucht* entfernt.

Schritt 4:

Der Algorithmus hält an, wenn der aktuelle Knoten B war oder wenn der kleinste Abstand zwischen den Knoten in der *unbesuchten* Menge unendlich ist (in diesem Fall gibt es keine Verbindung zwischen A und B).

Schritt 5:

Aus der Menge der *unbesuchten* Knoten wird ein Knoten ausgewählt, der mit dem kleinsten *vorläufigen Abstand* markiert ist, und als „aktuellen Knoten“ festgelegt.

Schritt 6:

Weiter mit Schritt 2.

Man beachte, dass der Algorithmus häufig dazu verwendet wird, den kleinsten Abstand von A zu allen anderen Knoten zu bestimmen, und dass die Kennzeichnung „vorläufige Entfernung“ den kleinsten Abstand angibt, wenn die betreffenden Knoten aus der Menge der *unbesuchten Knoten* entfernt werden. Im allgemeinen Fall kann die Laufzeit des Algorithmus als quadratisch in Bezug auf die Anzahl der Knoten angesehen werden, wenn die Kosten nicht negativ sind, d. h. sie liegt bei $O(n^2)$, wenn n die Anzahl der Knoten ist. Genauere Abschätzungen sind möglich, wenn eine bestimmte Datenstruktur betrachtet wird oder sowohl die Anzahl der Kanten als auch die Anzahl der Knoten berücksichtigt wird. (Für die Laufzeitschätzung $O(n^2)$ wird berücksichtigt, dass die Anzahl der Kanten maximal n^2 beträgt.) Für die Berechnung aller kürzesten Pfade (für alle Knotenpaare) eines Graphen kann die Laufzeit durch geeignete Datenstrukturen auf $O(n^2 \log n)$ reduziert werden (Takaoka, 2013).

Probleme des kürzesten Weges werden heute in verschiedenen Softwaretools unterstützt. Sie bilden insbesondere das Rückgrat der heutigen GPS-basierten Navigationssysteme. Solche Tools verfügen über eine Darstellung der realen Straßennetzes in ihren Daten und ermöglichen es, die aktuelle Position eines Fahrzeugs oder einer Person zu bestimmen. Bei Eingabe eines Ziels kann die Software den kürzesten oder schnellsten Weg zum Ziel berechnen, wobei sie einige Annahmen über die Geschwindigkeit des Fahrzeugs (oder der Person) trifft. Dabei werden auch die spezifischen Arten von Straßen (z. B. kleine, langsame Straßen oder schnellere Autobahnen), Informationen über die unterschiedlichen Geschwindigkeiten auf den Straßen je nach Fahrtrichtung (oder die Nichtzulassung einer bestimmten Richtung bei Einbahnstraßen) und in einigen Fällen Informationen über die aktuelle Verkehrsbelastung berücksichtigt.

Eine Schwierigkeit bei dieser Art von Problemen ist die beträchtliche Größe realer Straßennetze. Obwohl ein Algorithmus für den kürzesten Weg recht schnell ist, würde das Problem bei einem realen Straßennetz zu viel Zeit in Anspruch nehmen, zumindest wenn A und B relativ weit voneinander entfernt sind. Da der Benutzer eine schnelle Berechnung des kürzesten Weges erwartet, bedient sich eine in Navigationstools eingebettete Software weiterer Verfeinerungen wie einer Vereinfachung des Straßennetzes, um die Antwort auf eine Anfrage fast in Echtzeit zu liefern. Auch bei den Algorithmen für den kürzesten Weg werden häufig Anpassungen vorgenommen, insbesondere basieren diese Algorithmen häufig auf A*-Heuristiken, die Informationen für ein vielversprechendes Durchsuchen des Netzes nutzen. Andere heuristische Ansätze, die zur Beschleunigung eingesetzt werden, sind zum Beispiel Branch Pruning (das Nichtverfolgen bestimmter möglicher Verbindungen) und die Zerlegung des Suchproblems (Fu et al., 2006).

3.3 Das Travelling-Salesman-Problem

Das Travelling-Salesman-Problem ist ein sehr bekanntes Problem auf dem Gebiet der Transportplanung. Aber auch viele Probleme aus anderen Bereichen lassen sich als Travelling-Salesman-Problem formulieren. Darüber hinaus ist das Problem

recht einfach zu beschreiben, aber dennoch rechenintensiv (NP-schwer), so dass es zu einem der beliebtesten Probleme für Forscher geworden ist, die versuchen, gute Heuristiken oder Approximationen für schwierige Probleme zu finden. Für das TSP in seiner allgemeinen Formulierung wird jedoch angenommen, dass es nicht einmal effiziente Approximationsalgorithmen mit bekannter Qualität gibt (Lenstra & Kan, 1981).

Das Problem kann wie folgt beschrieben werden: Während einer einzigen Fahrt sollen mehrere Orte besucht werden, und es soll eine kürzeste Folge dieser Orte bestimmt werden. Es wird angenommen, dass die Fahrt an einem bestimmten Ort, dem Depot, beginnt und endet und dass kein Ort zweimal besucht werden darf. Die Entfernungen zwischen den verschiedenen Orten, einschließlich des Startpunkts, werden als gegeben angenommen. Sehr oft werden diese Entfernungen in Form einer (quadratischen) Matrix ausgedrückt, deren Zeilen und Spalten den betrachteten Orten entsprechen. Eine andere Möglichkeit besteht darin, diese Entfernungen zu berechnen, z. B. mit einer Metrik wie der euklidischen Metrik und unter Verwendung vorgegebener Koordinaten der Orte.

In jedem Fall wird angenommen, dass eine Menge von Orten $L = \{l_0, l_1, \ldots, l_n\}$ zusammen mit den Entfernungen (oder den zugehörigen Kosten) gegeben ist, d. h. eine reellwertige Entfernungsfunktion $d{:}L{\times}L \rightarrow \mathrm{R}$. Außerdem wird gewöhnlich angenommen, dass die Abstände nicht negativ sind, d. h. $d(a,b) \geq 0$ für alle $a, b{\in}L$, und dass die Dreiecksungleichung gilt, d. h. $d(a,b) + d(b,c) \geq d(a,c)$ für alle $a, b, c{\in}L$. In dieser Schreibweise wird l_0 üblicherweise verwendet, um das Depot, d. h. den Start- und Endort der Tour, anzugeben. Da Start- und Endort feststehen, reicht es in der Regel aus, eine Tour nur durch die Abfolge der dazwischen besuchten Orte zu spezifizieren, die durch eine Permutation π der Indexzahlen $\{1, \ldots, n\}$ ausgedrückt werden kann, die die Folge $(\pi(1), \ldots, \pi(n))$ darstellen.

Beispiel:

Ein Lieferant möchte vom Lager (Standort A) aus Waren an drei Kunden (Standorte B, C und D) liefern. Die gesamten Sendungen passen in das verwendete Fahrzeug. Die Entfernungen (in Minuten) werden durch die folgende Matrix der Fahrzeiten angegeben (Tab. 3.1):

Ziel ist es nun, die Reihenfolge zu bestimmen, in der die Kunden bedient werden sollen, um die Gesamtfahrzeit zu minimieren.

Bevor wir uns mit der Lösung des Problems befassen, sei darauf hingewiesen, dass die obige Matrix symmetrisch ist, d. h. ein Matrixkoeffizient in Zeile i und Spalte j ist immer gleich dem Koeffizienten in Zeile j und Spalte i. Das heißt, der Weg von einem Ort zu einem anderen Ort dauert immer gleich lang wie der Weg in

Tab. 3.1 Beispiel einer Distanzmatrix

	A	B	C	D
A		12	35	25
B	12		25	10
C	35	25		20
D	25	10	20	

die entgegengesetzte Richtung. In der Realität ist dies natürlich oft nicht der Fall. Beispielsweise können eine unterschiedliche Verkehrsbelastung je nach Fahrtrichtung, Probleme durch Baustellen, die nur eine Richtung betreffen, oder Einbahnstraßen zu unterschiedlichen Zeiten je nach gewählter Richtung zwischen zwei Punkten führen. Natürlich kann auch eine asymmetrische Formulierung des TSP in Betracht gezogen werden.

Betrachtet man die Lösung des TSP, so stellt man fest, dass es grundsätzlich $n!$ Sequenzen für den Besuch von n Kunden gibt, die alle zulässig sind, sich aber in der Regel hinsichtlich der Zielfunktion unterscheiden. Wenn Symmetrie angenommen wird (wie in der obigen Matrix), reduziert sich die Anzahl der wirklich unterschiedlichen Lösungen auf $n!/2$. (Eine Tour nach einer beliebigen Sequenz ist genauso lang wie die Reise in der umgekehrten Reihenfolge).

In unserem Beispiel gibt es sechs mögliche Lösungen, die wie folgt lauten:

1. ABCDA: 78′ (=12 + 25 + 20 + 21)
2. ABDCA: 77′ (=12 + 10 + 20 + 35)
3. ACBDA: 95′ (=35 + 25 + 10 + 25)
4. ACDBA: 77′ (entspricht der Lösung 2 aufgrund der Symmetrie)
5. ADBCA: 95′ (entspricht der Lösung 3)
6. ADCBA: 78′ (entspricht der Lösung 1)

Offensichtlich sind die Lösungen 2 und 4 die besten.

Der Abstand zwischen zwei Orten, die in der Matrix angegeben sind, kann eine grobe Annäherung sein, kann aber auch auf der Lösung eines Problems des kürzesten Weges a priori beruhen, d. h. es können zunächst die kürzesten Wege zwischen allen in Betracht kommenden Orten berechnet werden.

Wie der Name Travelling Salesman andeutet, könnte ein typisches Szenario für ein solches Problem die Planung einer Route für einen Verkäufer sein, der während einer Tour mehrere Kunden besuchen möchte. Es wird davon ausgegangen, dass die Kunden in beliebiger Reihenfolge besucht werden können, so dass es sinnvoll ist, eine Reihenfolge mit der geringsten Fahrzeit, den geringsten Fahrtkosten oder der kürzesten Gesamtstrecke zu bestimmen.

Ein anderes Szenario könnte die Lieferung von Gütern von einem Ort (z. B. einer Produktionsstätte) an mehrere Kunden (z. B. Geschäfte) oder die Abholung von Gütern (z. B. Briefe oder Pakete) an mehreren Orten sein. Da das Problem nicht von einer Kapazitätsgrenze ausgeht, ist die Verwendung einer TSP-Formulierung jedoch nur dann geeignet, wenn die Kapazität keine Rolle spielt. Es wird also angenommen, dass alles, was transportiert werden muss, immer in das Fahrzeug passt.

Weitere Restriktionen, die das TSP erschweren können, sind z. B. einzuhaltende Zeitfenster für Kundenbesuche oder die zusätzliche Berücksichtigung von Abfertigungszeiten an Halteorten (z. B. zum Entladen von Sendungen aus dem Fahrzeug).

Betrachten wir das obige Beispiel unter der Annahme, dass für jede Entladung einer Sendung 10 Minuten benötigt werden. Die Optimalität der obigen Lösungen ist davon jedoch nicht betroffen. Jede Tour dauert lediglich 30 Minuten länger, da drei Stopps einschließlich der Entladezeit zu planen sind. Wenn wir jedoch von Zeitfenstern ausgehen, kann sich die Situation ändern: Nehmen wir an, dass eine

Tour um 8:00 Uhr im Lager beginnt und dass D die Ware zwischen 8:15 und 8:45 Uhr benötigt, während C die Sendung nicht vor 9:00 Uhr annehmen kann.

Unsere beiden oben genannten optimalen Lösungen würden wie folgt geplant werden:

1. Optimale Lösung:

$$\begin{array}{lllll} A_{\text{abfahrt}}\,8:00 & B_{\text{ankunft}}\,8:12 & B_{\text{abfahrt}}\,8:22 & D_{\text{ankunft}}\,8:32 & D_{\text{abfahrt}}\,8:42 \\ C_{\text{ankunft}}\,9:02 & C_{\text{abfahrt}}\,9:12 & A_{\text{ankunft}}\,9:47 & & \end{array}$$

2. Optimale Lösung:

$$\begin{array}{lllll} A_{\text{abfahrt}}\,8:00 & C_{\text{anreise}}\,8:35 & C_{\text{abfahrt}}\,8:45 & D_{\text{anreise}}\,9:05 & D_{\text{abfahrt}}\,9:15 \\ B_{\text{anreise}}\,9:25 & B_{\text{abfahrt}}\,9:35 & A_{\text{anreise}}\,9:47 & & \end{array}$$

Während die erste optimale Lösung (ohne Änderungen) realisiert werden kann, erfüllt die zweite, bisher optimale Lösung nicht die Zeitfenster-Bedingungen.

Am Ende dieses Abschnitts sei darauf hingewiesen, dass es verschiedene Varianten des Standard-TSPs gibt. So könnte man beispielsweise annehmen, dass die Standorte von einem von mehreren fahrenden Verkäufern oder Fahrzeugen angefahren werden und dass diese von verschiedenen Depots aus operieren (multiple depot, multiple travelling salesmen problem; siehe z. B. Yadlapalli et al., 2009). Ferner gibt es TSP-Varianten, die Abholungen und Zustellungen von Gütern berücksichtigen (siehe z. B. Gendreau et al., 1999) oder die nicht den Besuch aller Orte erfordern (Feillet et al., 2005). Travelling-Salesman-Probleme können mit verschiedenen oder auch mehreren Zielen formuliert werden (siehe z. B. Shim et al., 2011) und einige der Informationen aus dem TSP können als stochastisch oder unscharf modelliert werden. So wird beispielsweise in Crisan und Nechita (2008) eine unscharfe Distanzmatrix verwendet. Zum Teil dienen diese Erweiterungen und Varianten des TSP den Anforderungen aus praktischen Problemen. Wir werden einige von ihnen in Abschn. 3.5 genauer betrachten, wo wir Vehicle-Routing-Probleme behandeln, die in vielen Fällen näher an praktischen Anwendungen sind als das TSP.

3.4 Methoden zur Lösung des Travelling-Salesman-Problems

Wie bereits erwähnt, gehört das TSP zur Klasse der NP-schweren Probleme. Das bedeutet, dass kein Algorithmus bekannt ist, der das Problem in einer polynomiell von der Problemgröße abhängigen Zeit optimal lösen kann (und dass es einen solchen Algorithmus wahrscheinlich auch nicht gibt). Das bedeutet, dass es bei nicht zu kleinen Problemen nicht möglich ist, das Problem in einer begrenzten Zeit bis zur Optimalität zu lösen, und selbst Approximationen mit definierter Qualität bleiben schwierig.

Aus diesem Grund spielen Heuristiken und Metaheuristiken eine wichtige Rolle bei der Behandlung des Problems, und in den letzten Jahrzehnten wurden viele entsprechende Algorithmen für das TSP entwickelt und erforscht. Obwohl solche Algorithmen keine Optimal-Lösung garantieren (zumindest nicht mit subexponentieller Laufzeit), ist die erhaltene Lösung in der Regel gut genug für praktische Zwecke. Selbst für größere Probleme konnte in vielen Fällen gezeigt werden, dass optimale oder nahezu optimale Lösungen erzielt werden können. Sehr oft wurden die Algorithmen mit Hilfe einer weithin bekannten Bibliothek von Testproblemen namens TSPLIB-A Travelling Salesman Problem Library (vgl. Reinelt, 1991) getestet, was den Vergleich der verschiedenen Algorithmen erleichtert.

Wie bei vielen anderen schwierigen Optimierungsproblemen gibt es drei grundlegende Klassen von Methoden, die zur Lösung des TSP verwendet werden: exakte (mathematische) Methoden, Metaheuristiken und andere CI-Ansätze sowie einfache Heuristiken. Zu den exakten Verfahren gehören Branch and Bound, Branch and Cut und ähnliche Verfahren. Einfache Heuristiken basieren zum Beispiel auf Routenverbesserungsoperationen, die auch in komplexere Metaheuristiken integriert werden können. Bei den Metaheuristiken wurde für das TSP im Grunde das gesamte Spektrum der zu dieser Klasse gehörenden Methoden untersucht.

3.4.1 Heuristiken für das Travelling-Salesman-Problem

Bevor wir uns mit weiteren Einzelheiten solcher Methoden befassen, wollen wir einige Grundsätze einfacher Heuristiken erörtern. Zunächst einmal gibt es Konstruktionsheuristiken wie den Ansatz des „nächsten Nachbarn“: Ausgehend vom Depot wird zunächst der nächstgelegene Kundenstandort aufgesucht. Von dort aus wird der nächstgelegene der verbleibenden (nicht besuchten) Standorte besucht und so weiter, bis alle Standorte für die Tour geplant sind.

Zweitens gibt es Verbesserungsheuristiken. Die einfachste von ihnen ist die Austauschheuristik, die als 2-opt bezeichnet wird. Nehmen wir an, dass es zwei Positionen in einer Tour gibt, $p, q \in \{1, \ldots, n\}$, mit $p < q$, und dass die folgende Bedingung für ihre Entfernungen oder Reisezeiten erfüllt ist: $d(l_{p-1}, l_p) + d(l_q, l_{q+1}) > d(l_{p-1}, l_q) + d(l_p, l_{q+1})$, dann reduziert das Umordnen der Tour durch Umdrehen der Teilfolge $(l_p, \ldots, l_q)$ zur neuen Tour $(l_0, \ldots, l_{p-1}, l_q, l_{q-1}, \ldots, l_{p+1}, l_p, l_{q+1}, \ldots, l_{n-1})$ reduziert die gesamte Reisezeit (bzw. Kosten). Die jeweilige Heuristik sucht wiederholt nach solchen Situationen in der Reihenfolge der Orte und modifiziert die Reihenfolge entsprechend, bis ein lokales Optimum erreicht ist. In ähnlicher Weise kann eine 3-Opt-Heuristik mit dem Austausch von drei Kanten definiert werden.

Die Lin-Kernighan-Heuristik versucht, eine Folge solcher einzelner Umkehroperationen zu bilden, um die Tour zu verbessern. Der Hauptunterschied besteht darin, dass nicht für jede einzelne Operation eine Verbesserung der Tour erforderlich ist. Erst nach Durchführung der gesamten Sequenz sollte die resultierende Tour besser sein als die ursprüngliche Reihenfolge. Die Erstellung solcher Sequenzen

kann z. B. mit einem Backtracking-Ansatz erfolgen (Applegate et al., 2003). Natürlich ist es sinnvoll, die Backtracking-Tiefe oder die maximale Länge der Sequenzen zu begrenzen, um eine nahezu vollständige Erkundung des Suchraums zu vermeiden. Wie Applegate et al. (2003) gezeigt haben, führt die Verwendung von Sequenzen mit bis zu 25 einzelnen Flip-Operationen auch bei großen TSP-Instanzen zu sehr guten Ergebnissen. Zusammen mit einigen weiteren Verbesserungen wird der Ansatz üblicherweise als verkettete Lin-Kernighan-Heuristik bezeichnet.

3.4.2 Evolutionäre Algorithmen für das Travelling-Salesman-Problem

Sehr oft verwenden Metaheuristiken, die auf das TSP (und andere Probleme der Tourenbildung) angewandt werden, mehrere solcher einfachen Heuristiken. Diskutieren wir dies am Beispiel der evolutionären Algorithmen, die in Studien zum TSP eine wichtige Rolle spielten und in vielen Varianten verwendet wurden, wobei die frühesten Veröffentlichungen auf die 1980er-Jahre zurückgehen (siehe z. B. Grefenstette et al., 1985; Braun, 1991; Larrañaga et al., 1999; Tsai et al., 2004; Snyder & Daskin, 2006).

Einschlägige Gestaltungsprinzipien evolutionärer Algorithmen (und anderer Metaheuristiken) sind insbesondere die auf Zufall basierenden Variationsoperatoren Mutation und Crossover. Wenn diese Operatoren nicht auf einer geeigneten Kodierung von Lösungen beruhen und/oder eine problemspezifische Anpassung ihrer Standardformen aufweisen, führen sie in der Regel zu unzulässigen Lösungen (siehe Kap. 5 über Scheduling für weitere Beispiele), was den Rechenaufwand erhöht und die Effizienz der evolutionären Methode verringert. Daher ist es sinnvoll, solche Operatoren zu konstruieren, die automatisch zulässige Lösungen erzeugen, d. h. Touren, bei denen jeder Knoten genau einmal in der Tour vorkommt.

Eine Möglichkeit, solche Anforderungen beim Entwurf geeigneter Variationsoperatoren zu vermeiden, die die Zulässigkeit aufrechterhalten (aber die entsprechenden Algorithmen komplexer machen und zusätzliche Rechenzeit erfordern), ist die Verwendung einer geeigneteren Problemrepräsentation, z. B. das Random Key Encoding (Snyder & Daskin, 2006, vgl. auch Kap. 5 über Scheduling). Hier wird jeder Ort mit einer reellen Zahl (Gleitkommazahl) assoziiert, die seine Position in einer entsprechenden Reihenfolge bestimmt. Das bedeutet, dass zur Bestimmung der einer Lösung entsprechenden Reihenfolge die Orte nach den ihnen zugeordneten Zahlen sortiert werden müssen. Variationen von Sequenzen ergeben sich aus Variationen der zugeordneten Zahlen und es können die üblichen Mutations- und Crossover-Operatoren angewendet werden. Zum Beispiel kann die reelle Zahl normalverteilten Mutationen unterworfen werden (z. B. wie im Konzept der Evolutionsstrategien), d. h. normalverteilter Zufallsvariablen werden zu den jeweiligen realen Werten addiert, die die spätere Position der Orte in einer Reihenfolge bezeichnen. Crossovers finden nur für das Array der zugeordneten Zahlen statt,

z. B. in Form von Ein-Punkt-Crossover oder Zwei-Punkt-Crossover. In jedem Fall ist das Ergebnis einer solchen Variation wieder eine Reihenfolge ohne doppelte oder fehlende Orte. Sollen weitere Nebenbedingungen berücksichtigt werden (z. B. Kapazitätsbeschränkungen), muss dies natürlich zusätzlich beachtet werden.

Als Nachteile dieser Repräsentation werden gelegentlich der zusätzliche Aufwand für das Sortieren einer Sequenz (vor allem wenn sie lang ist) und mögliche starke Auswirkungen der Variationsoperatoren genannt (Chatterjee et al., 1996). Zum zweiten Nachteil ist anzumerken, dass dieser Effekt bei Verwendung von $N(0,\sigma)$-verteilten Mutationen wirksam kontrolliert werden kann und insbesondere kleine σ-Werte nur zu kleineren Veränderungen in einer Reihenfolge führen sollten.

In den meisten Studien zum Einsatz evolutionärer Algorithmen für das TSP (und ähnliche Probleme) wird jedoch eine „direkte“ Darstellung der Lösungen in Form von Reihenfolgen gewählt und die evolutionären Operatoren werden in geeigneter Weise angepasst:

Bei einfachen Variationsoperatoren wie Mutationen in evolutionären Algorithmen muss darauf geachtet werden, dass eine möglicherweise zufällige Änderung nicht zu einer unerwünschten doppelten Berücksichtigung desselben Ortes oder zum Vergessen eines Ortes führt. Wenn eine Problemlösung als eine Folge von Orten (bzw. deren Indizes) dargestellt wird, müssen bei einer Mutation mindestens zwei Positionen verändert werden, z. B. ein Austausch von zwei zufällig ausgewählten Knoten in einer Tour (Austauschmutation).

Weitere häufig verwendete Möglichkeiten von Mutationsoperatoren sind Inversionen, Ersetzungsmutationen, Einfügemutationen und Mutationen, bei denen Teile der Reihenfolge durcheinander gewürfelt werden (Scramble-Mutationen) (siehe z. B. Larrañaga et al., 1999; Ahmed, 2010). Bei der Inversionsmutation (manchmal auch als einfache Inversionsmutation bezeichnet) wird eine Tour zwischen zwei zufällig ausgewählten Positionen umgedreht, z. B. wenn für eine Sequenz (ohne Depot)

$$(2,\ 6,\ 5,\ 4,\ 3,\ 1)$$

die Positionen 3 und 5 bestimmt werden, ergibt sich folgende Reihenfolge:

$$(2,\ 6,\ 3,\ 4,\ 5,\ 1).$$

Der Einfügemutationsoperator wählt eine zufällige Position in einer betrachteten Sequenz aus, entfernt sie aus der Sequenz und fügt sie an einer neuen, zufällig bestimmten Position wieder ein. Diese Idee wird durch den Ersetzungsmutationsoperator verallgemeinert, der eine zufällige Teilsequenz aus einer Lösung auswählt und sie an eine neue, zufällig bestimmte Position in der Tour verschiebt. Eine andere Art der Mutation (manchmal auch verwirrend als Inversionsmutation bezeichnet) kombiniert die Verdrängungs- und die einfache Inversionsmutation, d. h. eine zufällig ausgewählte Teilsequenz wird entfernt und an einer neuen zufälligen Position wieder eingefügt, allerdings in umgekehrter Reihenfolge. Schließlich sei noch der

Scramble-Mutationsoperator erwähnt, der eine zufällige Teilsequenz bestimmt und diese Teilsequenz zufällig neu anordnet.

Was das Crossover betrifft, so können die Standardoperatoren nicht für reihenfolgebasierte Repräsentationen verwendet werden: Die Teilung von zwei Sequenzen an der gleichen Stelle (Ein-Punkt-Crossover) und die Verbindung der verschiedenen Teile der beiden Lösungen (Ein-Punkt-Crossover) führt in der Regel zu einer unzulässigen Lösung aufgrund von doppelten oder fehlenden Orten in den resultierenden Reihenfolgen. Daher werden spezifische Crossover-Operatoren benötigt, die die Zulässigkeit sicherstellen. Eine Übersicht über verschiedene geeignete Rekombinations- und Crossover-Operatoren findet sich in Larrañaga et al. (1999): Ein häufig verwendeter Typ ist das Partial Mapped Crossover (PMX). Bei einem entsprechenden Crossover werden zwei Schnittpunkte für die beiden zu rekombinierenden Elternlösungen zufällig ausgewählt. In den Nachkommen werden die jeweiligen Reihenfolgen zwischen den Schnittpunkten für die beiden Nachkommenlösungen ausgetauscht. Für die anderen Positionen in einer Tour werden die ursprünglichen Positionen der jeweiligen Eltern verwendet, es sei denn, diese Positionen befinden sich bereits in der ausgetauschten Sequenz. In diesem Fall werden die jeweiligen Werte durch die Originalwerte der Positionen in der Tauschfolge ersetzt. Betrachten wir ein Beispiel:

Wenn für die Sequenzen (2, 6, 5, 4, 3, 1) und (1, 2, 6, 4, 3, 5) die Schnittpunkte zwischen der ersten und der zweiten und zwischen der dritten und der vierten Position gewählt werden, werden für die Nachkommenlösung zunächst die Zwischenbereiche mit den Positionen zwei und drei aus dem „gegenüberliegenden“ Elternteil gesetzt, d. h. (*, 2, 6, *, *, *) und (*, 6, 5, *, *, *). Dann werden für die übrigen Positionen des ersten Nachkommens die entsprechenden Werte vom ersten Elternteil kopiert. An der Position eins stellen wir fest, dass „2“ bereits in der Austauschsequenz vorkommt (an Position 2), und daher sollte der ursprüngliche Wert von dieser Position verwendet werden, d. h. „6“. Da dieser Wert auch in der ausgetauschten Folge an Position 3 vorkommt, verwenden wir den ursprünglichen Wert dieser Position, also „5“. Die Werte an den Positionen 4–6 können verwendet werden, da sie in der ausgetauschten Sequenz nicht vorkommen. Für den zweiten Nachkommen können alle Originalpositionen außer Position 6 verwendet werden. Hier führen ähnliche Überlegungen wie zuvor zu einer Platzierung von „2“ an dieser Position. Das heißt, die Nachkommenlösungen ergeben sich als (5, 2, 6, 4, 3, 1) und (1, 6, 5, 4, 3, 2).

Der Operator Order Crossover (OX1) funktioniert zunächst ähnlich: Für zwei Elternlösungen werden zwei Schnittpunkte zufällig bestimmt. Die Schnittpunkte für die beiden Elternlösungen können unterschiedlich sein, sollten aber eine gleich lange Sequenz enthalten. Dann werden die Sequenzen zwischen den Schnittpunkten gegen die jeweiligen Nachkommenlösungen ausgetauscht. Danach werden, beginnend nach dem zweiten Schnittpunkt, die verbleibenden Positionen schrittweise mit den nächsten Stellen in den ursprünglichen Sequenzen aufgefüllt, wobei Stellen, die bereits in den ausgetauschten Sequenzen vorkommen, ausgelassen werden.

Wenn zum Beispiel für die Sequenzen (2, 6, 5, 4, 3, 1) und (1, 2, 6, 4, 3, 5) die Schnittpunkte nach den Positionen eins und drei (erstes Elternteil) und nach den

Positionen drei und fünf (zweites Elternteil) ausgewählt werden, dann werden für die Nachkommenlösungen zunächst diese Positionen gesetzt, d. h. (*, 4, 3, *, *, *) und (*, *, *, 6, 5, *). Beginnend mit dem ersten Nachkommen an Position 4 können die ursprünglichen Werte an dieser und der nächsten Position, d. h. „4“ und „3“, nicht verwendet werden, da sie im ausgetauschten Teil vorkommen. Daher wird „1“ gewählt, dann „2“, „6“ und „5“, und die resultierende Folge ist (5, 4, 3, 1, 2, 6). Zu beachten ist, dass am Ende der Sequenz der Prozess am Anfang der Sequenz fortgesetzt wird, bis die erste Schnittposition erreicht ist und alle Positionen gesetzt sind. Für den zweiten Nachkommen sieht das Ergebnis wie folgt aus: (2, 4, 3, 6, 5, 1).

Eine ähnliche Idee der Ordnungserhaltung wird beim ordnungsbasierten Crossover (OX2) verwendet. Hier werden einige Positionen in einer Elternlösung zufällig bestimmt. Für die Nachkommen der anderen Elternlösung werden alle Positionen kopiert, mit Ausnahme der Positionen, die den von der anderen Elternlösung bestimmten Positionen entsprechen. Diese Positionen werden nun durch diese Positionen ersetzt, jedoch in der Reihenfolge, in der sie vom anderen Elternteil stammen.

Nehmen wir an, dass für unser Beispiel mit den Reihenfolgen (2, 6, 5, 4, 3, 1) und (1, 2, 6, 4, 3, 5) die Positionen 1, 3 und 5 vom zweiten Elternteil bestimmt werden. Diese entsprechen den Positionen „1“, „6“ und „3“. Für die Nachkommen des ersten Elternteils werden also zunächst nur die verbleibenden Orte festgelegt, d. h. (2, *, 5, 4, *, *). Die Positionen „1“, „6“ und „3“ werden in der Reihenfolge des zweiten Elternteils gesetzt und wir erhalten die Sequenz (2, 1, 5, 4, 6, 3). Wenn man die gleichen zufälligen Positionen für den ersten Elternteil betrachtet, führt die Lösung für den zweiten Nachkommen zu folgendem Ergebnis: (1, 2, 6, 4, 5, 3).

Beim positionsbasierten Crossover (POS) wird ebenfalls zunächst ein zufälliger Satz von Positionen aus den Elternlösungen bestimmt. Für den ersten Nachkommen werden die Positionen an diesen Stellen vom zweiten Elternteil kopiert, während – beginnend am Anfang der Reihenfolge – die restlichen Positionen schrittweise mit Positionen entsprechend der Reihenfolge des ersten Elternteils aufgefüllt werden, wobei darauf geachtet wird, dass keine Position zweimal gewählt wird.

Wenn in unseren Beispielsequenzen (2, 6, 5, 4, 3, 1) und (1, 2, 6, 4, 3, 5) wiederum die Positionen 1, 3 und 5 zufällig bestimmt werden, erhalten wir für den ersten Nachkommen zunächst die Sequenz (1, *, 6, *, 3, *), die zu (1, 2, 6, 5, 3, 4) aufgefüllt wird. Für den zweite Nachkommen erhält man zunächst (2, *, 5, *, 3, *) und dann (2, 1, 5, 6, 3, 4).

Zu den verschiedenen anderen Crossover-Operatoren, die in der Literatur zu finden sind (siehe z. B., Larrañaga et al., 1999) zählt beispielsweise der Zyklus-Crossover-Operator (CX), heuristische Crossover-Operatoren (die Wahrscheinlichkeitsverteilungen auf der Grundlage der Kantenkosten verwenden), der genetische Kanten-Rekombinations-Crossover (ER), der Kanten-Rekombinations-Crossover (ERX), der sortierte Match-Crossover-Operator, das maximal erhaltende Crossover (MPX), das Voting-Rekombinations-Crossover (VR), das alternierende Positions-Crossover (AP) und das verallgemeinerte N-Punkt-Crossover (GNX).

Entsprechende Mutations- und Crossover-Operatoren können wesentlich komplexer sein. So wird beispielsweise in Albayrak und Allahverdi (2011) ein „gieriger“ Subtour-Mutationsoperator vorgestellt, der auf sechs Parametern basiert, die

ihn steuern. Hier sehen wir eine Idee, die häufig in Metaheuristiken verwendet wird: nicht nur zufällige Änderungen durchzuführen, sondern zu versuchen, möglichst gute lokale Verbesserungen zu finden. Diese Idee wird auch in einem anderen Vorschlag für ein Crossover in Ahmed (2010) verwendet, das als sequentielles konstruktives Crossover (SCX) bezeichnet wird. Hier werden die Nachkommenchromosomen aus den Elternchromosomen konstruiert, indem versucht wird, die ursprüngliche Reihenfolge beizubehalten, aber nach den besten Knoten zu suchen, die an eine Tour angehängt werden können. Zu beachten ist, dass diese Art von Crossover keine Zufallsmechanismen verwendet. Andere, komplexere Operatoren könnten auf einer lokalen Suche beruhen, wie sie zum Beispiel im Konzept der memetischen Algorithmen betrachtet wird. Solche eingebetteten lokalen Suchansätze könnten zum Beispiel einen 2-Opt-Ansatz oder eine Lin-Kernighan-Heuristik verwenden.

3.4.3 Andere Metaheuristiken und neuronale Netze für das Travelling-Salesman-Problem

Einige andere Metaheuristiken wie Ant Colony Optimization (ACO) können auf recht einfache Weise auf ein TSP angewendet werden: In ACO werden alle möglichen Verbindungen in einer Tour (d. h. alle Kanten in einem entsprechenden Graphen) mit Werten versehen, die die Pheromonniveaus repräsentieren, welche die Präferenz für die Verwendung einer entsprechenden Kante angeben. Während des Durchlaufs des Algorithmus ändern sich diese Pheromonwerte, indem sie verstärkt werden, wenn die jeweilige Kante von einer Ameise genutzt wird, und andernfalls aufgrund von Pheromonverdunstung verringert werden. Eine tatsächliche Tour (die einer Ameise in der Ameisenkolonie entspricht) wird dann schrittweise mit Hilfe einer probabilistischen Funktion erstellt, die die Präferenz für die Nutzung der jeweiligen Kanten zwischen den Orten während der Tour bestimmt. Die probabilistische Funktion begünstigt hohe Pheromonwerte an Kanten und kurze Entfernungen zwischen Orten, lässt aber aufgrund ihrer stochastischen Natur auch gelegentlich weniger günstige Verbindungen zu.

Andere Metaheuristiken erfordern stärkere Anpassungen oder verwenden Modifikationen, die denen der evolutionären Ansätze ähneln. So sind beispielsweise bei der üblichen Particle Swarm Optimization (PSO) auf der Grundlage von reellen Zahlen Anpassungen erforderlich, um die kombinatorische Natur des TSP zu berücksichtigen. So wird beispielsweise in Shi et al. (2007) eine Darstellung der Lösungen als Permutationen verwendet, was weitere Anpassungen der PSO-Operatoren erfordert, z. B. die Berücksichtigung von Lösungsunterschieden als Anzahl von Transpositionen, und die Bewegung von Lösungen wird durch einen Verschiebeoperator spezifiziert. Außerdem werden in diesem Ansatz spezielle Crossover-Operatoren verwendet. In einem anderen PSO-basierten Ansatz von Zhong et al. (2007) werden für die Lösungsrepräsentation $n \times n$-*Matrizen* verwendet, die allen Kanten zwischen Orten entsprechen, genauer gesagt: um Wahrscheinlichkeiten für

die Verwendung bestimmter Verbindungen darzustellen. Diese reellwertigen Lösungsrepräsentationen können für eine recht unkomplizierte Anwendung der Standard-PSO-Konzepte verwendet werden.

Die Anpassungsanforderungen für einige andere Metaheuristiken können begrenzt sein. So müssen Pepper et al. (2002) in ihrer Studie zur Verwendung von Simulated Annealing und ähnlichen Metaheuristiken für das TSP nur den Schritt überdenken, wie neue Lösungen generiert werden. Anstatt völlig zufällige Nachbarn der aktuellen Lösungen zu verwenden, wenden sie ein Verfahren mit zwei Austauschvorgängen an, beschränken aber die möglichen Austauschvorgänge auf die im Voraus berechneten nächsten Nachbarn der einzelnen Knoten.

Neben evolutionären Algorithmen wurden auch verschiedene andere Metaheuristiken häufig auf das TSP und seine Varianten angewandt:

- Ant Colony Optimization (Dorigo & Gambardella, 1997a, b)
- GRASP (Marinakis et al., 2005)
- Particle Swarm Optimization (Li et al., 2006)
- Simulated Annealing, Threshold Accepting und ähnliche Algorithmen (Pepper et al., 2002)
- variable Nachbarschaftssuche (Mladenović et al., 2012)
- memetische Algorithmen (Buriol et al., 2004; Ulder et al., 1991)

Auch weniger bekannte oder neuere Vorschläge zu Metaheuristiken wurden verwendet, z. B.:

- Algorithmen mit künstlichen Bienenvölkern (Karaboga & Gorkemli, 2011)
- Cuckoo Search (Ouaarab et al., 2014)
- evolutionäre Ameisenregeln (Tsai & Tsai, 2002)
- der Firefly-Algorithmus (Jati & Suyanto, 2011)
- Jump Search (Tsubakitani & Evans, 1998),
- neuro-immune Netzwerke (Pasti und & Castro, 2006)
- die Noising-Methode (Charon & Hudry, 2000)

Auch neuronale Netze wurden wiederholt zur Lösung von TSP-Instanzen verwendet (siehe Übersichten in Potvin, 1993 und Steinhaus, 2015). Insbesondere Kohonen-Netze [auch als selbstorganisierende Karten (self-organizing map, SOM) oder selbstorganisierende Merkmalskarten (self-organizing feature map, SOFM) bezeichnet] sind populär geworden, weil ihre Datenstrukturen geografische Orte und deren Entfernungen darstellen können. In Kohonen-Netzen sind alle Neuronen miteinander verbunden und mit euklidischen Koordinaten verknüpft, die während des Lernprozesses des neuronalen Netzes angepasst werden. Für die Repräsentation eines Tourenbildungsproblems mit n Standorten sollte ein entsprechendes Netz mit $m \subseteq\geq n$ Knoten vorhanden sein. Typischerweise ist die Anzahl der Neuronen m im Netz deutlich größer als n, z. B. $m = 2n$ (Créput & Koukam, 2009). In Kohonen-Netzen führt die Dynamik des Netzwerks schließlich zu einer Lösung, die eine Tour darstellt, indem einige der Neuronen auf der Grundlage der geometrischen Nähe mit

Zielen assoziiert werden. Neben SOM-Ansätzen wird das ähnliche Konzept der elastischen Netze gelegentlich auf entsprechende Tourenbildungsprobleme angewandt. Auch Hopfield-Netze (HNN), eine Art vollständig verbundener Feedforward-Netze, können für die Repräsentation eines Transportnetzes verwendet werden und helfen bei der Ableitung einer geeigneten Tour (Jolai & Ghanbari, 2010).

3.4.4 Über die Leistungsfähigkeit von Lösungsansätzen

Im Allgemeinen ist es trotz der relativ guten Basis an Testproblemen schwierig zu beurteilen, welche der betrachteten Methoden (und andere Ansätze) am besten zur Lösung des TSP geeignet sind. Dies liegt z. B. an der Schwierigkeit, Laufzeiten unter verschiedenen Bedingungen oder im Hinblick auf bestimmte Verfahrensdetails und Parametereinstellungen zu vergleichen. In vielen Studien gibt es Vergleiche oder Benchmark-Ergebnisse mit anderen Verfahren, aber die rechnerischen Untersuchungen sind nicht ausreichend klar hinsichtlich experimenteller Details. Darüber hinaus kann in der Regel eine Verzerrung aufgrund intensiver Bemühungen zur Anpassung (von Parametern, Operatoren usw.) der von einem Autor neu vorgeschlagenen Methodik angenommen werden, während eine bestehende Methode ohne eine solche Anpassung oder mit einigen willkürlich gewählten Einstellungen verwendet werden kann, die für das betrachtete Problem möglicherweise nicht sehr geeignet sind.

Wir möchten jedoch einige vergleichende Ergebnisse anführen: In einem direkten Vergleich eines neuronalen Netzes, das dem Hopfield-Netz ähnelt, eines Ansatzes mit elastischen Netzen, eines genetischen Algorithmus, eines Simulated Annealing-Ansatzes und eines hybriden Ansatzes, der auf Simulated Annealing und vollständiger lokaler Suche beruht, schnitt der genetische Algorithmus in einer älteren Studie von Peterson (1990) am besten ab, gefolgt von dem hybriden Ansatz, dem elastischen Netz und dem anderen neuronalen Netz, während Simulated Annealing am schlechtesten abschnitt. In dem neueren Vergleich von Stützle et al. (2000) wurden folgende Methoden verglichen: ein genetischer Algorithmus mit abstandserhaltendem Crossover (DPX), ein reparaturbasierter genetischer Algorithmus, Ant Colony Optimization, iterierte lokale Suche mit fitnessdistanzbasierter Diversifizierung (ILS-FDD). Alle Algorithmen verwenden eine spezielle 3-Opt-Heuristik für die lokale Suche. Während alle Algorithmen sehr gute Ergebnisse erzielten, war ILS-FDD bei den meisten großen Testproblemen am besten. In der vergleichenden Studie von Weise et al. (2014) wurde festgestellt, dass „die lokale Suche reine globale Suchmethoden im TSP übertreffen kann, aber dass mit lokaler Suche hybridisierte [evolutionäre] Methoden sogar noch besser sein können." Eine populationsbasierte Ant Colony Optimization (PACO) und ein auf lokaler Suche basierender Ansatz namens Multi-Neighborhood Search (MNS) erwiesen sich in den jeweiligen Tests auf kleineren TSP-Instanzen als beste Ansätze.

3.5 Das Vehicle-Routing-Problem

Das Vehicle Routing Problem (VRP), auch als Capacitated Vehicle Routing Problem (CVRP) bezeichnet, ist aufgrund seiner realistischeren Annahmen eine der beliebtesten Formulierungen von Transportproblemen. Einige gute Übersichten über das VRP und Varianten finden sich in Anbuudayasankar et al. (2014), Golden et al. (2008), Liong et al. (2008) und Toth und Vigo (2014).

Der VRP geht davon aus, dass während einer Tour eine Reihe von Orten angefahren werden sollen (genau wie im Falle des TSP), aber es müssen Kapazitätsbeschränkungen der Fahrzeuge berücksichtigt werden. Ein typisches Szenario, in dem dies wichtig ist, ist die Auslieferung von Waren an Kunden oder das Abholen von Waren an verschiedenen Orten (z. B. die Abholung von Briefen oder Paketen), um sie zu einem gemeinsamen Ziel zu bringen.

In seiner Grundform kann das VRP für den Fall der Auslieferung von Gütern aus einem Depot wie folgt formuliert werden: Bei einer Kapazität q des Fahrzeugs und n Transportaufträgen zu den Standorten $l_1, l_2, \ldots, l_n$, mit Gütern, die jeweils eine Kapazität von a_i benötigen, $i = 1, \ldots, n$, sind eine oder mehrere Folgen von Standorten zu finden, die alle Transportaufträge bedienen und die Kapazität des Fahrzeugs einhalten.

Wie beim TSP beginnt und endet eine Route an einem bestimmten Ort, der in der Regel als Depot (der Fahrzeuge) bezeichnet wird. Das Ziel des Problems besteht in der Regel darin, mehrere möglichst kurze (oder kostengünstige) Routen (oder Touren) zu finden.

Mathematisch lässt sich das Problem als ganzzahliges Optimierungsproblem formulieren (siehe z. B. Kallehauge et al., 2005):

$$\min\ c(X) = \sum_{k \in V} \sum_{i \in N} \sum_{j \in N} c_{ij}\, x_{ijk} \tag{3.6}$$

so dass

$$\sum_{k \in V} \sum_{j \in N} x_{ijk} = 1, \text{ für alle } i \in \{1, \ldots, n\}, \tag{3.7}$$

$$\sum_{i \in N - \{0\}} a_i \sum_{j \in N} x_{ijk} \leq q, \text{ für alle } k \in V, \tag{3.8}$$

$$\sum_{j \in N} x_{0jk} = 1, \text{ für alle } k \in V, \tag{3.9}$$

$$\sum_{i \in N} x_{ihk} - \sum_{j \in N} x_{hjk} = 0, \text{ für alle } h \in \{1, \ldots, n\}, k \in V, \tag{3.10}$$

$$\sum_{i \in N} x_{i,n+1,k} = 1, \text{ für alle } k \in V, \tag{3.11}$$

$$x_{ijk} \in \{0,1\} \text{ für alle } i, j \in N, k \in V, \tag{3.12}$$

Die Entscheidungsvariablen sind x_{ijk}, die in (3.12) als binäre Variablen definiert sind. Diese Variablen geben an, ob das Fahrzeug k *den* Ort j nach dem Ort i besucht. In diesem Fall ist $x_{ijk} = 1$, andernfalls ist es 0. Es wird angenommen, dass eine Menge V von Fahrzeugen verfügbar ist und die Menge der Standorte N durch entsprechende Indexwerte ausgedrückt wird, d. h. $N = \{0, \ldots, n\}$, wobei 0 das Depot angibt.

In der Problemformulierung (3.6) wird das Ziel der Minimierung der gesamten Transportkosten angegeben. Gl. (3.7) stellt sicher, dass jeder Kunde genau einmal angefahren wird. Die Kapazitätsgrenze der Fahrzeuge wird durch die Nebenbedingung (3.8) ausgedrückt. Gl. (3.9) stellt sicher, dass jedes Fahrzeug das Depot verlässt, während (3.10) garantiert, dass jedes Fahrzeug einen Kunden verlässt, bei dem es zuvor in der Tour angekommen ist. Die Bedingung (3.11) drückt aus, dass die Fahrzeuge am Ende der Tour zum Depot zurückkehren müssen.

Es ist zu beachten, dass die Einhaltung der Kapazität unabhängig von der Reihenfolge der Knotenpunkte innerhalb einer Route ist. Wir brauchen nur zu prüfen, ob jede einzelne Tour Kunden bedient, deren aufsummierter Kapazitätsbedarf kleiner (oder gleich) ist als die verfügbare Kapazität. Wir sollten auch erwähnen, dass das Problem ohne Nebenbedingungen (3.8) eine mathematische Formulierung des TSP ist.

In der Regel wird das CVRP als Problem interpretiert, mehrere Rundfahrten für mehrere Fahrzeuge (jeweils für eines der Fahrzeuge) zu erstellen, die hinsichtlich der Kapazität als identisch angenommen werden.

Wir könnten das Problem auch so interpretieren, dass ein Fahrzeug (oder eine geringere Anzahl von Fahrzeugen als die erforderliche Anzahl von Touren) mehrere Rundfahrten durchführen muss. Dieses einzelne Fahrzeug müsste gegebenenfalls zu seinem Ausgangsort zurückkehren (wenn aufgrund der Kapazitätsgrenze nicht alle Güter in das Fahrzeug passen) und eine zweite Rundfahrt durchführen (Multitrip-VRP). Die Problemformulierung würde sich nicht ändern, lediglich die Touren müssten sequentiell und überschneidungsfrei durchgeführt werden, anstatt dass mehrere Fahrzeuge gleichzeitig fahren könnten. Die unterschiedliche Auslegung wäre nur relevant, wenn weitere Aspekte zu beachten sind (z. B. Zeitfenster, siehe unten).

3.5.1 Das Vehicle-Routing-Problem mit Zeitfenstern

Eine verbreitete Variante des Vehicle-Routing-Problems erfordert zusätzlich die Berücksichtigung von Zeitfenstern für die Anlieferung von Waren an den geplanten Zielorten der Transportaufträge. Das heißt, für jeden Kunden i wird gefordert, dass die Waren innerhalb eines vorgegebenen Zeitintervalls $[s_i^{\min}, s_i^{\max}]$ eintreffen. Sind Zeitfenster nur für einen Teil der Kunden erforderlich, so kann man für Kunden ohne Zeitfenster immer noch die allgemeine Formulierung mit der Zuweisung eines Zeitfensters $[-\infty,\infty]$ verwenden.

Die Zeitfenster können in dem mathematischen Optimierungsproblem (3.6)–(3.12) durch zwei zusätzliche Nebenbedingungen ausgedrückt werden:

$$x_{ijk}\left(s_{ik} + t_{ij} - s_{jk}\right) \leq 0 \text{ für alle } i, j \in N, k \in V, \tag{3.13}$$

$$s_i^{\min} \leq s_{ik} \leq s_i^{\max} \text{ für alle } i \in N, k \in V, \tag{3.14}$$

Die Bedingung (3.13) definiert die Ankunfts- (und Abfahrts-) Zeiten der Fahrzeuge an bestimmten Zielorten und stellt die Beziehungen zwischen den Abfahrtszeiten der Fahrzeuge von einem Kunden und seinem unmittelbaren Nachfolger her. Die Bedingung (3.14) stellt sicher, dass die Zeitfenster eingehalten werden. Man beachte, dass die nichtlineare Bedingung (3.13) linearisiert werden kann, um sie für Standardoptimierungsverfahren besser handhabbar zu machen (Kallehauge et al., 2005).

3.5.2 Das Vehicle-Routing-Problem mit mehreren Fahrzeugen

Wie bereits erwähnt, wird beim VRP in der Regel nicht unterschieden, ob die berechneten Mehrfachtouren von einem oder mehreren identischen Fahrzeugen (homogene Fahrzeuge) durchgeführt werden, da die Zulässigkeitsbedingungen und Zielwerte (im Falle der Gesamt-Routenlänge) bei beiden Interpretationen immer identisch wären, obwohl im ersten Fall die Touren nacheinander von einem einzigen Fahrzeug durchgeführt werden müssten, während im zweiten Fall die Transporte von mehreren Fahrzeugen parallel durchgeführt werden könnten. Anders sieht es aus, wenn weitere Aspekte zu berücksichtigen sind, z. B. Zeitfenster wie oben beschrieben, oder wenn die Fahrzeuge unterschiedlich sind und insbesondere unterschiedliche Kapazitäten haben (heterogene Fahrzeuge).

Im Allgemeinen erfordert eine Situation mit expliziter Berücksichtigung mehrerer Fahrzeuge, die die Transportaufträge ausführen können, die Festlegung von Routen für jedes Fahrzeug, die die zugewiesenen Aufträge abdecken. Für jedes Fahrzeug und die zugeordneten Aufträge muss geprüft werden, ob die Kapazitäten eingehalten werden.

Es ist anzumerken, dass diese Art von Problemen unter Berücksichtigung einer Fahrzeugflotte mit heterogenen Kapazitäten (und möglicherweise weiteren ausstattungsbedingten Unterschieden) realistischer zu sein scheint als die Standard-VRP-Annahmen. Diese Variante des VRP kann natürlich auch mit Zeitfenstern formuliert werden.

Bei Routing-Problemen mit mehreren Fahrzeugen können auch andere Ziele verfolgt werden. Anstatt die Gesamtlänge der Touren zu minimieren, kann man versuchen, die kleinste Anzahl von Fahrzeugen zu finden, so dass ihre Touren alle Lieferungen umfassen und die Kapazitätsgrenzen einhalten.

3.5.3 *Das Vehicle-Routing-Problem mit mehreren Depots*

Eine andere Variante des Vehicle-Routing-Problems besteht darin, verschiedene Depots für die Fahrzeuge zu berücksichtigen. In der Regel wird davon ausgegangen, dass ein Transportauftrag von jedem Depot aus zugestellt werden kann. Es ist also notwendig, jeden Transportauftrag einem Depot zuzuordnen, von dem aus er erfüllt werden soll, und dann entsprechende Touren zu bilden. Bei heterogenen Fahrzeugen ist es stattdessen erforderlich, die Transportaufträge individuellen Fahrzeugen zuzuordnen, von denen in der Regel angenommen wird, dass sie einem bestimmten Depot zugeordnet sind (als Teil der Problemformulierung).

Anstatt eine mathematische Problemformulierung zu erörtern, betrachten wir ein Beispiel für ein Multi-Depot-Vehicle-Routing-Problem, wie in Abb. 3.1 dargestellt. In der Abbildung sind drei Depots durch Sterne und 15 Kunden durch Punkte gekennzeichnet. In der dargestellten Lösung werden acht der Kunden von demselben Depot in zwei Touren bedient, während für jedes andere Depot nur eine Tour definiert ist.

Man könnte also sagen, dass das Multi-Depot-Vehicle-Routing-Problem (ähnlich dem Vehicle-Routing-Problem mit heterogenen Fahrzeugen) zwei zu lösende Aufgaben enthält: die Zuordnung von Transportaufträgen zu bestimmten Depots (oder einzelnen Fahrzeugen, die mit einem bestimmten Depot verbunden sind) und die Erstellung einer oder mehrerer Touren für jedes Depot. Einige Lösungsansätze machen sich diese Problemstruktur zunutze und lösen das Problem auf hierarchische Weise, indem sie den Transportaufträgen zunächst bestimmte Ressourcen (z. B. Depots oder Fahrzeuge) zuordnen und dann für jede Ressource eine Tour erstellen.

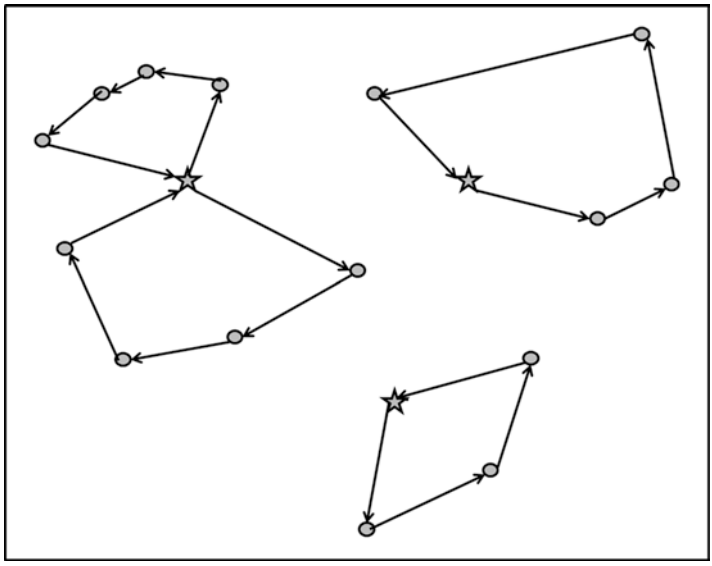


Abb. 3.1 Beispiel für die Lösung eines Multi-Depot-Vehicle-Routing-Problems

Auch wenn kein solcher zweiphasiger Ansatz angewandt wird, müssen beide Arten von lösungsbezogenen Informationen in einer entsprechenden Methode kodiert werden. Im GA-basierten Ansatz verwenden Li et al. (2016) beispielsweise ein einzelnes Chromosom zur Darstellung einer Lösung, das aus einem Teil für jedes Depot (mit einem Gen, das das Depot angibt), gefolgt von der Tourreihenfolge für das Depot, besteht. Es ist anzumerken, dass der Zuweisungsteil (auch als Clustering bezeichnet) manchmal nicht in den Lösungsansatz einbezogen wird. So wird entweder eine einfache Heuristik a priori angewandt, z. B. die Zuweisung von Kunden zum nächstgelegenen Depot, oder, wie in Liu et al. (2014), die Zuweisung zu einem Depot erfolgt, nachdem die Touren durch eine Metaheuristik bestimmt wurden.

3.5.4 Weitere Problemvarianten

Komplexere Varianten des VRP können weitere Einschränkungen hinsichtlich der durchführbaren Transporte berücksichtigen. Zum Beispiel können Kapazitäten nicht nur durch skalare Werte für Kapazitätsanforderungen definiert werden, bei denen eine Zahl die verfügbare Kapazität eines Fahrzeugs ausdrückt. Stattdessen kann das Problem von definierten zwei- oder dreidimensionalen Merkmalen der Lasten ausgehen, die in einem entsprechend definierten Laderaum des Fahrzeugs untergebracht werden müssen. Das Problem ist dann eine Kombination aus einem Tourbildungsproblem und einem Packproblem (siehe z. B. Leung et al., 2011).

Andere Problemformulierungen können die Kapazitäten als alternative Beladungsformen der Fahrzeuge berücksichtigen. So wird beispielsweise in Hanne et al. (2009) der Transport von Patienten in Klinikfahrzeugen betrachtet, die den Transport von z. B. einem liegenden Patienten und einer sitzenden Person oder alternativ von drei sitzenden Personen (z. B. Patienten mit Rollstühlen) ermöglichen. Darüber hinaus können weitere Beschränkungen für die Zusammenlegung von Ladungen gelten, z. B. Beschränkungen für den Sammeltransport verschiedener Tierkategorien bei Tiertransporten, siehe z. B. Oppen und Løkketangen (2006). Weitere Beispiele für eine ausgeschlossene Sammeltransportmöglichkeiten finden sich beim Transport ansteckender Patienten oder von Gütern, die unterschiedliche Handhabungs- oder Transportbedingungen erfordern, z. B. Kühlung.

Weitere praxisrelevante Aspekte können sich aus der für den Transport erforderlichen spezifischen Ausrüstung ergeben, z. B. Lüftungs- oder Kühlgeräte, die entweder spezifische Fahrzeuge oder die explizite Modellierung zusätzlicher Geräte für die Zuordnung zu den Transporten erfordern. Dies kann bedeuten, dass aus einer Fahrzeugflotte nicht jedes Fahrzeug trotz ausreichender Kapazität geeignet ist.

Eine andere Klasse von VRP-Varianten, die in jüngster Zeit mehr Aufmerksamkeit erhalten haben, sind Multi-Echelon-Vehicle-Routing-Probleme und insbesondere das Zwei-Echelon-Vehicle-Routing-Problem (2E-VRP), bei dem davon ausgegangen wird, dass die Fracht von Depots zu Zwischenstandorten (Satelliten) geliefert wird, von wo aus sie dann zu den Kunden befördert wird. Solche Probleme können z. B. auftreten, wenn aus Kosten- oder Kapazitätsgründen oder aufgrund

von Fahrbeschränkungen für große Fahrzeuge (z. B. auf kleinen Straßen in einem Stadtzentrum) unterschiedliche Fahrzeuge eingesetzt werden. Eine Einführung in diese Art von Problemen findet sich in Perboli et al. (2011).

Abschließend sei erwähnt, dass natürlich verschiedene Aspekte von Vehicle-Routing-Problemen als unsicher, vage oder zeitlich variierend angesehen werden können. In ihrem Überblick über dynamische und stochastische Vehicle-Routing-Probleme unterscheiden Ritzinger et al. (2016) drei Hauptquellen der Unsicherheit: stochastische Reisezeiten, stochastische Nachfrage und stochastische Kunden. Häufig werden Situationen mit Unsicherheit oder Vagheit durch Fuzzy-Ansätze modelliert. Für einen älteren Überblick über die Verwendung von Fuzzy-Techniken für verschiedene Arten von Problemen in der Transportplanung und -steuerung verweisen wir auf Teodorović (1999). Neuere Studien über Fuzzy-VRP werden beispielsweise in Erbao und Mingyong (2009) in Bezug auf Fuzzy-Nachfrage, in Xu et al. (2011) in Bezug auf Fuzzy-Zeitfenster und in Zheng und Liu (2006) in Bezug auf Fuzzy-Transportzeiten vorgestellt.

3.6 Lösungsansätze für Vehicle-Routing-Probleme

Als Verallgemeinerungen des Travelling-Salesman-Problems sind das kapazitierte Vehicle-Routing-Problem und Varianten davon ebenfalls NP-schwer (Lenstra & Kan, 1981). Es wird daher davon ausgegangen, dass es keine Polynomialzeit-Algorithmen zur exakten Lösung solcher Probleme gibt, und es wurden erhebliche Anstrengungen unternommen, Heuristiken und Metaheuristiken zu finden, die ausreichend gute Lösungen liefern sollten. Die Qualität von Algorithmen wird in der Regel in Experimenten mit Testproblemen gemessen. Entsprechende Testprobleme für das VRP und einige andere Erweiterungen des TSP sind auch in der erwähnten TSPLIB-Bibliothek verfügbar. Weitere Testprobleme werden in mehreren Veröffentlichungen diskutiert und verwendet. Insbesondere die Sammlung von Testproblemen für VRPTW, die von Solomon (2014) bereitgestellt wird, wird häufiger verwendet. Eine weitere Sammlung von Testproblemen ist in der OR-Bibliothek von Beasley (2014) enthalten. Die allgemeine Abdeckung dieser Probleme durch Standard-Benchmark-Instanzen ist jedoch nicht so gründlich wie für das TSP.

Wie bei anderen Optimierungsproblemen können wir grob drei Arten von Methoden unterscheiden: einfache Heuristiken, Metaheuristiken und mathematische Ansätze. Einen guten Überblick über einfache Heuristiken geben Laporte und Semet (2001), die zwischen konstruktiven Heuristiken, zweiphasigen Heuristiken, die aus einer Clustering-Phase (Zuweisung von Transportaufträgen zu Routen oder Fahrzeugen) und einer Routenbildungsphase bestehen, und Verbesserungsheuristiken unterscheiden. Die Verbesserungsheuristiken können recht einfach sein, wie die oben diskutierten 2-opt (und Verallgemeinerungen), können aber auch den Austausch von Standorten zwischen verschiedenen Routen berücksichtigen. Was die mathematischen Ansätze betrifft, so ähneln die typischen Techniken denen für andere schwierige kombinatorische Probleme, z. B. Column Generation, Branch-and-

Bound- oder Branch-and-Price-Ansätze. Im Folgenden wird die Anwendung von Metaheuristiken näher betrachtet.

Wie beim TSP können Lösungen für das VRP und Varianten als eine Folge von Orten (bzw. ihren jeweiligen Indizes) repräsentiert werden. Einfache Variationsoperatoren wie Mutation und Crossover in evolutionären Algorithmen müssen darauf achten, dass eine möglicherweise zufällige Änderung einer Sequenz nicht zum Weglassen eines Ortes oder zur doppelten Berücksichtigung eines Ortes führt. Eine typische Änderung würde daher auf dem Austausch von mindestens zwei Orten in einer Reihenfolge basieren (siehe Abschn. 3.4.2). Andere Nebenbedingungen wie Kapazitätsgrenzen oder Zeitfenster müssen bei solchen Änderungen zusätzlich berücksichtigt werden, um die Zulässigkeit von Lösungen zu erhalten.

Generell ist festzustellen, dass die jeweiligen Anpassungen der Anwendbarkeit von CI-Ansätzen auf VRP-Probleme einschließlich der Lösungsrepräsentationen denen ähneln, die oben für das TSP diskutiert wurden. Insbesondere wurde eine ähnlich große Anzahl verschiedener Metaheuristiken wie für das TSP auf das VRP und seine Varianten angewendet.

In ihrer kategorisierten Bibliographie hinsichtlich des Einsatzes von Metaheuristiken für verschiedene VRP-Typen (Standard-VRP, VRP mit Zeitfenstern, VRP mit Rücktransporten, Abhol- und Zustellproblemen, VRP mit Mehrfachnutzung von Fahrzeugen, mit heterogenen Fahrzeugen, mit mehreren Depots und periodischen Problemen sowie dynamische VRP) berücksichtigen Gendreau et al. (2008) die folgende Anzahl von Veröffentlichungen für verschiedene Arten von Metaheuristiken (mit möglicher Doppelzählung, wenn ein Ansatz für verschiedene Problemtypen verwendet wird):

- Tabu-Suche: 65
- genetische Algorithmen: 43
- Ant Colony Optimization: 10
- Simulated Annealing: 10
- variable Nachbarschaftssuche: 4
- Greedy Randomized Adaptive Search Procedure (GRASP): 3
- andere: 51

Abgesehen von der beeindruckenden Gesamtzahl der auf Metaheuristiken basierenden Ansätze fällt auf, dass die große Zahl anderer Metaheuristiken die anhaltende Suche nach neuen Lösungsansätzen in diesem Bereich widerspiegelt, von denen die meisten bisher nur in ein oder zwei Veröffentlichungen verwendet wurden. Einige Beispiele für diese Metaheuristiken sind die Particle Swarm Optimization, „adaptive large neighborhood search“, die iterierte lokale Suche und das Path-Relinking, aber wir finden auch einige neuere und weniger bekannte metaheuristische Ansätze wie einen heuristischen Blasenalgorithmus oder den Imperialist Competitive Algorithm. Oft handelt es sich bei den angewandten Techniken um memetische Algorithmen oder andere hybride Ansätze.

Ergebnisse, die mit denen von Gendreau et al. (2008) vergleichbar sind, werden in einer Übersicht von Jozefowiez et al. (2008) über Mehrziel-Probleme im Vehicle-Routing vorgestellt: Evolutionäre Algorithmen erscheinen neben einigen Arten von

einfachen Heuristiken als die am häufigsten verwendeten Ansätze. Andere häufig verwendete Ansätze sind verschiedene Arten lokaler Suchheuristiken und Ant Colony Optimization, während andere Ansätze eher sporadisch verwendet werden. Der Mehrzielaspekt wird in der Regel entweder durch Gewichtung der Ziele, durch eine Formulierung des Goal Programming oder durch eine Approximation der Pareto-Menge behandelt (wie dies häufig bei multikriteriellen evolutionären Algorithmen geschieht).

Auch neuronale Netze wurden gelegentlich für die Behandlung von Vehicle-Routing-Problemen verwendet. Im Vergleich zu ihrer Anwendung auf das TSP sind entsprechende Studien jedoch weniger häufig, was wahrscheinlich darauf zurückzuführen ist, dass sich weitere Nebenbedingungen nur schwer in ein auf neuronalen Netzen basierendes Modell einbetten lassen. Unter den entsprechenden Studien finden wir Modares et al. (1999), die eine Selbstorganisierende Karte (Kohonen-Netz) auf solche Probleme anwenden, die erfolgreicher ist als einige der anderen untersuchten neuronalen Netzvarianten. Ähnliche Ansätze werden in Steinhaus et al. (2015) und Steinhaus (2015) verwendet. Die Kapazitätsbeschränkung wird hier berücksichtigt, indem die Neuronen verschiedenen Ringen zugewiesen werden, die separaten Touren entsprechen. Außerdem verwendet der Ansatz einen verbesserten Bias-Term und nutzt einen neuartigen eingeschränkten Mechanismus zur Verbesserung der Zulässigkeit von Lösungen. Ein dynamisches VRP wurde auch mit einem Kohonen-Netz-Ansatz gelöst, der mit einem evolutionären Algorithmus hybridisiert wurde (Créput et al., 2012). In einer Reihe von Experimenten wurde gezeigt, dass der resultierende Algorithmus eine auf einem genetischen Algorithmus basierende Methode und einen Ameisenkolonie-Ansatz übertrifft.

Hinsichtlich der Leistungsfähigkeit verschiedener Algorithmentypen sind allgemeine Aussagen schwieriger als für das TSP, da das Problem weniger standardisiert ist und ein weniger gut etablierter Kanon von Testproblemen existiert. Dennoch wollen wir einige vergleichende Ergebnisse aus der Literatur vorstellen: Evolutionäre Algorithmen scheinen die dominierenden komplexen Metaheuristiken zu sein, aber viele Studien haben gezeigt, dass gutes Design und problemspezifische Anpassungen Schlüsselfaktoren für überlegene Leistung sind.

Dorigo und Gambardella (1997a, b) zeigten eine konkurrenzfähige oder überlegene Leistung von ACO im Vergleich zu anderen Metaheuristiken. Für eine Reihe von mittelgroßen Testproblemen konnten optimale Ergebnisse abgeleitet werden. Die Entwicklung neuronaler Netze hingegen blieb trotz einiger vielversprechender erster Ergebnisse in Richtung konkurrenzfähiger Ansätze stecken. Erst in den letzten Jahren wurden diese Konzepte mit einigem Erfolg wieder aufgegriffen. Sehr gute Ergebnisse wurden auch für hybride Metaheuristiken erzielt, insbesondere für verschiedene Metaheuristiken, die mit lokaler Suche kombiniert wurden (siehe z. B. Nguyen et al., 2007), z. B. memetische Algorithmen, die einen evolutionären Algorithmus mit lokaler Suche zur Verbesserung neu erzeugter Zwischenlösungen kombinieren. Typischerweise werden für die lokale Suche einfache Heuristiken wie 2-opt, 3-opt oder die komplexeren Lin-Kernighan-Verbesserungsoperationen verwendet. Einerseits sind solche Verbesserungsoperationen auch bei alleiniger Anwendung sehr effektiv. Andererseits können metaheuristische Verfahren bessere

Perturbationsmechanismen zum Verlassen eines lokalen Optimums enthalten als die in etablierten Lin-Kernighan-Methoden verwendeten zufälligen Störungen . Das bedeutet, dass die Zufallsmechanismen eines evolutionären Algorithmus und eine diversifizierte Population ein effektiveres Verlassen von lokalen Optima oder lokaler Einzugsgebiete ermöglichen können.

3.7 Abhol- und Zustellprobleme

Das Abhol- und Zustellproblem ist eine weitere häufige Form des Transportproblems. Im Gegensatz zum VRP sind hier nicht nur die Liefer- oder Abholaktivitäten innerhalb der Route zu planen, sondern beide Aktivitäten. Das bedeutet, dass das Fahrzeug in der Regel leer an einem Depot startet und an verschiedenen Orten entlang seiner Route (gemäß den vorgegebenen Transportaufträgen) Güter abholen und zu den jeweiligen Zielorten bringen muss. Dies ist der typische Geschäftsfall eines Transportdienstleisters, der Waren für verschiedene Kunden transportiert, während das Standard-VRP eher Transportaktivitäten berücksichtigt, die von einem Unternehmen auf eigene Rechnung durchgeführt werden, z. B. die Verteilung von Waren aus einem Lager, das auch als Depot für eigene Fahrzeuge dient. Ein Sonderfall eines Abhol- und Lieferproblems liegt vor, wenn der Transportdienstleister nur an einem Ort Güter abholt, die zu mehreren Kunden transportiert werden sollen. In diesem Fall beginnt die Route am Depot des Dienstleisters, fährt dann zum Standort des Auftraggebers, um die Ware aufzunehmen, bedient dessen Kunden in einer sinnvollen Reihenfolge und kehrt schließlich zum Depot zurück.

Wie bei VRP müssen auch bei Abhol- und Zustellproblemen in der Regel die Kapazitätsbeschränkungen des Fahrzeugs beachtet werden. Das kann bedeuten, dass einige Orte zweimal angefahren werden müssen, z. B. ein Ort, an dem Waren für verschiedene Kunden abgeholt werden sollen. Dies ist dann der Fall, wenn nicht alle Güter, die an diesem Ort abgeholt werden müssen, gleichzeitig in das Fahrzeug passen. Genau wie beim kapazitierten VRP ist es oft notwendig, Zeitfenster zu beachten, jetzt möglicherweise sowohl für Abhol- als auch für Lieferorte.

Betrachten wir ein Beispiel mit der in Tab. 3.2 angegebenen Matrix der Fahrzeiten. Ein Transportdienstleister, dessen Fahrzeugdepot sich in D befindet, muss Sendungen an mehreren Orten abholen und zu verschiedenen Orten bringen. Für das Be- und Entladen an jedem Ort werden im Allgemeinen 10 Minuten angenommen.

Tab. 3.2 Beispiel einer Distanzmatrix

	A	B	C	D
A		12	35	25
B	12		25	10
C	35	25		20
D	25	10	20	

Die folgenden Transporte sind auszuführen:

1. eine Sendung von A nach B,
2. eine Sendung von C nach B,
3. eine Sendung von C nach A.

Wenn die Kapazität der Fahrzeuge ausreicht, um diese Sendungen gleichzeitig zu transportieren, können wir zur Vereinfachung des Problems folgende Überlegung anstellen: Wenn möglich, sollte jeder Ort nur einmal angefahren werden. Das bedeutet hier, dass die beiden Sendungen von C während desselben Stopps abgeholt werden sollten, für A sollte die Abholung der ersten Sendung und die Entladung der dritten Sendung während desselben Stopps erfolgen und die beiden Sendungen für B sollten gemeinsam zugestellt werden. Daraus ergibt sich die folgende „natürliche“ Tour:

$$D_{\text{abfahrt}}\,0'\ C_{\text{anreise}}\,20'\ C_{\text{abfahrt}}\,30'\ A_{\text{anreise}}\,65'\ A_{\text{abfahrt}}\,75'\ B_{\text{anreise}}\,87'$$
$$B_{\text{abfahrt}}\,97'\ D_{\text{anreise}}\,107'$$

Wenn z. B. die Sendung 2 die gesamte Kapazität des Fahrzeugs benötigt, ist diese Lösung nicht mehr zulässig. Wir können dann davon ausgehen, dass zwei Teiltouren C → B und C → A → B zu einer Gesamttour kombiniert werden müssen, die in D beginnt und endet:

$$D_{\text{abfahrt}}\,0'\ C_{\text{ankunft}}\,20'\ C_{\text{abfahrt}}\,30'\ B_{\text{ankunft}}\,55'\ B_{\text{abfahrt}}\,65'\ C_{\text{ankunft}}\,90'C_{\text{abfahrt}}\,100'$$
$$A_{\text{ankunft}}\,135'A_{\text{abreise}}\,145'\ B_{\text{ankunft}}\,157'\ B_{\text{abfahrt}}\,167'\ D_{\text{ankunft}}\,177'$$

$$D_{\text{abfahrt}}\,0'\ C_{\text{ankunft}}\,20'\ C_{\text{abfahrt}}\,30'\ A_{\text{ankunft}}\,65'\ A_{\text{abfahrt}}\,75'\ B_{\text{ankunft}}\,87'\ B_{\text{abfahrt}}\,97'$$
$$C_{\text{ankunft}}\,122'C_{\text{abfahrt}}\,132'\ B_{\text{ankunft}}\,157'\ B_{\text{abfahrt}}\,167'\ D_{\text{ankunft}}\,177'$$

Wenn wir davon ausgehen, dass die Tour um 8:00 Uhr beginnt und für die Abholung der Waren bei A ein Zeitfenster von 9:15 bis 9:45 Uhr eingehalten werden muss, wäre die erste Tour nicht mehr durchführbar, da die entsprechende Abholung zwischen 10:15 und 10:25 Uhr stattfindet. Die zweite Lösung kann realisiert werden, wenn eine Wartezeit von 10′ bei A berücksichtigt wird (Warten von 9:05 Ankunftszeit bis 9:15, dem Beginn des Zeitfensters). Die gesamte Tour würde sich dann um diese 10′ verzögern und um 11:07 Uhr enden.

Sollte ansonsten ein Zeitfenster von 8:30 bis 9:00 Uhr für die Abholung von Waren bei C eingehalten werden, gäbe es keine zulässige Lösung, da C zweimal angefahren werden muss und keine der jeweiligen Lieferungen innerhalb von 30′ einschließlich der Rückfahrt zu C erfolgen kann. Der Konflikt könnte durch eine Änderung der Problemstellung gelöst werden, z. B. durch den Einsatz von zwei Fahrzeugen, durch die Möglichkeit der Zwischenlagerung einer Sendung in der Nähe von C oder durch die Vereinbarung eines größeren Zeitfensters.

Es sei darauf hingewiesen, dass in der Literatur zu Abhol- und Zustellproblemen, die den VRPs ähnlich sind, auch andere Varianten betrachtet werden, z. B. meh-

rere Fahrzeuge, spezifische Beschränkungen für die Beladung oder zusätzlich benötigten Geräte oder Probleme mit Unsicherheit oder vagen Informationen. Eine spezielle Variante von Abhol- und Zustellproblemen hat in der Literatur mehr Aufmerksamkeit auf sich gezogen. Dieses als Dial-a-Ride-Problem bezeichnete Problem erfordert eine zusätzliche Einschränkung, dass die Transportdauer jeder Ladung begrenzt ist (die maximale Dauer ist festgelegt). Die Beschränkung soll Lösungen verhindern, bei denen zwischen der Abholung und der Zustellung eines Gutes zu viele andere Abholungen und Lieferungen innerhalb einer Tour geplant sind. Solche Begrenzungen der Transportdauer können beim Transport von Menschen relevant sein oder sogar auf gesetzlichen Beschränkungen beruhen, z. B. bei Tiertransporten.

3.8 Netzwerk-Flussprobleme

Eine andere Art von Transportproblemen beruht auf der Annahme, dass die Warenströme in einem Netz optimiert werden müssen. Das Netz wird durch einen gerichteten (oder ungerichteten) Graphen dargestellt. Das Problem wird durch Netzknoten mit einer Nachfrage (Senken) und Netzknoten mit einem Angebot an Gütern (Quellen) spezifiziert.

Netzwerk-Flussprobleme gibt es in zwei grundlegenden Varianten in Bezug auf die Nebenbedingungen und die Zielfunktion. Eine Variante geht davon aus, dass die Kapazität der Kanten im Graphen begrenzt ist, d. h. $f(u,v) \leq c(u,v)$, wobei f der Fluss auf einer Kante $(u.v)$ und $c(u,v)$ die jeweilige Kapazitätsgrenze ist. Ziel ist es, die maximale Last zu bestimmen, die durch das Netz von einem Startort a zu einem Zielort b transportiert werden kann. Unter der Annahme, dass alle über die verschiedenen Kanten zu transportierenden Mengen reelle Variablen sind, kann das Problem als lineares Optimierungsproblem formuliert und mit etablierten Methoden gelöst werden. Eine häufig betrachtete Erweiterung des Grundproblems ist das Multicommodity-Flow-Problem (MCFP) (siehe z. B. McBride, 1998 für einen Überblick). Hier werden verschiedene Arten von Materialien betrachtet, die durch das Netzwerk transportiert werden müssen. Bei all diesen Problemen werden die spezifischen Transportaktivitäten (z. B. die zu verwendenden Fahrzeuge) nicht berücksichtigt. Außerdem kann die Lösung erfordern, dass eine große Anzahl von Kanten für die Transportaufgabe verwendet wird, was zur Folge hat, dass die Fracht aufgeteilt und mit verschiedenen Fahrzeugen auf verschiedenen Routen transportiert werden muss. Dies kann zu relativ großen Entfernungen und hohen Transportkosten führen. Hinsichtlich der Berechnungskomplexität ist zu erwähnen, dass reellwertige Probleme in polynomialer Zeit gelöst werden können, während sich ganzzahlige Varianten als NP-schwer erweisen (Even et al., 1975; Hall et al., 2007). Anders als bei den meisten anderen betrachteten Transportplanungsproblemen spielen Metaheuristiken bei der Lösung von Netzwerk-Flussproblemen, die meist Lösungsansätze aus dem Bereich der mathematischen Programmierung anziehen, noch eine untergeordnete Rolle.

Eine andere Art von Netzwerk-Flussproblem kann entstehen, wenn das Transportproblem aus der Sicht eines Unternehmens betrachtet wird, das Transportdienstleister für die Durchführung der Transporte in Anspruch nehmen möchte. Mit Hilfe moderner IT-Technologien ist es relativ einfach geworden, die Transportkosten elektronisch von den jeweiligen Dienstleistern zu erhalten. Der Nutzen von Transportdienstleistungen ist nicht an der konkreten Durchführung der Transporte (wie z. B. eingesetzte Fahrzeuge) interessiert, sondern kann die jeweiligen Kosteninformationen und weitere Informationen wie z. B. die jeweiligen Kapazitäten oder Transportzeiten erfragen. Möglicherweise sind weitere praxisrelevante Aspekte zu beachten, z. B. Zeitfenster, Anforderungen an Sammeltransporte der Ladung, Crossdocking oder Scheduling-Aspekte. Das Ziel ist in der Regel die Minimierung einer Art Kostenfunktion. Ein verwandter Ansatz wird von Bryan und McIntire (2015) vorgestellt, die eine cloudbasierte Architektur verwenden, die verschiedene Arten von Metaheuristiken in einem agentenbasierten Rahmen einsetzt.

Literatur

Ahmed, Z. H. (2010). Genetic algorithm for the traveling salesman problem using sequential constructive crossover operator. *International Journal of Biometrics and Bioinformatics (IJBB), 3*(6), 96–105.

Albayrak, M., & Allahverdi, N. (2011). Development a new mutation operator to solve the traveling salesman problem by aid of genetic algorithms. *Expert Systems with Applications, 38*(3), 1313–1320.

Anbuudayasankar, S. P., Ganesh, K., & Mohapatra, S. (2014). *Models for practical routing problems in logistics*. Springer.

Applegate, D., Cook, W., & Rohe, A. (2003). Chained Lin-Kernighan for large traveling salesman problems. *INFORMS Journal on Computing, 15*(1), 82–92.

Beasley, J. E. (2014). *OR-library*. http://people.brunel.ac.uk/~mastjjb/jeb/info.html. Zugegriffen am 12.03.2016.

Braun, H. (1991). On solving travelling salesman problems by genetic algorithms. In H.-P. Schwefel & R. Männer (Hrsg.), *Parallel problem solving from nature* (S. 129–133). Springer.

Bryan, J., & McIntire, J. (2015). *Leveraging distributed metaheuristics to balance flexibility and complexity*. Invited talk at IEEE TENCON 2015: Technical track 11, IEEE.

Buriol, L., França, P. M., & Moscato, P. (2004). A new memetic algorithm for the asymmetric traveling salesman problem. *Journal of Heuristics, 10*(5), 483–506.

Burkard, R. E., & Cela, E. (1999). Linear assignment problems and extensions. In D.-Z. Du & P. M. Pardalos (Hrsg.), *Handbook of combinatorial optimization* (S. 75–149). Springer.

Charon, I., & Hudry, O. (2000). Application of the noising method to the travelling salesman problem. *European Journal of Operational Research, 125*(2), 266–277.

Chatterjee, S., Carrera, C., & Lynch, L. A. (1996). Genetic algorithms and traveling salesman problems. *European Journal of Operational Research, 93*(3), 490–510.

Créput, J. C., & Koukam, A. (2009). A memetic neural network for the Euclidean traveling salesman problem. *Neurocomputing, 72*(4), 1250–1264.

Créput, J. C., Hajjam, A., Koukam, A., & Kuhn, O. (2012). Self-organizing maps in population based metaheuristic to the dynamic vehicle routing problem. *Journal of Combinatorial Optimization, 24*(4), 437–458.

Crisan, G. C., & Nechita, E. (2008). Solving fuzzy TSP with ant algorithms. *International Journal of Computers, Communications and Control, 3*, 228–231.

Dorigo, M., & Gambardella, L. M. (1997a). Ant colonies for the travelling salesman problem. *BioSystems, 43*(2), 73–81.

Dorigo, M., & Gambardella, L. M. (1997b). Ant colony system: A cooperative learning approach to the traveling salesman problem. *IEEE Transactions on Evolutionary Computation, 1*(1), 53–66.

Erbao, C., & Mingyong, L. (2009). A hybrid differential evolution algorithm to vehicle routing problem with fuzzy demands. *Journal of Computational and Applied Mathematics, 231*(1), 302–310.

Even, S., Itai, A., & Shamir, A. (1975). *On the complexity of time table and multi-commodity flow problems*. 16th annual symposium on foundations of computer science, 1975, IEEE, S. 184–193.

Feillet, D., Dejax, P., & Gendreau, M. (2005). Traveling salesman problems with profits. *Transportation Science, 39*(2), 188–205.

Fu, L., Sun, D., & Rilett, L. R. (2006). Heuristic shortest path algorithms for transportation applications: State of the art. *Computers & Operations Research, 33*(11), 3324–3343.

Gendreau, M., Laporte, G., & Vigo, D. (1999). Heuristics for the traveling salesman problem with pickup and delivery. *Computers & Operations Research, 26*(7), 699–714.

Gendreau, M., Potvin, J. Y., Bräumlaysy, O., Hasle, G., & Løkketangen, A. (2008). Metaheuristics for the vehicle routing problem and its extensions: A categorized bibliography. In B. L. Golden, S. Raghavan, & E. A. Wasil (Hrsg.), *The vehicle routing problem: Latest advances and new challenges* (S. 143–116). Springer.

Golden, B. L., Raghavan, S., & Wasil, E. A. (Hrsg.). (2008). *The vehicle routing problem: Latest advances and new challenges* (Operations research/computer science interfaces series, Bd. 43). Springer.

Grefenstette, J., et al. (1985). *Genetic algorithms for the traveling salesman problem*. Proceedings of the first international conference on genetic algorithms and their applications, Lawrence Erlbaum, S. 160–168.

Hall, A., Hippler, S., & Skutella, M. (2007). Multicommodity flows over time: Efficient algorithms and complexity. *Theoretical Computer Science, 379*(3), 387–404.

Hanne, T., Melo, T., & Nickel, S. (2009). Bringing robustness to patient flow management through optimized patient transports in hospitals. *Interfaces, 39*(3), 241–255.

Jati, G. K., & Suyanto, S. (2011). Evolutionary discrete firefly algorithm for travelling salesman problem. In *Adaptive and intelligent systems* (Lecture Notes in Computer Science, Bd. 6943, S. 393–403). Springer.

Jolai, F., & Ghanbari, A. (2010). Integrating data transformation techniques with Hopfield neural networks for solving travelling salesman problem. *Expert Systems with Applications, 37*(7), 5331–5335.

Jozefowiez, N., Semet, F., & Talbi, E. G. (2008). Multi-objective vehicle routing problems. *European Journal of Operational Research, 189*(2), 293–309.

Kallehauge, B., Larsen, J., Madsen, O. B., & Solomon, M. M. (2005). Vehicle routing problem with time windows. In G. Desaulniers, J. Desrosiers, & M. M. Solomon (Hrsg.), *Column generation* (S. 67–98). Springer.

Karaboga, D., & Gorkemli, B. (2011). *A combinatorial artificial bee colony algorithm for traveling salesman problem*. 2011 international symposium on Innovations in Intelligent Systems and Applications (INISTA), IEEE, S. 50–53.

Laporte, G., & Semet, F. (2001). Classical heuristics for the capacitated VRP. In P. Toth & D. Vigo (Hrsg.), *The vehicle routing problem* (S. 109–128). Society for Industrial and Applied Mathematics.

Larrañaga, P., Kuijpers, C. M. H., Murga, R. H., Inza, I., & Dizdarevic, S. (1999). Genetic algorithms for the travelling salesman problem: A review of representations and operators. *Artificial Intelligence Review, 13*(2), 129–170.

Lenstra, J. K., & Kan, A. H. G. (1981). Complexity of vehicle routing and scheduling problems. *Networks, 11*(2), 221–227.

Leung, S. C., Zhou, X., Zhang, D., & Zheng, J. (2011). Extended guided tabu search and a new packing algorithm for the two-dimensional loading vehicle routing problem. *Computers & Operations Research, 38*(1), 205–215.

Li, J., Li, Y., & Pardalos, P. M. (2016). Multi-depot vehicle routing problem with time windows under shared depot resources. *Journal of Combinatorial Optimization, 31*(2), 515–532.

Li, X., Tian, P., Hua, J., & Zhong, N. (2006). A hybrid discrete particle swarm optimization for the traveling salesman problem. In T.-D. Wang, X. Li, & X. Wang (Hrsg.), *Simulated evolution and learning* (S. 181–188). Springer.

Liong, C. Y., Wan, R. I., Khairuddin, O., & Zirour, M. (2008). Vehicle routing problem: Models and solutions. *Journal of Quality Management and Analysis, 4*, 205–218. http://pkukmweb.ukm.my/~ppsmfst/jqma/Vol4_Is1/PDF/JQMA4(1)-19-liong-drk.pdf. Zugegriffen am 12.03.2016.

Liu, T., Jiang, Z., & Geng, N. (2014). A genetic local search algorithm for the multi-depot heterogeneous fleet capacitated arc routing problem. *Flexible Services and Manufacturing Journal, 26*(4), 540–564.

Marinakis, Y., Migdalas, A., & Pardalos, P. M. (2005). Expanding neighborhood GRASP for the traveling salesman problem. *Computational Optimization and Applications, 32*(3), 231–257.

McBride, R. D. (1998). Advances in solving the multicommodity-flow problem. *Interfaces, 28*(2), 32–41.

Mladenović, N., Urošević, D., & Ilić, A. (2012). A general variable neighborhood search for the one-commodity pickup-and-delivery travelling salesman problem. *European Journal of Operational Research, 220*(1), 270–285.

Modares, A., Somhom, S., & Enkawa, T. (1999). A self-organizing neural network approach for multiple traveling salesman and vehicle routing problems. *International Transactions in Operational Research, 6*(6), 591–606.

Nguyen, H. D., Yoshihara, I., Yamamori, K., & Yasunaga, M. (2007). Implementation of an effective hybrid GA for large-scale traveling salesman problems. *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics), 37*(1), 92–99.

Oppen, J., & Løkketangen, A. (2006). *The livestock collection problem*. Proceedings of the 21st European conference on Operational Research.

Ouaarab, A., Ahiod, B., & Yang, X. S. (2014). Discrete cuckoo search algorithm for the travelling salesman problem. *Neural Computing and Applications, 24*(7–8), 1659–1669.

Pasti, R., & De Castro, L. N. (2006). *A neuro-immune network for solving the traveling salesman problem*. International joint conference on Neural Networks, 2006. IJCNN'06, IEEE, S. 3760–3766.

Pepper, J. W., Golden, B. L., & Wasil, E. A. (2002). Solving the traveling salesman problem with annealing-based heuristics: A computational study. *IEEE Transactions on Systems, Man and Cybernetics, Part A: Systems and Humans, 32*(1), 72–77.

Perboli, G., Tadei, R., & Vigo, D. (2011). The two-echelon capacitated vehicle routing problem: Models and math-based heuristics. *Transportation Science, 45*(3), 364–380.

Peterson, C. (1990). Parallel distributed approaches to combinatorial optimization: Benchmark studies on traveling salesman problem. *Neural Computation, 2*(3), 261–269.

Potvin, J. Y. (1993). State-of-the-art survey – The traveling salesman problem: A neural network perspective. *ORSA Journal on Computing, 5*(4), 328–348.

Reinelt, G. (1991). TSPLIB – A traveling salesman problem library. *ORSA Journal on Computing, 3*(4), 376–384.

Ritzinger, U., Puchinger, J., & Hartl, R. F. (2016). A survey on dynamic and stochastic vehicle routing problems. *International Journal of Production Research, 54*(1), 215–231.

Shi, X. H., Liang, Y. C., Lee, H. P., Lu, C., & Wang, Q. X. (2007). Particle swarm optimization-based algorithms for TSP and generalized TSP. *Information Processing Letters, 103*(5), 169–176.

Shim, V. A., Tan, K. C., Chia, J. Y., & Chong, J. K. (2011). Evolutionary algorithms for solving multi-objective travelling salesman problem. *Flexible Services and Manufacturing Journal, 23*(2), 207–241.

Snyder, L. V., & Daskin, M. S. (2006). A random-key genetic algorithm for the generalized traveling salesman problem. *European Journal of Operational Research, 174*(1), 38–53.

Solomon, M. M. (2014). *VRPTW benchmark problems*. http://w.cba.neu.edu/~msolomon/problems.htm. Zugegriffen am 12.03.2016.

Steinhaus, M. (2015). *The application of the self organizing map to the vehicle routing problem*. Open access dissertations, University of Rhode Island.

Steinhaus, M., Shirazi, A. N., & Sodhi, M. (2015). *Modified self organizing neural network algorithm for solving the vehicle routing problem*. 2015 I.E. 18th international conference on Computational Science and Engineering (CSE), IEEE, S. 246–252.

Stützle, T., Grün, A., Linke, S., & Rüttger, M. (2000). A comparison of nature inspired heuristics on the traveling salesman problem. In M. Schoenauer, K. Deb, G. Rudolph, X. Yao, E. Lutton, J. J. Merelo & H.-P. Schwefel (Hrsg.), *Parallel problem solving from nature PPSN VI* (S. 661–670). Springer.

Takaoka, T. (2013). A simplified algorithm for the all pairs shortest path problem with $O(n^2 \log n)$ expected time. *Journal of Combinatorial Optimization, 25*(2), 326–337.

Teodorović, D. (1999). Fuzzy logic systems for transportation engineering: The state of the art. *Transportation Research Part A: Policy and Practice, 33*(5), 337–364.

Toth, P., & Vigo, D. (Hrsg.). (2014). *Vehicle routing: Problems, methods, and applications* (Bd. 18). Society for Industrial and Applied Mathematics.

Tsai, C. F., & Tsai, C. W. (2002). *A new approach for solving large traveling salesman problem using evolutionary ant rules*. Proceedings of the 2002 international joint conference on Neural Networks, 2002 (IJCNN'02), Bd. 2, IEEE.

Tsai, H. K., Yang, J. M., Tsai, Y. F., & Kao, C. Y. (2004). An evolutionary algorithm for large traveling salesman problems. *IEEE Transactions on Systems, Man, and Cybernetics, Part B: Cybernetics, 34*(4), 1718–1729.

Tsubakitani, S., & Evans, J. R. (1998). Optimizing tabu list size for the traveling salesman problem. *Computers & Operations Research, 25*(2), 91–97.

Ulder, N. L., Aarts, E. H., Bandelt, H. J., van Laarhoven, P. J., & Pesch, E. (1991). Genetic local search algorithms for the traveling salesman problem. In H.-P. Schwefel & R. Männer (Hrsg.), *Parallel problem solving from nature* (S. 109–116). Springer.

Weise, T., Chiong, R., Lassig, J., Tang, K., Tsutsui, S., Chen, W., et al. (2014). Benchmarking optimization algorithms: An open source framework for the traveling salesman problem. *IEEE Computational Intelligence Magazine, 9*(3), 40–52.

Xu, J., Yan, F., & Li, S. (2011). Vehicle routing optimization with soft time windows in a fuzzy random environment. *Transportation Research Part E: Logistics and Transportation Review, 47*(6), 1075–1091.

Yadlapalli, S., Malik, W. A., Darbha, S., & Pachter, M. (2009). A Lagrangian-based algorithm for a multiple depot, multiple traveling salesmen problem. *Nonlinear Analysis: Real World Applications, 10*(4), 1990–1999.

Zheng, Y., & Liu, B. (2006). Fuzzy vehicle routing model with credibility measure and its hybrid intelligent algorithm. *Applied Mathematics and Computation, 176*(2), 673–683.

Zhong, W. L., Zhang, J., & Chen, W. N. (2007). *A novel discrete particle swarm optimization to solve traveling salesman problem*. IEEE Congress on Evolutionary Computation, 2007. CEC 2007, IEEE, S. 3283–3287.

Kapitel 4
Bestandsplanung und Losgrößenbestimmung



4.1 Die Notwendigkeit einer Bestandsplanung

Die Verwaltung von Vorräten gehört zu den Kerntätigkeiten der Logistik. Die Lagerung von Materialien in der Nähe des Ortes, an dem sie voraussichtlich zu einem späteren Zeitpunkt benötigt werden, kann von Vorteil sein, insbesondere wenn ihre Bereitstellung (z. B. durch Herstellung oder Transport) zum gewünschten Zeitpunkt zu kostspielig, zu riskant oder einfach nicht möglich ist.

Genauer gesagt können wir drei Hauptgruppen von Gründen für die Lagerung von Waren unterscheiden: beschaffungsbezogene Aspekte, unternehmensinterne Aspekte und Aspekte, die mit den Kunden eines Unternehmens zusammenhängen. Die Beschaffungsaspekte beziehen sich auf die Interaktionen eines Unternehmens mit seinen Lieferanten. Insbesondere wirtschaftliche Erwägungen führen in der Regel dazu, dass größere Mengen beschafft werden, als unmittelbar benötigt werden. Der Kauf größerer Mengen kann den Preis senken und zu weniger Beschaffungsaktivitäten und Transportvorgängen führen, was ebenfalls zu geringeren Beschaffungskosten führen sollte. Die Verwaltung eines Lagerbestands der beschafften Artikel ist eine Folge der Berücksichtigung solcher beschaffungsbezogenen Kosten.

Ein weiterer lieferantenbezogener Grund ist die Reduktion von Unsicherheiten bei der Beschaffung: Die bestellten Materialien könnten zu spät eintreffen, die Qualität könnte unzureichend sein, oder die Materialien könnten in zu geringen Mengen oder gar nicht eintreffen. Für ein Unternehmen könnte eine Möglichkeit, mit solchen Versorgungsproblemen umzugehen, darin bestehen, eine ausreichende Menge des benötigten Materials auf Lager zu haben. Aber auch ohne solche Kosten- oder Risikoerwägungen kann ein Lagerbestand der benötigten Artikel notwendig sein, wenn die Vorlaufzeit für die Beschaffung unerwünscht hoch ist.

Eine weitere Gruppe von Gründen für Lagerbestände sind die innerbetrieblichen Abläufe in einem Unternehmen. Sehr oft ist es so, dass die Produktion nicht in einem Schritt oder in einem kontinuierlichen Prozess ablaufen kann. Infolgedessen

© Der/die Autor(en), exklusiv lizenziert an Springer Nature Switzerland AG 2023

T. Hanne, R. Dornberger, *Computational Intelligence in Logistik und Supply Chain Management*, https://doi.org/10.1007/978-3-031-21452-3_4

gibt es Zwischenprodukte, die zumindest eine kurze Zeit der Lagerung (Pufferung) erfordern. Ein weiterer Grund kann die Produktionstechnologie sein, die bestimmte Losgrößen erfordert, z. B. Mindestmengen von Materialien, die auf einmal produziert werden müssen. Mindestlosgrößen können auch aus wirtschaftlichen Erwägungen resultieren, wie z. B. der Senkung der Produktionskosten bei einer zusammenhängenden Produktion einer größeren Menge (Größenvorteile). Ein Grund dafür könnte die Vermeidung von Rüstzeiten sein, die bei der Umstellung von einem Produkt auf ein anderes anfallen. Infolgedessen kann ein Unternehmen eine größere Menge eines Produkts vorrätig haben, als derzeit benötigt wird. Die Überproduktion wird in den Lagerbestand aufgenommen.

Schließlich gibt es mehrere Gründe, die die Kunden eines Unternehmens betreffen. Erstens ist in vielen Branchen, insbesondere in der Konsumgüterindustrie, die genaue künftige Nachfrage ungewiss. Wenn ein Unternehmen ausreichende Mengen eines Produkts zur Verfügung haben will, um die unsichere Kundennachfrage stets befriedigen zu können, muss es einen Vorrat an diesen Produkten anlegen. Ein solcher Bestand wird auch als Sicherheitsbestand bezeichnet.

Ein weiterer Grund für eine Vorratshaltung könnte der Betrieb von Lagern in der Nähe der Kunden sein, um eine schnelle Befriedigung der Nachfrage zu ermöglichen. Solche Lager können dann auch die damit verbundenen Transportkosten senken, da größere Mengen zu geringeren Kosten in die Gebiete der erwarteten Kunden transportiert werden können. Auch eine stark saisonale Nachfrage nach Produkten kann zu einer Lagerhaltung führen. Wenn z. B. die Nachfrage nur in kurzen Perioden des Jahres auftritt, kann es (aus Kostengründen) sinnvoll sein, die entsprechenden Waren nicht erst kurz vor diesen Perioden zu produzieren, sondern die Produktion auszugleichen, d. h. die Waren gleichmäßig über das Jahr zu produzieren. Auf diese Weise werden in Zeiten, in denen keine Nachfrage besteht, Vorräte aufgebaut und in Zeiten der Nachfrage abgebaut.

Ein weiterer Zweck der Lagerung kann die Spezialisierung von Produkten im Lager sein, z. B. die Anbringung von länderspezifischen Kabeln, Netzteilen oder Handbüchern im europäischen Zentrallager eines US-amerikanischen IT-Herstellers (van Hoek, 2001). Auch können sich manche Güter während ihrer Lagerzeit verbessern, z. B. bei Käse oder Wein (Reifungsprozesse). Ein weiterer guter Grund für die Lagerhaltung könnte die Erwartung höherer Preise in der Zukunft sein. Solche spekulativen Gründe können insbesondere für die Einlagerung von Edelmetallen oder Kunstwerken gelten.

Die Erwartung höherer Preise oder einer Qualitätsverbesserung während der Lagerhaltung ist jedoch die Ausnahme. Häufiger verschlechtert sich die Qualität im Laufe der Zeit, z. B. bei den meisten Lebensmitteln. In anderen Fällen kann die Qualität unverändert bleiben, aber aufgrund des technischen Fortschritts oder veränderter Kundenpräferenzen sind die Menschen nicht mehr bereit, das Produkt zu kaufen, oder nur zu einem niedrigeren Preis. So sind beispielsweise Computer oder Mobiltelefone, die einige Jahre alt sind, in der Regel nicht mehr an die Kunden verkäuflich.

Die Veralterung der gelagerten Güter führt häufig zu erheblichen Lagerkosten. Weitere Lagerkosten ergeben sich aus der Notwendigkeit eines physischen Lagers,

das Baukosten oder entsprechende Abschreibungen oder Mieten erfordert. Hinzu kommen die Betriebskosten eines Lagers, z. B. für Licht, Heizung oder Kühlung, Strom für die Technik und Lohnkosten. Weitere Betriebskosten entstehen durch die Einlagerung und Auslagerung von Artikeln, einschließlich der damit verbundenen Transportkosten.

Ein weiterer möglicher Kostentreiber ist der Schwund (Diebstahl) von Artikeln aus dem Lager. Der wichtigste Kostenaspekt ist jedoch häufig die Kapitalbindung der gelagerten Güter. Diese Kosten lassen sich ähnlich wie ein Zinssatz für jede andere Art von Investition berechnen.

So gibt es einerseits verschiedene Kostenkomponenten, die eine Lagerhaltung kostspielig machen und die es nahelegen, eine Lagerhaltung nach Möglichkeit zu vermeiden. Andererseits sind zu geringe Bestände auch unerwünscht. Stock-outs, die entstehen, wenn bestimmte Güter nicht oder nicht in ausreichender Menge auf Lager sind, wenn die Kunden sie kaufen wollen, verursachen Opportunitätskosten, d. h. Kosten durch entgangene Einnahmen. In den meisten Planungsmodellen wird davon ausgegangen, dass die Kundennachfrage immer (soweit möglich) gedeckt werden sollte, obwohl wirtschaftliche Überlegungen auch dafür sprechen können, die Nachfrage nicht zu decken, wenn der Deckungsbeitrag der betreffenden Aktivitäten negativ wird.

Aus diesen Gründen gibt es einen starken Bedarf an guten Planungstechniken für den Bestand, die die Lagerkosten minimieren und gleichzeitig sicherstellen, dass die logistischen Anforderungen ausreichend erfüllt werden. Grundlegende Fragen der Lagerverwaltung betreffen die Lagerbestände und damit die wirtschaftlichen Bestellmengen.

4.2 Wirtschaftliche Bestellmengen und Sicherheitsbestände

Zur Erfüllung der Hauptaufgabe der Logistik, nämlich die richtigen Artikel in der richtigen Menge am richtigen Ort zur Verfügung zu haben, sind zwei grundlegende Strategien möglich: die Waren in ausreichender Menge dort zu lagern, wo sie benötigt werden, oder sie zu diesem Ort zu transportieren, wenn sie gebraucht werden. Theoretisch könnte also die Lagerhaltung – zumindest teilweise – durch den Einsatz von Transportmitteln vermieden werden. Diese Idee ist unter dem Schlagwort Just-in-Time-Produktion (JiT) populär geworden, bei der Unternehmen ihre bisher großen Lagerbestände an beschafften Materialien abbauen und mit den Lieferanten vereinbaren, die benötigten Waren pünktlich, d. h. kurz bevor sie gebraucht werden, zu liefern (siehe z. B. Waters-Fuller, 1995).

Dieses Konzept hat seinen Ursprung in der Autoindustrie, und es wurde gelegentlich argumentiert, dass es durch die größere Marktmacht der Autohersteller im Vergleich zu ihren Zulieferern erzwungen wurde. Dieses Argument beruht auf der Annahme, dass die Lagerhaltungskosten (und andere Kosten) von den Kunden (den Automobilherstellern) auf die Zulieferer verlagert werden. Befürworter von JiT argumentieren jedoch, dass eine bessere Zusammenarbeit zwischen Lieferanten

und Kunden die Gesamtlagermenge und die damit verbundenen Lagerkosten von Lieferanten und Kunden verringern kann, so dass beide Seiten von einer Win-Win-Situation profitieren können.

Ein offensichtliches Problem des JiT-Konzepts ist jedoch, dass es häufige Lieferungen vom Lieferanten erfordert. Für die Produktion am selben Tag ist z. B. jeden Morgen eine Lieferung der erforderlichen Materialien erforderlich, wenn kein eigener Bestand an diesen Materialien vorhanden ist. Dies mag für die Automobilherstellung, bei der große Mengen von Materialien verarbeitet werden, angemessen sein, aber bei kleineren Mengen von Materialien können die Transportkosten der häufigen Lieferungen die Kosteneinsparungen aufzehren, die sich aus reduzierten (oder aufgehobenen) Lagerbeständen ergeben.

Dies führt uns zu der Frage, wie oft ein Material geliefert werden sollte, oder umgekehrt zu der Frage, was eine wirtschaftliche Bestellmenge ist. Um darauf eine Antwort zu finden, müssen Lager- und Transportkosten zusammen betrachtet werden: Je kleiner die bestellten Mengen sind, desto geringer sind die Lagerhaltung und die damit verbundenen Kosten, aber desto höher sind die Transportkosten aufgrund der häufigeren Lieferungen. Je größer die bestellten Mengen sind, desto geringer sind die Transportkosten, aber desto höher sind die Lagerhaltungskosten.

Formulieren wir ein einfaches mathematisches Modell, um diese Situation zu analysieren und zu eindeutigeren Ergebnissen zu gelangen: Nehmen wir an, dass der Verbrauch oder die Nachfrage nach dem Lagerbestand stetig ist, d. h. die Anzahl der pro Zeiteinheit aus dem Lager entnommenen Artikel ist im Laufe der Zeit konstant (keine Nachfrageschwankungen). Außerdem nehmen wir an, dass das Unternehmen immer die gleiche Menge des betrachteten Materials, bezeichnet mit x, bestellt und dass neue Bestellungen gerade dann eintreffen, wenn der alte Bestand aufgebraucht ist. Die Entwicklung des Lagerbestands kann dann durch Abb. 4.1 veranschaulicht werden

In einer solchen Situation beträgt der durchschnittliche Bestand $x/2$, d. h. die Hälfte der bestellten Menge.

Nehmen wir an, dass wir den Bestand für einen Planungshorizont von T Perioden (z. B. T Tage) planen wollen und dass während dieses Planungshorizonts eine Gesamtmenge von D benötigt wird. Die Lagerkosten werden mit h pro Einheit des Materials und pro Periode angenommen. Die durchschnittlichen Lagerkosten pro Periode sind dann $h \cdot x/2$ und die gesamten Lagerkosten im Planungshorizont sind $T \cdot h \cdot x/2$.

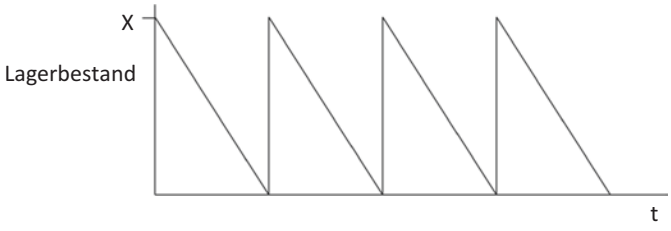


Abb. 4.1 Entwicklung des Lagerbestands bei gleichbleibendem Verbrauch

Für die Berechnung der Transportkosten treffen wir eine weitere grob vereinfachte Annahme: Es wird angenommen, dass jede Lieferung fixe Kosten von u verursacht, d. h. die Kosten der Lieferung sind unabhängig von der gelieferten Menge. Da während des Planungshorizonts D Artikel benötigt werden, sind D/x Lieferungen erforderlich. Dies führt zu Transportkosten von $u \cdot$D/x. Es sei darauf hingewiesen, dass dasselbe Modell auch für die Planung der Produktion (anstelle der externen Beschaffung) angewandt werden könnte, wobei sich u dann auf die Rüstkosten eines Produktionsprozesses bezieht.

Die Gesamtkosten der Beschaffung sind die gesamten Lagerkosten plus die gesamten Bestellkosten, d. h. $u \cdot D/x + x/2 \cdot h \cdot T$. Die Kosten für die Materialien selbst werden nicht berücksichtigt, da wir davon ausgehen, dass immer insgesamt die Menge D zu einem bestimmten Preis bestellt wird, unabhängig von größeren oder kleineren Bestellmengen. Die gleiche Überlegung gilt für den Fall der Eigenproduktion, wobei die variablen Produktionskosten als konstant angenommen werden, so dass es für sie unerheblich ist, ob kleinere oder größere Losgrößen produziert werden.

Da wir die Gesamtkosten der Beschaffung minimieren wollen, können wir einfach die erste und zweite Ableitung dieser Kostenfunktion (nach der Variablen x) überprüfen und erhalten so die folgende bekannte Quadratwurzelformel für x, die die notwendige und hinreichende Bedingung für ein Minimum erfüllt (x_o ist die wirtschaftliche Bestellmenge, d. h. der Wert für x, der zu den niedrigsten Beschaffungskosten führt):

$$x_o = \sqrt{2Du/hT} \tag{4.1}$$

In der Literatur gibt es andere Versionen dieser Formel, die auf etwas anderen Formulierungen des Bestandsmodells beruhen. So kann man beispielsweise die Lagerhaltungskosten in Form eines Lagerkostensatzes pro Jahr ausdrücken, der mit h^r bezeichnet wird, ähnlich wie ein Zinssatz. Dabei wird angenommen, dass die Kosten aufgrund der Kapitalbindung die dominierende Komponente der Lagerkosten sind. Die Lagerhaltungskosten eines Jahres lassen sich dann durch Multiplikation von l mit dem durchschnittlichen Lagerbestand multipliziert mit dem Einkaufspreis für eine Einheit des Materials, bezeichnet mit p, ausdrücken:

$$x_o = \sqrt{2Du/ph^r} \tag{4.2}$$

wobei u die Fixkosten pro Lieferung sind (wie zuvor) und D die benötigte Menge des Materials pro Jahr ist.

Das obige Bestandsmodell lässt sich durch die folgenden drei Annahmen zusammenfassen: konstante Bestandsverbrauchsraten über die Zeit, Lagerkosten proportional zu den Bestandsniveaus und Beschaffungskosten proportional zur Anzahl der Beschaffungsvorgänge (d. h. konstante Kosten für jeden Beschaffungsvorgang, wie z. B. Rüstkosten für eine entsprechende Produktion oder feste Transportkosten im Falle der Fremdbeschaffung). Dieses Modell geht auf die grundlegenden Beiträge

von Harris (1913) und Andler (1929) zurück. Es wird auch als Losgrößenmodell mit festen Rüstkosten und konstanten Lagerstückkosten, als Uncapacitated Single Item Lot-Sizing Problem (USILSP) oder als Formel zur Berechnung der wirtschaftlichen Losgröße oder der wirtschaftlichen Bestellmenge (EOQ) bezeichnet.

Es scheint einfach zu sein, diese Ergebnisse zu verwenden, die in einer recht einfachen Gleichung für die wirtschaftlichen Bestellmengen ausgedrückt werden können. Leider sind die Annahmen des Modells in den meisten Fällen nicht sehr realistisch, so dass es nicht geeignet zu sein scheint. Einige der unrealistischen Annahmen könnten recht einfach behoben werden. So ist es z. B. unkompliziert, Lieferfristen, Meldebestände oder bestimmte Sicherheitsbestände (Mindestbestände) zu berücksichtigen. In anderen Fällen sind schwierigere Modelle und Berechnungen erforderlich.

Betrachten wir nun einige der einfachen Erweiterungen des Modells: Wir gehen von einer Vorlaufzeit von LT-Perioden aus, d. h. es dauert LT-Perioden von der Bestellung des Materials bis zu seinem Eingang. d sei die (durchschnittliche) Nachfrage pro Periode, d. h. $d = D/T$. Dann ist der Meldebestand (reorder point) RP gegeben durch

$$RP = dLT, \tag{4.3}$$

d. h. wenn der Bestand dieses Niveau erreicht, ist eine Nachschubbestellung für das Material erforderlich.

Eine weitere Standarderweiterung des Modells ist die Berücksichtigung von Sicherheitsbeständen. Ein Sicherheitsbestand kann als Mindestbestand aufgrund von Nachfrageschwankungen oder anderen unvorhergesehenen Umständen beschrieben werden. Wenn ein solcher Sicherheitsbestand (safety stock) SS angewendet werden soll, ist der Meldebestand

$$RP = dLT + SS. \tag{4.4}$$

Diese Formulierung führt bereits über das einfache Bestandsmodell hinaus, das von einer konstanten und damit sicheren Nachfrage pro Periode ausgeht. Bei den meisten Unternehmen ist die Kundennachfrage und auch die interne Nachfrage jedoch weder konstant noch sicher.

In diesem Fall ist die nicht konstante Nachfrage das kleinere Problem. Die Annahme eines durchschnittlichen Lagerbestands von $x/2$ kann immer noch ein guter Schätzwert für die tatsächliche Situation sein, und bekannte Nachfrageschwankungen können bei der Berechnung von Meldebeständen gut berücksichtigt werden. Die Berücksichtigung der stochastischen Nachfrage ist schwieriger. Das gebräuchlichste Modell zur Berücksichtigung dieses Sachverhalts sieht wie folgt aus. Es wird davon ausgegangen, dass das Unternehmen eine Art Vorhersagemodell für die zukünftige Nachfrage verwendet. Dieses muss nicht sehr ausgefeilt sein. Auch die Annahme einer konstanten zukünftigen Nachfrage würde

funktionieren. Unter der Annahme, dass die tatsächliche Nachfrage normalverteilt ist, lässt sich der Vorhersagefehler für die Nachfrage berechnen

$$\sigma_d = \frac{1}{n}\sum_{t=1}^{n}\left(d_t - \hat{d}_t\right)^2 \tag{4.5}$$

wobei d_t die tatsächliche Nachfrage im Zeitraum t, $\hat{d}_t$ die vorhergesagte Nachfrage im Zeitraum t und n die Anzahl der betrachteten Perioden in der Vergangenheit ist. Auf ähnliche Weise kann der Vorhersagefehler für die Vorlaufzeiten bestimmt werden (siehe z. B. Chopra et al. (2004) für eine ausführlichere Diskussion):

$$\sigma_{LT} = \frac{1}{m}\sum_{t=1}^{m}\left(LT_t - \overline{LT}\right)^2 \tag{4.6}$$

wobei LT_t die Durchlaufzeit der *t-ten* vergangenen Lieferung, $\overline{LT}$ die durchschnittliche (oder geplante) Vorlaufzeit und m die Anzahl der berücksichtigten vergangenen Lieferungen ist. Der Meldebestand RP kann dann wie folgt berechnet werden:

$$RP = \overline{d}\,\overline{LT} + Z\sqrt{\overline{LT}\sigma_d^2 + \overline{d}^2\sigma_{LT}^2} \tag{4.7}$$

Die in dieser Gleichung verwendeten Parameter lauten wie folgt:

$\overline{LT}$: Durchschnittliche Vorlaufzeit
$\overline{d}$: Durchschnittliche Nachfrage pro Periode
Z: Service-Level (Lieferbereitschaftsgrad)
σ_d: Standardabweichung der Nachfrage
σ_{LT}: Standardabweichung der Vorlaufzeit (Leadtime)

Aufgrund der strengen Annahmen dieses Modells ist es in der Praxis oft nicht brauchbar. So können beispielsweise die Beschaffungskosten von den bestellten Mengen in Form einer konstanten oder linearen Funktion mit Sprüngen abhängen. In solchen Situationen lassen sich optimale Ergebnisse nicht einfach durch eine Formel berechnen. Stattdessen sind komplexere Optimierungsmodelle abzuleiten, die z. B. durch Metaheuristiken gelöst werden können.

Eine realistischere, aber auch komplexere Variante des Losgrößenmodells ist seine dynamische Version mit der Annahme einer gegebenen zeitlich variierenden Nachfrage für die kommenden Perioden. Dieses Modell wurde erstmals von Wagner und Whitin (1958) untersucht, die es in einer Version mit dynamischer Programmierung formulierten und einen entsprechenden Algorithmus für seine Lösung vorschlugen. Für eine einfachere Formulierung und auch für die Berücksichtigung von Varianten des dynamischen Losgrößenproblems wurden verschiedene spezifische Heuristiken entwickelt, wie z. B. die Heuristik von Silver und Meal (1973).

4.3 Losgrößenprobleme mit beschränkten Kapazitäten

Das Losgrößenproblem wird deutlich komplexer, wenn zusätzliche Nebenbedingungen berücksichtigt werden müssen, insbesondere Kapazitätsbeschränkungen für die benötigten Ressourcen. Das Single-Item-Capacitated-Lot-Sizing-Problem (SICLSP) kann wie folgt definiert werden (siehe z. B. Mateus et al., 2010):

$$\min \sum_{t=1}^{T} \left(c_t x_t + h_t i_t + u_t y_t \right) \tag{4.8}$$

$$i_{t-1} + x_t = d_t + i_t, t \in \{1, \ldots, T\} \tag{4.9}$$

$$ax_t + by_t \leq w_t, t \in \{1, \ldots, T\} \tag{4.10}$$

$$ax_t \leq w_t y_t, t \in \{1, \ldots, T\} \tag{4.11}$$

$$y_t \in \{0, 1\} \quad t \in \{1, \ldots, T\}, \tag{4.12}$$

$$x_t, i_t \in N, \quad t \in \{1, \ldots, T\} \tag{4.13}$$

Dabei ist T der Planungshorizont, der die Perioden $\{1, \ldots, T\}$ umfasst, c_t sind die Stückkosten der Produktion, h_t die Stückkosten (pro Periode) der Lagerhaltung und u_t die Rüstkosten für die Produktion. Die Entscheidungsvariablen sind x_t, die produzierten Mengen in der Periode t, und die Variablen y_t, die 1 sind, wenn in t produziert wird, und 0 andernfalls. Die Lagerbestände i_t sind weitere Entscheidungsvariablen, die jedoch durch die Gleichung der Lagerbestände (4.9) bestimmt werden, die die Beziehung zwischen den früheren Beständen, der Produktion, der Nachfrage und den daraus resultierenden neuen Beständen angibt. Möglicherweise sind ein Anfangsbestand i_0 und ein gewünschter Endbestand i_T gegeben. Weitere Parameter für das Problem sind die Nachfragewerte d_t. Das Ziel ist die Minimierung der Kosten (4.8), die Produktions-, Lager- und Rüstkosten umfassen. Gl. (4.10) spezifiziert die Kapazitätsbeschränkung, die davon ausgeht, dass die Produktion jeder Einheit eine Kapazität (wie Zeit) von a erfordert, während die Einrichtung der Produktion eine Kapazität von b erfordert, die durch die gesamte verfügbare Kapazität w_t in jeder Periode t gedeckt werden muss. Gl. (4.11) soll sicherstellen, dass y_t gleich 1 ist, wenn $x_t > 0$, und andernfalls gleich 0.

Weitere Einschränkungen Gl. (4.12) und (4.13) betreffen den Bereich der Variablen, die boolesche Werte für y_t und natürliche Zahlen (nichtnegative ganze Zahlen) für x_t und i_t sind. Insbesondere dürfen diese Werte nicht negativ werden. Alternativ könnte man annehmen, dass die letzten Variablen nichtnegative reelle Werte annehmen.

Ohne Gl. (4.10) wird das Problem als unkapazitiertes Single Item-Losgrößenproblem (uncapacitated single item lot-sizing problem) bezeichnet, das in kleiner polynomieller Zeit gelöst werden kann, während die zusätzliche Kapazitäts-

beschränkung das Problem NP-schwer macht (Brahimi et al., 2006; Florian et al., 1980) und somit ein interessanter Kandidat für die Anwendung von Metaheuristiken ist.

Wegen ihrer praktischen Bedeutung werden häufig Versionen von Losgrößenproblemen für mehrere Materialien betrachtet. Eine allgemeine Erweiterung der obigen SICLSP-Gleichungen (4.8)–(4.13) ist dann das folgende Losgrößenproblem (multi-item capacitated lot-sizing problem, MICLSP):

$$\min \sum_{p=1}^{P}\sum_{t=1}^{T}\left(c_{pt}x_{pt} + h_{pt}i_{pt} + u_{pt}y_{pt}\right) \tag{4.14}$$

$$i_{p(t-1)} + x_{pt} = d_{pt} + i_{pt}, t \in \{1, \ldots, T\}, p \in \{1, \ldots, P\} \tag{4.15}$$

$$\sum_{p=1}^{P}\left(a_p x_{pt} + b_p y_{pt}\right) \le w_t, t \in \{1, \ldots, T\} \tag{4.16}$$

$$a_p x_{pt} \le w_t y_{pt}, t \in \{1, \ldots, T\}, p \in \{1, \ldots, P\} \tag{4.17}$$

$$y_{pt} \in \{0, 1\} \quad t \in \{1, \ldots, T\}, p \in \{1, \ldots, P\} \tag{4.18}$$

$$x_{pt}, i_{pt} \in N, \quad t \in \{1, \ldots, T\}, p \in \{1, \ldots, P\} \tag{4.19}$$

Dabei wird angenommen, dass es P Produkte mit Indizes in {1, …, P} gibt, die während des Planungshorizonts T zu planen sind. Die Zielfunktion (4.14) ist wiederum die Minimierung der Kosten, die sich aus den Produktions-, Lager- und Rüstkosten für alle Produkte und alle Perioden zusammensetzen. Die Bestandsgleichungen (4.15) und die logische Bedingung für die y-Variablen (4.17) sind nun für alle Produkte zu beachten. Für die Kapazitätsbeschränkung (4.16) wird der Kapazitätsverbrauch durch Produktion und Rüsten für alle Produkte aufsummiert, d. h. wir gehen weiterhin davon aus, dass es eine einzige Kapazitätsbeschränkung für alle Produkte gibt. Eine ähnliche Version des MICLSP mit zusätzlichem Backordering findet sich z. B. in Mateus et al. (2010).

Man beachte, dass die unkapazitierte Version, d. h. das obige Problem ohne Gl. (4.16), noch polynomial ist, da es durch die Lösung eines unkapazitierten Losgrößenproblems für nur ein Produkt separat für jedes Produkt gelöst werden kann. Das MICLSP (4.14)–(4.19) ist natürlich NP-schwer, genau wie das SICLSP, das ein Spezialfall davon ist.

Typischerweise sind die in einem Unternehmen zu planenden Materialien in eine hierarchische Produktionsstruktur eingebettet, d. h. die zu produzierenden Endartikel erfordern Komponenten, die ebenfalls produziert oder beschafft werden müssen. Die Beziehungen zwischen den Materialien werden in den Stücklisten beschrieben, die in der Regel von einer festen Menge verschiedener Komponenten ausgehen, die zur Herstellung einer Einheit des betreffenden Artikels erforderlich sind (siehe Abb. 4.2 als Beispiel). In der Regel wird davon ausgegangen, dass der resultierende

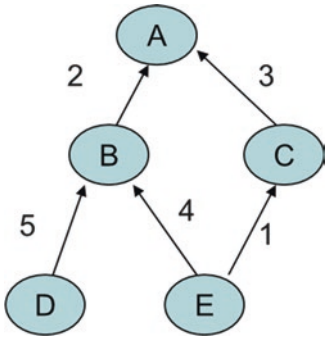


Abb. 4.2 Beispiel einer Stückliste (Bill of Materials. BOM). Die *Zahlen neben den Pfeilen geben* die jeweiligen Produktionskoeffizienten an. Zur Herstellung von 1 Stück A werden 2 Stücke B, 3 Stücke C, 10 Stücke D und 11 Stücke E benötigt

Graph azyklisch ist. (Besondere Situationen mit zyklischer Produktionsstruktur können in der Chemie- oder Lebensmittelindustrie auftreten, wo wir auch gekoppelte Produktion finden, d. h. mehrere, manchmal unerwünschte Produkte, die aus einem einzigen Produktionsprozess hervorgehen.)

Infolge einer hierarchischen Produktionsstruktur besteht die Nachfrage nach einem Artikel nicht nur aus der externen Nachfrage (auch als Primärnachfrage bezeichnet), die als gegeben angenommen wird (z. B. durch Prognosemodelle), sondern zusätzlich aus der internen Nachfrage (Sekundärnachfrage), die sich daraus ergibt, dass wir planen, nachgeordnete Artikel in der Produktionsstruktur zu produzieren, die der betrachtete Artikel benötigt.

Eine einfache Variante eines entsprechenden Planungsproblems ist das Multi-Level-Uncapacitated-Lot-Sizing-Problem (MLULSP, manchmal auch einfach als Multi-Level-Lot-Sizing-Problem oder MLLSP bezeichnet), das wie folgt definiert werden kann: Die Produktion von P Artikeln soll geplant werden, die durch eine gegebene Produktionsstruktur verbunden sind, so dass jede externe Nachfrage während T Perioden befriedigt wird. Das Ziel Gl. (4.20) ist dabei die Minimierung der Gesamtkosten, die sich aus Lagerhaltungs- und Rüstkosten zusammensetzen (siehe z. B. Homberger, 2010).

$$\min \sum_{p=1}^{P} \sum_{t=1}^{T} \left(h_p i_{pt} + u_p y_{pt} \right) \tag{4.20}$$

$$i_{pt-1} + x_{pt} = d_{pt} + i_{pt},\ p \in \{1, \ldots, P\},\ t \in \{1, \ldots, T\} \tag{4.21}$$

$$d_{pt} = \sum_{j=1}^{P} v_{pj} x_{jt} + ed_{pt},\ p \in \{1, \ldots, P\},\ t \in \{1, \ldots, T\} \tag{4.22}$$

$$x_{pt} \leq M y_{pt},\ t \in \{1, \ldots, T\},\ p \in \{1, \ldots, P\} \tag{4.23}$$

$$y_{pt} \in \{0, 1\} \tag{4.24}$$

$$x_{pt}, i_{pt} \in N,\ t \in \{1, \ldots, T\} \tag{4.25}$$

Im Vergleich zum MICLSP besteht der Hauptunterschied in der Berechnung der Gesamtnachfrage. Es wird davon ausgegangen, dass eine externe Nachfrage ed_{pt} nach der Ware p in der Periode t gegeben ist. v_{pj} sind die Produktionskoeffizienten, die angeben, wie viele Einheiten der Ware p für die Produktion einer Einheit der Ware j erforderlich sind. Wenn eine Ware j kein Nachfolger von p ist, dann ist $v_{pj} = 0$. Die Gesamtnachfrage nach Artikel p in der Periode t ergibt sich dann aus Gl. (4.22), in der die externe und die interne Nachfrage addiert werden, die sich aus der Multiplikation der Nachfolger-Produktionsmengen mit den Produktionskoeffizienten ergeben. Anstatt über alle Güter zu summieren, könnte man auch einfach die Summe über alle Nachfolger des Gutes p bilden.

Weitere Unterschiede zum MICLSP bestehen darin, dass die Produktionskosten vernachlässigt werden, was möglich ist, wenn die Produktionsstückkosten in jeder Periode für jedes Produkt identisch sind. (Eine Version des Problems, die die Produktionskosten einbezieht, wird z. B. in Dellaert et al. (2000) behandelt). Die fehlende Kapazitätsbeschränkung führt zu einer Umformulierung von Gl. (4.23) im Vergleich zu Gl. (4.17), bei der die Kapazitätsgrenze durch eine hinreichend große Zahl M ersetzt wird. Häufig werden die ganzzahligen Annahmen in Gl. (4.25) weggelassen, wobei weiterhin angenommen wird, dass x_{pt} und i_{pt} nicht negativ sind.

Wie Arkin et al. (1989) gezeigt haben, ist das MLULSP NP-schwer für Produktionsstrukturen, bei denen jeder Artikel mehr als einen Nachfolger und Vorgänger haben kann. Die Einführung von Kapazitätsbeschränkungen ist eine weitere, praktisch relevantere Version des Problems und auch eine Erweiterung des MICLSP. Dieses Problem wird manchmal als allgemeines Losgrößenproblem mit Kapazitäten (General Capacitated Lot-Sizing Problem, GCLSP) bezeichnet, aber es gibt mehrere Varianten dieses Problems mit zusätzlichen Nebenbedingungen und anderen Modifikationen, z. B. die Berücksichtigung von Überstundenkapazitäten und -kosten (siehe z. B. Xie & Dong, 2002).

Eine weitere Erweiterung des MICLSP ist das coordinated capacitated lot-sizing problem (CCLSP), das zusätzlich individuelle und gemeinsame Rüstkosten bei der Produktion in einer Periode berücksichtigt (Robinson et al., 2009). Während das CCLSP NP-schwer ist, gilt dies auch für seine in Bezug auf Kapazitäten erleichterte Version, das koordinierte unkapazitierte Losgrößenproblem (CULSP) (Arkin et al., 1989).

4.4 Lösungsansätze für kapazitätsbezogene Losgrößenprobleme

Wie wir gesehen haben, sind Losgrößenprobleme mit beschränkten Kapazitäten im Allgemeinen, aber auch einige Varianten ohne Berücksichtigung von Kapazitäten, NP-schwer und daher sind Metaheuristiken interessante Methoden, um sie zu lösen. Natürlich gibt es auch viele Beiträge, die sich auf exakte Ansätze wie Branch & Bound, Branch & Cut und Lagrange-Relaxation konzentrieren oder eher einfache

problemabhängige Heuristiken verwenden. Weitere Komplikationen ergeben sich bei der Betrachtung eines Problems für mehrere Materialien (möglicherweise mit gemeinsamen Ressourcen für die Fertigung verschiedener Materialien) oder bei der Betrachtung mehrstufiger Situationen (wie in Abb. 4.2). Abgesehen von der Komplexität können diese Probleme recht groß werden, da die Anzahl der Variablen ein Produkt aus der Anzahl der Artikel (einschließlich der Vorgänger in der Produktionsstruktur oder der zu beschaffenden Materialien) und der Anzahl der Planungsperioden ist. Es sei darauf hingewiesen, dass es verschiedene weitere Varianten von Losgrößenproblemen gibt, von denen einige das Problem mit anderen betrieblichen Problemen wie dem Scheduling verknüpfen (Karimi et al., 2003).

Bei den oben diskutierten Losgrößenproblemen ist die Repräsentation von Lösungen in einer Metaheuristik eine wichtige Entscheidung für den Erfolg. Für die Entscheidungsvariablen ist es oft ein einfacher und effizienter Weg, sie durch entsprechende Datenstrukturen darzustellen, z. B. Größen wie x_{pt} als Ganzzahl, wenn sie als ganzzahlig angenommen werden, als Fließkommazahl, wenn sie als reelle Werte angenommen werden, oder als binäre Variablen in der verwendeten Programmiersprache im Falle von binären Variablen wie y_{pt}. Da es eine gewisse Redundanz zwischen den Variablen gibt (y_{pt} ist 1, wenn $x_{pt} > 0$ und ansonsten 0), könnten wir in Erwägung ziehen, die y-Variablen in einer Lösungsrepräsentation auszulassen. Dann sollten wir jedoch überlegen, wie es hinreichend wahrscheinlich wird, 0-Werte für die Variable x_{pt} zu erhalten. Wenn wir beispielsweise diese Variablen während des Algorithmus durch zufällige reelle Werte ändern (wie z. B. bei verschiedenen evolutionären Algorithmen), ist es eher unwahrscheinlich, genau den neuen Wert 0 zu erreichen. Wenn wir die y-Variablen beibehalten, müssen wir natürlich sicherstellen, dass solche Bedingungen wie Gl. (4.17) oder (4.23) eingehalten werden. Dies könnte durch einen Abgleich zwischen den x_{pt} und den y_{pt} Werten erreicht werden. Zum Beispiel werden zunächst die Variablen y_{pt} bestimmt. Wenn $y_{pt} = 0$ ist, wird auch x_{pt} auf 0 gesetzt. Andernfalls muss sichergestellt werden, dass x_{pt} auf einen positiven Wert gesetzt wird.

Für die Beachtung der anderen Nebenbedingungen sind im Allgemeinen verschiedene Ansätze geeignet. Abgesehen von allgemeinen Ansätzen wie der Neuberechnung von Variablenwerten oder der Verwendung eines Straffunktionsansatzes (Bestrafung von verletzten Nebenbedingungen in einer modifizierten Zielfunktion) haben sich spezifische Reparaturverfahren oft als überlegen erwiesen. Für die Losgrößenprobleme gibt es einige recht offensichtliche Ansätze. Wenn Kapazitätsbeschränkungen wie Gl. (4.16) verletzt werden, könnte es ein einfacher Ansatz sein, die zugehörige Produktionsmenge x_{pt} für ein oder mehrere Produkte zu reduzieren (möglicherweise können Zufallsfaktoren entscheiden, welche Produkte zu reduzieren sind). Lagerbestandsgleichungen wie (4.15) können hauptsächlich dazu verwendet werden, die Lagerbestände von Periode zu Periode zu aktualisieren, wenn d_{pt} gegeben ist und x_{pt} festgelegt wird. Ein eventuell entstehender negativer Lagerbestand in einer Periode könnte durch eine Erhöhung der Produktion in der betreffenden Periode „repariert“ werden. Ist dies nicht möglich (wegen Kapazitätsengpässen) oder erwünscht (weil $y_{pt} = 0$ ist und nach Möglichkeit nicht verändert werden soll), könnte die Produktion in den vorangegangenen Perioden erhöht

werden. Natürlich könnte dies aufgrund weiterer erschöpfter Kapazitäten in den Vorperioden (oder am Anfang des Planungshorizonts) auch nicht möglich sein. Bei einer Version des Losgrößenproblems mit beschränkten Kapazitäten ist es immer möglich, dass es keine zulässige Lösung gibt, d. h. die Nachfrage kann nicht in jeder Periode vollständig befriedigt werden. Daher könnte es eine gute Idee sein, die nicht befriedigte Nachfrage als Strafterm für die Zielfunktion zu berücksichtigen. Gelegentlich finden wir diesen Gedanken bereits in der jeweiligen mathematischen Problemformulierung. Unbefriedigte Nachfrage wird dann in der Regel als Fehlmengenkosten ausgedrückt, die zu einer Komponente der Zielfunktion werden (siehe z. B. Absi & Kedad-Sidhoum, 2008). Eine weitere Möglichkeit, mit den Nebenbedingungen umzugehen, zeigen Haase und Kohlmorgen (1996), die eine (reellwertige) Random Key-Kodierung zur Darstellung von Lösungen im Rahmen eines genetischen Algorithmus verwenden. Die Koeffizienten des Random Key-Vektors helfen dabei, den Beginn der Produktionstätigkeiten im Einklang mit den Kapazitätsbeschränkungen zu bestimmen und die angenommene Nachfrage zu decken.

Unter den Metaheuristiken, die zur Lösung von Losgrößenproblemen eingesetzt werden, finden sich die folgenden Methoden:

- genetische Algorithmen (Akrami et al., 2006; Dellaert et al., 2000; Gutierrez et al., 2001; Gaafar, 2006; Homberger, 2008; Kämpf & Köchel, 2006; Haase & Kohlmorgen, 1996; Woarawichai et al., 2011)
- Tabu-Suche (Akrami et al., 2006; Berretta et al., 2005; Gopalakrishnan et al., 2001; Xie & Dong, 2002)
- Nachbarschaftssuche (Almada-Lobo & James, 2010; Clark, 2000; Muller et al., 2012)
- memetische Algorithmen (Berretta & Rodrigues, 2004; Torabi et al., 2006)
- hybride Metaheuristiken (Berretta et al., 2005; Staggemeier et al., 2002; Mishra et al., 2011)
- Particle Swarm Optimization (Dye & Hsieh, 2013)
- Simulated Annealing (Homberger, 2010)
- GRASP mit Path-Relinking (Nascimento et al., 2010),
- Ant Colony Optimization (Ameisenkolonie-Optimierung) (Pitakaso et al., 2006)
- eine auf Kreuzentropie basierende Metaheuristik (Caserta & Rico, 2009)

Es sei darauf hingewiesen, dass es auch problemspezifisch entwickelte Heuristiken gibt, wie z. B. in França et al. (1997). Neuere Übersichten über mathematische und metaheuristische Ansätze finden sich bei Jans und Degraeve (2007), Buschkühl et al. (2010) und Goren et al. (2010). Die letzte Übersicht konzentriert sich auf genetische Algorithmen als eine der am häufigsten verwendeten Methoden für komplexe Losgrößenprobleme.

Anhand der veröffentlichten Ergebnisse ist es nicht einfach zu beurteilen, wie Metaheuristiken im Vergleich abschneiden. In vielen der Studien werden keine Methodenvergleiche durchgeführt. Eine weitere Schwierigkeit besteht darin, dass es für die meisten der betrachteten Problemtypen keine allgemein akzeptierten Sammlungen von Testproblemen gibt, die zum Benchmarking der verschiedenen

vorgeschlagenen Methoden herangezogen werden könnten. Neuere Veröffentlichungen zeigen, dass Testprobleme mittlerweile häufig wiederholt verwendet werden und sich als Benchmark-Instanzen etablieren.

In Akrami et al. (2006) zeigen sowohl ein genetischer Algorithmus als auch Tabu-Suche deutlich bessere Ergebnisse für zufällig generierte Testinstanzen eines kombinierten Losgrößen- und Scheduling-Problems im Vergleich zu einer „optimalen Aufzählungsmethode“. In Almada-Lobo und James (2010) werden variable Nachbarschaftssuche (Variable Neighborhood Search) und Tabu-Suche mit einer Standard-Optimierungssoftware (CPLEX) verglichen, die auf einem Branch & Bound-Verfahren basiert. Bei größeren, zufällig generierten Testinstanzen schneiden beide Metaheuristiken besser ab als die Standardmethode, wobei die Tabu-Suche bei den betrachteten Problemgrößen meist überlegen bleibt. Unter Verwendung einer Reihe von Testproblemen (mit unterschiedlichen Eigenschaften) wird in Berretta und Rodrigues (2004) ein memetischer Algorithmus mit optimalen CPLEX-Lösungen, mit den problemspezifischen Heuristiken in França et al. (1997) und mit denen einer anderen Lagrange-basierten Heuristik verglichen. Im Allgemeinen zeigte das Ergebnis, dass der memetische Algorithmus in der Lage war, bessere Ergebnisse zu berechnen als die verglichenen früheren Heuristiken. Ähnliche Ergebnisse für eine hybride Metaheuristik wurden in Berretta et al. (2005) und für genetische Algorithmen in Staggemeier et al. (2002) und Woarawichai et al. (2011) gezeigt. Dellaert et al. (2000) erzielen gute Ergebnisse für einen GA im Vergleich zu zwei einfachen Heuristiken und einer zeitbeschränkten Branch & Bound-Implementierung. Darüber hinaus werden in Gaafar (2006) bessere Ergebnisse im Vergleich zu einer modifizierten Silver-Meal-Heuristik gezeigt. Weitere Verbesserungen des genetischen Algorithmus werden in Homberger (2008) untersucht und mit anderen GA-Varianten und einem Ansatz basierend auf Ant Colony System (ACS) verglichen. Zuvor hat dieser ACS-Ansatz (Pitakaso et al., 2006) im Vergleich zu einigen anderen heuristischen Ansätzen in der Literatur überlegene Ergebnisse gezeigt. In Homberger (2008) schneidet der neue GA sehr gut ab und ist in der Lage, einige neue beste Lösungen für einige etablierte Testinstanzen zu berechnen. Weitere konkurrenzfähige Ergebnisse unter einem ähnlichen Bewertungsrahmen werden für Simulated Annealing in Homberger (2010) erzielt. In Nascimento et al. (2010) führen Experimente mit Ansatz basierend GRASP mit Path-Relinking zu einigen neuen besten Lösungen im Vergleich zu denen aus der Literatur für das Problem der Losgrößenbestimmung mit beschränkten Kapazitäten.

4.5 Planung des Lagerbetriebs

Neben der Beantwortung der Frage, wie viel produziert und vorrätig gehalten werden sollte, wurde in der wissenschaftlichen Literatur der Planung bestimmter Vorgänge im Zusammenhang mit der Lagerhaltung und Bestandsverwaltung große Aufmerksamkeit geschenkt. Eine dieser Tätigkeiten ist die Kommissionierung. (Die Einlagerung von Materialien in das Lager wirft ähnliche Probleme auf, ist aber in

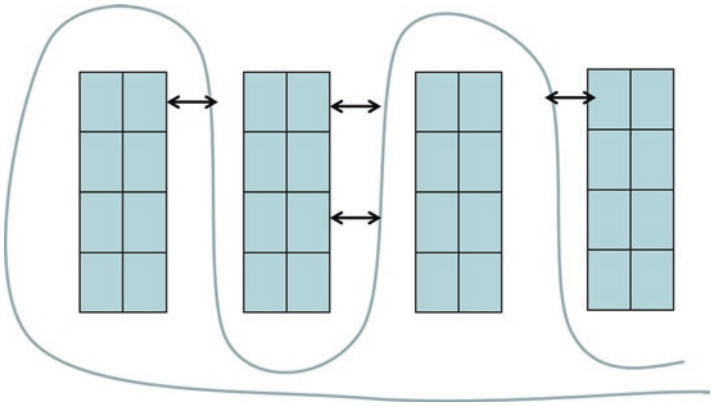


Abb. 4.3 Beispiel für einen Rundgang durch die Lagergänge mit vier Entnahmen von Artikeln

der Regel weniger zeitkritisch und wird daher in Optimierungsansätzen nicht so häufig berücksichtigt.) Grundsätzlich kann die Kommissionierung als eine bestimmte Art von Transport, kombiniert mit einigen zusätzlichen Aktivitäten, betrachtet werden. Zum Beispiel geht ein Mitarbeiter durch das Lager zu bestimmten Lagerplätzen mit Artikeln, die für einen Auftrag benötigt werden, entnimmt diese und liefert sie an einen vordefinierten Ort (siehe Abb. 4.3).

Diese Art von Problem ist ein typisches Round-Trip-Problem wie das Vehicle Routing-Problem mit Kapazitätsbeschränkung (CVRP) und kann mit ähnlichen Techniken gelöst werden (siehe oben). Kapazitätsgrenzen werden durch die physikalischen Beschränkungen beim Transport von Lasten verursacht oder ergeben sich aus Kapazitätsbeschränkungen der verwendeten Hilfsmittel (z. B. einem Behälter oder einem Kommissionierwagen). Aufgrund der Kapazitätsbeschränkungen müssen große Aufträge möglicherweise aufgeteilt werden, während kleinere Aufträge zusammengefasst und in einer einzigen Tour ausgeführt werden können (wenn dies aus organisatorischer Sicht unterstützt wird). Es gibt jedoch Unterschiede gegenüber dem Standard-CVRP. Erstens kann ein benötigter Artikel an mehreren Lagerorten verfügbar sein, was weitere Optimierungsmöglichkeiten zulässt. Zweitens spielt die Reihenfolge der Auftragserfüllung eine Rolle, insbesondere wenn Aufträge gleichzeitig oder überlappend bearbeitet werden.

In Lagern werden automatisierte Systeme wie automatische Regalbediengeräte (RBG) häufig für die Kommissionierung oder verwandte Tätigkeiten wie die Versorgung von „Ware-zum-Mann“-Kommissionierplätzen eingesetzt. Bei einem Regalbediengerät handelt es sich in der Regel um ein computergesteuertes Gerät, das auf einer Schiene im Gang zwischen zwei Regalen läuft. Es ist mit einem Mast ausgestattet, der die Höhe der Regale abdeckt und an dem sich ein Shuttle vertikal bewegt, um definierte Lagerplätze anzufahren und Lasten aus dem Regal aufzunehmen oder dort abzulegen (siehe Abb. 4.4). Moderne Regalbediengeräte haben oft eine Kapazität von mehr als 1, so dass mehrere Lasten gleichzeitig transportiert werden können. Dies erlaubt es auch, ihre Operationen als Rundfahrtprobleme zu

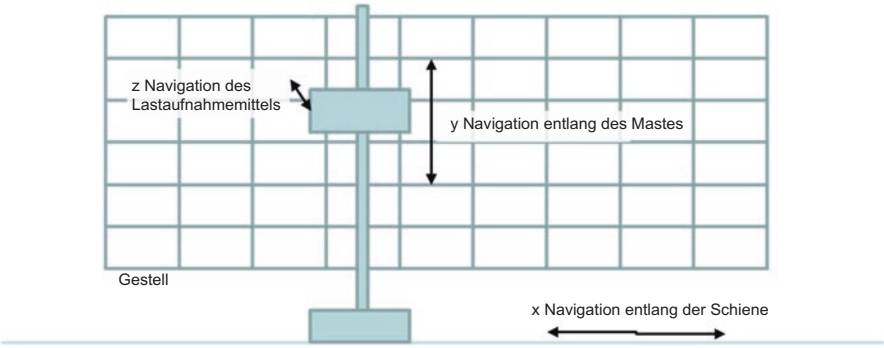


Abb. 4.4 Schematischer Aufbau eines Regalbediengerätes (RBG)

formulieren. Manchmal haben sie nicht nur einen einzigen Punkt für die Interaktion mit anderen logistischen Systemen, so dass mehr Freiheiten in Bezug auf die Start- und Zielorte einer Fahrt berücksichtigt werden können. Auch hier ist die Reihenfolge der einzelnen Transportaufträge und deren Verschachtelung wichtig für die Optimierung der Abläufe. Insbesondere die Kombination von Einlagerungs- und Auslagerungsbewegungen trägt dazu bei, Leerfahrten zu vermeiden.

4.6 Lagerorte

Ein weiterer Aspekt, der die Effizienz der Kommissionierung und anderer Tätigkeiten maßgeblich beeinflusst, ist die gezielte Festlegung von Lagerplätzen, auch Lagerplatzzuordnungsproblem genannt. Für die Bestimmung eines geeigneten Lagerplatzes wurden traditionell die physikalischen Eigenschaften der Waren sowie die Warengruppen berücksichtigt. Die Lagerung ähnlicher Produkte in unmittelbarer Nähe zueinander ist sinnvoll, da der Kunde die gewünschten Produkte so leichter finden kann. Dieses Konzept ist im Einzelhandel, wo die Kunden ihre Einkäufe selbst zusammenstellen, immer noch vorherrschend. In Lagern, in denen die Kommissionierung durch qualifiziertes Personal erfolgt, wird die Aufgabe des Auffindens der Produkte heute in der Regel durch die automatisierte Bereitstellung von Informationen über die spezifischen Standorte unterstützt, so dass eine feste Zuweisung von Lagerplätzen nicht mehr erforderlich ist. Stattdessen können die Lagerplätze willkürlich und nach dem Zufallsprinzip zugewiesen werden, solange die Lagerplätze die physikalischen Anforderungen an die Ware (Größe, Gewicht etc.) erfüllen. Eine rein zufällige Zuweisung von Lagerorten ist natürlich nicht unbedingt vorteilhaft. Stattdessen ist es sinnvoll, die Lagerorte so zuzuweisen, dass die entsprechenden Tätigkeiten wie die Kommissionierung effizienter werden. Eine einfache Regel könnte sein, häufig benötigte Artikel an leicht (schnell) zugänglichen Orten zu lagern, während weniger häufig benötigte Güter an weniger günsti-

gen Orten gelagert werden können. Bei einer solchen Überlegung spielt auch der Platzbedarf pro Artikel eine Rolle.

Zunächst einmal gibt es viele spezifische Heuristiken zur Lösung eines solchen Problems, siehe z. B. Goetschalckx und Ratliff (1990). Die bekannteste dieser Heuristiken und Regeln ist die Heuristik des Cube-per-Order-Index (COI) von Heskett (1963). Der COI ist das Verhältnis zwischen dem Volumen eines Artikels (Platzbedarf im Lager) und seiner Nachfragemenge (Umschlag pro Periode). Produkte mit kleinem COI sollten näher am Versandbereich und solche mit größerem COI weiter entfernt gelagert werden.

Bessere Ergebnisse sind zu erwarten, wenn die Lagerplatzzuweisung als Optimierungsproblem (Stock Location Assignment Problem, SLAP) gelöst wird. In der Literatur werden zwei grundlegende Ansätze untersucht: Die individuelle Zuordnung von Lagerplätzen für Güter und die Zuordnung von Gütern zu Klassen ähnlicher Güter (z. B. ähnlicher Bedarf oder ähnlicher COI) und die Bestimmung des Platzes für diese Gruppen (klassenbasierte Lagerplatzzuordnung).

Im Bereich der Metaheuristiken wird das Problem beispielsweise durch genetische Algorithmen (Park & Seo, 2010), durch einen Genetischen Algorithmus für Pareto-Optimierung (Pareto and Niche Genetic Algorithm, Li et al., 2008), durch Simulated Annealing (Muppani & Adil, 2008) und durch hybride Metaheuristiken wie eine Kombination aus Particle Swarm Optimization und Artificial Bee Colonies (Hu et al., 2012) gelöst.

Das Problem der Lagerplatzzuweisung kann auch mit anderen Aspekten der Optimierung von Lagerabläufen kombiniert werden, z. B. mit der effizienten Verschachtelung von Bewegungen von Regalbediengeräten, z. B. in Chen et al. (2011), wo dieses Problem mit einem Tabu-Suche-Ansatz angegangen wird.

Ein weiterer Aspekt bei der Zuweisung von Lagerplätzen ist die Notwendigkeit, Standorte aufgrund von saisonalen Schwankungen und anderen Veränderungen der Nachfrage oder des Produktmixes neu zuzuweisen. In Kofler et al. (2011) wird gezeigt, dass solche Reorganisationen periodisch ohne allzu großen betrieblichen Aufwand durchgeführt werden können.

Ähnlich wie bei Transportproblemen gibt es bei bestandsbezogenen Optimierungsproblemen viele weitere Varianten, von denen sich einige auf spezifische Details konzentrieren (z. B. Aspekte des Lagerbetriebs), andere verbinden die Probleme mit untergeordneten Aspekten wie zugehörigen Transportprozessen (auch als Inventory--Routing bezeichnet) oder berücksichtigen mehrere Lager-Standorte und andere Aspekte einer Lieferkette.

4.7 Inventory Routing

Die meisten der bisherigen Probleme in der Bestandsplanung vernachlässigen mehr oder weniger die Tatsache, dass die Lagerhaltung in der Regel eng mit den Transportaktivitäten verknüpft ist. Beispielsweise erfordert die Erfüllung von Kundenaufträgen erstens die Verfügbarkeit der benötigten Artikel (oder die Möglichkeit, sie

zu produzieren oder zu beschaffen) und zweitens ihren Transport zu den Kunden. Bei Inventory Routing-Problemen wird versucht, beide Aktivitäten in einem einzigen Planungs- und Optimierungsmodell zu kombinieren.

Der einfachste Fall ist das für die EOQ beschriebene Modell, das die Transport- und Lagerkosten auf einfachste Weise berücksichtigt (siehe Abschn. 4.2). Aber schon leicht abweichende Situationen, in denen es immer noch um einen einzigen Bestand und einen einzigen Kunden geht, können sehr viel komplizierter sein. Wenn z. B. Transporte mit Fahrzeugen mit begrenzter Kapazität durchgeführt werden und die Transportkosten nur von der Anzahl der Fahrzeugtouren abhängen, z. B. feste Transportkosten pro Fahrzeug, kann sich die Situation ändern und die Berechnung der Ableitung einer Zielfunktion (wie in den EOQ-Formeln angenommen) führt nicht mehr zur optimalen Lösung.

Eine Version des Problems mit mehreren Waren wird als Single Link Shipping Problem (SLSP) bezeichnet (siehe Bertazzi & Speranza, 2012). Hier muss ein Anbieter eine Menge von P Produkten an einen Kunden mit einer konstanten Nachfrage d_p, $p \in \{1, \ldots, P\}$ liefern. Für den Lieferanten wird angenommen, dass die Produkte mit der gleichen konstanten Menge pro Periode d_p erzeugt werden. Die Produkte sollen mit kapazitätsbeschränkten Fahrzeugen ausgeliefert werden, und die Transportkosten werden für jede Fahrt mit c angenommen (unabhängig von der genutzten Kapazität des Fahrzeugs). Zusätzlich zu den Transportkosten entstehen Lagerkosten, wobei angenommen wird, dass für jeden Artikel Lagerkosten h_p gegeben sind, unabhängig davon, ob er an der Quelle oder am Zielort gelagert wird. Die zu bestimmenden Variablen sind die ausgelieferten Mengen x_{pt} für den Artikel p in der Periode t und in Abhängigkeit davon die Anzahl der eingesetzten Fahrzeuge y_t. Ziel ist es, die Gesamtkosten zu minimieren und dabei die gesamte Nachfrage in allen Perioden zu befriedigen. Mathematisch kann das Problem wie folgt formuliert werden (modifiziert nach Bertazzi & Speranza, 2002; vgl. auch Speranza & Ukovich, 1994):

$$\min \sum_{t=1}^{T} \sum_{p=1}^{P} h_p \left(i_{pt}^A + i_{pt}^B \right) + \sum_{t=1}^{T} c y_t \tag{4.26}$$

$$\sum_{t=1}^{T} x_{pt} = d_p T, \quad p \in \{1, \ldots, P\} \tag{4.27}$$

$$\sum_{p=1}^{P} a_p x_{pt} \leq q y_t, \quad t \in \{1, \ldots, T\} \tag{4.28}$$

$$i_{pt}^A = i_{p0}^A + d_p t - \sum_{t'=1}^{t} x_{pt'}, \quad t \in \{1, \ldots, T\}, \; p \in \{1, \ldots, P\} \tag{4.29}$$

$$i_{pt}^B = i_{p0}^B + \sum_{t'=1}^{t} x_{pt'} - d_p t, \quad t \in \{1, \ldots, T\}, \; p \in \{1, \ldots, P\} \tag{4.30}$$

$$i_{pt}^{A} \geq 0, \quad t \in \{1, \ldots, T\}, \, p \in \{1, \ldots, P\} \tag{4.31}$$

$$i_{pt}^{B} - x_{pt} \geq 0, \quad t \in \{1, \ldots, T\}, \, p \in \{1, \ldots, P\} \tag{4.32}$$

Die Zielfunktion in Gl. (4.26) beschreibt die Minimierung der Summe aus Lagerkosten und Transportkosten. Dabei ist der Gesamtbestand jedes Produkts gleich dem Bestand am Ausgangsort plus dem Bestand am Zielort, d. h.

$$i_{pt} = i_{pt}^{A} + i_{pt}^{B}, \, p \in \{1, \ldots, P\}, \, t \in \{1, \ldots, T\} \tag{4.33}$$

Gl. (4.27) stellt sicher, dass die Summe aller transportierten Waren die Gesamtmenge der im betrachteten Zeithorizont verfügbaren Produkte abdeckt. Gl. (4.28) dient der Einhaltung der Kapazitätsbeschränkungen für die eingesetzten Fahrzeuge: a_p ist der Kapazitätsbedarf (Volumen) pro Produkteinheit p, während q die verfügbare Kapazität eines Fahrzeugs ist. Der Bestand am Herkunftsort A wird in Gl. (4.29) berechnet als die Summe aus Anfangsbestand und den bis zur Periode t verfügbar werdenden Mengen abzüglich der bis t ausgelieferten Mengen. Gl. (4.30) beschreibt den Bestand am Bestimmungsort B als Anfangsbestand plus ankommende Lieferungen minus Nachfrage bis zur betrachteten Periode t. Die Gl. (4.31) und (4.32) stellen sicher, dass der Bestand am Ausgangs- und am Zielort nicht negativ werden kann. Es ist zu beachten, dass die ankommende Lieferung vom Bestand in Gl. (4.32) abgezogen wird, da die entsprechende Variable den Wert nach der Ankunft beschreibt.

In Bertazzi und Speranza (2002) werden mehrere Varianten des Single-Link-Problems betrachtet, insbesondere eine zeitkontinuierliche Version und eine zeitdiskrete Version mit angenommener konstanter Häufigkeit der Transporte.

Dieser Single-Link-Fall ist natürlich nicht besonders realistisch. Aus diesem Grund wird meist eine Situation angenommen, die die Belieferung mehrerer Kunden berücksichtigt (Abb. 4.5).

Ein solches Inventory-Routing-Problem kann kurz wie folgt beschrieben werden (siehe Archetti et al., 2007): Es wird angenommen, dass es eine Menge von Kunden (z. B. Einzelhändlern) {1, …, N} gibt, die von einem einzigen Lieferanten über einen Zeithorizont {1, …, T} zu beliefern sind. Beim Lieferanten wird zu jedem Zeitpunkt t eine Menge d_{0t} produziert (oder beschafft), und beim Kunden n wird zu jedem Zeitpunkt t eine Menge d_{nt} benötigt (und verbraucht). Der Lagerbestand bei jedem Kunden wird als i_{nt} bezeichnet, und wir nehmen an, dass die Anfangsbestände i_{n0} gegeben sind. Außerdem wird davon ausgegangen, dass bei jedem Kunden i *ein* maximaler Lagerbestand (U_i) zu beachten ist. Die Lagerbestände beim Lieferanten werden mit i_t bezeichnet, beginnend mit einem gegebenen Lagerbestand von i_0. Die Lagerhaltungskosten pro Einheit beim Lieferanten werden mit h_0 bezeichnet, während h_n die Lagerhaltungskosten pro Einheit beim Kunden n sind.

Die Waren werden vom Lieferanten zu den Kunden mit einem Fahrzeug mit der Kapazität q befördert. Die Kunden können in beliebiger Reihenfolge bedient werden, indem eine geeignete Route gebildet wird. Die Transportkosten werden als c_{ij}

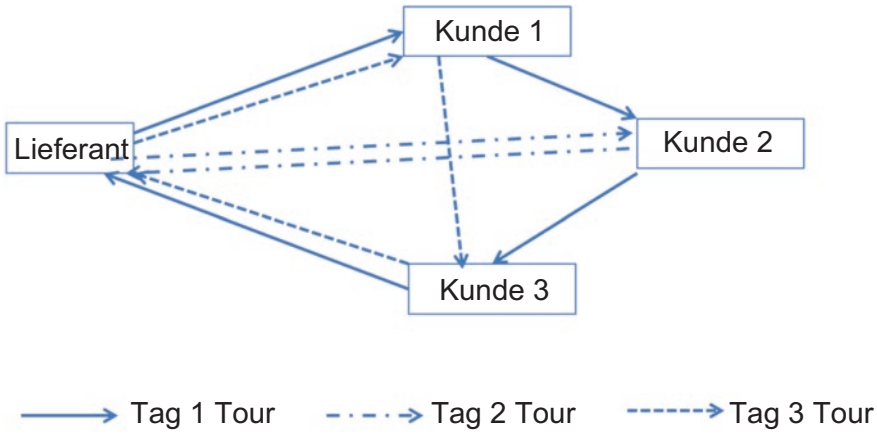


Abb. 4.5 Beispiel für ein Inventory Routing-Problem mit einem Lieferanten und drei Kunden, das zu drei verschiedenen Routen an verschiedenen Tagen führt

für die Standorte $i,j \in \{0, \ldots, n\}$ bezeichnet, die den Standort des Lieferanten (Indexwert 0) oder einen Standort der Kunden (Indexwert ≥0) bezeichnen. Die mathematische Formulierung des Problems lautet dann wie folgt:

$$\min \sum_{t=1}^{T+1} h_0 i_t + \sum_{n=1}^{N} \sum_{t=1}^{T+1} h_n i_{nt} + \sum_{i=0}^{N} \sum_{j=0}^{i-1} \sum_{t=1}^{T} c_{ij} y_{ij}^t \tag{4.34}$$

$$i_t = i_{t-1} + d_{0t-1} - \sum_{i=1}^{N} x_{it-1}, \quad t \in \{1, \ldots, T+1\} \tag{4.35}$$

$$i_t \geq \sum_{i=1}^{N} x_{it}, \quad t \in \{1, \ldots, T\} \tag{4.36}$$

$$i_{it} = i_{it-1} + x_{it-1} - d_{it-1}, \; i \in \{1, \ldots, N\}, t \in \{1, \ldots, T+1\} \tag{4.37}$$

$$i_{it} \geq 0, \quad i \in \{1, \ldots, N\}, t \in \{1, \ldots, T+1\} \tag{4.38}$$

$$\sum_{i=1}^{N} x_{it} \leq q, \quad t \in \{1, \ldots, T\} \tag{4.39}$$

$$\sum_{i=1}^{N} x_{it} \leq q z_{0t}, \quad t \in \{1, \ldots, T\} \tag{4.40}$$

$$\sum_{j=1}^{i-1} y_{ij}^t + \sum_{j=i+1}^{N} y_{ji}^t = 2 z_{it}, \quad i \in \{1, \ldots, N\}, t \in \{1, \ldots, T\} \tag{4.41}$$

$$x_{it} \geq 0, \quad i \in \{1, \ldots, N\}, t \in \{1, \ldots, T\} \tag{4.42}$$

$$y_{ij}^{t} \in \{0, 1\}, \quad i, j \in \{1, \ldots, N\}, t \in \{1, \ldots, T\} \tag{4.43}$$

$$z_{it} \in \{0, 1\}, \quad i \in \{0, \ldots, N\}, t \in \{1, \ldots, T\} \tag{4.44}$$

Die Zielfunktion (4.34) besteht aus den Lagerkosten beim Lieferanten, den Lagerkosten aller Kunden und den Transportkosten. Zu beachten ist, dass die Summen der Lagerkosten die Periode $T + 1$ einschließen, um den Bestand am Ende des Planungshorizonts zu berücksichtigen. Die Berechnung der Transportkosten beinhaltet binäre Variablen y^{t}_{ij}, die definieren, ob zum Zeitpunkt t eine Route mit Transport von Knoten i zu Knoten j vorliegt. In dieser Problemformulierung werden ungerichtete Kanten im Graph angenommen.

Die Gl. (4.35) und (4.37) sind die Bestandsgleichungen für den Lieferanten und die Kunden. Gl. (4.36) dient der Vermeidung von Fehlbeständen beim Lieferanten, während Gl. (4.38) der Vermeidung negativer Bestände bei den Kunden dient.

Gl. (4.39) stellt sicher, dass die Fahrzeugkapazität beachtet wird, während Gl. (4.40) sie mit den binären Variablen z_{it} in Beziehung setzt, die definieren, ob zum Zeitpunkt t *der* Knoten i besucht wird ($z_{it} = 1$) oder nicht ($z_{it} = 0$) (hier für den Lieferanten $i = 0$).

Gl. (4.41) dient zur Festlegung konsistenter Touren, was voraussetzt, dass es für jeden besuchten Kunden (und auch für den Lieferanten) zwei Kanten in der Tour gibt, eine für die Fahrt zu den Kunden und eine für die Fahrt von ihnen weg.

Oft gibt es zusätzliche Beschränkungen, z. B. für eine so genannte „Order-up-to-Level"-Bestandsstrategie, die besagt, dass, wenn ein Kunde zum Zeitpunkt t beliefert wird, der resultierende Bestand ein bestimmtes Niveau erreichen sollte:

$$x_{it} = \left(U_i - i_{it}\right) z_{it}, i \in \{1, \ldots, N\}, t \in \{1, \ldots, T\} \tag{4.45}$$

Weitere Nebenbedingungen können für eine effiziente Modellierung des Problems in einer entsprechenden Standardsoftware nützlich sein, z. B. eine Bedingung für die Eliminierung von Untertouren (Archetti et al., 2007).

Wie man sich leicht vorstellen kann, sind neben diesen Standardformulierungen von Inventory Routing-Problemen viele Varianten möglich, um praktische Situationen besser zu repräsentieren. Eine Möglichkeit zur Unterscheidung verschiedener Arten von Inventory Routing-Problemen ist die Berücksichtigung der Zeit. Die Zeit kann entweder diskret sein, d. h. es werden nur Zeiträume wie z. B. Tage betrachtet, oder die Zeit wird in kontinuierlicher Form gemessen. Ein weiterer Aspekt betrifft den Zeithorizont: Es gibt Ein-Perioden-Modelle, Mehr-Perioden-Modelle und Modelle mit unendlichem Zeithorizont (siehe Moin & Salhi, 2007).

Weitere Unterscheidungen betreffen die Netzstruktur. Neben den oben genannten Fällen mit einer einzigen Verbindung oder einem einzigen Lieferanten, aber mehreren Kunden, kann es natürlich auch Situationen mit mehreren Lieferanten und Kunden geben. Alternative Modelle für die Lieferungen sind Modelle nur mit direkten Lieferungen (wie im Fall einer einzigen Verbindung), Routen mit mehreren Kunden

und möglicherweise Lieferungen ohne ein einziges, zentrales Depot, d. h. mit möglichen Lieferungen zwischen Kunden. Der letztere Fall wird in der Literatur (siehe z. B. Coelho et al., 2013) auch als kontinuierlicher Fall oder als Modell mit Umladungen (Transshipments) bezeichnet.

Sehr oft sind bei der Lösung von Inventory Routing-Problemen zusätzliche Aspekte zu berücksichtigen. Insbesondere ist es oft gewünscht, dass die Lieferpläne zyklische Muster oder definierte Lieferfrequenzen aufweisen, z. B. Lieferungen alle 3 Tage. Andere Anforderungen können maximale Lagerbestände bei den Kunden, nur Lkw-Komplettladungen (Full Truck Load), Verbote für bestimmte Routen, z. B. nur Direktlieferungen, oder eine vorhandene Aufteilung der zu bedienenden Kunden in einer Route sein (siehe Bertazzi und Speranza (2012) für weitere Beispiele für solche Einschränkungen).

Weitere zusätzliche Beschränkungen können sich auf die Bestandsauffüllungspolitik beziehen: Insbesondere müssen Höchstbestände an den Bestimmungsorten beachtet werden, oder es gibt eine „Order-up-to-Level"-Bestandsstrategie (siehe oben). Eine weitere mögliche Strategie ist das Zero Inventory Ordering (ZIO), das es erlaubt, die Bestände bei einem Kunden nur dann aufzufüllen, wenn sein Lagerbestand auf Null gesunken ist.

Unzureichende Lieferungen (die nicht zu einem negativen Bestand führen sollten) können als entgangener Umsatz betrachtet werden oder zu Auftragsrückständen (Back Orders) führen (d. h. die Lieferungen sollten so schnell wie möglich nachgeholt werden). Schließlich könnten die Transportmittel differenzierter betrachtet werden, d. h. es könnte ein einzelnes Fahrzeug, eine bestimmte Anzahl von Fahrzeugen oder eine unbeschränkte Anzahl von Fahrzeugen geben, letzteres möglicherweise mit dem Ziel, so wenige Fahrzeuge wie möglich einzusetzen. Werden mehrere Fahrzeuge betrachtet, so können diese identisch oder in Bezug auf die Kapazität heterogen sein. Neben der Berücksichtigung der Ladekapazität der Fahrzeuge kann auch eine Full Truck Load-Strategie angewandt werden, die es erlaubt, Fahrzeugtouren nur mit voll ausgelasteten Kapazitäten zu starten.

Was die Lösungsansätze angeht, so wird die Literatur trotz der hohen rechnerischen Komplexität immer noch von traditionellen mathematischen Optimierungsansätzen einerseits und einfachen oder problemspezifischen Ansätzen andererseits dominiert, die gelegentlich kombiniert werden. So wird z. B. in Campbell und Savelsbergh (2004) das Problem in zwei Teilprobleme zerlegt, von denen eines (die Bestimmung, welche Kunden an welchem Tag und in welcher Menge zu bedienen sind) durch einen Branch-and-Bound-Ansatz gelöst wird, während die zweite Phase (Bestimmung der Routen) durch eine Einfügungsheuristik gelöst wird. Ein weiteres Beispiel für einen zweiphasigen Ansatz ist Kang und Kim (2010), wo in beiden Phasen spezifische Heuristiken verwendet werden.

Ein Beispiel für einen zweiphasigen Ansatz, bei dem in der zweiten Phase eine Metaheuristik zum Einsatz kommt, ist Sindhuchao et al. (2005). In der zweiten Phase verwenden sie einen Ansatz der Very Large-Scale Neighbourhood Search (VLSN), während die erste Phase durch eine Column Generation-Methode gelöst wurde.

García et al. (2013) verwenden zwei Metaheuristiken zur Lösung eines Mehrzielproblems für Inventory Routing: variable Nachbarschaftssuche (VNS) und den evolutionären Algorithmus NSGA-II. Die betrachteten Ziele sind die Transportkosten und die Lagerkosten. Unter Verwendung einiger realer Problemdaten erzielte die VNS-Metaheuristik bessere Ergebnisse. Vidović et al. (2014) schlagen einen auf einer konstruktiven Heuristik basierenden Ansatz des variablen Nachbarschaftsabstiegs (variable neighborhood descent, VND) vor. Der Ansatz verwendet zwei Arten der VND-Suche, zunächst eine lokale Intra-Perioden-Suche durch Verbesserung der Routen (unter Verwendung des Austauschs von Destinationen zwischen den Routen, des Entfernens und Wiedereinfügens von Orten innerhalb einer Route und des Austauschs von Routensegmenten), dann eine große Inter-Perioden-Nachbarschaftssuche, die auf der Idee basiert, die Lieferung eines Ortes aus einer Route zu entfernen und die Liefermenge auf einen anderen Tag zu verschieben. Im Vergleich zu einer gemischt-ganzzahligen Problemformulierung und Lösung mit der Software CPLEX führt VND bei kleinen Problemgrößen zu Ergebnissen mit weniger als 4 % höheren Kosten bei deutlich kürzeren Rechenzeiten, während größere Problemfälle nur mit VND gelöst werden konnten.

In Liu und Chen (2011) wird die Tabu-Suche mit verschiedenen Ansätzen der Nachbarschaftssuche zur Lösung eines erweiterten Inventory-Routing-Problems verwendet. In Lee et al. (2006) wird die Tabu-Suche ebenfalls zur Lösung von Inventory-Routing-Problemen eingesetzt. Numerische Experimente zeigen, dass die erhaltenen Lösungen nicht mehr als 4 % schlechter sind als die optimalen Lösungen, während sie angemessene Rechenzeiten benötigen.

Ein weiterer Tabu-Search-Lösungsansatz wird in Toptal et al. (2013) untersucht. In verschiedenen Experimenten erzielte dieser Ansatz optimale oder nahezu optimale Lösungen mit einer Abweichung von 0,31 % von den jeweiligen unteren Schranken.

Die Tabu-Suche wird auch von Zhao et al. (2007) verwendet, wo sie zur Bestimmung der Partitionierung der Kunden für die Routenbildung dient. Ein ähnlicher Ansatz wird in Zhao et al. (2008) untersucht, wo die Partitionen durch eine Methode der variablen Nachbarschaftssuche (VNS) ermittelt werden.

Gelegentlich wurden auch spezifische KI-Ansätze verwendet, um Unsicherheiten oder unvollständige Informationen während des Modellierungsprozesses von Inventory-Routing-Problemen zu berücksichtigen. So wird in Huang und Lin (2010) Ant Colony Optimization auf ein Inventory Routing-Problem mit mehreren Produkten und unsicherer Nachfrage angewendet. Für die Testläufe wird hier die Nachfrage als normalverteilte Variable mit bekannten Parametern berechnet. Ein weiteres Beispiel ist ein Fuzzy-Modell einer Multi-Echelon-Lagersituation in einer Lieferkette, das in Giannoccaro et al. (2003) untersucht wird. Hier dient ein Fuzzy-Ansatz zur Modellierung von Unsicherheiten sowohl bei der Nachfrage als auch bei den angenommenen Lager- und Backorder-Kosten. Ein Ansatz für das Bestandsmanagement auf der Grundlage eines Markov-Entscheidungsprozessmodells und unter Verwendung von Reinforcement Learning wird in Giannoccaro und Pontrandolfo (2002) untersucht.

Generell ist jedoch zu beobachten, dass Fuzzy-Modelle, aber auch traditionellere stochastische Modelle bei der Untersuchung von Inventory-Routing-Problemen vernachlässigt wurden. Ein kurzer Überblick über diesen Bereich findet sich in Bertazzi et al. (2013).

Literatur

Absi, N., & Kedad-Sidhoum, S. (2008). The multi-item capacitated lot-sizing problem with setup times and shortage costs. *European Journal of Operational Research, 185*(3), 1351–1374.

Akrami, B., Karimi, B., & Hosseini, S. M. (2006). Two metaheuristic methods for the common cycle economic lot sizing and scheduling in flexible flow shops with limited intermediate buffers: The finite horizon case. *Applied Mathematics and Computation, 183*(1), 634–645.

Almada-Lobo, B., & James, R. J. (2010). Neighbourhood search meta-heuristics for capacitated lot-sizing with sequence-dependent setups. *International Journal of Production Research, 48*(3), 861–878.

Andler, K. (1929). *Rationalisierung der Fabrikation und optimale Losgröße*. Oldenbourg.

Archetti, C., Bertazzi, L., Laporte, G., & Speranza, M. G. (2007). A branch-and-cut algorithm for a vendor-managed inventory-routing problem. *Transportation Science, 41*(3), 382–391.

Arkin, E., Joneja, D., & Roundy, R. (1989). Computational complexity of uncapacitated multi-echelon production planning problems. *Operations Research Letters, 8*(2), 61–66.

Berretta, R., & Rodrigues, L. F. (2004). A memetic algorithm for a multistage capacitated lot-sizing problem. *International Journal of Production Economics, 87*(1), 67–81.

Berretta, R., França, P. M., & Armentano, V. A. (2005). Metaheuristic approaches for the multilevel resource-constrained lot-sizing problem with setup and lead times. *Asia-Pacific Journal of Operational Research, 22*(02), 261–286.

Bertazzi, L., & Speranza, M. G. (2002). Continuous and discrete shipping strategies for the single link problem. *Transportation Science, 36*(3), 314–325.

Bertazzi, L., & Speranza, M. G. (2012). Inventory routing problems: An introduction. *EURO Journal on Transportation and Logistics, 1*(4), 307–326.

Bertazzi, L., Bosco, A., Guerriero, F., & Lagana, D. (2013). A stochastic inventory routing problem with stock-out. *Transportation Research Part C: Emerging Technologies, 27*, 89–107.

Brahimi, N., Dauzere-Peres, S., Najid, N. M., & Nordli, A. (2006). Single item lot sizing problems. *European Journal of Operational Research, 168*(1), 1–16.

Buschkühl, L., Sahling, F., Helber, S., & Tempelmeier, H. (2010). Dynamic capacitated lot-sizing problems: A classification and review of solution approaches. *OR Spectrum, 32*(2), 231–261.

Campbell, A. M., & Savelsbergh, M. W. (2004). A decomposition approach for the inventory-routing problem. *Transportation Science, 38*(4), 488–502.

Caserta, M., & Rico, E. Q. (2009). A cross entropy-Lagrangean hybrid algorithm for the multi-item capacitated lot-sizing problem with setup times. *Computers & Operations Research, 36*(2), 530–548.

Chen, L., Langevin, A., & Riopel, D. (2011). A tabu search algorithm for the relocation problem in a warehousing system. *International Journal of Production Economics, 129*(1), 147–156.

Chopra, S., Reinhardt, G., & Dada, M. (2004). The effect of lead time uncertainty on safety stocks. *Decision Sciences, 35*(1), 1–24.

Clark, A. R. (2000). A local search approach to lot sequencing and sizing. In M. Tucci & M. Garetti (Hrsg.), *Proceedings of the third international workshop of the IFIP WG 5.7 special interest group on „Advanced techniques in production planning & control“* (S. 145–152). Firenze University Press.

Coelho, L. C., Cordeau, J. F., & Laporte, G. (2013). Thirty years of inventory routing. *Transportation Science, 48*(1), 1–19.

Dellaert, N., Jeunet, J., & Jonard, N. (2000). A genetic algorithm to solve the general multi-level lot-sizing problem with time-varying costs. *International Journal of Production Economics, 68*(3), 241–257.

Dye, C. Y., & Hsieh, T. P. (2013). A particle swarm optimization for solving lot-sizing problem with fluctuating demand and preservation technology cost under trade credit. *Journal of Global Optimization, 55*(3), 655–679.

Florian, M., Lenstra, J. K., & Rinnooy Kan, A. H. G. (1980). Deterministic production planning: Algorithms and complexity. *Management Science, 26*(7), 669–679.

França, P. M., Armentano, V. A., Berretta, R. E., & Clark, A. R. (1997). A heuristic method for lot-sizing in multi-stage systems. *Computers & Operations Research, 24*(9), 861–874.

Gaafar, L. (2006). Applying genetic algorithms to dynamic lot sizing with batch ordering. *Computers & Industrial Engineering, 51*(3), 433–444.

García, I., Pacheco, J., & Alvarez, A. (2013). Optimizing routes and stock. *Journal of Heuristics, 19*(2), 157–177.

Giannoccaro, I., & Pontrandolfo, P. (2002). Inventory management in supply chains: A reinforcement learning approach. *International Journal of Production Economics, 78*(2), 153–161.

Giannoccaro, I., Pontrandolfo, P., & Scozzi, B. (2003). A fuzzy echelon approach for inventory management in supply chains. *European Journal of Operational Research, 149*(1), 185–196.

Goetschalckx, M., & Ratliff, H. D. (1990). Shared storage policies based on the duration stay of unit loads. *Management Science, 36*(9), 1120–1132.

Gopalakrishnan, M., Ding, K., Bourjolly, J. M., & Mohan, S. (2001). A tabu-search heuristic for the capacitated lot-sizing problem with set-up carryover. *Management Science, 47*(6), 851–863.

Goren, H. G., Tunali, S., & Jans, R. (2010). A review of applications of genetic algorithms in lot sizing. *Journal of Intelligent Manufacturing, 21*(4), 575–590.

Gutierrez, E., Hernandez, W., & Süer, G. A. (2001). Genetic algorithms in capacitated lot sizing decisions. In *Computing research conference,* March 31, 2001, Puerto Rico. http://www.ece.uprm.edu/crc/crc2001/papers/eliecer_gutierrez.PDF. Zugegriffen am 12.03.2016.

Haase, K., & Kohlmorgen, U. (1996). Parallel genetic algorithm for the capacitated lot-sizing problem. In P. Kleinschmidt, A. Bachem, U. Derigs, D. Fischer, U. Leopold-Wildburger, & R. Möhring (Hrsg.), *Operations research proceedings 1995* (S. 370–375). Springer. http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.45.4394&rep=rep1&type=pdf. Zugegriffen am 12.03.2016.

Harris, F. W. (1913). How many parts to make at once. *Factory, The Magazine of Management, 10*(2), 135–136, 152, reprinted in *Operations Research, 38*(6), 947–950.

Heskett, J. L. (1963). Cube-per-order index-a key to warehouse stock location. *Transportation and Distribution Management, 3*(1), 27–31.

van Hoek, R. I. (2001). The rediscovery of postponement. *A literature review and directions for research. Journal of Operations Management, 19*, 161–184.

Homberger, J. (2008). A parallel genetic algorithm for the multilevel unconstrained lot-sizing problem. *INFORMS Journal on Computing, 20*(1), 124–132.

Homberger, J. (2010). Decentralized multi-level uncapacitated lot-sizing by automated negotiation. *4OR: A Quarterly Journal of Operations Research, 8*(2), 155–180.

Hu, W., Wang, Y., & Zheng, J. (2012). Research on warehouse allocation problem based on the Artificial Bee Colony inspired particle swarm optimization (ABC-PSO) algorithm. In *5th International Symposium in Computational Intelligence and Design (ISCID)* (Bd. 1, S. 173–176). IEEE.

Huang, S. H., & Lin, P. C. (2010). A modified ant colony optimization algorithm for multi-item inventory routing problems with demand uncertainty. *Transportation Research Part E: Logistics and Transportation Review, 46*(5), 598–611.

Jans, R., & Degraeve, Z. (2007). Meta-heuristics for dynamic lot sizing: A review and comparison of solution approaches. *European Journal of Operational Research, 177*(3), 1855–1875.

Kämpf, M., & Köchel, P. (2006). Simulation-based sequencing and lot size optimisation for a production-and-inventory system with multiple items. *International Journal of Production Economics, 104*(1), 191–200.

Kang, J. H., & Kim, Y. D. (2010). Coordination of inventory and transportation managements in a two-level supply chain. *International Journal of Production Economics, 123*(1), 137–145.

Karimi, B., Ghomi, S. F., & Wilson, J. M. (2003). The capacitated lot sizing problem: A review of models and algorithms. *Omega, 31*(5), 365–378.

Kofler, M., Beham, A., Wagner, S., Affenzeller, M., & Achleitner, W. (2011). Re-warehousing vs. healing: Strategies for warehouse storage location assignment. In *3rd IEEE international symposium on logistics and industrial informatics* (S. 77–82). IEEE.

Lee, Y. H., Jung, J. W., & Lee, K. M. (2006). Vehicle routing scheduling for cross-docking in the supply chain. *Computers & Industrial Engineering, 51*(2), 247–256.

Li, M., Chen, X., & Liu, C. (2008). Pareto and niche genetic algorithm for storage location assignment optimization problem. In *3rd International Conference in Innovative Computing Information and Control. ICICIC'08* (S. 465). IEEE.

Liu, S. C., & Chen, J. R. (2011). A heuristic method for the inventory routing and pricing problem in a supply chain. *Expert Systems with Applications, 38*(3), 1447–1456.

Mateus, G. R., Ravetti, M. G., de Souza, M. C., & Valeriano, T. M. (2010). Capacitated lot sizing and sequence dependent setup scheduling: An iterative approach for integration. *Journal of Scheduling, 13*(3), 245–259.

Mishra, N., Kumar, V., Kumar, N., Kumar, M., & Tiwari, M. K. (2011). Addressing lot sizing and warehousing scheduling problem in manufacturing environment. *Expert Systems with Applications, 38*(9), 11751–11762.

Moin, N. H., & Salhi, S. (2007). Inventory routing problems: A logistical overview. *Journal of the Operational Research Society, 58*(9), 1185–1194.

Muller, L. F., Spoorendonk, S., & Pisinger, D. (2012). A hybrid adaptive large neighborhood search heuristic for lot-sizing with setup times. *European Journal of Operational Research, 218*(3), 614–623.

Muppani, V. R., & Adil, G. K. (2008). Efficient formation of storage classes for warehouse storage location assignment: a simulated annealing approach. Omega, 36(4), 609–618.

Nascimento, M. C., Resende, M. G., & Toledo, F. M. (2010). GRASP heuristic with path-relinking for the multi-plant capacitated lot sizing problem. *European Journal of Operational Research, 200*(3), 747–754.

Park, C., & Seo, J. (2010). Comparing heuristic algorithms of the planar storage location assignment problem. *Transportation Research Part E: Logistics and Transportation Review, 46*(1), 171–185.

Pitakaso, R., Almeder, C., Doerner, K. F., & Hartl, R. F. (2006). Combining population-based and exact methods for multi-level capacitated lot-sizing problems. *International Journal of Production Research, 44*(22), 4755–4771.

Robinson, P., Narayanan, A., & Sahin, F. (2009). Coordinated deterministic dynamic demand lot-sizing problem: A review of models and algorithms. *Omega, 37*(1), 3–15.

Silver, E. A., & Meal, H. C. (1973). A heuristic for selecting lot size quantities for the case of a deterministic time-varying demand rate and discrete opportunities for replenishment. *Production and Inventory Management, 14*(2), 64–74.

Sindhuchao, S., Romeijn, H. E., Akçali, E., & Boondiskulchok, R. (2005). An integrated inventory-routing system for multi-item joint replenishment with limited vehicle capacity. *Journal of Global Optimization, 32*(1), 93–118.

Speranza, M. G., & Ukovich, W. (1994). Minimizing transportation and inventory costs for several products on a single link. *Operations Research, 42*(5), 879–894.

Staggemeier, A., Clarke, A., Aickelin, U., & Smith, J. (2002). A hybrid genetic algorithm to solve a lot sizing and scheduling problem. *Internal research report MS-2002-4*. University of the West of England.

Toptal, A., Koc, U., & Sabuncuoglu, I. (2013). A joint production and transportation planning problem with heterogeneous vehicles. *Journal of the Operational Research Society, 65*(2), 180–196.

Torabi, S. A., Ghomi, S. F., & Karimi, B. (2006). A hybrid genetic algorithm for the finite horizon economic lot and delivery scheduling in supply chains. *European Journal of Operational Research, 173*(1), 173–189.

Vidović, M., Popović, D., & Ratković, B. (2014). Mixed integer and heuristics model for the inventory routing problem in fuel delivery. *International Journal of Production Economics, 147*, 593–604.

Wagner, H. M., & Whitin, T. M. (1958). Dynamic version of the economic lot size model. *Management Science, 5*(1), 89–96.

Waters-Fuller, N. (1995). Just-in-time purchasing and supply: A review of the literature. *International Journal of Operations & Production Management, 15*(9), 220–236.

Woarawichai, C., Kullpattaranirun, T., & Rungreunganun, V. (2011). Inventory lot-sizing problem with supplier selection under storage space and budget constraints. *IJCSI International Journal of Computer Science Issues, 8*(2), 250–255.

Xie, J., & Dong, J. (2002). Heuristic genetic algorithms for general capacitated lot-sizing problems. *Computers & Mathematics with Applications, 44*(1), 263–276.

Zhao, Q. H., Wang, S. Y., & Lai, K. K. (2007). A partition approach to the inventory/routing problem. *European Journal of Operational Research, 177*(2), 786–802.

Zhao, Q. H., Chen, S., & Zang, C. X. (2008). Model and algorithm for inventory/routing decision in a three-echelon logistics system. *European Journal of Operational Research, 191*(3), 623–635.

Kapitel 5
Scheduling



5.1 Einleitung

Bei den im vorigen Kapitel behandelten Produktionsproblemen geht es um Planungsaufgaben auf einer eher groben Ebene. Die Planung erfolgt für eine Reihe von Zeiträumen wie Tage, Wochen oder Monate, und es wird nur geplant, was in welcher Menge produziert werden soll. Die Frage, wie die Produktion durchgeführt wird, d. h. welche spezifischen Ressourcen eingesetzt werden, wird weitgehend ausgeklammert. Auch die spezifischen Zeiten der Produktionsaktivitäten wie Stunden und Minuten werden nicht berücksichtigt.

Bei der Planung auf einer feinen Zeitskala ist es wichtiger, sicherzustellen, dass die Pläne kohärent und durchführbar sind. Zum Beispiel muss die Planung der Ressourcen deren gegebene Verfügbarkeit und Kapazitäten im Zeitablauf berücksichtigen. Auch die Arbeitszeiten, Abwesenheiten, Pausenzeiten und andere erforderliche Leerlaufzeiten der Mitarbeiter müssen ebenso berücksichtigt werden wie ihre Kapazitäten, z. B. die Möglichkeit, nur einen Artikel zur gleichen Zeit zu bearbeiten. Arbeitszeiten können auch für Maschinen gelten, z. B. aufgrund von Wartungspausen oder definierten Instandhaltungszeiten. Außerdem beeinflussen bereits eingeplante Aktivitäten die zeitliche Verfügbarkeit der jeweiligen Ressourcen.

Die detaillierte Planung von Aufträgen, Aufgaben oder Tätigkeiten, die bestimmte Ressourcen erfordern, wird als Scheduling bezeichnet. Anstelle von Ressourcen wird in der Terminologie des Scheduling häufig der Ausdruck „Maschinen“ verwendet, unabhängig davon, ob es sich um echte Maschinen handelt oder nicht. Das Ergebnis des Planungsprozesses, der Schedule oder Terminplan, ist ein Plan, der die Zuordnung der Aufträge zu den Ressourcen und die konkreten Ausführungszeitpunkte der Aufträge enthält. In manchen Fällen reicht es aus, die Jobs einem Pool von Ressourcen zuzuweisen (ohne dass eine einzelne Ressource angegeben werden muss), oder die Reihenfolge der Jobs, die auf einer Ressource ausgeführt werden sollen, ist für ihre spätere Ausführung ausreichend. Der Grundgedanke ist

© Der/die Autor(en), exklusiv lizenziert an Springer Nature Switzerland AG 2023

T. Hanne, R. Dornberger, *Computational Intelligence in Logistik und Supply Chain Management*, https://doi.org/10.1007/978-3-031-21452-3_5

jedoch, dass die Jobs auf einer detaillierten Ebene geplant werden, die für ihre endgültige Ausführung verwendet werden kann.

Scheduling-Probleme treten in vielen verschiedenen Bereichen auf. Neben der Planung von Produktions-, Logistik- und anderen Geschäftsprozessen werden Schedules auch in rein technischen Bereichen wie der automatischen Ausführung von Aufträgen unter Nutzung der IT-Infrastruktur benötigt, z. B. bei der Planung der Ausführung von Jobs auf einer CPU. Weitere zu planende technische Ressourcen können Maschinen, Servicestationen, Geräte und Werkzeuge, Kommunikationskanäle oder Speicher sein (siehe z. B. Brucker und Brucker (2007) für eine allgemeine Einführung in Scheduling-Probleme und Lösungsansätze).

Traditionell wurden Scheduling-Probleme in der Wirtschaft manuell gelöst, d. h. ein Mensch wies die Aufgaben den Ressourcen zu und kümmerte sich darum, ein praktikables Zeitfenster zu finden. In der Regel wurden Werkzeuge wie große Tafeln mit Gantt-Diagrammen (oder andere Formen der Visualisierung) verwendet. Die angewandte Planungslogik war eher eine Daumenregel oder eine nicht näher spezifizierte Heuristik als eine genau definierte Planungsstrategie oder ein Algorithmus. Obwohl die Ergebnisse von der Erfahrung des Planers profitieren konnten, waren keineswegs optimale Ergebnisse zu erwarten. Dies ist kaum verwunderlich, wenn man bedenkt, dass viele solche Planungsprobleme NP-schwer sind (wie wir später noch ausführlicher diskutieren werden).

Der Fortschritt in der Computertechnologie und die Notwendigkeit der automatischen Planung aufgrund von Geschwindigkeitsanforderungen (z. B. Planung der Ausführung von Computerprogrammen) förderten den verstärkten Einsatz von Scheduling-Algorithmen. Solche Algorithmen werden heute häufig in Bereichen eingesetzt, in denen noch vor nicht allzu langer Zeit die rein menschliche Planung dominierte.

5.2 Einfache Regeln und Heuristiken

Ein Scheduling-Problem erfordert die Angabe einer Reihe von Merkmalen der zu planenden Aufträge (Jobs). So kann beispielsweise der Auftrag i durch die folgenden Daten beschrieben werden:

- Ankunftszeit at_i (Zeit, ab der der Auftrag zur Ausführung bereitsteht)
- Bearbeitungszeit p_i (Dauer, die für die Ausführung des Auftrags erforderlich ist, kann von der spezifischen Wahl einer Ressource abhängen)
- Fälligkeitsdatum (due date) dd_i (Zeitpunkt, zu dem der Auftrag abgeschlossen sein sollte)
- Frist (deadline) dl_i (Zeitpunkt, nach dem die Ausführung des Auftrags nicht mehr erlaubt oder nicht mehr möglich ist)
- Startzeit s_i (noch zu planen)
- Endzeitpunkt (Fertigstellungszeitpunkt) C_i (zu planen)
- mögliche Prioritäten der Aufträge

Darüber hinaus kann es Abhängigkeiten zwischen Aufträgen geben, insbesondere Abhängigkeiten in Bezug auf die mögliche Reihenfolge der Aufträge. Die häufigste Situation ist, dass ein Auftrag abgeschlossen sein muss, bevor ein anderer Auftrag begonnen werden kann (Ende-Start-Beziehungen). So muss beispielsweise beim Anstreichen die Grundierung fertig sein, bevor die Endanstrich erfolgt. Obwohl Ende-Start-Beziehungen die häufigsten Beziehungen zwischen Aufträgen sind, kann es auch Start-Start-Beziehungen (ein Auftrag kann erst beginnen, wenn ein anderer begonnen hat), Start-Ende-Beziehungen (ein Auftrag muss begonnen haben, bevor ein anderer abgeschlossen werden kann) oder Ende-Ende-Beziehungen (ein Auftrag kann erst beendet werden, wenn ein anderer Auftrag abgeschlossen ist) geben.

Bei all diesen Beziehungen können zusätzliche Mindest- oder Höchstzeiten zu berücksichtigen sein. So muss beispielsweise eine Arbeit mindestens x Stunden abgeschlossen sein, bevor eine andere Arbeit beginnen kann. In dem oben erwähnten Beispiel muss die Grundierung getrocknet sein (was eine bestimmte Zeit erfordert), bevor der Endanstrich aufgetragen werden kann.

In diesem Beispiel wird eine Mindestzeit zwischen zwei Aufträgen berücksichtigt. Ein Beispiel für eine maximale Zeit könnte wie folgt aussehen: Wenn Dämmplatten angebracht werden, sollte dies geschehen, bevor der Kleber getrocknet ist. In diesem Fall darf eine Arbeit nicht später als x Stunden nach Abschluss einer anderen Arbeit beginnen.

Weitere Planungseinschränkungen können sich aus den benötigten Ressourcen ergeben. Insbesondere sind einige Ressourcen nur für bestimmte Arten von Aufgaben geeignet. Zum Beispiel benötigt ein Mitarbeiter eine bestimmte Qualifikation für eine bestimmte Aufgabe, z. B. das Führen eines Fahrzeugs.

Außerdem können einige Aufträge mehrere Ressourcen nacheinander oder parallel erfordern, z. B. eine Maschine und einen Bediener. Die Ressourcen können unterschiedliche Kapazitäten haben, z. B. Maschinen, die mehrere Artikel gleichzeitig bearbeiten können.

Schließlich ist die zeitliche Verfügbarkeit von Ressourcen zu berücksichtigen (freie Kapazitäten im Zeitablauf z. B. durch Schichten, Pausen, Wartungszeiten, bereits fest eingeplante oder begonnene Aufträge, etc.)

Für Scheduling-Probleme können verschiedene Ziele relevant sein: Typische Ziele können sich auf geringe Durchlaufzeiten, einen fairen Ausgleich der Lasten zwischen den Ressourcen, einen hohen Durchsatz oder die Einhaltung von Lieferterminen (due dates, deadlines) beziehen.

Für jeden dieser allgemeinen Aspekte gibt es in der Regel verschiedene Möglichkeiten, die Ziele zu spezifizieren. Zum Beispiel kann die Durchlaufzeit eines einzelnen Auftrags als die Dauer vom Eintreffen des Auftrags (Ankunftszeit) bis zu seiner Beendigung (Endzeitpunkt) definiert werden. Für eine zu planende Menge von Aufträgen könnte man dann die durchschnittliche Durchlaufzeit oder die maximale Durchlaufzeit der betrachteten Aufträge betrachten. Diese maximale Durchlaufzeit wird auch als Länge eines Schedules oder einfach als Zeitspanne C_{max} bezeichnet. Auch die Durchlaufzeit aller Jobs zusammen (Summe der Durchlaufzeiten) kann betrachtet werden.

Im Hinblick auf die Einhaltung der Liefertermine kann man für einen einzelnen Auftrag dessen Verspätung und/oder Verfrühung betrachten. Ein Auftrag ist verspätet, wenn er nach seinem Fälligkeitsdatum abgeschlossen wird ($c_i > dd_i$). Er ist verfrüht, wenn das Fälligkeitsdatum nach seinem Endtermin liegt ($c_i < dd_i$).

Für eine Gruppe von Aufträgen können die maximale Verspätung

$$max\{c_1 - dd_1, \ldots, c_n - dd_n, 0\} \tag{5.1}$$

und/oder maximale Verfrühung

$$max\{dd_1 - c_1, \ldots, dd_n - c_n, 0\} \tag{5.2}$$

von Interesse sein, oder die Summe der Verspätungen L der verspäteten Aufträge

$$la = \sum_{i=1}^{n} \max\{c_i - dd_i, 0\} \tag{5.3}$$

und/oder die Summe der Verfrühungen E der frühen Aufträge

$$ea = \sum_{i=1}^{n} \max\{dd_i - c_i, 0\}. \tag{5.4}$$

Wenn sowohl Verfrühungen als auch Verspätungen wichtig sind, könnte man möglicherweise beide berücksichtigen und die jeweiligen Werte als Unpünktlichkeitsindikator P aufsummieren, wobei Verspätungen stärker gewichtet werden könnte, wenn es wichtiger ist, Verspätungen zu vermeiden als Verfrühungen:

$$P = w_L \cdot la + w_E \cdot ea, \; with \; 0 < w_E \leq w_L. \tag{5.5}$$

Ein anderer Ansatz könnte darin bestehen, einfach die Anzahl der verspäteten oder verfrühten Aufträge zu berechnen.

Im Hinblick auf eine gute Ressourcennutzung könnte man versuchen, so viele Aufträge wie möglich in einem bestimmten Zeitintervall zu verarbeiten (Maximierung der Leistung) oder eine bestimmte Menge von Aufträgen mit so wenig Ressourcen wie möglich zu planen (Minimierung der Kosten).

Der Lastausgleich kann durch Fairnessüberlegungen motiviert sein, insbesondere wenn es sich bei den Ressourcen um Mitarbeiter handelt. Aber auch aus technischen Gründen kann es sinnvoll sein, die vorhandenen Maschinen annähernd gleichmäßig zu nutzen.

Weitere allgemeine Aspekte von Scheduling-Problemen sind, wie sie im Hinblick auf die verfügbaren Informationen gelöst werden und welche Grundvoraussetzungen für Lösungen zu berücksichtigen sind.

Ein grobes Unterscheidungsmerkmal von Scheduling-Problemen ist die Frage, ob es erlaubt ist, Aufträge zu unterbrechen. Wenn die Unterbrechung von Aufträgen

erlaubt ist, wird die Planung auch als präemptives Scheduling bezeichnet. Eine typische Situation für präemptives Scheduling ist die Verarbeitung von Aufträgen auf einem Computer. Typische Multitasking-Betriebssysteme weisen jedem Auftrag winzige Zeitscheiben zu, nach denen die Aufträge in eine Warteschlange gestellt werden und auf ihre nächste Zeitscheibe warten. Das heißt, die Aufträge werden eigentlich sequentiell abgearbeitet (auf einem Einzelprozessor- oder Einzelkernsystem) und nur durch kurze Zwischenwartezeiten entsteht der Eindruck einer parallelen Ausführung.

Bei nicht-präemptiven Scheduling ist es nicht erlaubt, einen bereits begonnenen Auftrag zu unterbrechen. In einigen Bereichen kann dies aus technischen oder organisatorischen Gründen erforderlich sein. In anderen Situationen kann das Unterbrechen von Aufträgen zwar möglich sein, aber da das Anhalten und erneute Starten eines Auftrags mit erheblichen Rüstzeiten (und Kosten) verbunden sein kann, ist dies eher unerwünscht. So erfordert beispielsweise die Unterbrechung eines Transports, die Durchführung eines weiteren Transports und die Fortsetzung des ersten Transports zusätzliche Fahrstrecken und Zeit. Die Unterbrechung eines Produktionsprozesses kann Reinigungsarbeiten, einen Wechsel der Werkzeuge oder einfach die Anpassung eines Arbeiters an neue Aufgaben erfordern. Bei der Erstellung eines Schedules sollten daher Unterbrechungen vermieden werden.

Eine weitere grundsätzliche Frage ist, ob zum Zeitpunkt der Planung alle erforderlichen Informationen verfügbar sind. Bei einfachen Scheduling-Problemen wird in der Regel davon ausgegangen, dass alle Aufträge und sonstigen erforderlichen Informationen zum Zeitpunkt der Erstellung des Plans bekannt sind. In der Praxis sind diese Informationen jedoch oft nur teilweise bekannt und können sich im Laufe der Zeit ändern. So kann beispielsweise nur ein Teil der Aufträge bekannt sein, wenn der Zeitplan für den nächsten Tag erstellt wird. Im Laufe des Tages können dann neue Aufträge (z. B. aufgrund neuer Kundenaufträge) hinzukommen. Möglicherweise werden einige Aufträge gestrichen oder verschoben. Oder die Parameter der Aufträge sind nicht bekannt oder können sich ändern. Es kann z. B. zu unerwarteten Verzögerungen bei den Bearbeitungszeiten kommen.

All diese praktischen Aspekte verwandeln das (statische) Planungsproblem (auch Offline-Planungsproblem genannt) in ein dynamisches Problem, das auch als Online-Planungsproblem bezeichnet wird (Ouelhadj & Petrovic, 2009; Pruhs et al., 2004; Albers, 2003). Bei der Bewältigung einer dynamischen Planungssituation können verschiedene Strategien verfolgt werden. Ein extremer Ansatz wäre, die gegebenen Informationen zu berücksichtigen und das Problem wie ein statisches zu behandeln. Wenn neue Informationen auftauchen (z. B. neue zu planende Aufträge), wird ein völlig neuer Plan berechnet (ohne Berücksichtigung des vorherigen Plans). Ein anderer extremer Ansatz wäre, alle geplanten Aktivitäten beizubehalten und nur den neuen Auftrag irgendwie zum aktuellen Plan hinzuzufügen (oder Aufträge mit geänderten Informationen werden neu geplant, z. B. aufgrund von Verzögerungen bei zuvor geplanten Aufträgen). Beide Ansätze haben ihre unterschiedlichen Vor- und Nachteile. Der erste Ansatz ermöglicht einen völlig neuen Plan, der nach den neuen Informationen optimiert wird. Er ist jedoch kostspieliger in Bezug auf den Planungsaufwand und die Berechnungszeiten und kann zu häufigen und drastischen

Planänderungen führen. Der zweite Ansatz ist weniger kostspielig und führt zu Plänen, die im Zeitablauf robuster sind. Allerdings können die Ergebnisse im Hinblick auf die Planungsziele schlecht sein. Eine bessere Lösung kann auf einem „Kompromiss“ zwischen diesen beiden Ideen beruhen. So werden beispielsweise nur bestimmte Teile des Plans unverändert beibehalten (z. B. „Einfrieren“ der für die nahe Zukunft geplanten Aufträge), während die anderen neu geplant werden. Oder es werden neue Pläne in der „Nachbarschaft“ des aktuellen Plans hergeleitet (z. B. durch einen Ansatz der Nachbarschaftssuche).

5.3 Standard-Scheduling-Probleme

Wie bereits erwähnt, lassen sich Scheduling-Probleme durch verschiedene Merkmale unterscheiden. Auf der Grundlage eines Vorschlags von Graham et al. (1979) hat sich der folgende Klassifizierungsansatz zur Beschreibung verschiedener Arten von Planungssituationen durchgesetzt: Ein Scheduling-Problem wird durch ein 5-Tupel $\alpha/\beta/\gamma/\delta/\varepsilon$ bezeichnet, wobei jeder der griechischen Buchstaben ein Platzhalter für einen Aspekt des Scheduling-Problems ist:

- α steht für die Anzahl der Aufträge
- β bezeichnet die Anzahl der Ressourcen (Maschinen) mit beschränkter Kapazität
- γ bezieht sich auf Nebenbedingungen (z. B. Ankunftszeiten von Aufträgen, Fälligkeitsdaten, Abhängigkeiten zwischen Aufträgen)
- δ bezieht sich auf die Bearbeitungszeiten von Aufträgen (die von der spezifischen Zuweisung von Ressourcen abhängen können)
- ε bezieht sich auf die Zielfunktion (oder Zielfunktionen, wenn mehrere optimiert werden sollen)

Für einige Arten von Scheduling-Problemen, wie z. B. Probleme mit nur einer Maschine ($\beta = 1$), können einfache Regeln zur Optimierung der betrachteten Zielfunktionen zur Verfügung stehen. Wenn beispielsweise die durchschnittliche Durchlaufzeit minimiert werden soll, führt die so genannte Shortest Processing Time (SPT)-Regel (auch als Shortest Job First-Strategie bezeichnet) zu optimalen Ergebnissen. Die Regel besagt, dass (verarbeitungsbereite) Aufträge in der Reihenfolge nicht abnehmender Bearbeitungszeiten ausgeführt werden, d. h. der Auftrag mit der kürzesten Bearbeitungszeit wird zuerst ausgeführt.

Beispiel: Angenommen, es sind drei Aufträge mit einer Bearbeitungszeit von 60 min, 20 min und 10 min auszuführen. Die Ausführung der Aufträge in dieser Reihenfolge führt zu einer durchschnittlichen Durchlaufzeit von (60 min + 80 min + 90 min)/3 = 76,66 min. Die Ausführung der Aufträge in umgekehrter Reihenfolge, d. h. nach der SPT-Regel, führt zu einer durchschnittlichen Durchlaufzeit von (10 min + 30 min + 90 min)/3 = 43,33 min.

Ein Nachteil dieser Regel ist, dass lange Aufträge (mit einer langen Bearbeitungszeit) „verhungern“ können. Dies kann passieren, wenn ständig neue Aufträge eintreffen, während die bereits verfügbaren Aufträge verarbeitet werden. Die Regel

würde verlangen, dass neu eintreffende Aufträge vor bereits wartenden Aufträgen ausgeführt werden, wenn sie kürzere Bearbeitungszeiten haben. Daher kann es sehr lange dauern, bis die Aufträge mit langen Bearbeitungszeiten endlich abgeschlossen sind.

Offensichtlich minimiert die SPT nicht die maximale Durchlaufzeit der Aufträge. Dieses Ziel wird optimiert, wenn die Aufträge in der Reihenfolge ihrer Ankunftszeiten so früh wie möglich ausgeführt werden. Diese Regel kann auch Abhängigkeiten zwischen Aufträgen berücksichtigen. In diesem Fall sind die Aufträge so früh wie möglich in der Reihenfolge ihrer Ankunftszeiten auszuführen, wenn ihre Vorgängeraufträge geplant sind.

Eine ähnliche Regel kann angewandt werden, um die maximale Verspätung einer Gruppe von Aufträgen zu minimieren (Earliest Due Date-Scheduling) (Jackson, 1955). Unter der Annahme, dass es keine Abhängigkeiten zwischen den Aufträgen gibt und dass alle Aufträge zur gleichen Zeit verfügbar sind (oder unterbrochen werden können), sind die Aufträge in der Reihenfolge der nicht abnehmenden Fälligkeitstermine auszuführen.

Im Falle von Abhängigkeiten zwischen Aufträgen kann eine Modifikation dieser Regel angewendet werden, die sicherstellt, dass Aufträge erst dann geplant werden, wenn ihre Vorgänger ausgeführt wurden.

Die am häufigsten betrachteten schwierigeren Planungsprobleme sind das Job-Shop-Scheduling-Problem, das Flow-Shop-Scheduling-Problem und das Open-Shop-Scheduling-Problem, die in den nächsten Unterabschnitten behandelt werden.

5.3.1 Job-Shop-Scheduling

Das Job-Shop-Scheduling-Problem kann wie folgt beschrieben werden: Es gibt n Aufträge, die auf m verschiedenen Maschinen geplant werden müssen. Jeder Auftrag besteht aus einer Reihe von m Vorgängen (die jeder der Maschinen entsprechen), und die Reihenfolge dieser Vorgänge ist vorgegeben, aber nicht unbedingt für alle Aufträge identisch. Jeder Vorgang ist durch die benötigte Maschine und eine bestimmte Bearbeitungszeit gekennzeichnet.

Es gibt weitere Einschränkungen für die Arbeitsplätze und Maschinen, wie z. B.,

- ein Auftrag wird nicht zweimal an dieselbe Maschine geschickt;
- es gibt keine Abhängigkeiten zwischen den Vorgängen verschiedener Aufträge;
- Operationen dürfen nicht unterbrochen werden (nicht-präemptives Scheduling);
- jede Maschine kann jeweils nur einen Auftrag verarbeiten;
- alle Aufträge sind zur gleichen Zeit verfügbar;
- Fälligkeitstermine sind nicht vorgegeben.

Abb. 5.1 zeigt ein kleines Beispiel für ein Job-Shop-Scheduling-Problem mit drei Aufträgen, die auf drei Maschinen bearbeitet werden sollen. In dem Beispiel sollen die Maschinen in einer bestimmten Reihenfolge eingesetzt werden, die für jeden

	Reihenfolge der Maschinen und Bearbeitungszeiten		
Auftrag 1	1, 4 min	2, 3 min	3, 2 min
Auftrag 2	1, 4 min	3, 4 min	2, 3 min
Auftrag 3	2, 4 min	1, 3 min	3, 2 min

Abb. 5.1 Beispiel für ein Job-Shop-Scheduling-Problem mit drei Aufträgen, die aus drei Vorgängen bestehen

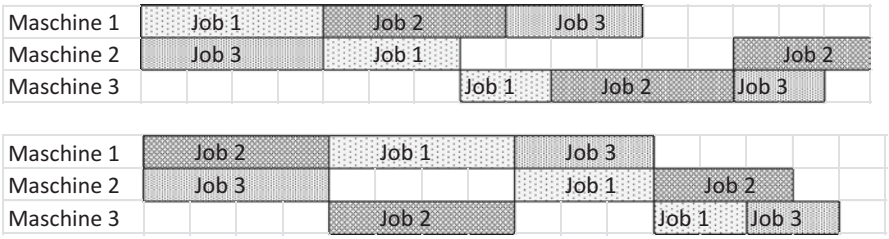


Abb. 5.2 Zwei Gantt-Diagramme mit möglichen Lösungen für das Problem aus Abb. 5.1

Auftrag unterschiedlich ist. Zum Beispiel muss Auftrag 3 zuerst auf Maschine 2, dann auf Maschine 1 und schließlich auf Maschine 3 bearbeitet werden. Die Aufgabenstellung umfasst auch die Angabe der Bearbeitungszeiten der einzelnen Arbeitsgänge.

Die Gantt-Diagramme in Abb. 5.2 zeigen zwei mögliche Lösungen für das Beispielproblem. Es ist zu beachten, dass die zweite Lösung zu einer kleineren Gesamtdurchlaufzeit führt, d. h. zu einer kürzeren Fertigstellungszeit für alle drei Aufträge.

Das übliche Ziel besteht darin, die Gesamtdurchlaufzeit zu minimieren, d. h. die Zeit, die benötigt wird, um alle Aufträge zu erledigen. Es wurde bewiesen, dass dieses Problem NP-schwer ist (Garey et al., 1976).

Eine wichtige Verallgemeinerung des obigen Job Shop Scheduling-Problems stellt das flexible Job Shop Scheduling-Problem dar, bei dem ein Vorgang auf mehr als einer Maschine bearbeitet werden kann, d. h. die Maschinen können mehr als eine Art von Vorgang ausführen (siehe z. B. Gambardella & Mastrolilli, 2000). Wie beim Job Shop Scheduling-Problem steht für jeden Vorgang mindestens eine Maschine zur Verfügung. Aufgrund der Auswahlmöglichkeiten der Maschinen, die für einen Vorgang verwendet werden können, könnte man das flexible Job Shop Scheduling-Problem auch als Zuordnungsproblem (Bestimmung der für jeden Vorgang verwendeten Maschinen) in Kombination mit einem traditionellen Job Shop Scheduling-Problem bezeichnen. Auch dieses Problem ist NP-schwer.

In Anbetracht der modernen Produktionstechnologie und der Idee flexibler Fertigungssysteme (Basnet & Mize, 1994) wird das Szenario des flexiblen Job Shop Scheduling-Problems in der Regel als realistischer angesehen als die traditionelle Problemformulierung. Eine weitere realistische Variante des Problems ist das flexible Job Shop Scheduling-Problem mit Work Centers, bei der davon ausgegangen

wird, dass Ressourcen für identische Tätigkeiten (z. B. Pressen oder Schweißen) als Gruppe (d. h. Work Centers oder Arbeitsplätze) organisiert sind und das erste und das letzte Work Center, das für einen Auftrag verwendet werden soll, im Voraus festgelegt sind (Behnke & Geiger, 2012).

5.3.2 *Flow-Shop-Scheduling*

Das Flow Shop Scheduling-Problem ist ein Spezialfall des Job Shop Scheduling-Problems, das durch eine bestimmte Abfolge von Vorgängen gekennzeichnet ist, die für alle Aufträge gilt. Es gibt n Aufträge, die nacheinander auf m verschiedenen Maschinen (manchmal auch Stufen genannt) eingeplant werden müssen. Wie beim Job Shop Scheduling-Problem sind die Bearbeitungszeiten für jeden Auftrag auf jeder Maschine vorgegeben, die Aufträge dürfen nicht unterbrochen werden, die Maschinen können zu einem Zeitpunkt nur einen Auftrag bearbeiten usw.

Mathematisch ausgedrückt, muss jeder der n Aufträge aus der Menge $J = \{1,\ldots, n\}$ auf m Maschinen in der Reihenfolge $\{1,\ldots, m\}$ bearbeitet werden. Jeder Auftrag $j, j \in J$, besteht aus einer Folge von m Vorgängen oder Operationen, die der Bearbeitung von Auftrag j auf Maschine i während einer ununterbrochenen Bearbeitungszeit $p_{ij} > 0$ entspricht. Das Ziel des Flow Shop Scheduling-Problems ist in der Regel die Minimierung der Gesamtdurchlaufzeit, obwohl auch andere Ziele oder eine Mehrzielvariante praktisch relevant sein können.

Unter geeigneten Bedingungen kann gezeigt werden, dass es ausreicht, nur Permutationspläne zu betrachten. Ein Permutationsplan ist ein Plan, bei dem die Aufträge auf allen Maschinen in der gleichen Reihenfolge abgearbeitet werden und eine Maschine nicht untätig bleibt, wenn ein Auftrag für die Maschine zur Bearbeitung bereitsteht. Die Lösung des Problems kann dann einfach als eine Permutation der Aufträge ausgedrückt werden, die mit $\pi = (j_1,\ldots, j_n)$ bezeichnet wird, wobei $j_i \in \{1,\ldots, n\}$ die Nummer des Auftrags angibt, der an der i-ten Position auf jeder Maschine zu bearbeiten ist. Die Fertigstellungszeit des Vorgangs k von Auftrag j_i, $C(j_i, k)$, kann dann wie folgt berechnet werden (siehe z. B. Liu et al., 2007):

$$C\left(j_1, 1\right) = p_{1j_1},$$

$$C\left(j_i, 1\right) = C\left(j_{i-1}, 1\right) + p_{1j_i},$$

$$C\left(j_1, k\right) = C\left(j_1, k-1\right) + p_{kj_1},$$

$$C\left(j_i, k\right) = \max\left\{C\left(j_{i-1}, k\right), C\left(j_i, k-1\right)\right\} + p_{kj_i},$$

für $i = 2,\ldots, n$, und $k = 2,\ldots, m$. Die resultierende Gesamtdurchlaufzeit ist

$$C_{\max}\left(\pi\right) = C\left(j_n, m\right).$$

Man beachte, dass es Fälle des allgemeinen Flow Shop Scheduling -Problems gibt, in denen Permutationslösungen nicht optimal sind (Dudek et al., 1992). In Lösungsansätzen werden jedoch häufig nur Lösungen berücksichtigt, die durch Permutationen von Aufträgen beschrieben werden. Daher wird eine entsprechende Version des Flow-Shop-Scheduling-Problems, die die zulässigen Lösungen auf Permutationen beschränkt, auch als Permutations-Flow-Shop-Scheduling-Problem bezeichnet.

Eine weitere Variante (und Verallgemeinerung) des Flow-Shop-Scheduling-Problems ist das hybride Flow-Shop-Scheduling-Problem. Bei diesem Problem gibt es mehrere Phasen der Aufträge (die aufeinanderfolgenden Vorgängen entsprechen), wobei für jede Phase mehrere Maschinen verfügbar sein können. Somit ermöglicht dieses Problem eine größere Flexibilität bei der Bearbeitung von Vorgängen, ähnlich wie beim flexiblen Job Shop Scheduling-Problem. Gleichzeitig kann das Problem als eine Verallgemeinerung des Scheduling-Problems für parallele Maschinen betrachtet werden (Ribas et al., 2010). Hybride Flow-Shop-Scheduling-Probleme gibt es in verschiedenen Untervarianten, z. B. mit identischen Maschinen auf jeder Stufe, mit einheitlichen Maschinen, die sich durch unterschiedliche Geschwindigkeiten auszeichnen, oder mit sogenannten unverbundenen Maschinen, die dadurch gekennzeichnet sind, dass die Bearbeitungszeiten für jede Maschine auf einer bestimmten Stufe spezifisch sind.

Eine weitere häufig untersuchte Variante des Flow-Shop-Problems wird als kontinuierliches Flow-Shop-Scheduling-Problem oder No-Wait-Flow-Shop-Scheduling-Problem bezeichnet (Czogalla & Fink, 2012) und erfordert, dass Aufträge ohne Wartezeiten zwischen den Vorgängen abgearbeitet werden, d. h. wenn ein Auftrag gestartet wird, muss sichergestellt werden, dass beim Verlassen einer Maschine die nächste Maschine bereits verfügbar ist.

5.3.3 Open Shop Scheduling

Das Open Shop Scheduling-Problem ähnelt der Job Shop- und Flow Shop-Situation: Es wird davon ausgegangen, dass es n Aufträge gibt, die auf m verschiedenen Maschinen eingeplant werden müssen. Das Open Shop Scheduling-Problem zeichnet sich jedoch dadurch aus, dass es keine Beschränkungen für die Reihenfolge der Vorgänge gibt. Jeder Auftrag muss von jeder Maschine bearbeitet werden, und die jeweiligen Bearbeitungszeiten sind gegeben. Wie zuvor dürfen die Aufträge nicht unterbrochen werden, und jede Maschine kann zu jedem Zeitpunkt nur einen Auftrag bearbeiten (und umgekehrt kann jeder Auftrag zu einem Zeitpunkt nur von einer Maschine bearbeitet werden).

In den meisten Studien zu diesem Problem ist das Ziel die Minimierung der Gesamtdurchlaufzeit – genau wie bei den oben genannten Problemtypen. In einigen Arbeiten wird die Summe der Fertigstellungszeitpunkte der Aufträge (oder, äquivalent dazu, die durchschnittliche Fertigstellungszeit) als Minimierungsziel betrachtet. Beide Problemvarianten sind im allgemeinen Fall für $m > 2$ NP-schwer (Andresen et al., 2008) – genau wie die Job-Shop- und Flow-Shop-Probleme. Allerdings ist

der Suchraum des Open-Shop-Problems aufgrund der viel größeren Freiheit in Bezug auf mögliche Vorgangssequenzen an den Maschinen viel größer, was insbesondere für Lösungsansätze, die auf einer geschickten Exploration des Suchraums beruhen, wie z. B. Branch & Bound-Ansätze, Schwierigkeiten mit sich bringt. Im Gegensatz dazu können Algorithmen, die von der Parallelität Gebrauch machen, von dieser Situation profitieren (Prins, 2000).

Aus praktischer Sicht können Open-Shop-Probleme als stärker anwendungsorientiert angesehen werden als die beiden vorherigen Standardprobleme, das Job-Shop- und das Flow-Shop-Problem in ihren kanonischen Versionen. So bieten beispielsweise workcenter-basierte Fertigung, Wartung oder Reparatur oft einen großen Spielraum bei der Reihenfolge der Vorgänge. Dennoch können einige Einschränkungen für die Reihenfolge gelten, und darüber hinaus können in der Praxis mehrere für bestimmte Vorgänge geeignete Ressourcen zur Verfügung stehen. Daher kann eine Variante eines Open-Shop-Problems oder eine der oben genannten Varianten von Standard-Planungsproblemen für die Modellierung solcher praktischen Situationen besser geeignet sein.

5.4 Spezifische Scheduling-Probleme in der Logistik

Es sei darauf hingewiesen, dass es viele andere Arten von Scheduling-Problemen gibt, die für Logistikanwendungen relevant sein können. Einschlägige Beispiele finden sich sicherlich auf dem Gebiet der ressourcenbeschränkten Projektplanung (Herroelen et al., 1998; Brucker et al., 1999). Solche Probleme spiegeln allgemeine Scheduling-Situationen mit knappen Ressourcen und Aufträgen mit verschiedenen Arten von Abhängigkeiten wider, wie sie in der Produktion, im Bauwesen, bei der Projektarbeit, aber auch bei spezifischen Logistikaktivitäten auftreten können. Andere Standard-Scheduling-Probleme wie das Job-Shop-Scheduling-Problem können als Sonderformen der ressourcenbeschränkten Projektplanung betrachtet werden. Die Vielfalt der ressourcenbeschränkten Projektplanungsprobleme kann z. B. auch verschiedene Modi im Sinne von Verarbeitungsalternativen für Aufträge oder verschiedene Arten von Ressourcen wie erneuerbare und nicht erneuerbare Ressourcen berücksichtigen.

Ein weiterer Planungsbereich mit erheblicher Relevanz für die Logistikpraxis ist sicherlich die Personaleinsatzplanung, die auch als Dienstplanung bezeichnet wird. Das Problem geht davon aus, dass es verschiedene mögliche Zeitfenster gibt, in denen Mitarbeiter arbeiten könnten, und dass eine entsprechende Planung notwendig ist (im Gegensatz zur Selbstplanung oder Selbstverwaltung, die keine zentrale Planungslogik voraussetzt). Dienstplanungsprobleme treten vor allem dort auf, wo Mitarbeiter im Schichtbetrieb arbeiten, z. B. in Krankenhäusern, in großen Industriebetrieben oder bei Transportdienstleistern wie der Bahn. In einigen Bereichen können solche Probleme mit klar definierten Zeitfenstern für die Schichten formuliert werden. In anderen Bereichen, z. B. im Bahnbetrieb, können die Zeitfenster im Zeitablauf variieren, da die Dienste nicht willkürlich unterbrochen werden können.

Stattdessen kann es Vorgaben für die maximale Schichtdauer und Regeln für den Ausgleich langer Schichten durch kürzere Schichten oder freie Tage geben.

Abgesehen von solchen Komplikationen, die sich aus den Anforderungen der zugrundeliegenden Arbeitstätigkeiten ergeben, können Dienstplanungsprobleme weitere Beschränkungen enthalten, z. B. hinsichtlich der Zulässigkeit bestimmter Schichtzuweisungen, der Gesamtzahl der Einsätze einer Person, der Zeit zwischen zwei aufeinanderfolgenden Einsätzen oder bezüglich der Zuweisung bestimmter Einsätze (z. B. Nachtschichten). Die Zielfunktion könnte ein Maß für die Verletzung allgemeiner Beschränkungen enthalten oder auch individuelle Präferenzen der Mitarbeiter berücksichtigen.

Eine ähnliche Art von Scheduling-Problemen mit definierten Zeitfenstern für Aktivitäten findet sich im Bereich der Stundenplanerstellung. Hier gibt es in der Regel feste Zeitfenster für Aktivitäten gleicher oder unterschiedlicher Länge, und die Aktivitäten müssen geeigneten Zeitfenstern zugewiesen werden, oft auf wiederkehrender Basis (z. B. wöchentlich). Typische Beispiele sind die Stundenplanprobleme in Schulen, Universitäten oder anderen Bildungseinrichtungen. Aktivitäten wie Kurse müssen sorgfältig geplant werden, um Zeitkonflikte (wie sich überschneidende Zuweisungen) für Teilnehmer wie Schüler, Klassen und Lehrer zu vermeiden. Auch andere Ressourcen wie Räume oder andere Infrastruktur müssen berücksichtigt werden, und es können weitere Anforderungen gelten, z. B. die minimalen oder maximalen zeitlichen Abstände zwischen geplanten Aktivitäten oder die einer Person zugewiesene maximale oder minimale Zeit pro Tag.

Bei bestimmten Vorgängen, wie z. B. im Transportmanagement, sind die Aufgaben der Planung und des Dienstplans mit anderen Aktivitäten, wie z. B. Transportvorgängen, verknüpft. Das bedeutet, dass bei der Planung nicht nur die zeitliche Zuweisung von Ressourcen berücksichtigt wird, sondern auch die Durchführung des Transports und folglich der Wechsel der Standorte berücksichtigt werden muss. Die Transportzeiten ergeben sich beispielsweise aus den Entfernungen zwischen den Standorten, der angenommenen Geschwindigkeit und der Zeit, die für verwandte Zwecke benötigt wird, z. B. für Pausen, Umladetätigkeiten oder Fahrzeugwechsel. Bei der Gesamtplanung von Aktivitäten kann es erforderlich sein, Ressourcen (Fahrzeuge, Fahrer usw.) zunächst zu den Orten der Aktivitäten (z. B. Abholorte) zu bringen und sie nach Beendigung der Aufgaben wieder zu den definierten Orten (z. B. Depots) zurückzubringen. Aufgrund dieser besonderen Situationen werden Transportplanungsaktivitäten in der Regel als ganz andere Planungsprobleme betrachtet und meist als Routing oder Vehicle Routing bezeichnet (siehe Kap. 3).

Für die Anbieter von Linienverkehren stellen sich verschiedene Planungsprobleme. Die angebotenen Verkehrsdienste haben eine wiederkehrende Struktur, die darauf beruht, dass sie in einer Abfolge, auch Linie genannt, an verschiedenen Orten ankommen. Im Linienverkehr gibt es oft Fahrpläne, die diese Orte mit bestimmten Taktstrukturen bedienen, z. B. ein Zug pro Stunde. Bei der Planung des Linienbetriebs und der Ausarbeitung von Fahrplänen müssen unter anderem der erwartete

Beförderungsbedarf, die verfügbaren Ressourcen und Kosten sowie die erforderliche Zeit für die Fortbewegung, die Halte und andere Zwecke berücksichtigt werden.

Eine weitere Komplikation bei der Planung ergibt sich aus der Koordinierung der verschiedenen Linien: Ein größeres Netz von Orten kann von mehreren Linien bedient werden, die es Fahrgästen (oder Gütern) ermöglichen, von einer Linie auf eine andere umzusteigen (bzw. umgeschlagen zu werden). Es ist ratsam, unnötige Wartezeiten an Umsteigeorten zu vermeiden, z. B. indem zwei Verkehrsmittel gleichzeitig an einem gemeinsamen Ort halten und warten, bis der Umsteigevorgang beendet ist. Dieses Konzept hat jedoch neben der erforderlichen Umsteigezeit auch einige Nachteile: Wenn sich ein Verkehrsmittel verspätet, verspätet sich auch der Betrieb des anderen Verkehrsmittels, wenn es auf den entsprechenden Anschluss wartet. Dies kann zu weiteren Verspätungen bei nachfolgenden Verbindungen führen, und möglicherweise wird die Verspätung schließlich auf das gesamte Netz verteilt. Wenn in den Betriebsabläufen keine Wartezeiten vorgesehen sind, kann eine Verspätung oft zu verpassten Anschlüssen führen. Daher kann das Verspätungsmanagement ein wichtiges Thema bei der Planung solcher Verkehrsnetze und der Berechnung von Fahrplänen sein (siehe z. B. Liebchen et al., 2010).

5.5 Lösung von Scheduling-Problemen mit Techniken der Computational Intelligence

Die diskutierten Standardplanungsprobleme (Job Shop, Flow Shop und Open Shop) können nur in speziellen Fällen effizient gelöst werden, die in der Regel von der Problemgröße abhängen (Garey et al., 1976; Williamson et al., 1997). So kann beispielsweise das Open-Shop-Scheduling-Problem durch einen polynomiellen Algorithmus gelöst werden, wenn es entweder nur zwei Aufträge oder zwei Maschinen gibt oder wenn alle Bearbeitungszeiten identisch sind. Im Allgemeinen ist das Problem NP-schwer, was es zu einem guten Kandidaten für die Anwendung metaheuristischer Algorithmen macht.

Infolgedessen gibt es eine Vielzahl von Veröffentlichungen, die sich mit Metaheuristiken und anderen Techniken aus dem Bereich Computational Intelligence zur Lösung von Scheduling-Problemen befassen. Während Fuzzy-Ansätze häufig verwendet werden, um eine gewisse Unschärfe in der Formulierung von Scheduling-Problemen zu modellieren (z. B. ungenaue Bearbeitungszeiten, unscharfe Fälligkeitstermine oder flexible Nebenbedingungen, siehe z. B. Dubois et al. (2003)), scheinen neuronale Netze im Scheduling eine untergeordnete Rolle zu spielen. In den folgenden Unterkapiteln konzentrieren wir uns auf eine geeignete Repräsentation von Scheduling-Problemen, wie sie für die Anwendung von Lösungstechniken benötigt wird, und fügen einige Informationen über die aktuelle Bedeutung des Einsatzes von Metaheuristiken hinzu.

5.5.1 Aspekte der Kodierung

Für die Behandlung von Scheduling-Problemen mit einem geeigneten Optimierungsverfahren oder einer Heuristik ist es in der Regel nicht erforderlich, die genauen Zeiten für die Ausführung von Aufträgen oder Vorgängen zu bestimmen. Stattdessen reicht es meist aus, die Reihenfolge der Ausführung (bei Zuordnung zu einer bestimmten Maschine) zu bestimmen. Mit dieser Information ist es nicht schwierig, die genauen Zeiten zu berechnen, wenn man davon ausgeht, dass die Aufträge in der vorgegebenen Reihenfolge so früh wie möglich ausgeführt werden. Zum Beispiel wird der erste Auftrag so früh wie möglich ausgeführt, d. h. nicht vor seiner Ankunftszeit und sobald die Maschine verfügbar ist (es könnten bereits andere Aufträge für diese Maschine festgelegt sein). Wenn man die Bearbeitungszeit des Auftrags kennt, kann man berechnen, wann die Maschine wieder frei wird (auch hier möglicherweise unter Berücksichtigung der bereits festgelegten Aufträge). Zu diesem Zeitpunkt kann der zweite Auftrag in der Reihenfolge gestartet werden – oder später, wenn die Maschine zu diesem Zeitpunkt noch nicht verfügbar ist, und so weiter. (Siehe oben für ein Beispiel für diesen Ansatz im Fall des Flow Shop Scheduling in Abschn. 5.3.2).

Ein solches Verfahren zur Berechnung der Start- und Endzeiten von Aufträgen könnte auch Vorrangbeziehungen berücksichtigen, wenn diese nicht bereits in der Reihenfolge enthalten sind. So kann man beispielsweise bei einem Job Shop Scheduling-Problem berücksichtigen, dass der zweite Vorgang eines Auftrags nicht begonnen werden kann, bevor der erste Vorgang abgeschlossen ist. Dies kann leicht erreicht werden, wenn die konkreten Pläne für die verschiedenen Maschinen schrittweise erstellt werden, wobei immer mit der nächsten frei werdenden Maschine fortgefahren wird (zu Beginn, wenn vielleicht alle Maschinen frei sind, spielt die Reihenfolge der betrachteten Maschinen keine Rolle).

Die Darstellung von Schedules in dieser abstrakteren Form (d. h. nur die Zuweisung zu den Maschinen und die Reihenfolge der ausgeführten Aufträge pro Maschine) hat erhebliche Vorteile. Der Suchraum ist sehr viel kleiner und das Problem der Berechnung von nicht durchführbaren Lösungen durch Veränderungsoperationen (z. B. Mutationen in einem evolutionären Algorithmus) wird entschärft. Nachteilig ist natürlich, dass die Bewertung einer Lösung (z. B. die Berechnung der Gesamtdurchlaufzeit) zusätzliche Rechenzeit benötigt.

Beispiel
Auftrag 1: Vorgang 1: 8 min; Vorgang 2: 10 min
Auftrag 2: Vorgang 1: 15 min; Vorgang 2: 5 min
Auftrag 3: Vorgang 1: 5 min; Vorgang 2: 10 min
Maschine 1: Auftrag 1; Auftrag 2; Auftrag 3
Maschine 2: Auftrag 3; Auftrag 1; Auftrag 2

Daraus ergibt sich der folgende Zeitplan:

Maschine 1: Auftrag 1: 0–8 min, Auftrag 2: 8–23 min; Auftrag 3:23–28 min.
Maschine 2: Auftrag 3: 0–10 min; Auftrag 1: 10–20 min; Auftrag 2: 23–28 min

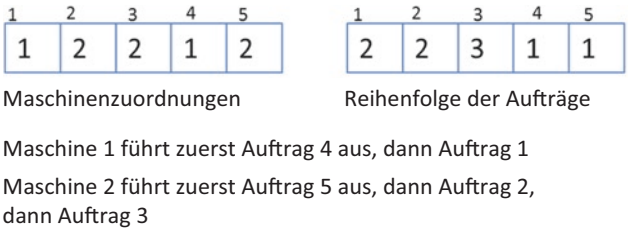


Abb. 5.3 Beispiel für die Kodierung der Lösung eines Scheduling-Problems mit fünf Aufträgen. Das erste Array definiert die Zuweisungen der Aufträge an Maschinen, während das zweite Array die Reihenfolge für die jeweiligen Maschinen festlegt

Aus dem oben genannten Grund sind für die Repräsentation der Lösung eines Scheduling-Problems hauptsächlich zwei Dinge erforderlich: die Zuweisung bestimmter Maschinen für Aufgaben oder Vorgänge (falls dies nicht von vornherein klar ist, wie z. B. bei Job-Shop-Problemen) und die Reihenfolge der Aufgaben oder Vorgänge an den zugewiesenen Maschinen. Beispielsweise gibt es ein ganzzahliges Array der Länge o mit den Werten $j \in \{1,\ldots, m\}$ an der Position i, das besagt, dass die Operation $i \in \{1,\ldots, o\}$ der Maschine j zugewiesen ist. Eine andere Möglichkeit ist, dass die Werte j Indexwerte sind, die die *j-te* Maschine von denen darstellen, die für die Operation i in Frage kommen (Abb. 5.3) (vgl. Zhang et al., 2010).

Der reihenfolgespezifische Teil einer Lösung, der Sequenzen oder Permutationen von Objekten (Aufgaben, Vorgänge) darstellt, könnte auf einfache Weise wie folgt konstruiert werden: Man könnte z. B. die Objekte 1,…, 5 durch einen String (Zeichenkette) der Länge 5 darstellen, z. B. (2, 4, 5, 3, 1), wenn zuerst Vorgang 2 ausgeführt wird, dann Vorgang 4, usw. Eine entsprechende Lösungsrepräsentation könnte möglicherweise auch vereinfacht werden: Beispielsweise wird in Zhang et al. (2010) für den Fall eines flexiblen Job-Shop-Scheduling-Problems nur die Auftragsnummer für deren Reihenfolge angegeben, nicht aber für die einzelnen Vorgänge, da die Reihenfolge der Vorgänge innerhalb eines Auftrags vorgegeben ist.

Eine weitere Möglichkeit zur Darstellung eines Zeitplans, z. B. für Open-Shop-Probleme, ist die Rangmatrix mit ihren Vorteilen der Vermeidung von Redundanz und der eindeutigen Darstellung von Lösungen (Andresen et al., 2008). Ein rein auf Reihenfolgen basierender Ansatz wird in Prins (2000) für den Fall des Open Shop Scheduling-Problems untersucht. Die Reihenfolge der Aufträge kann als Prioritätsliste interpretiert werden. Auf der Grundlage dieser Prioritätsliste und unter Verwendung bestimmter einfacher Heuristiken wird ein Schedule erstellt. So wird beispielsweise eine Heuristik ohne Verzögerung für die Planung des nächsten Auftrags verwendet oder eine Heuristik, die auf dem FIFO-Prinzip (First-in-First-out) basiert.

Eine solche direkte Darstellung von Reihenfolgen kann jedoch im Rahmen einer Suchstrategie nachteilig sein: Betrachten wir eine kleine Änderung einer Lösung, z. B. die Änderung des letzten Elements in einer Sequenz von 1 auf 2 aufgrund einer zufälligen Mutation. Die sich daraus ergebende Repräsentation (2, 4, 5, 3, 2) entspricht einer unzulässigen Lösung, da „2“ zweimal in der Sequenz vorkommt, wäh-

rend „1“ überhaupt nicht vorkommt. Es sind also mindestens zwei „synchronisierte“ Änderungen in der Sequenz notwendig, um wieder eine zulässige Lösung zu erhalten.

Eine alternative Repräsentation einer Reihenfolge, die als Random Key-Kodierung bezeichnet wird, hilft, solche Zulässigkeitsprobleme zu vermeiden (siehe Bean, 1994): Nehmen wir an, dass die Reihenfolge von Elementen durch reelle Werte dargestellt wird, z. B. (1:0,75, 2:0,08, 3:0,72, 4:0,33, 5:0,51). Der reelle Wert an der Position *i* im Vektor gibt seine „Priorität“ in der Reihenfolge an. Sortiert man die Elemente nach dieser Zahl, ergibt sich die dargestellte Reihenfolge (2:0,08, 4:0,33, 5:0,51, 3:0,72, 1:0,75). In diesem Beispiel gehen wir davon aus, dass niedrigere Werte eine frühere Ausführung einer Aufgabe anzeigen.

Eine Änderung dieser Werte führt immer zu einer zulässigen Lösung, und kleine Änderungen des Wertes führen in der Regel zu kleinen Änderungen in der Lösung, z. B. eine Verringerung der Werte für Objekt 1 um 0,1 tauscht ihn mit Objekt 3 (dem letzten in der betrachteten Reihenfolge), während eine Erhöhung überhaupt keine Auswirkungen hat.

Auch bei der Berechnung von Lösungskombinationen (z. B. durch Rekombination oder Crossover in einem evolutionären Algorithmus) wird das Ergebnis bei Verwendung der direkten Repräsentation von Reihenfolgen in der Regel unzulässig, während die Random Key-Kodierung dazu beiträgt, die Zulässigkeit zu bewahren.

Die Random Key-Kodierung wird häufig im Rahmen genetischer Algorithmen verwendet (Bean, 1994; Mendes et al., 2009; Gonçalves et al., 2011), kann aber auch in Verbindung mit anderen Metaheuristiken eingesetzt werden, z. B. bei der differentiellen Evolution (Nearchou & Omirou, 2006), der lokalen Suche (Gonçalves et al., 2005), der variablen Nachbarschaftssuche (Roshanaei et al., 2009), der Particle Swarm Optimization (Lin et al., 2010; Kuo et al., 2009) oder bei memetischen Algorithmen (Liu et al., 2007).

Während die Random Key-Kodierung die Zulässigkeit im Hinblick auf die Beibehaltung einer wohldefinierten Reihenfolge während der Variationsoperationen in den Suchalgorithmen gewährleisten kann, werden weitere Einschränkungen, wie z. B. Vorrangsbedingungen, nicht automatisch beachtet. Daher könnte man über fortgeschrittenere Operatoren nachdenken, die die Zulässigkeit in einem allgemeineren Sinne berücksichtigen.

Betrachten wir zunächst noch einmal das einfachere Problem der Repräsentation von Reihenfolgen und deren Zulässigkeit während des Crossovers oder der Rekombination in einem evolutionären Algorithmus. Wir gehen davon aus, dass die Sequenzen „direkt“ dargestellt werden, d. h. für die Darstellung der Sequenz von *n* Jobs haben wir ein Chromosom der Länge *n* mit Einträgen, die die Job-Indizes darstellen, d. h. der erste Eintrag ist der Index des zuerst ausgeführten Jobs und so weiter. Im Beispiel in Abb. 5.4 sollen zwei Elternlösungen, die Reihenfolgen von acht Jobs repräsentieren, durch ein Ein-Punkt-Crossover rekombiniert werden. Nach Bestimmung einer zufälligen Crossover-Position (im Beispiel nach Chromosom 4) wird der erste Teil jeder Elternlösung in den entsprechenden Teil einer Nachkommenlösung kopiert. Anders als beim regulären Crossover wird der zweite Teil der Nachkommenlösung nicht mit Daten des zweiten Elternteils an denselben Chromosomenpositionen aufgefüllt. Stattdessen wird die Lösung des zweiten Elternteils

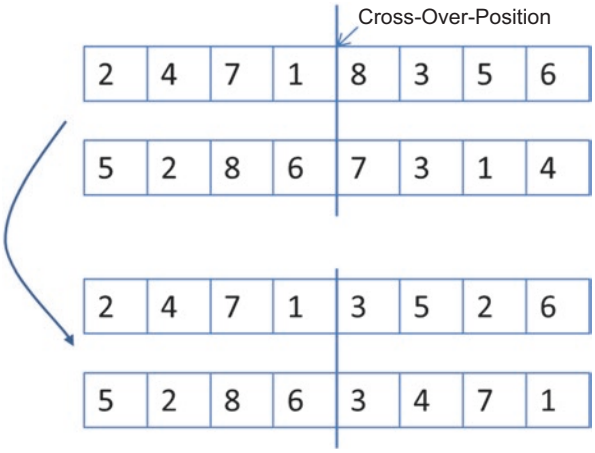
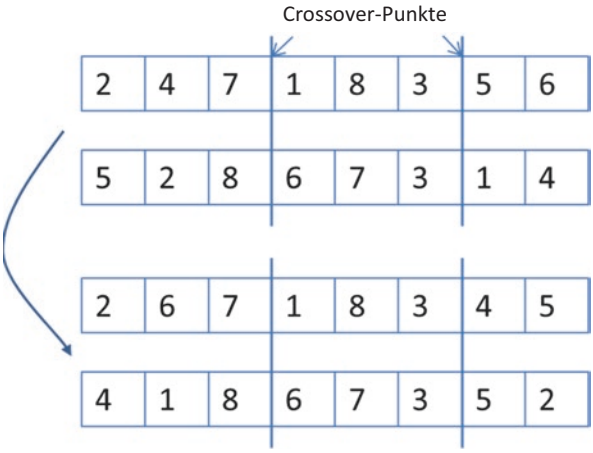


Abb. 5.4 Beispiel für eine Ein-Punkt-Crossover für eine Reihenfolge von acht Aufträgen

Abb. 5.5 Beispiel für ein Zwei-Punkt-Crossover OX für eine Reihenfolge von acht Aufträgen



(beginnend an der Kreuzungsstelle und nach rechts) nach Einträgen durchsucht, die im ersten Teil der Nachkommenlösung noch nicht enthalten sind, um die verbleibenden Chromosomenstellen aufzufüllen. Wenn das Ende des Chromosoms erreicht ist, wird der Prozess an der ersten Position fortgesetzt. Schließlich erscheint jede Zahl {1,…, 8} genau einmal in jeder Nachkommenlösung.

In dem Beispiel in Abb. 5.5 sollen zwei Elternlösungen, die Sequenzen von acht Aufträgen darstellen, durch eine Zweipunkt-Crossover genannt OX neu kombiniert werden. Zunächst werden zwei Crossover-Positionen zufällig bestimmt. Der mittlere Teil zwischen den beiden Crossover-Stellen wird von jeder Elternlösung in eine entsprechende Nachkommenlösung kopiert. Die restlichen Teile werden ähnlich wie bei dem Ein-Punkt-Crossover in Abb. 5.4 aufgefüllt. Das heißt, ab dem zweiten Crossover-Punkt (und nach rechts) wird geprüft, ob die Einträge bereits in der Nachkommenlösung enthalten sind. Wenn nicht, werden sie für die aktuelle Posi-

tion verwendet, die in der Nachkommenlösung gefüllt werden muss. Wenn das Ende des Elternchromosoms erreicht ist, wird der Prozess mit der ersten Position der Elternlösung fortgesetzt. In ähnlicher Weise wird der Prozess mit der ersten Position der Nachkommenlösung fortgesetzt, wenn der letzte Nachkommen-Eintrag gefüllt ist, um auch den Teil vor der ersten Crossover-Position zu füllen.

Es sei angemerkt, dass diese Operatoren recht effizient arbeiten. Aus algorithmischer Sicht erfolgen die Konstruktion neuer Lösungen und die Kontrolle, ob keine Jobs vergessen wurden oder doppelt in einer Sequenz vertreten sind, gleichzeitig ohne relevanten Mehraufwand. In einem Array kann für jeden Auftrags-Index festgehalten werden, ob er bereits in einer Nachkommenlösung enthalten ist oder nicht. Damit entfällt bei der Konstruktion einer neuen Lösung die wiederholte Suche, ob ein aktuell betrachteter Eintrag bereits enthalten ist.

Bei der Betrachtung komplexerer Scheduling-Probleme mit weiteren Nebenbedingungen, wie z. B. Reihenfolgebeschränkungen, können ähnliche Konzepte zur Erhaltung der Zulässigkeit verwendet werden. Im Rahmen evolutionärer Algorithmen werden insbesondere zwei Kodierungsansätze und verwandte Operatoren verwendet: das präzedenzerhaltende ordnungsbasierte Crossover (precedence preserving order-based crossover, POX) und die präzedenzserhaltende Verschiebungsmutation (precedence preserving shift mutation, PPS) (Lee et al., 1998; vgl. auch Pezzella et al., 2008). Die POX-Operation arbeitet mit zwei Lösungsrepräsentationen (Eltern) und leitet, wie bei Crossover üblich, zwei Nachkommenlösungen ab. Zunächst wird ein Vorgang aus dem ersten Elternteil ausgewählt und zusammen mit allen anderen Vorgängen, die zum selben Auftrag gehören, in die erste Nachkommenlösung kopiert. Danach wird die Lösung mit den verbleibenden Vorgängen vervollständigt, und zwar in der gleichen Reihenfolge, wie sie im zweiten Elternteil dargestellt ist. Für die zweite Nachfolgelösung wird das Verfahren analog angewendet. Auf diese Weise bewahrt POX die Vorrangsbedingungen.

Es sei darauf hingewiesen, dass es mehrere andere Rekombinationsoperatoren gibt, die die Zulässigkeit der resultierenden Lösungen berücksichtigen, z. B. das Partial-Mapped Crossover (PMX) oder das oben diskutierte Order Crossover (OX), die als Zwei-Punkt-Crossover (Austausch zufällig ausgewählter Teilstrings) mit einem anschließenden Reparaturmechanismus interpretiert werden können (Cheng et al., 1999). Weitere in der Literatur gelegentlich verwendete Rekombinationsoperatoren sind positionsbasiertes Crossover, reihenfolgebasiertes Crossover (order-based crossover), zyklisches Crossover (cycle crossover, CX), Linear Order Crossover (LOX), Teilsequenz-Austausch-Crossover (subsequence exchange crossover), auftragsbasiertes Reihenfolge-Crossover (job-based order crossover), Teil-Schedule-Austausch-Crossover (partial schedule exchange crossover), Teilstring-Austausch-Crossover (substring exchange crossover) (Cheng et al., 1999). Es sei erwähnt, dass Crossover-Operatoren für reihenfolgebasierte Repräsentationen für Scheduling-Problemlösungen den entsprechenden Operatoren im Kontext des TSP ähneln, vgl. Abschn. 3.4.2.

PPS (Lee et al., 1998) wählt einen Vorgang aus einem einzelnen Elternchromosom aus und verschiebt ihn an eine andere Position, wobei die Vorrangsbedingungen für diesen Vorgang beachtet werden. Andere Möglichkeiten für (zufällige)

Änderungen sind die Inversionsmutation, bei der ein durch zwei zufällige Stellen definierter Teilstring im Chromosom invertiert wird. In ähnlicher Weise kann eine zufällige Teilzeichenkette an eine neue zufällige Position verschoben werden. Ein weiterer Mutationsansatz ist die Austauschmutation, bei der die Gene an zwei zufälligen Positionen ausgetauscht werden (Cheng et al., 1999).

Eine weitere wichtige Voraussetzung für die Anwendung verschiedener Metaheuristiken ist die Definition einer Nachbarschaft, z. B. für die lokale Suche, die Tabu-Suche oder die variable Nachbarschaftssuche (Gambardella & Mastrolilli, 2000; Zhang et al., 2007, 2008, 2010). Eine kleine Nachbarschaft kann durch das Verschieben eines Vorgangs definiert werden. Da Lösungen mit kürzerer Gesamtdurchlaufzeit gesucht werden, kann man die Nachbarschaft auf Verschiebungen von Vorgängen auf dem kritischen Pfad beschränken. Größere Nachbarschaften (wie sie z. B. für die variable Nachbarschaftssuche VNS erforderlich sind) können durch die Verschiebung von zwei Vorgängen auf einem kritischen Pfad definiert werden, z. B. in Form von zwei Austauschvorgängen (Zhang et al., 2007; Gao et al., 2008).

Ein anderer Ansatz für die Nachbarschaftssuche mit zwei Nachbarschaften (ein iteratives Abstiegsverfahren) sieht folgendermaßen aus: Eine Nachbarschaft wird definiert, indem ein Auftrag von einer Maschine mit maximaler Zeitspanne einer anderen Maschine zugewiesen wird. Die andere Nachbarschaft wird durch die Vertauschung der Maschinenzuweisung von zwei Aufträgen definiert. In der Metaheuristik des iterativen Abstiegs werden zunächst sukzessive Verbesserungen im ersten Nachbarschaftstyp gesucht. Wenn sich keine weiteren Verbesserungen ergeben, werden sukzessive Verbesserungen durch die Erkundung von Nachbarschaften des Typs 2 angestrebt.

5.5.2 Verwendung von Metaheuristiken im Scheduling

Job Shop Scheduling-Probleme wurden mit fast allen bekannteren und sogar einigen weniger bekannten CI-Techniken behandelt. Aufgrund dieser umfangreichen Studien zu Scheduling-Problemen versuchen wir nicht, diesen Aspekt im Detail zu behandeln.

Ein kurzer Blick in die Literatur zeigt, dass unter den dominierenden Metaheuristiken genetische Algorithmen zu finden sind, die z. B. in Della Croce et al. (1995) verwendet werden. Erste Anwendungen genetischer Algorithmen gehen auf die 1980er-Jahre zurück und gehören damit auch zu den ältesten Metaheuristiken, die zur Lösung von Scheduling-Problemen eingesetzt wurden (siehe z. B. Davis, 1985).

Ruiz und Vázquez-Rodríguez (2010) geben einen interessanten Überblick über die Verwendung verschiedener Lösungstechniken für das hybride Flow-Shop-Problem. Ihren Untersuchungen zufolge dominieren spezifische Heuristiken und Dispositionsregeln in den Veröffentlichungen, die sich mit den entsprechenden Problemen befassen (50 %), während mathematische Programmierung und Branch & Bound-Ansätze an zweiter Stelle stehen (25 %). Unter den Anwendungen von Metaheuristiken werden genetische Algorithmen am häufigsten verwendet (8 %). Auf

den Plätzen 2 und 3 der Metaheuristiken liegen die Tabu-Suche (6 %) und das Simulated Annealing (5 %). Es sei darauf hingewiesen, dass die Studie Veröffentlichungen seit dem Jahr 1970 umfasst und dass die Verwendung von Metaheuristiken in den letzten 15 Jahren erheblich zugenommen hat.

In einem Überblick ab 2005 über neuere Ansätze für das flexible Job Shop Scheduling-Problem (Ye & Ma, 2015) basieren zehn von 25 Ansätzen (40 %) auf evolutionären Algorithmen (insbesondere genetischen Algorithmen), drei auf Ant Colony Optimization (12 %), weitere drei auf anderen Metaheuristiken (12 %), während neun auf mathematischen Ansätzen oder problemspezifischen Heuristiken (36 %) beruhen. Somit beruhen in der neueren Erhebung 64 % der Ansätze auf Metaheuristiken.

Leider ist es nicht einfach, ähnlich zusammengestellte Ergebnisse für andere Arten von Scheduling-Problemen zu finden, die Aufschluss über den Einsatz, die Häufigkeit und die Verteilung der verschiedenen Klassen von Methoden geben. Die Ergebnisse für das flexible Job Shop Scheduling-Problem scheinen jedoch nicht untypisch für die aktuelle Bedeutung von Metaheuristiken zu sein, während ältere Bibliographien (z. B. Hoogeveen et al., 1997) einen deutlich geringeren Anteil an Metaheuristiken zeigen.

Lösungsalgorithmen für Scheduling-Probleme werden häufig anhand etablierter Benchmark-Instanzen getestet, wie sie beispielsweise von Taillard (1993) bereitgestellt wurden. Obwohl dies immer noch eine der wichtigsten Quellen für Scheduling-Probleme zum Vergleich verschiedener Algorithmen ist, wurden von verschiedenen Autoren neue Sammlungen hinzugefügt. Obwohl in Studien zu verschiedenen Metaheuristiken (und anderen Methoden) oft identische Testprobleme verwendet werden, ist es nicht einfach, allgemeine Ergebnisse darüber abzuleiten, welche Methoden am besten geeignet sind (insbesondere im Hinblick auf die Leistung). Ein Grund dafür sind die meist unterschiedlichen Testbedingungen (z. B. Computerhardware). Ein weiterer Grund ist, dass die Leistung weitgehend von der Anpassung der Methode an die Besonderheiten des Problems wie Kodierung, Anpassung der Operatoren und Parametereinstellungen abhängt. Es scheint jedoch einige Hinweise darauf zu geben, dass für einige Problemtypen nachbarschaftsorientierte Ansätze (wie Tabu-Suche) am besten funktionieren, während für andere parallele Suchansätze wie genetische Algorithmen Vorteile zu haben scheinen. Auf jeden Fall gibt es klare Hinweise auf die Wirksamkeit einiger Metaheuristiken, da sie es ermöglichten, Testprobleme zu lösen, die zuvor praktisch unlösbar waren, oder die Lösungszeiten zu verkürzen.

Literatur

Albers, S. (2003). Online algorithms: A survey. *Mathematical Programming, 97*(1–2), 3–26.

Andresen, M., Bräsel, H., Mörig, M., Tusch, J., Werner, F., & Willenius, P. (2008). Simulated annealing and genetic algorithms for minimizing mean flow time in an open shop. *Mathematical and Computer Modelling, 48*(7), 1279–1293.

Basnet, C., & Mize, J. H. (1994). Scheduling and control of flexible manufacturing systems: A critical review. *International Journal of Computer Integrated Manufacturing, 7*(6), 340–355.

Bean, J. C. (1994). Genetic algorithms and random keys for sequencing and optimization. *ORSA Journal on Computing, 6*(2), 154–160.

Behnke, D., & Geiger, M. J. (2012). *Test instances for the flexible job shop scheduling problem with work centers*. Working paper. Helmut-Schmidt-Universität.

Brucker, P., & Brucker, P. (2007). *Scheduling algorithms* (5. Aufl.). Springer.

Brucker, P., Drexl, A., Möhring, R., Neumann, K., & Pesch, E. (1999). Resource-constrained project scheduling: Notation, classification, models, and methods. *European Journal of Operational Research, 112*(1), 3–41.

Cheng, R., Gen, M., & Tsujimura, Y. (1999). A tutorial survey of job-shop scheduling problems using genetic algorithms, part II: Hybrid genetic search strategies. *Computers & Industrial Engineering, 36*(2), 343–364.

Czogalla, J., & Fink, A. (2012). Fitness landscape analysis for the no-wait flow-shop scheduling problem. *Journal of Heuristics, 18*(1), 25–51.

Davis, L. (1985). Job shop scheduling with genetic algorithms. In *Proceedings of an international conference on genetic algorithms and their applications* (Bd. 140). Carnegie-Mellon University.

Della Croce, F., Tadei, R., & Volta, G. (1995). A genetic algorithm for the job shop problem. *Computers & Operations Research, 22*(1), 15–24.

Dubois, D., Fargier, H., & Fortemps, P. (2003). Fuzzy scheduling: Modelling flexible constraints vs. coping with incomplete knowledge. *European Journal of Operational Research, 147*(2), 231–252.

Dudek, R. A., Panwalkar, S. S., & Smith, M. L. (1992). The lessons of flowshop scheduling research. *Operations Research, 40*(1), 7–13.

Gambardella, L. M., & Mastrolilli, M. (2000). Effective neighborhood functions for the flexible job shop problem. *Journal of Scheduling, 3*(1), 3–20.

Gao, J., Sun, L., & Gen, M. (2008). A hybrid genetic and variable neighborhood descent algorithm for flexible job shop scheduling problems. *Computers & Operations Research, 35*(9), 2892–2907.

Garey, M. R., Johnson, D. S., & Sethi, R. (1976). The complexity of flowshop and jobshop scheduling. *Mathematics of Operations Research, 1*(2), 117–129.

Gonçalves, J. F., de Magalhães Mendes, J. J., & Resende, M. G. (2005). A hybrid genetic algorithm for the job shop scheduling problem. *European Journal of Operational Research, 167*(1), 77–95.

Gonçalves, J. F., Resende, M. G., & Mendes, J. J. (2011). A biased random-key genetic algorithm with forward-backward improvement for the resource constrained project scheduling problem. *Journal of Heuristics, 17*(5), 467–486.

Graham, R. L., Lawler, E. L., Lenstra, J. K., & Kan, A. H. G. (1979). Optimization and approximation in deterministic sequencing and scheduling: A survey. *Annals of Discrete Mathematics, 5*, 287–326.

Herroelen, W., De Reyck, B., & Demeulemeester, E. (1998). Resource-constrained project scheduling: A survey of recent developments. *Computers & Operations Research, 25*(4), 279–302.

Hoogeveen, J. A., Lenstra, J. K., & Van de Velde, S. L. (1997). *Sequencing and scheduling: An annotated bibliography*. Eindhoven University of Technology, Department of Mathematics and Computing Science.

Jackson, J. R. (1955). *Scheduling a production line to minimize maximum tardiness*. Research report 43, Management Sciences Research Project. University of California.

Kuo, I. H., Horng, S. J., Kao, T. W., Lin, T. L., Lee, C. L., Terano, T., & Pan, Y. (2009). An efficient flow-shop scheduling algorithm based on a hybrid particle swarm optimization model. *Expert Systems with Applications, 36*(3), 7027–7032.

Lee, K. M., Yamakawa, T., & Lee, K. M. (1998). A genetic algorithm for general machine scheduling problems. In *Knowledge-based intelligent electronic systems, 1998. Proceedings KES'98. 1998 second international conference on* (Bd. 2, S. 60–66). IEEE.

Liebchen, C., Schachtebeck, M., Schöbel, A., Stiller, S., & Prigge, A. (2010). Computing delay resistant railway timetables. *Computers and Operations Research, 37*(5), 857–868.

Lin, T. L., Horng, S. J., Kao, T. W., Chen, Y. H., Run, R. S., Chen, R. J., Lai, J. L., & Kuo, I. H. (2010). An efficient job-shop scheduling algorithm based on particle swarm optimization. *Expert Systems with Applications, 37*(3), 2629–2636.

Liu, B., Wang, L., & Jin, Y. H. (2007). An effective PSO-based memetic algorithm for flow shop scheduling. *IEEE Transactions on Systems, Man, and Cybernetics Part B: Cybernetics, 37*(1), 18–27.

Mendes, J. J. D. M., Gonçalves, J. F., & Resende, M. G. (2009). A random key based genetic algorithm for the resource constrained project scheduling problem. *Computers & Operations Research, 36*(1), 92–109.

Nearchou, A. C., & Omirou, S. L. (2006). Differential evolution for sequencing and scheduling optimization. *Journal of Heuristics, 12*(6), 395–411.

Ouelhadj, D., & Petrovic, S. (2009). A survey of dynamic scheduling in manufacturing systems. *Journal of Scheduling, 12*(4), 417–431.

Pezzella, F., Morganti, G., & Ciaschetti, G. (2008). A genetic algorithm for the flexible job-shop scheduling problem. *Computers & Operations Research, 35*(10), 3202–3212.

Prins, C. (2000). Competitive genetic algorithms for the open-shop scheduling problem. *Mathematical Methods of Operations Research, 52*(3), 389–411.

Pruhs, K., Sgall, J., & Torng, E. (2004). Online scheduling. In J. Y.-T. Leung (Hrsg.), *Handbook of scheduling: Algorithms, models, and performance analysis*. CRC Press. Chapter 15.

Ribas, I., Leisten, R., & Framiñan, J. M. (2010). Review and classification of hybrid flow shop scheduling problems from a production system and a solutions procedure perspective. *Computers & Operations Research, 37*(8), 1439–1454.

Roshanaei, V., Naderi, B., Jolai, F., & Khalili, M. (2009). A variable neighborhood search for job shop scheduling with set-up times to minimize makespan. *Future Generation Computer Systems, 25*(6), 654–661.

Ruiz, R., & Vázquez-Rodríguez, J. A. (2010). The hybrid flow shop scheduling problem. *European Journal of Operational Research, 205*(1), 1–18.

Taillard, E. (1993). Benchmarks for basic scheduling problems. *European Journal of Operational Research, 64*(2), 278–285.

Williamson, D. P., Hall, L. A., Hoogeveen, J. A., Hurkens, C. A. J., Lenstra, J. K., Sevast'Janov, S. V., & Shmoys, D. B. (1997). Short shop schedules. *Operations Research, 45*(2), 288–294.

Ye, J., & Ma, H. (2015). Multi-objective joint optimization of production scheduling and maintenance planning in the flexible job-shop problem. *Mathematical Problems in Engineering*. https://doi.org/10.1155/2015/725460. Zugegriffen am 12.03.2016.

Zhang, C., Li, P., Guan, Z., & Rao, Y. (2007). A tabu search algorithm with a new neighborhood structure for the job shop scheduling problem. *Computers & Operations Research, 34*(11), 3229–3242.

Zhang, G., Gao, L., Li, X., & Li, P. (2008). Variable neighborhood genetic algorithm for the flexible job shop scheduling problems. In C. Xiong, Y. Huang, & Y. Xiong (Hrsg.), *Intelligent robotics and applications. First international conference, ICIRA 2008 Wuhan, China* (S. 503–512). Springer.

Zhang, G., Gao, L., & Shi, Y. (2010). A genetic algorithm and tabu search for multi objective flexible job shop scheduling problems. In *2010 International Conference on Computing, Control and Industrial Engineering (CCIE)* (Bd. 1, S. 251–254). IEEE.

Kapitel 6
Standortplanung und Netzwerkgestaltung



Die Planung eines neuen Standorts gehört zu den komplexesten und schwierigsten Planungsproblemen der Entscheidungsfindung. Zudem gehört sie zum strategischen Bereich der Planung, der sich mit den wichtigsten und weitreichendsten Entscheidungen beschäftigt. Während die in den vorangegangenen Kapiteln betrachteten Probleme meist auf der operativen Planungsebene untersucht werden, sind Entscheidungen über einzelne Standorte oder die Struktur eines Liefernetzwerks meist langfristiger Natur und können für das zukünftige Überleben und den Erfolg eines Unternehmens entscheidend sein.

6.1 Standortplanung als multikriterielles Entscheidungsproblem

Wenn eine neue Anlage, z. B. ein neues Lager oder eine neue Produktionsstätte, errichtet werden soll, gibt es in der Regel viele (oft unendlich viele) praktisch mögliche Standorte. Diese Standorte unterscheiden sich in vielerlei Hinsicht, z. B. hinsichtlich der Entfernung zu Lieferanten, Kunden oder anderen Standorten, der Grundstückspreise, der Verfügbarkeit und Qualität von Arbeitskräften, der örtlichen Vorschriften und Steuern, der Infrastruktur usw. All diese Aspekte beeinflussen die künftige Rentabilität des neuen Standorts, und oft ist es schwierig, ihren Einfluss zu quantifizieren und die künftige Entwicklung dieser Einflussfaktoren abzuschätzen.

Darüber hinaus ist es angesichts der großen Zahl möglicher Standorte schwierig, alle relevanten Daten selbst für die aktuelle Situation zu erhalten. In einigen Fällen werden heute geographische Informationssysteme (GIS) eingesetzt, um die Standortentscheidung mit den benötigten Daten zu untermauern (siehe z. B. Cheng et al. (2007)). Dennoch bleibt die Bereitstellung ausreichend vollständiger Daten und die Ableitung entscheidungsrelevanter Kriterien ein großes Problem bei Standortentscheidungen.

© Der/die Autor(en), exklusiv lizenziert an Springer Nature Switzerland AG 2023

T. Hanne, R. Dornberger, *Computational Intelligence in Logistik und Supply Chain Management*, https://doi.org/10.1007/978-3-031-21452-3_6

Neben dem Problem der Beschaffung der relevanten Daten führt der multikriterielle Charakter eines Entscheidungsproblems dazu, dass es in der Regel nicht nur eine beste Lösung gibt, die andere Lösungen in allen Kriterien übertrifft. Stattdessen gibt es typischerweise eine mehr oder weniger umfangreiche Menge von Lösungen, die als Pareto-optimal, effizient oder nicht-dominiert bezeichnet werden. Für diese Lösungen gibt es keine andere Lösung, die in mindestens einem Kriterium besser und in allen anderen nicht schlechter ist. In den letzten Jahrzehnten wurden verschiedene Methoden vorgeschlagen, die zusätzliche Informationen eines Entscheidungsträgers nutzen, um einen bevorzugten Kompromiss unter diesen Lösungen zu ermitteln (siehe z. B. Hanne, 2012). Solche präferenzbasierten Informationen zur Lösung des Entscheidungsproblems können beispielsweise Gewichtungen für die Kriterien, Informationen über wünschenswerte Kriterienwerte wie Referenzpunkte oder die Ermittlung einer Nutzenfunktion umfassen. Im Folgenden wird eine Reihe von Studien zu Standortplanungsproblemen diskutiert, die einen multikriteriellen Modellierungs- und Lösungsansatz verwenden.

So berücksichtigen Awasthi et al. (2011) in einem Modell für die Planung von Standorten für städtische Distributionszentren die folgenden 11 Kriterien: Erreichbarkeit, Sicherheit, Anbindung an den multimodalen Verkehr, Kosten, Umweltauswirkungen, Kundennähe, Lieferantennähe, Ressourcenverfügbarkeit, Konformität mit den Vorschriften für nachhaltigen Güterverkehr, Erweiterungsmöglichkeiten und Servicequalität. In Doerner et al. (2009) werden nur vier Kriterien berücksichtigt: das Minisum Facility Location Criterion, das Maximal Covering Location Criterion (MCLC), ein Risikokriterium (die Minimierung der Tsunami-Wahrscheinlichkeit) und die Gesamtkosten, die sich aus den Einrichtungskosten einer Anlage und den zusätzlichen Kosten für die erforderliche Kapazität zusammensetzen. Die Minisum-Zielfunktion misst die Summe der Entfernungen der Kunden, die sich innerhalb einer maximalen Entfernung von der zugewiesenen Einrichtung befinden, während MCLC die Anzahl der Kunden außerhalb dieser Entfernung zählt. Beide Kriterien werden mit Hilfe eines gewichteten Durchschnitts zu einem einzigen Kriterium zusammengefasst, so dass nur drei Kriterien übrigbleiben.

Chen (2001) stellt einen Fuzzy-Ansatz vor, der auf ein Beispiel mit fünf Kriterien angewandt wird: die Investitionskosten, die Expansionsmöglichkeiten, die Verfügbarkeit der benötigten Materialien, die personellen Ressourcen und die Nähe zum Nachfragemarkt. Ein weiterer unscharfer multikriterieller Ansatz wird von Chou et al. (2008b) untersucht, die 21 Kriterien für die Auswahl eines internationalen Hotelstandorts für Touristen berücksichtigen. Die Kriterien beziehen sich auf Aspekte der geografischen Lage, der Verkehrsbedingungen, der Hoteleigenschaften und des Betriebsmanagements. Chou et al. (2008a) analysieren einen Fuzzy-Ansatz, der traditionelle multikriterielle Entscheidungsfindungstechniken und ein Gruppenentscheidungsszenario umfasst. In einer anderen Fuzzy-Gruppenentscheidungsstudie berücksichtigen Kahraman et al. (2003) fünf Bewertungskriterien für die Auswahl von Betriebsstandorten, nämlich Kundennähe, Infrastruktur, Qualität der Arbeitskräfte, Freihandelszonen und Wettbewerbsvorteile.

In Harris et al. (2009) werden drei Kriterien für ein Standortproblem berücksichtigt: Minimierung der Kosten, Minimierung der Umweltauswirkungen und Minimierung der ungedeckten Nachfrage.

Eine Übersicht über multikriterielle Standortprobleme findet sich in Farahani et al. (2010). Insgesamt listet die Studie 63 Publikationen zu multikriteriellen Standortproblemen auf, aber es kann davon ausgegangen werden, dass es sicherlich mehr gibt, die unterhalb der Erkennungsschwelle liegen, die durch die auf der Scopus-Plattform, einer der größten Abstract- und Zitationsdatenbanken für wissenschaftliche Literatur mit Peer-Review, gelisteten Publikationen definiert ist. In der Übersicht werden die Publikationen in Multiattribute-Probleme, die durch eine endliche, explizit gegebene Menge von Alternativen charakterisiert sind, und in Multiobjective-Probleme mit einer unendlichen oder sehr großen Menge von Alternativen, die durch Zielfunktionen und Restriktionen charakterisiert sind, unterteilt. Der Sonderfall der Zweiziel-Probleme wird gesondert betrachtet. Die gefundenen Kriterien werden unterschieden in Kostenkriterien, Umweltrisiken, Abdeckung des relevanten Gebiets und Gleichheitsaspekte, Servicelevel und Effektivität, Gewinn und andere Kriterien. Bei Multiattribute-Problemen sind die Kriterien oft vielfältiger und können vielseitigere Aspekte in Bezug auf Kosten oder Nutzen, verfügbare Ressourcen, Zugang zu öffentlichen Einrichtungen, politische Fragen und Vorschriften, Wettbewerbsaspekte, weitere wirtschaftliche Kriterien, Bevölkerungsaspekte und Aspekte in Bezug auf Entfernungen umfassen.

Obwohl es naheliegend ist, bei komplexen Standortplanungsproblemen mehrere Kriterien zu berücksichtigen, wird häufig nur ein einziges Ziel in der Planung verwendet. Ein Grund dafür ist, dass Standortprobleme in der Regel bereits hinreichend komplex und mit einer einzigen Zielsetzung schwer zu lösen sind. Die meisten der untersuchten multikriteriellen Varianten von Standortproblemen basieren auf dem Fall, dass eine eher kleine endliche Menge von Alternativen gegeben ist, während Situationen, die entweder auf einer möglicherweise unendlichen Menge oder einer sehr großen endlichen Menge beruhen, die durch verschiedene Einschränkungen gekennzeichnet ist, typisch für die Studien mit einem Ziel sind.

Der andere Grund ist, dass bei Problemen mit nur einem Ziel häufig eine komplexe Zielfunktion verwendet wird, die mehrere Ziele zusammenfasst, so dass diese nicht explizit behandelt werden.

In den folgenden Abschnitten werden wir uns auf grundlegende Varianten von Standortproblemen konzentrieren, die in der Regel von einer einzigen Zielfunktion, Entscheidungen unter Gewissheit und einigen anderen oft nicht sehr realistischen Annahmen ausgehen. Am Ende des Kapitels kehren wir zu einer Diskussion über den Einsatz von Computational Intelligence für komplexere und realistischere Varianten von Standortproblemen zurück.

Während es sich bei den meisten der betrachteten Probleme um Standortprobleme auf einer einzigen Ebene einer Lieferkette handelt (z. B. um die Entscheidung über eine bestimmte Anzahl offener Distributionszentren für die Belieferung von Kunden), berücksichtigen andere Probleme die Beziehungen zwischen mehreren Ebenen oder dienen der Planung von Einrichtungen auf mehreren Ebenen

einer Supply Chain. Insbesondere solche komplexeren Multi-Facility-Standortprobleme werden am Ende des Kapitels behandelt.

Für eine ausführlichere Behandlung der mathematischen Standorttheorie verweisen wir auf Nickel und Puerto (2006). Eine weitere hervorragende Übersicht findet sich in Drezner und Hamacher (2001).

6.2 Diskrete Standortprobleme

Meistens wird bei Standortproblemen davon ausgegangen, dass die möglichen Standorte von Einrichtungen auf diskrete Weise spezifiziert werden, d. h. es wird eine endliche Anzahl möglicher Standorte betrachtet. Solche Standorte können als explizit gegeben angenommen werden oder es wird ein Graph angegeben, der die Netzstruktur beschreibt. In diesem Fall wird häufig davon ausgegangen, dass sich die Anlagen an den Knotenpunkten des Graphen befinden, was unter den üblichen Annahmen wiederum zu einer diskreten Menge führt. Eine weitere Voraussetzung für solche Probleme sind Informationen über Entfernungen oder damit zusammenhängende Kosten, die explizit gegeben sein können (z. B. in Form einer Entfernungsmatrix) oder die durch eine Metrik (z. B. auf der Grundlage gegebener Koordinaten von Standorten) oder unter Verwendung eines Ansatzes für den kürzesten Weg bei einer graphbasierten Darstellung berechnet werden können.

6.2.1 Das p-Median-Problem

6.2.1.1 Problemstellung

Bei dieser Art von Problem sind p von m möglichen Standorten auszuwählen, um die gesamten (gewichteten) Entfernungen oder Transportkosten zu minimieren. Alle anderen Aspekte (z. B. die Errichtungskosten für eine neue Anlage) werden nicht berücksichtigt, d. h. alle möglichen Standorte werden abgesehen von den transportbezogenen Aspekten als gleichwertig betrachtet.

Das Problem kann wie folgt formuliert werden: Es sei L eine Menge von m möglichen Einrichtungen (oder Standorten), C eine Menge von n Kunden (oder Nachfrageorten), und d_{ij}, $i \in \{1,\ldots,n\}$, $j \in \{1,\ldots,m\}$, die gegebenen Entfernungen zwischen Einrichtungen und Kunden. Das Ziel besteht darin, p Standorte aus L zu bestimmen, um die Summe dieser Entfernungen (oder Transportkosten) von den ausgewählten Standorten zu den Kunden zu minimieren:

$$\min \sum_{i=1}^{n} \min_{j \in P} \quad d_{ij} \tag{6.1}$$

mit $P \subseteq L$ und $|P| = p$.

Unter Verwendung von binären Entscheidungsvariablen zur Beurteilung, welche der Anlagenstandorte genutzt werden (d. h. zu P gehören) und welche Kunden von welcher Anlage beliefert werden, können wir das folgende ganzzahlige Optimierungsproblem formulieren:

$$\min \sum_{i=1}^{n} \sum_{j=1}^{m} d_{ij} x_{ij} \tag{6.2}$$

so dass

$$\sum_{j=1}^{m} x_{ij} = 1, i \in \{1, \ldots, n\} \tag{6.3}$$

$$x_{ij} \leq y_j, i \in \{1, \ldots, n\}, j \in \{1, \ldots, m\} \tag{6.4}$$

$$\sum_{j=1}^{m} y_j = p \tag{6.5}$$

$$x_{ij}, y_j \in \{0, 1\}, i \in \{1, \ldots, n\}, \; j \in \{1, \ldots, m\} \tag{6.6}$$

$y_j \in \{0, 1\}$ für $j \in \{1,\ldots, m\}$ gibt an, ob der Standort j als offen ausgewählt ist ($y_j = 1$) oder nicht ($y_j = 0$, d. h. Standort j ist geschlossen). $x_{ij} \in \{0, 1\}$ mit $i \in \{1,\ldots, n\}$ und $j \in \{1,\ldots,m\}$ gibt an, ob Kunde i von Standort j bedient wird ($x_{ij} = 1$) oder nicht ($x_{ij} = 0$). Nebenbedingung (6.3) stellt sicher, dass jeder Kunde von genau einem Standort bedient wird, Gl. (6.4) stellt sicher, dass nur offene Standorte (die zu P gehören) einen Kunden bedienen können. Nebenbedingung (6.4) schließlich garantiert, dass genau p Standorte offen sind. Man beachte, dass die Zielfunktion modifiziert werden könnte, indem man eine spezifische Nachfrage a_i jedes Kunden berücksichtigt (siehe z. B. ReVelle und Swain (1970)), um die Entfernungen zu „gewichten“:

$$\min \sum_{i=1}^{n} \sum_{j=1}^{m} a_i d_{ij} x_{ij} \tag{6.7}$$

In Abb. 6.1 ist ein Beispiel für ein Standortproblem dargestellt, bei dem 15 Kunden 3 Einrichtungen zugewiesen werden, die aus insgesamt 6 möglichen Standorten ausgewählt wurden. Das p-Median-Problem ist erwiesenermaßen ein NP-schweres Optimierungsproblem (Kariv & Hakimi, 1979). Es wird in der Regel im Zusammenhang mit der Standortplanung interpretiert, d. h. der Bestimmung des Standorts von Fabriken, Lagern oder anderen Standorten, aber es kann auch in anderen Zusammenhängen verwendet werden, z. B. für die Modellierung und Lösung von Clustering-Problemen.

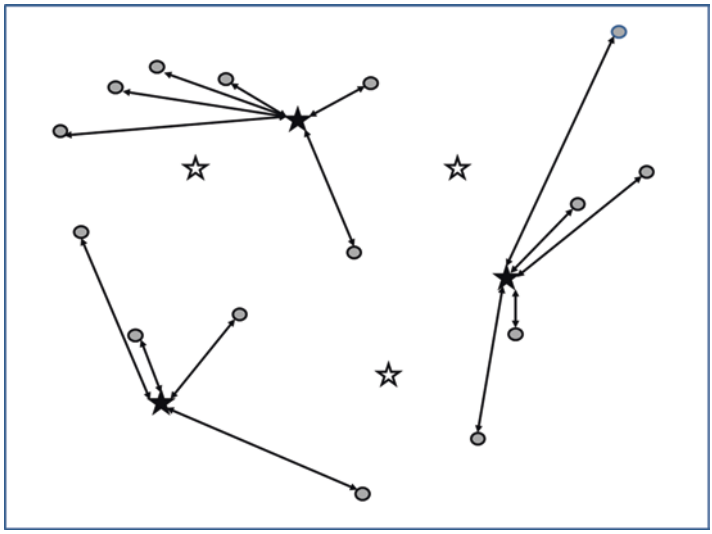


Abb. 6.1 Beispiel für ein Standortproblem mit 15 Kunden, wobei 3 von 6 möglichen Standorten ausgewählt werden (*offen*)

Es sei darauf hingewiesen, dass es verschiedene Varianten des p-Median-Problems gibt, die in der Literatur untersucht wurden. Das bedingte p-Median-Problem geht beispielsweise davon aus, dass einige der p zu planenden Anlagen bereits feststehen (Berman & Drezner, 2008). Eine weitere praktisch wichtige Variante des Problems ist das p-Median-Problem mit Kapazitäten, bei dem davon ausgegangen wird, dass jede Anlage eine bestimmte Kapazität q_j hat. In zusätzlichen Nebenbedingungen wird gefordert, dass die zugewiesene Nachfrage für jede Anlage diese Kapazitätsgrenze nicht überschreitet, die wie in Gl. (6.8) formuliert werden kann.

$$\sum_{i=1}^{n} a_i x_{ij} \leq q_j y_j, \quad j \in \{1, \ldots, m\} \tag{6.8}$$

Schließlich gibt es auch stochastische, unscharfe und dynamische Versionen des Problems, die in der Literatur untersucht werden. Ein Modell mit unscharfer Nachfrage (d. h. unscharfer Existenz von Nachfragepunkten) wird beispielsweise in Canós et al. (2001) untersucht. Dies ermöglicht eine bessere Berücksichtigung vager oder zusätzlicher Informationen aus der Realität, die im Standard-P-Median-Problem nicht vorhanden sind, und die Berechnung von Lösungen, die sonst nicht zulässig wären. Dies kann zu einer besseren Kostenbetrachtung führen, d. h. zu Lösungen mit geringeren Kosten, die im Standardmodell nicht als zulässig angesehen würden.

6.2.1.2 Lösungsansätze

Testprobleme für die Analyse und das Benchmarking von Algorithmen sind in verschiedenen Bibliotheken zu finden. So werden beispielsweise in Mladenović et al. (2007) sechs verschiedene Problemsammlungen erwähnt.

Wie bei NP-harten Problemen üblich, finden wir als mögliche Lösungsansätze für das p-Median Problem einfache Heuristiken, Metaheuristiken und mathematische Ansätze. Während in der Vergangenheit lange Zeit mathematische Ansätze dominierten (mit Ansätzen wie Lagrangescher Relaxation, Branch-and-Bound, Branch-and-Cut-and-Price, Column Generation und dynamischer Programmierung), haben metaheuristische Ansätze in den letzten 15 Jahren an Popularität gewonnen.

Beginnen wir jedoch mit der Betrachtung einiger einfacher Heuristiken. Diese Ansätze können in Konstruktionsheuristiken und Verbesserungsheuristiken unterschieden werden. Konstruktionsheuristiken beginnen z. B. mit einer Menge von nur einer Anlage (d. h. einer Lösung des 1-Median-Problems) und fügen dann schrittweise weitere Anlagen hinzu, bis p erreicht ist. Jede Anlage wird so hinzugefügt, dass die Kosten maximal sinken. In ähnlicher Weise ist es möglich, mit einer Lösung zu beginnen, bei der alle m Einrichtungen geöffnet sind, und dann die Anzahl der Einrichtungen in jeder Iteration um eine zu verringern, so dass die Kostensteigerung am geringsten ist. Dieses Verfahren endet, wenn nur noch p Anlagen geöffnet sind.

Verbesserungsheuristiken oder lokale Suchansätze können auf ähnlichen Überlegungen für Lösungsänderungen beruhen. Die populärste Art von Verbesserungsheuristiken könnte als „facility exchange“ oder „vertex substitution“ bezeichnet werden, die zwei, z. B. zufällig ermittelte, Einrichtungen betrachtet, von denen eine aus der offenen Menge stammt und die andere geschlossen ist, und prüft, ob deren Austausch in der betrachteten Lösung die Kosten reduziert (siehe z. B. Chiyoshi und Galvao (2000)).

Während die Zuordnung von Kunden zu Einrichtungen für das Standard-p-Median-Problem eher trivial ist (da immer die nächste gewählte Einrichtung zugewiesen wird), wird sie für eine Version des Problems mit Kapazitätsbeschränkungen komplexer. In diesem Fall gehört eine Heuristik für den Austausch von Kunden zwischen Einrichtungen zu den typischen Verbesserungsschritten.

Solche lokalen Such- oder Verbesserungsheuristiken können zur Einbettung in andere, komplexere Ansätze und z. B. zur Definition von Nachbarschaften in Metaheuristiken verwendet werden, wie sie z. B. beim Simulated Annealing, der Tabu-Suche oder der variablen Nachbarschaftssuche (VNS) eingesetzt werden. So werden z. B. im VNS-Ansatz von Hansen und Mladenović (1997) verschiedene, zunehmend größere Nachbarschaften durch eine zunehmende Anzahl von Einrichtungen aus der aktuellen Lösung definiert, die durch noch nicht geöffnete Einrichtungen ausgetauscht werden (Austausch-Nachbarschaftsstruktur).

Im Hinblick auf die Kodierung von Lösungen für eine Metaheuristik kann die folgende Vereinfachung für das Standard-p-Median-Problem verwendet werden: Während es in der Problemformulierung (6.2)–(6.6) grundsätzlich zwei Arten von

Variablen gibt, kann es ausreichend sein, bei der Behandlung von Lösungen in einer Metaheuristik nur eine zu kodieren, nämlich die offenen Einrichtungen. Die Zuordnung von Kunden zu Einrichtungen kann relativ einfach erfolgen, indem jeder Kunde der nächstgelegenen (oder kostengünstigsten) offenen Einrichtung zugewiesen wird (was in O(p) erledigt werden kann). Dieses Konzept verkleinert insbesondere den Lösungsraum und sollte die Leistungsfähigkeit des betrachteten Lösungsansatzes erhöhen. In dem von Alp et al. (2003) verwendeten genetischen Algorithmus werden Lösungen beispielsweise durch Strings der Länge *p* dargestellt, die die Indizes der offenen Einrichtungen enthalten. Für diese Art von Lösungsrepräsentationen würden modifizierte kanonische Variationsoperatoren in evolutionären Algorithmen funktionieren, z. B. Mutationen, die eine Einrichtung im String durch eine geschlossene Einrichtung (die noch nicht im String enthalten ist) oder 1-Punkt- oder Mehrpunkt-Crossovers ersetzen. Bei den Crossover-Operationen muss darauf geachtet werden, dass Einrichtungen nicht doppelt in der resultierenden Zeichenkette vertreten sind. In Correa et al. (2004) werden beispielsweise die Mengen von ausschließlich durch einen Elter repräsentierten Einrichtungen für jeden Elternteil bestimmt und dann eine zufällige Anzahl von Einrichtungen aus diesen Mengen für die Nachkommenlösungen ausgetauscht. In der Literatur finden sich jedoch weitere, nicht standardisierte Implementierungen. Die in Alp et al. (2003) verwendete Rekombination basiert darauf, dass zunächst eine Vereinigung der Einrichtungen von zwei Eltern gebildet wird und anschließend deren Anzahl auf *p* in einer „gierigen Weise“ reduziert wird. Das bedeutet, dass in jedem Schritt ermittelt wird, welche Anlagenschließung die geringste Kostensteigerung verursacht. Was die Mutationen betrifft, so wurde dieser Operator in der Studie wegen unzureichender Leistung ausgelassen, was möglicherweise daran lag, dass die Populationsgröße recht hoch gewählt wurde, so dass eine zusätzliche Diversitätserzeugung nicht erforderlich war.

Ein solches gieriges Vorgehen könnte als eine in den evolutionären Algorithmus eingebettete lokale Suche interpretiert werden. Eine ähnliche Idee findet sich in Correa et al. (2004), die sie Hypermutation nennen: Dieser rechenintensive und selten angewandte evolutionäre Operator sucht für jede offene Anlage eine Teilmenge von Lösungen, um sie durch eine beste bisher geschlossene Anlage zu ersetzen.

In Bezug auf Metaheuristiken wurden in der Literatur die folgenden Ansätze für das p-Median-Problem und Varianten untersucht:

- evolutionäre Algorithmen (Alp et al., 2003; Correa et al., 2004)
- Path Relinking (Díaz & Fernandez, 2006)
- Scatter Search (Díaz & Fernandez, 2006)
- Simulated Annealing (Al-Khedhairi, 2008; Chiyoshi & Galvao, 2000)
- Tabu-Suche (Rolland et al., 1997)
- variable Nachbarschaftssuche (Hansen & Mladenović, 1997)

Auch einige weniger bekannte Ansätze wie bionomische Algorithmen, die eine Art von memetischen Algorithmen sind, wurden verwendet (Maniezzo et al., 1998). Wir finden auch eine Reihe weiterer Studien zu p-Median-Problemen, die verschiedene heuristische und metaheuristische Konzepte kombinieren.

6.2.2 Das p-Center-Problem

6.2.2.1 Problemstellung

Das p-Center-Problem ist dem p-Median-Problem ähnlich. Auch hier soll eine bestimmte Anzahl von p Einrichtungen aus einer Menge von m Einrichtungen ausgewählt werden, um n Kunden zu bedienen. Das Ziel ist jedoch nicht die Minimierung der Summe der Entfernungen oder der damit verbundenen Kosten, sondern die Minimierung der maximalen Entfernung zwischen einem der Kunden und der zugewiesenen Einrichtung.

Das p-Center-Problem kann mathematisch wie folgt formuliert werden:

$$\min \max_{i \in C} \min_{j \in P} \quad d_{ij} \tag{6.9}$$

mit $P \subseteq L$ und $|P| = p$.

Wie oben gehen wir davon aus, dass L eine Menge von m möglichen Einrichtungen (oder Standorten) ist und P ⊆ L eine Teilmenge von p der als offen ausgewählten Einrichtungen ist. C ist eine Menge von n Kunden (oder Nachfrageorten), und d_{ij}, $i \in \{1,\ldots, n\}$, $j \in \{1,\ldots, m\}$, sind gegebene Entfernungen zwischen Einrichtungen und Kunden. Ziel ist es, p Standorte aus L so zu finden, dass die maximale Entfernung zwischen einem Kunden und seiner nächsten offenen Einrichtung minimiert wird. Es sei darauf hingewiesen, dass wir d_{ij} mit den jeweiligen kundenspezifischen Gewichten (z. B. für einzelne Frachtgewichte oder Kostenfaktoren) multiplizieren können, ohne das Problem wesentlich zu verändern.

Wenn wir binäre Entscheidungsvariablen verwenden, um zu beurteilen, welche Standorte offen sind (d. h. zu P gehören) und welche Kunden von welchem Standort beliefert werden, können wir das folgende ganzzahlige Optimierungsproblem formulieren:

$$\min z \tag{6.10}$$

so dass

$$\sum_{j=1}^{m} x_{ij} = 1, i \in \{1, \ldots, n\} \tag{6.11}$$

$$x_{ij} \leq y_j, i \in \{1, \ldots, n\}, \; j \in \{1, \ldots, m\} \tag{6.12}$$

$$\sum_{j=1}^{m} y_j = p \tag{6.13}$$

$$z \geq \sum_{j=1}^{m} d_{ij} x_{ij}, i \in \{1, \ldots, n\} \tag{6.14}$$

$$x_{ij}, y_j \in \{0,1\}, i \in \{1,\ldots,n\}, j \in \{1,\ldots,m\} \tag{6.15}$$

$y_j \in \{0, 1\}$ für $j \in \{1,\ldots, m\}$ gibt an, ob der Standort j geöffnet ($y_j = 1$) oder geschlossen ($y_j = 0$) ist. $x_{ij} \in \{0, 1\}$ mit $i \in \{1,\ldots, n\}$ und $j \in \{1,\ldots, m\}$ gibt an, ob Kunde i von Standort j bedient wird ($x_{ij} = 1$) oder nicht ($x_{ij} = 0$). Die Bedingung (6.11) stellt sicher, dass jeder Kunde von einem Standort bedient wird. Gl. (6.12) garantiert, dass nur offene Standorte Kunden bedienen. Gl. (6.13) stellt sicher, dass genau p Standorte offen sind. Gl. (6.14) legt fest, dass die Hilfsvariable z größer oder gleich der Entfernung zwischen den Kunden und den ihnen zugewiesenen Standorten ist. Zusammen mit dem Ziel, z zu minimieren, wird angenommen, dass die Kunden den nächstgelegenen offenen Einrichtungen zugewiesen werden.

Ähnlich wie beim p-Median-Problem gibt es verschiedene Varianten des Problems, wie z. B. eine Version mit Kapazitäten (mit zusätzlichen Beschränkungen für offene Einrichtungen und deren zugewiesener Nachfrage), siehe z. B. Özsoy und Pınar (2006). In Contreras et al. (2012) wird eine Variante mit einer Budgetbeschränkung betrachtet. Darüber hinaus berücksichtigt das Modell die Einrichtungskosten von Anlagen und die Kosten für die Einrichtung neuer Verbindungen in der Graphen-Darstellung des angenommenen Netzes.

Eine weitere Variante – ähnlich dem bedingten p-Median-Problem – ist das bedingte p-Center-Problem, bei dem einige der p Orte vorgegeben sind (Berman & Drezner, 2008). In einigen anderen Problemformulierungen werden die Beziehungen zu Abdeckungsproblemen (coverage problems) oder Abdeckungsanforderungen untersucht. In der in Lim et al. (2004) analysierten Problemformulierung wird beispielsweise gefordert, dass die Standorte die Nachfrage von mindestens einer vorgegebenen Anzahl von Kunden abdecken müssen. Eine andere Variante des Problems geht davon aus, dass die minimale Entfernung zwischen Einrichtungen und Kunden maximiert werden sollte, was für unangenehme Einrichtungen sinnvoll ist (siehe z. B. Klein & Kincaid, 1994).

6.2.2.2 Lösungsansätze

Da das p-Center-Problem dem p-Median-Problem ähnlich ist, können auch ähnliche Ansätze zu seiner Lösung verwendet werden. Da das Problem ebenfalls NP-schwer ist (Megiddo & Supowit, 1984), haben metaheuristische Ansätze in den letzten Jahrzehnten deutlich an Bedeutung gewonnen.

Generell ist festzustellen, dass das p-Center-Problem weniger häufig mit Metaheuristiken behandelt wird als das p-Median-Problem und dass mathematische Ansätze immer noch dominieren. Ein Grund dafür könnte sein, dass das Problem im Allgemeinen weniger häufig analysiert wird als das p-Median-Problem.

Heuristiken nutzen oft eine Art von lokaler Suche und einfache Verbesserungsoperationen wie die Austauschheuristik (oder Vertex-Substitution). Beispielsweise können Metaheuristiken, die auf lokaler Suche basieren, wie die Tabu-Suche oder die variable Nachbarschaftssuche, auf ähnlichen oder verallgemeinerten Konzepten

beruhen (Mladenović et al., 2003). Ein weiteres Beispiel ist die „large-scale local search“, wie sie in Scaparra et al. (2004) diskutiert wird. Diese Methode basiert auf einer Multi-Austausch-Methode, die den zyklischen Austausch einer Reihenfolge von Kunden mit kapazitätsbeschränkten Anlagen und Pfadaustausche mehrerer Kunden nutzt. Evolutionäre Algorithmen oder Ansätze der Schwarmintelligenz werden nur selten auf p-Center-Probleme und Varianten angewandt. Ein Beispiel ist die Verwendung von Bee Colony Optimization in Davidović et al. (2011). Bee Colony Optimization kann grundsätzlich sowohl zur Konstruktion als auch zur Verbesserung einer Lösung verwendet werden. Der Verbesserungsschritt besteht darin, eine zufällig festgelegte Anzahl geschlossener Standorte auszuwählen, die zu einer betrachteten Lösung hinzugefügt werden, wobei berücksichtigt wird, dass der größte Abstand unter allen Abständen zwischen offenen Standorten und Kunden verringert wird. Danach wird die gleiche Anzahl offener Standorte auf „gierige Weise“ aus der Lösung entfernt, d. h. es werden Lösungen gewählt, deren Schließung die Zielwerte am wenigsten erhöht. Während der konstruktive Ansatz unbefriedigend arbeitet, schneidet die Nutzung von Bee Colony Optimization zur Verbesserung im Vergleich zu anderen Metaheuristiken in Bezug auf Lösungsqualität und Laufzeit erfolgreich ab.

Ein weiteres Beispiel für eine metaheuristische Anwendung ist die Verwendung eines angepassten Harmony Search-Algorithmus in Kaveh und Nasr (2011), der bei numerischen Experimenten im Vergleich zu variabler Nachbarschaftssuche, Tabu-Suche und Scatter Search recht konkurrenzfähig ist.

Ein Simulated Annealing-Ansatz wird in Hassin et al. (2003) untersucht, wo er zusammen mit gewöhnlicher lokaler Suche und mit einer neuartigen lexikographischen lokalen Suche verwendet wird, die den Austausch von Lösungen berücksichtigt, die nicht die Zielwerte (größter Abstand) verbessern, sondern die Anzahl der Orte, die dafür verantwortlich sind. Diese lexikografische lokale Suche zeigt eine verbesserte Leistung, auch wenn sie in andere heuristische Techniken eingebettet ist, aber die besten Ergebnisse, wenn sie mit Simulated Annealing verwendet wird.

6.2.3 Das Unkapazitierte Facility Location-Problem (UFLP)

Ein offensichtlicher Nachteil der obigen Formulierungen des p-Median-Problems und des p-Center-Problems ist, dass unter den vielen Vereinfachungen im Vergleich zu praktischen Standortentscheidungen die Einrichtungskosten von Anlagen nicht berücksichtigt werden. In der Praxis sind solche Einrichtungskosten, die für den Bau eines mittelgroßen Lagers leicht 50 Mio. US$ betragen können (siehe z. B. International, Inc., 2013), sicherlich nicht zu vernachlässigen. Außerdem wird die spezifische Anzahl der Standorte, p, in diesen Problemen als gegeben angenommen und nicht weiter berücksichtigt.

6.2.3.1 Problemstellung

Diese Annahmen werden bei den Standortproblemen geändert, die Rüstkosten berücksichtigen und keine bestimmte Anzahl von Anlagen vorschreiben. Stattdessen ergibt sich die Anzahl der offenen Anlagen aus der Lösung des Optimierungsproblems. Bei diesem Problem setzen sich die zu minimierenden Kosten aus einem Fixkostenanteil für die Errichtung einer Anlage an einem bestimmten Standort und den Transportkosten für die Beförderung der Waren zu den Kunden zusammen.

Nehmen wir an, dass es m mögliche Standorte für die Anlagen gibt, die durch die Indexwerte $\{1,\ldots, m\}$ bezeichnet werden, und n Kunden, die durch die Indexmenge $\{1,\ldots, n\}$ bezeichnet werden. Die Transportkosten sind durch eine Matrix $C = [c_{ij}]$ gegeben, wobei c_{ij} die Kosten für einen Transport vom Standort j zum Kunden i angibt. Alternativ könnte man auch die Entfernungen zwischen den Standorten verwenden und diese mit geeigneten Kostenfaktoren multiplizieren. Die Einrichtungskosten einer Anlage werden als f_j für $j \in \{1,\ldots, m\}$ bezeichnet.

Das Unkapazitierte Facility Location-Problem (UFLP) kann dann wie folgt spezifiziert werden (siehe z. B. Ghosh (2003)):

$$\min \sum_{j \in S} f_j + \sum_{i=1}^{n} \min_{j \in S} c_{ij} \tag{6.16}$$

für eine Menge S von zu bestimmenden Anlagenstandorten, $\varnothing \neq S \subseteq \{1,\ldots, m\}$.

Eine alternative Formulierung mit binären Entscheidungsvariablen würde wie folgt lauten:

$$\min \sum_{j=1}^{m} f_j y_j + \sum_{i=1}^{n} \sum_{j=1}^{m} c_{ij} x_{ij} \tag{6.17}$$

so dass

$$\sum_{j=1}^{m} x_{ij} = 1, \quad i \in \{1,\ldots, n\} \tag{6.18}$$

$$x_{ij} \leq y_j, \quad i \in \{1,\ldots, n\}, \; j \in \{1,\ldots, m\} \tag{6.19}$$

$$x_{ij} \in \{0, 1\}, \; y_j \in \{0, 1\}, \quad i \in \{1,\ldots, n\}, \; j \in \{1,\ldots, m\} \tag{6.20}$$

Die Entscheidungsvariablen y_j geben an, ob der jeweilige Standort geöffnet ist, während die Variablen x_{ij} angeben, ob Kunde i von Standort j bedient wird. Gl. (6.18) stellt sicher, dass jeder Kunde von genau einem Standort bedient wird, Gl. (6.19) stellt sicher, dass nur offene Standorte die Kunden bedienen. Gl. (6.20) legt fest, dass alle Entscheidungsvariablen binär sind. Die Zielfunktion (6.17) definiert die Gesamtkosten, die sich aus den Einrichtungskosten der offenen Standorte plus den Transportkosten von den zugewiesenen Standorten zu den Kunden zusammensetzen.

In der Literatur werden mehrere Varianten des Problems betrachtet, z. B. Probleme mit zusätzlichen Budgetbeschränkungen zur Begrenzung der Menge offener Standorte (siehe z. B. Ghaderi und Jabalameli (2013), Guha und Khuller (1999)), Probleme mit mehreren Ebenen (z. B. Stufen in einer Supply Chain) (Tcha & Lee, 1984) oder Probleme, die mögliche Ausfälle von Anlagen berücksichtigen (Guha et al., 2003).

6.2.3.2 Lösungsansätze

Es ist bekannt, dass das UFLP NP-schwer ist (Cornuéjols et al., 1983), aber es gibt eine Reihe von Approximationsalgorithmen mit bekannter Worst-Case-Qualität. Außerdem kann das Problem für einige Spezialfälle effizient gelöst werden (Jones et al., 1995).

Wie die zuvor betrachteten Standortprobleme kann auch das UFLP durch einfache und problemspezifische Heuristiken, durch Metaheuristiken und durch mathematische Ansätze gelöst werden.

Ein Beispiel für die Verwendung einer einfachen Heuristik ist Ghosh (2003), wo Verbesserungsschritte in Betracht gezogen werden, bei denen entweder ein neuer Standort hinzugefügt wird, der zuvor geschlossen war, oder bei denen ein offener und ein geschlossener Standort ausgetauscht werden. Solche lokalen Suchkonzepte sind in eine Tabu-Suche und einen Backtracking-Ansatz eingebettet. In Sun (2006) wird eine Tabu-Suchstrategie mit einfacheren Veränderungsschritten erforscht. Diese Veränderungen berücksichtigen nur das Hinzufügen und Entfernen von Standorten zur bzw. von der offenen Menge, aber der Ansatz schneidet in einer Reihe von numerischen Experimenten sehr gut ab. Eine ähnliche Version der Tabu-Suche mit Hinzufügen und Weglassen von Standorten wurde auch in Al-Sultan und Al-Fawzan (1999) untersucht. Ein weiterer Ansatz, der hauptsächlich auf lokaler Suche basiert, aber durch die Verwendung verschiedener Startlösungen Zufälligkeit einführt, wird in Cura (2010) untersucht. In einer Reihe von Experimenten schnitt dieser Ansatz trotz seiner Einfachheit vergleichbar mit einigen anderen Ansätzen ab, darunter mehrere Tabu-Suchvarianten und ein genetischer Algorithmus.

Genetische Algorithmen wurden auf den UFLP angewandt, zum Beispiel in Jaramillo et al. (2002). In diesem Ansatz können Lösungen repräsentiert werden, indem ein Bitstring nur für die Einrichtungen vorhanden ist, um anzuzeigen, welche offen und welche geschlossen sind. Die Zuweisung von Kunden ist nicht explizit erforderlich, da die Kunden der billigsten oder der nächstgelegenen offenen Einrichtung zugewiesen werden können. Auch genetische Operatoren können in kanonischer Weise angewendet werden. Aus Leistungsgründen werden jedoch ein spezieller Crossover-Operator (fitnessbasierte Verschmelzung) und Mutationen auf der Grundlage des Austauschs von Anlagenstandorten vorgeschlagen. Die gleiche Kodierung für GA-Lösungen wird in Kratica et al. (2001) verwendet, aber es werden andere genetische Operatoren eingesetzt. Der Crossover-Operator verwendet hier eine zufällig erstellte Maske für den Austausch von Bits zwischen zwei Elternlösungen. Der Mutations-Operator verwendet zufällige Bit-Änderungen mit einer

durch eine Heuristik gesteuerten (zufälligen) Häufigkeit. Eine weitere GA-Implementierung wird in Tohyama et al. (2011) erörtert, die dieselbe Lösungsrepräsentation, aber einfaches Ein-Punkt-Crossover und Bit-ändernde Mutationen verwendet. Als zusätzlicher Mutationsoperator wird in diesem Ansatz eine lokale Suche eingeführt.

Ein weiterer metaheuristischer Ansatz, der für die Lösung des UFLP verwendet werden kann, ist Particle Swarm Optimization (PSO), wie in Guner und Sevkli (2008) beschrieben. Der PSO-Algorithmus, der normalerweise kontinuierliche Variablen verwendet, wird modifiziert, indem binäre Variablen hinzugefügt werden, die angeben, ob eine Einrichtung geöffnet ist oder nicht. Diese binären Variablen werden durch ein geeignetes Mapping (basierend auf Abrunden und Runden) aus gewöhnlichen kontinuierlichen Variablen bestimmt. Außerdem wird eine diskrete PSO-Variante analysiert, die nur diskrete Repräsentationen von Lösungen verwendet. Diese Variante verwendet ebenfalls Variationsoperatoren, wie sie in evolutionären Algorithmen verwendet werden, Mutationen, Ein-Punkt- und Zwei-Punkt-Crossover. Außerdem wurden beide PSO-Varianten mit eingebetteter lokaler Suche untersucht. Während die reinen PSO-Varianten bei einer Reihe von numerischen Tests weniger überzeugend waren, schnitten die Varianten mit eingebetteter lokaler Suche auch im Vergleich mit einem genetischen Algorithmus und evolutionärem Simulated Annealing sehr erfolgreich ab. In Wang et al. (2008) wurde ein weiterer PSO-Algorithmus in einer Multi-Populations-Variante und unter Verwendung einer parallelen Implementierung verwendet.

In Marić et al. (2012) werden drei Metaheuristiken für eine spezielle Variante des UFLP untersucht: PSO, Simulated Annealing und eine Kombination aus reduzierter und standardmäßiger variabler Nachbarschaftssuche, bezeichnet als RVNS-VNS. Der letztgenannte Ansatz schneidet in einer Reihe von Berechnungsexperimenten am besten ab.

6.2.4 Das Capacitated Facility Location Problem (CFLP)

6.2.4.1 Problemstellung

Das Capacitated Facility Location Problem (CFLP) entspricht im Wesentlichen der Version ohne Kapazitäten (6.17)–(6.20) mit einer zusätzlichen Einschränkung bezüglich der Anlagenkapazitäten, die wie in Gl. (6.8) formuliert werden kann. Man beachte, dass das Problem manchmal auch als Single Source Capacitated Facility Location Problem (SSCFLP) bezeichnet wird, weil es davon ausgeht, dass die gesamte Nachfrage eines Kunden durch eine einzige zugewiesene Anlage befriedigt wird. In Anlehnung an diese Terminologie erlauben einige Autoren für das CFLP, dass die Variablen x_{ij} nicht notwendigerweise binär sind (sondern nicht-negative reelle Werte haben), was Lösungen mit aufgeteilter Nachfragebefriedigung zwischen den Einrichtungen (multiple sourcing) ermöglicht (siehe z. B. Sankaran, 2007).

Wie bei der UFLP gibt es auch bei der CFLP mehrere in der Literatur untersuchte Varianten, z. B. ein Zwei-Echelon-Modell (mit Anlagen auf verschiedenen Stufen der Supply Chain) (Tragantalerngsak et al., 1997), eine Mehrperioden-CFLP und eine Multicommodity-CFLP für die Berücksichtigung mehrerer Nachfragegüter (siehe z. B. Arostegui et al., 2006) sowie Probleme mit mehreren Anlagen am selben Standort (Ghiani et al., 2002).

6.2.4.2 Lösungsansätze

Das NP-schwere CFLP hat vor allem zur Anwendung mathematisch orientierter Lösungsansätze und mehr oder weniger einfacher Heuristiken (siehe z. B. Sridharan, 1995), aber auch einer Reihe metaheuristischer Ansätze geführt. Ein Ansatz der Nachbarschaftssuche (very large-scale neighborhood search) wird z. B. in Ahuja et al. (2004) untersucht. Nachbarschaften für die lokale Suche werden hier auf zwei Arten definiert, erstens eine Nachbarschaft, die auf dem Austausch von Standorten basiert, die Kunden zyklisch zugewiesen werden, und zweitens eine Nachbarschaft, die durch die Veränderung von Standorten definiert wird, d. h. durch das Öffnen von Standorten, das Schließen von Standorten oder den Austausch von geschlossenen und offenen Standorten. Im Vergleich zu einer Lagrangeschen Relaxationsheuristik führte dieser Ansatz zu vergleichbaren Ergebnissen.

In Arostegui et al. (2006) wurden drei Metaheuristiken für das CFLP (und zwei Problemvarianten) verglichen: Tabu-Suche, Simulated Annealing und ein genetischer Algorithmus. Während die Tabu-Suche bei identischer Laufzeit für das Problem am besten abschnitt und das Simulated Annealing am zweitbesten war, konnte der genetische Algorithmus bei einer geringeren Anzahl von ausgewerteten Lösungen bessere Ergebnisse erzielen (was auf eine ungünstige Implementierung des GA hindeuten könnte).

Eine komplexere hybride Metaheuristik wird in Chen und Ting (2008) erforscht. Bei diesem Ansatz werden wiederholt eine Lagrangesche Heuristik und ein Multiple Ant Colony System (MACS) abwechselnd eingesetzt. Der MACS-Teil des Ansatzes verwendet zwei Arten von Pheromon-Informationen und zwei Ansätze zur Lösungskonstruktion: Eine Art von Pheromon ist spezifisch für jeden Standort. Diese Pheromon-Information wird zusammen mit einem Präferenzindikator, der vom Verhältnis zwischen der bereitgestellten Kapazität und den Einrichtungskosten einer Anlage abhängt, für die probabilistische Auswahl der offenen Anlagen verwendet. Die Anzahl der auszuwählenden Einrichtungen wird ebenfalls auf probabilistische Weise unter Berücksichtigung der Gesamtnachfrage im Verhältnis zur durchschnittlichen Kapazität einer Einrichtung bestimmt. Der zweite Typ von Pheromonen ist spezifisch für Kanten im Graphen zwischen Standorten und Kunden und wird für die Zuweisung von Kunden zu offenen Standorten verwendet. Für die probabilistische Zuordnung von Standorten werden die jeweiligen Pheromonwerte zusammen mit den reziproken Kosten der jeweiligen

Zuordnung verwendet. Da der Ansatz die Zulässigkeit einer Lösung in Bezug auf die Kapazitätsbeschränkung nicht direkt berücksichtigt, wird die Zielfunktion durch einen Strafterm modifiziert, um Kapazitätsverletzungen zu bestrafen. Außerdem beinhaltet der Ansatz eine lokale Suche, die nur auf die beste Lösung jeder Iteration angewendet wird. Diese Suche basiert auf zwei Heuristiken, von denen eine die Neuzuweisung von Kunden (möglicherweise auch an noch geschlossene Standorte) und die andere den Austausch von Kunden in Bezug auf die ihnen zugewiesenen Standorte verwendet.

In Sun (2012) wird ein Ansatz der Tabu-Suche für eine Multi-Sourcing-Version des CFLP verwendet. Die Tabu-Suche bestimmt die Einstellung der Variablen y_j für den Status des Standorts (offen oder geschlossen) durch Veränderungen im Sinne von Hinzufügen und Verwerfen von Standorten, die die Zulässigkeit der Lösungen aufrechterhalten. Die Einstellungen der x_{ij} Variablen, von denen angenommen wird, dass sie reellwertig sind, können anschließend durch einen Netzwerkalgorithmus bestimmt werden. Die Tabu-Suche wird durch einen eingebetteten Prozess der Verstärkung von Lösungen mit Path-Relinking ergänzt. Ein weiterer hybrider Ansatz, der zwei verschiedene Methoden für die beiden Teilprobleme verwendet, wird in Harris et al. (2011) für eine zweikriterielle Problemformulierung erforscht. Bei der ersten Methode handelt es sich um einen GA, der Bit-Strings zur Bestimmung der y_j Werte verwendet, während die x_{ij} Variablenwerte durch Lagrange-Relaxation ermittelt werden.

Weitere Metaheuristiken, die auf das Problem angewandt werden, sind Scatter Search, die in Contreras und Díaz (2008) zusammen mit GRASP für die Diversifikation und Tabu-Suche zur Verbesserung der in Betracht gezogenen Lösungen verwendet wird. Ein reaktives GRASP wird für eine stochastische Problemvariante in Silva und De la Figuera (2007) verwendet. Eine weitere neuere Metaheuristik, die als Kernel Search bezeichnet wird, wurde in Guastaroba und Speranza (2014) auf das CFLP angewendet.

6.3 Kontinuierliche Standortprobleme

Im allgemeinen Fall erscheint es zu restriktiv, die Anzahl der Standortalternativen auf eine kleine, explizit vorgegebene Menge zu beschränken. Theoretisch könnten alle geografischen Orte (z. B. Koordinaten auf einer Ebene oder geografische Koordinaten) mögliche Standorte sein, obwohl natürlich die meisten Koordinaten unter Berücksichtigung verschiedener praktischer Aspekte als ungeeignet gelten würden. Eine andere Möglichkeit, mögliche Orte in einem allgemeineren Sinne zu beschreiben, könnte darin bestehen, sie an beliebiger Stelle in Graphen zu platzieren, die z. B. ein Straßennetz darstellen, d. h. nicht nur an Orten aus der endlichen Menge der Knoten eines Graphen, sondern möglicherweise auch entlang von Kanten (siehe z. B. Sherali und Nordai (1988)).

6.3.1 Das „Uncapacitated Multi-facility Weber Problem“ (UMWP)

Weber-Probleme, die zu den berühmtesten Problemen der Standorttheorie gehören, sind nach dem Wirtschaftswissenschaftler Alfred Weber benannt, der sich zum ersten Mal mit einem verwandten Standortproblem befasste. Gelegentlich werden solche Probleme auch als Fermat-Weber-Probleme bezeichnet, da der Mathematiker Pierre de Fermat schon früher über Probleme nachgedacht hat, bei denen es darum geht, Punkte zu finden, die die Summe der Entfernungen zu bestimmten Punkten minimieren, ohne dabei eine geografische Anwendung im Sinn zu haben. Manchmal wird das Problem in der Literatur auch als (Multi-Facility-)Standortallokationsproblem bezeichnet.

6.3.1.1 Problemstellung

Die Grundidee des Standortproblems besteht darin, einen oder mehrere Punkte in der Ebene zu finden, die die Summe der entfernungsabhängigen Transportkosten von diesen Punkten zu n vorgegebenen Kundenzielen minimieren.

Genauer gesagt wird angenommen, dass es eine Menge von n Kunden mit bekannten Standorten gibt, die als zweidimensionale Vektoren (Koordinaten in der Ebene) $a_i = (a_{i1}, a_{i2})$ bezeichnet werden. Außerdem gibt es eine Menge von m Einrichtungen mit zu bestimmenden Standorten, die als $x_j = (x_{j1}, x_{j2})$ für $j \in \{1,\ldots, m\}$ bezeichnet werden. Weitere Entscheidungsvariablen beschreiben die Zuordnung eines Kunden i zu einer Einrichtung j, die mit x_{ij} bezeichnet wird. Die Transportkosten pro Entfernungseinheit des Kunden i werden mit c_i bezeichnet (die in der Regel proportional zur Nachfragemenge sein sollten), und die Entfernung wird durch eine l_p Metrik berechnet. Man beachte, dass der Faktor c_i in den entsprechenden Problemformulierungen oft weggelassen wird, so dass bestimmte zu befördernde Mengen in der Zielfunktion nicht berücksichtigt werden. Das unkapazitierte Multi-Facility-Weber-Problem (UMWP) kann dann wie folgt formuliert werden:

$$\min \sum_{i=1}^{n}\sum_{j=1}^{m} x_{ij} c_i d\left(x_j, a_i\right) \tag{6.21}$$

$$\sum_{j=1}^{m} x_{ij} = 1, i \in \{1, \ldots, n\} \tag{6.22}$$

$$x_{ij} \in \{0, 1\}, \quad a_j \in \Re^2, \quad i \in \{1, \ldots, n\}, \; j \in \{1, \ldots, m\} \tag{6.23}$$

Die Zielfunktion (6.21) beschreibt die Kosten, die sich aus der Multiplikation der Entfernungen mit einem Kostenparameter c_i ergeben, der als proportional zu der versendeten Menge angenommen wird. Gl. (6.22) stellt sicher, dass jeder Kunde

einem Standort zugeordnet ist, und Gl. (6.23) definiert die zuordnungsspezifischen Entscheidungsvariablen als binäre und die standortspezifischen Entscheidungsvariablen als zweidimensionale reellwertige Vektoren.

Meistens werden die in der Problemstellung verwendeten Entfernungen unter Verwendung einer euklidischen Metrik berechnet, d. h.

$$d\left(x_{j,}a_i\right) = \sqrt[2]{\left(\mathrm{x}_{\mathrm{j1}} - \mathrm{a}_{\mathrm{i1}}\right)^2 + \left(\mathrm{x}_{\mathrm{j2}} - \mathrm{a}_{\mathrm{i2}}\right)^2}, \tag{6.24}$$

aber auch andere Metriken werden in der Literatur verwendet (z. B. die Manhattan-Metrik).

Häufig wird eine relaxierte Version des Problems betrachtet, bei der $x_{ij} \geq 0$ reelle Zahlen sind. Während das obige Problem ein gemischt ganzzahliges Optimierungsproblem ist, ist die relaxierte Version ein nichtlineares Optimierungsproblem mit linearen Nebenbedingungen. Das relaxierte Problem wird auch als Multisource-Weber-Problem bezeichnet, während die Gl. (6.21)–(6.23) auch als Single-Source-Weber-Problem bezeichnet werden können.

Eine weitere Vereinfachung des Problems besteht darin, $m = 1$ anzunehmen, was auch als „Single Facility Weber"-Problem bezeichnet wird. Es gibt auch Varianten, bei denen nicht von einer festen Anzahl von Anlagen *m ausgegangen wird*, sondern diese ebenfalls optimiert werden können, z. B. ähnlich wie beim obigen UFLP oder CFLP (siehe z. B. Brimberg et al., 2004).

6.3.1.2 Lösungsansätze

Es sei darauf hingewiesen, dass sich das UMWP wie die meisten der betrachteten Standortprobleme als NP-schwer erweist (Megiddo & Supowit, 1984). Die meisten Methoden zur Lösung des Problems stammen entweder aus dem Bereich der mathematischen Programmierung oder können als einfache Heuristiken oder Metaheuristiken beschrieben werden.

Wir wollen eine der Heuristiken etwas näher erläutern, da sie als Hintergrundkonzept in mehreren anderen Ansätzen verwendet wird: Die Heuristik von Cooper ,(1964) beruht auf der Beobachtung, dass die Standort- und die Zuordnungsaufgaben des unkapazitierten Weber-Problems ohne Schwierigkeiten getrennt gelöst werden können. Wenn die Kundenzuordnung bekannt ist, besteht das Problem in der Lösung von *m* Einzelstandort-Weber-Problemen, was effizient durchgeführt werden kann. Sind hingegen die Standorte definiert, kann das Zuordnungsproblem leicht gelöst werden, indem die Kunden der nächstgelegenen Einrichtung zugewiesen werden (unter der Annahme, dass keine Kapazitäts- oder sonstigen Beschränkungen bestehen). Die Cooper-Heuristik, die auch als alternative Standortzuweisungsmethode bezeichnet wird, löst diese beiden Probleme wiederholt, bis keine weiteren Verbesserungen mehr auftreten.

Neben diesem Ansatz gibt es weitere Heuristiken und Ansätze, die auf mathematischen Optimierungstechniken zur Lösung des Problems basieren. Auch

verschiedene Ansätze aus dem Bereich der CI, insbesondere Metaheuristiken, wurden für das UMWP vorgeschlagen: Ein Ansatz, der Ideen der Tabu-Suche verwendet, wird in Gamal und Salhi (2001) untersucht. Auf der Grundlage eines Konzepts, das der Cooper-Heuristik ähnelt, wählen sie die anfänglichen Standorte auf der Grundlage eines Konzepts der weitesten Entfernung aus, das schrittweise neue Standorte für Anlagen bestimmt, indem es nach Kundenstandorten sucht, die so weit wie möglich von den bereits bestimmten Standorten entfernt sind. Bei den neuen Standorten wird darauf geachtet, dass sie nicht zu nahe an früheren Kundenstandorten liegen (Tabu-Punkte). Ein anderer Ansatz der Tabu-Suche wird von Ohlemüller (1997) verwendet. Hier wird die Tabu-Suche zur Bestimmung der Zuordnungen von Kunden zu Standorten verwendet (ausgehend von einer Zufallslösung). Anschließend werden die Standortprobleme als Einzelstandort-Weber-Probleme gelöst und die Zielfunktion kann bewertet werden. Danach werden Nachbarlösungen, die durch die Neuzuordnung einzelner Kunden definiert sind, unter Berücksichtigung der Tabu-Liste untersucht. Um eine vorzeitige Konvergenz zu vermeiden, ist ein Diversifikationsverfahren in die Methode integriert.

Die früheste Anwendung evolutionärer Algorithmen auf das UMWP scheint von Houck et al. (1996) zu stammen. Zur Repräsentation von Lösungen werden Fließkommazahlen für die reellwertigen Koordinaten der Standorte verwendet. Zur Einschränkung ihrer Werte können untere und obere Schranken verwendet werden. Der EA verwendet sieben verschiedene Standard-Variationsoperatoren und darüber hinaus ist die Cooper-Heuristik in den Algorithmus eingebettet, um die Lösungen zu verbessern und die Fitnessfunktion zu bewerten. Ein anderer evolutionärer Algorithmus, der in Brimberg et al. (2000) vorgeschlagen wird, verwendet dieselbe Kodierung, aber ein anderes Crossover-Schema, das die Berücksichtigung eines minimalen Trennungsabstands der Nachkommenlösungen zu ihren Eltern einschließt, während der Mutationsoperator zugunsten der Verwendung der Cooper-Heuristik für Lösungsverbesserungen weggelassen wird. Ein ähnlicher Ansatz wird in Salhi und Gamal (2003) mit einem zusätzlichen Operator zur Einschleusung neuer Zufallslösungen während des Algorithmuslaufs angewandt, um die Vielfalt zu erhöhen und eine vorzeitige Konvergenz zu vermeiden. Die erzielten Ergebnisse sind im Vergleich zu früheren genetischen Algorithmen besser.

In Brimberg et al. (2006) wird eine variable Nachbarschaftssuche (VNS) auf der Grundlage eines Dekompositionsansatzes vorgeschlagen (Variable Neighborhood Decomposition Search oder VNDS), bei der Zufallslösungen aus den Nachbarschaften verwendet und anschließend durch lokale Suche verbessert werden. Die Dekomposition basiert auf der Auswahl einer kleinen Anzahl k von Standorten, die dann in Bezug auf ihre Koordinaten verschoben werden. Eine solche Gruppe von k Standorten kann durch Berücksichtigung der Nähe, nach dem Zufallsprinzip oder durch eine gemischte Strategie auf der Grundlage der beiden anderen gebildet werden.

Particle Swarm Optimization wird in Ghaderi et al. (2012) unter der Annahme von Ober- und Untergrenzen für die Standortkoordinaten verwendet, die sich aus den Kundenkoordinaten ergeben. Die Zulässigkeit von Lösungen wird durch zufälliges Zurücksetzen der Koordinaten in den Lösungsraum oder durch zufällige

Verwendung der Koordinaten eines Kunden erreicht. Der Ansatz wird mit mehreren lokalen Suchheuristiken hybridisiert: der Cooper-Heuristik und zwei Heuristiken, die auf dem Austausch von Anlagenstandorten mit Kundenstandorten basieren. Hybride Metaheuristiken, die auf zwei Varianten von VNS, einem genetischen Algorithmus und mehreren lokalen Suchheuristiken basieren, werden in Jabalameli und Ghaderi (2008) untersucht.

Auch neuronale Netze werden gelegentlich zur Lösung des Problems eingesetzt. Ein modifiziertes Kohonen-Netz (selbstorganisierende Karte) wird in Aras et al. (2006) zusammen mit einer als Vektorquantisierung bezeichneten Technik zur Lösung eines unkapazitierten Weber-Problems verwendet. Ein ähnlicher Ansatz findet sich in Hsieh und Tien (2004) und Lozano et al. (1998). Es wird angenommen, dass das Kohonen-Netz eine kanonische Darstellung des Problems liefert. Es besteht aus zwei Schichten, wobei die Eingabeschicht den zweidimensionalen Bedarfsorten entspricht. Die Ausgangsschicht besteht aus m Neuronen, die den Anlagenstandorten entsprechen. Jeder dieser Knoten befindet sich auf einer zweidimensionalen Ebene, deren Koordinaten als Gewichte interpretiert werden. Einer der Knoten wird aktiviert, wenn die Eingabewerte (d. h. die Kundenkoordinaten) den Gewichten am nächsten liegen. Dies entspricht einer entsprechenden Kunden-Anlagen-Zuordnung. In der Lernphase des Algorithmus wird versucht, die Abstände zwischen den Eingabewerten und den Gewichten des jeweiligen Neurons zu verringern, was dem Ziel entspricht, die Anlagenstandorte so nah wie möglich an den bedienten Kunden zu platzieren.

Ein Vergleich verschiedener Metaheuristiken, darunter Tabu-Suche, Simulated Annealing, variable Nachbarschaftssuche, ein evolutionärer Algorithmus und Ant Colony Optimization, führte zu den folgenden Ergebnissen für unkapazitierte Weber-Probleme, die mit einer verallgemeinerten Kostenfunktion gelöst wurden (Bischoff & Dächert, 2009): Während ACO für kleinere Problemgrößen am häufigsten die besten bekannten Lösungen fand, erwiesen sich Tabu-Suche und der evolutionäre Algorithmus für größere Instanzen als am leistungsfähigsten. Außerdem war der evolutionäre Algorithmus bei größeren Problemen am wenigsten zeitaufwändig.

6.3.2 Das Capacitated Multi-facility Weber Problem (CMWP)

6.3.2.1 Problemstellung

Das kapazitierte Multi-Facility-Weber-Problem (CMWP) ist im Grunde die unkapazitierte Version mit einer zusätzlichen Kapazitätsbeschränkung, die mit der spezifischen Nachfrage der Kunden in Einklang gebracht werden muss.

Dazu werden die Kunden durch eine bekannte Nachfrage d_i für $i \in \{1,\ldots, n\}$ zusätzlich zu ihren Standortkoordinaten charakterisiert. Die Anlagen sind durch gegebene Mengen q_j eines betrachteten Gutes für $j \in \{1,\ldots, m\}$ charakterisiert. Die Entscheidungsvariablen sind die Standorte der Anlagen, bezeichnet als $x_j = (x_{j1}, x_{j2})$, und zusätzlich die von einer Anlage j zu einem Kunden i zu transportierenden

Mengen, bezeichnet als q_{ij}. Die Transportkosten sind wiederum gegeben durch c_{ij} pro Einheit des Gutes und pro Entfernungseinheit, wobei die Entfernung durch eine geeignete Metrik berechnet wird. Es ist auch möglich, dass die Kosten nur von der Höhe der Nachfrage des Kunden i abhängen.

Mathematisch kann das Problem wie folgt formuliert werden:

$$\min \sum_{i=1}^{n} \sum_{j=1}^{m} q_{ij} c_{ij} d\left(x_j, a_i\right) \tag{6.25}$$

$$\sum_{j=1}^{m} q_{ij} = d_i, i \in \{1, \ldots, n\} \tag{6.26}$$

$$\sum_{i=1}^{n} q_{ij} = q_j, j \in \{1, \ldots, m\} \tag{6.27}$$

$$q_{ij} \geq 0, \quad i \in \{1, \ldots, n\}, j \in \{1, \ldots, m\} \tag{6.28}$$

Gl. (6.25) formuliert die zu minimierende Gesamtkostenfunktion. Gl. (6.26) stellt sicher, dass der gesamte Bedarf gedeckt wird. Gl. (6.27) garantiert, dass alle Transporte durch ausreichende Mengen an den Quellknoten gedeckt werden.

Außerdem geht das Modell davon aus, dass die gegebenen Mengen an den Standorten und die Gesamtnachfrage übereinstimmen, d. h.

$$\sum_{i=1}^{n} d_i = \sum_{j=1}^{m} q_j, \tag{6.29}$$

Ist dies nicht der Fall, können bei Übermengen in den Betrieben Dummy-Kunden eingeführt werden. Wenn die Gesamtnachfrage zu hoch ist, ist das Problem nicht lösbar. Dies kann jedoch leicht herausgefunden werden, bevor man mit der Lösung des Problems beginnt.

Es sei darauf hingewiesen, dass das CMWP dem UMWP die zusätzliche Kapazitätsbeschränkung (6.27) hinzufügt. Bei dieser Art von Problem ist es im Allgemeinen nicht mehr möglich, die gesamte Nachfrage eines Knotens durch die nächstgelegene Anlage zu befriedigen. Daher erfordert das Problem die Berücksichtigung expliziter Mengen q_{ij} des von den Unternehmensstandorten an die Kunden gelieferten Gutes, anstatt eine eindeutige Zuordnung von Kunden zu diesen Standorten anzunehmen. Diese Annahme der Mehrfachbeschaffung impliziert, dass auch Gl. (6.26) zusätzlich erforderlich ist.

Man beachte, dass es Varianten des Problems gibt, die ein Single-Sourcing voraussetzen, d. h. dass der gesamte Bedarf eines Kunden von einer einzigen Anlage gedeckt wird. Andere Varianten des CMWP umfassen zum Beispiel Formulierungen mit mehreren Gütern (Akyüz et al., 2010) oder Varianten, bei denen die Anzahl der (offenen) Anlagen nicht a priori festgelegt ist (Doong et al., 2007). Gong

et al. (1995) untersuchen ein CMWP mit Hindernissen, die als Polygone definiert sind, die als Standorte für Anlagen verboten sind und nicht überquert werden dürfen. In der Literatur werden auch stochastische und unscharfe Problemformulierungen diskutiert. So kann beispielsweise die Unsicherheit in Bezug auf die erwartete Kundennachfrage in Fuzzy-Modellen berücksichtigt werden, siehe z. B. Zhou und Liu (2007).

6.3.2.2 Lösungsansätze

Im Vergleich zum UMWP wird das CMWP in der Literatur weniger häufig betrachtet und auch Ansätze, die sich mit CI beschäftigen, werden seltener verwendet. In Manzour-al-Ajdad et al. (2012) wird Simulated Annealing für die Zuordnung von Kunden zu Anlagen verwendet. Ausgehend von einer durch die Cooper-Heuristik erhaltenen Anfangslösung ist das Simulated Annealing insbesondere für die Wiedererlangung der Zulässigkeit einer Lösung zuständig. Während des Durchlaufs des Simulated Annealing-Prozesses und danach werden verschiedene Verbesserungsheuristiken verwendet. Eine ähnliche Strategie wird in Öncan (2013) verwendet, wo Very Large Scale Neighborhood Search für die Kundenzuweisungen eingesetzt wird, während die Logik der Standortbestimmung auf der Cooper-Heuristik beruht.

Eine GRASP-Methode wird in Luis et al. (2011) verwendet. Das GRASP-Konzept wird zur Bestimmung der Standortkoordinaten der Anlagen verwendet, wobei nur die Kundenkoordinaten berücksichtigt werden, was das Problem zu einem diskreten Problem macht. Die Zuweisungen werden dann wie in der Heuristik von Cooper ermittelt. Da diese Heuristik keine Kapazitätsbeschränkungen berücksichtigt, wird ein zusätzlicher Ansatz angewandt, um Kunden im Falle von Kapazitätsverletzungen neu zuzuordnen. Außerdem wird ein Konzept der Einschränkung von Regionen verwendet, um zu vermeiden, dass zuvor gewählte Standorte während des Suchprozesses berücksichtigt werden.

In Gong et al. (1995) wird ein hybrider evolutionärer Algorithmus verwendet, um eine CMWP-Variante mit polygonalen Hindernissen zu lösen. Der EA wird für die Lösung des Standortaspekts des Problems verwendet, d. h. für die Bestimmung der Standorte der Anlagen, während ein anderes Optimierungsverfahren für die Zuweisung der Kunden zu den Standorten verwendet wird, d. h. für die Lösung des Zuweisungsteils. Der EA verwendet Repräsentationen auf der Basis von Fließkommazahlen für die Standortkoordinaten. Die Zulässigkeit der Lösungen wird durch die Reparatur einer Lösung sichergestellt: Befindet sich ein Standort in einem verbotenen Gebiet, werden die Koordinaten zum nächstgelegenen Eckpunkt des Hindernispolygons verschoben. Der Crossover-Operator verwendet intermediäre Rekombinationen, wie sie z. B. aus Evolutionsstrategien bekannt sind, d. h. die Standorte der Nachkommen werden als zufällige Punkte auf der Linie zwischen den beiden Elternstandorten definiert. Es werden zwei Arten von Mutationen berücksichtigt, und die Auswahl basiert auf einem elitären Konzept, wobei gleichzeitig vermieden wird, dass ähnliche Lösungen für die nächste Generation ausgewählt werden.

Ein ähnlicher hybrider Ansatz, der auf der Trennung von Standort- und Zuordnungsphase beruht, wird in Wen und Iwamura (2008) für eine Variante des CMWP betrachtet, bei der die Kundennachfrage als unscharf betrachtet wird. Zur Lösung einer umformulierten Version des Problems wird der Simplex-Algorithmus für die Lösung des Zuordnungsproblems verwendet, während für die Standortbestimmung ein genetischer Algorithmus eingesetzt wird.

Für ein Modell mit stochastischer Nachfrage wird in Zhou und Liu (2003) ein ähnlicher hybrider Ansatz verwendet, der ebenfalls einen genetischen Algorithmus für die Standortbestimmung und einen eingebetteten Simplex-Algorithmus für den Zuweisungsteil einsetzt. Die Stochastizität wird in den Algorithmen durch wiederholte Simulationen berücksichtigt.

In Doong et al. (2007) wird ein CMWP ohne eine feste Anzahl von Standorten und mit Einrichtungskosten für Einrichtungen ähnlich dem CFLP betrachtet. Bei dem vorgeschlagenen Lösungsansatz handelt es sich ebenfalls um eine hybride Metaheuristik, bei der ein GA zur Bestimmung der Standorte der Anlagen und eine Subgradientenmethode zur Lösung einer Lagrange-Relaxation des Zuordnungsproblems verwendet werden.

Ein anderer Ansatz wird in Mohammadi et al. (2010) betrachtet, der zwei verschachtelte genetische Algorithmen verwendet, von denen einer zur Bestimmung der Koordinaten der Anlagen und der andere zur Lösung des Zuordnungsproblems eingesetzt wird.

6.4 Location-Routing-Probleme

Im Allgemeinen verknüpfen die oben genannten Standortprobleme die Aufgabe der strategischen Standortplanung mit den nachfolgenden operativen Planungsaufgaben, wenn die jeweiligen Standorte genutzt werden, indem sie Transportentfernungen oder damit verbundene Kosten berücksichtigen. Diese Verknüpfung wird jedoch nur auf einer recht groben Ebene erreicht, da die Entfernungen zwischen Anlagen und Kunden meist als „gegeben“ angenommen oder mit einer Standardmetrik berechnet werden. Die Verwendung einer Metrik wie der euklidischen Metrik wird der Situation der Nutzung eines tatsächlichen Verkehrssystems wie eines Straßennetzes kaum gerecht. Manchmal wird angenommen, dass andere Metriken wie die Manhattan-Metrik ein besseres Maß für orthogonale Straßennetze darstellen, aber diese Situation ist in der Realität nicht sehr typisch. Ein noch schwerwiegenderer Nachteil des Entfernungskonzepts ist, dass es die Möglichkeiten der Tourenbildung in der Realität vernachlässigt. Wie in Kap. 3 erläutert, lassen sich im Transportwesen häufig kleinere Sendungen bündeln, die auf einer gemeinsamen optimierten Route transportiert werden. Abb. 6.2 zeigt ein Beispiel mit der Auswahl von 3 aus 6 möglichen Standorten und den dazugehörigen Touren zur Belieferung von 15 Kunden. Es ist zu beachten, dass aufgrund der Tourenoptimierung andere Standorte im Vergleich zu Abb. 6.1 als optimal erscheinen.

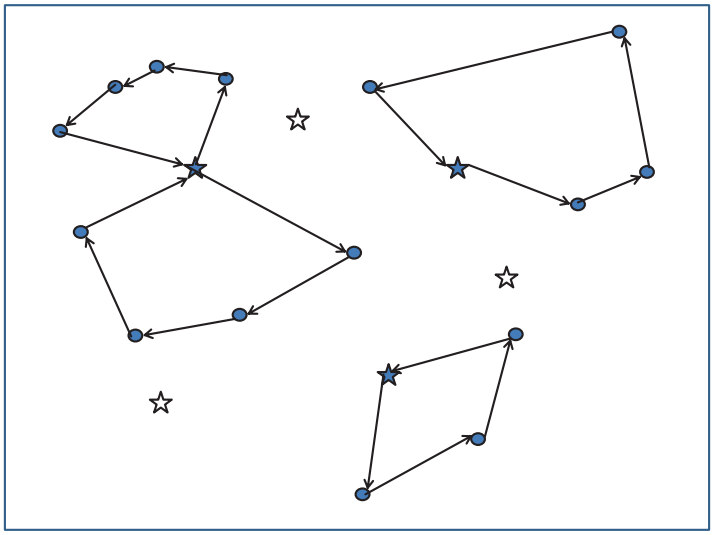


Abb. 6.2 Beispiel für ein Location-Routing-Problem, bei dem aus sechs möglichen Standorten drei ausgewählt und entsprechende Touren erstellt werden

Bei diesen sogenannten Location-Routing-Problemen wird dieser Aspekt durch die Kombination von Standort- und Routing-Problemen berücksichtigt. Das bedeutet, dass die Standorte der Anlagen bestimmt werden sollen, während die zu beachtenden Transportkosten aus der Lösung eines entsprechenden Vehicle-Routing-Problems resultieren. Wie in dem Übersichtsartikel von Nagy und Salhi (2007) erörtert, gibt es eine große Vielfalt von Location-Routing-Problemen, die es kaum erlaubt, einen Standardtypus zu benennen. Grob gesagt, können Location-Routing-Probleme aus jeder Art von Standortproblemen in Kombination mit jeder Art von Vehicle-Routing-Problemen erstellt werden. Einige Kombinationen sind jedoch aus theoretischer oder praktischer Sicht sinnvoller.

Meistens ist ein kapazitiertes Standortproblem der Ausgangspunkt für die Formulierung eines Location-Routing-Problems, d. h. ein Szenario mit einer endlichen Menge möglicher Standorte mit begrenzten Kapazitäten für die Belieferung von Kunden. In der Zielsetzung werden die Funktionen Transportkosten und Eröffnungskosten von Anlagen berücksichtigt. Die Transportkosten ergeben sich aus der Lösung eines Vehicle-Routing-Problems unter der Annahme, dass eine bestimmte Kundennachfrage befriedigt werden muss. In einigen Varianten des Problems kann die Anzahl der Anlagen vorgegeben sein, wie z. B. beim p-Median Problem. Für einen Überblick über verschiedenste Varianten des Location-Routing-Problems verweisen wir auf Drexl und Schneider (2015), wo beispielsweise dynamische und periodische Probleme, Probleme mit kontinuierlichen Standortentscheidungen, Multi-Echelon-Probleme, Mehrzielprobleme, Arc-Routing-Probleme, Pickup-and-Delivery-Varianten und Location-Routing-Probleme zusammen mit Entscheidungen über den Lagerbestand betrachtet werden.

Da es sich bei dem zugrundliegenden Standortproblem meist um ein Standortproblem für mehrere Standorte handelt, muss das zugehörige Vehicle-Routing-Problem auch mit mehreren Depots, d. h. Anlagestandorten, umgehen. Das in der Studie von Prodhon und Prins (2014) betrachtete Standard-Location-Routing-Problem geht von einer (entweder zu bestimmenden oder vorgegebenen) Anzahl verfügbarer Fahrzeuge aus, die in Bezug auf die Kapazität homogen sind und feste Kosten pro verwendetem Fahrzeug verursachen (abgesehen von variablen Transportkosten).

Was die Lösungsansätze betrifft, so scheint es naheliegend zu sein, die beiden Problemteile getrennt zu lösen. Dies würde typischerweise einen hierarchischen Ansatz voraussetzen, bei dem ein Lösungsverfahren für das Standortproblem (äußere Schleife) mit einem eingebetteten zweiten Verfahren zur Lösung des Vehicle-Routing-Problems verwendet wird, wenn Routen zur Ermittlung der Kosten (innere Schleife) ermittelt werden sollen. Da bereits das Vehicle-Routing-Problem schwierig ist, könnte man annehmen, dass der Zeitaufwand bei solchen Lösungsansätzen nicht realisierbar sein könnte. Wenn jedoch innerhalb der äußeren Schleife meist nur kleine Änderungen vorgenommen werden, kann ein großer Teil der Lösung des Tourenplanungsproblems beibehalten werden. Da andererseits die Planungsdaten für die angenommene Kundennachfrage als unsicher angesehen werden können, könnte möglicherweise ein schneller, aber weniger genauer Ansatz für das eingebettete Vehicle-Routing-Problem ausreichend sein.

In der Umfrage von Nagy und Salhi (2007) wurden Metaheuristiken noch nicht einmal als eine Klasse möglicher Lösungsansätze diskutiert (obwohl mehrere Anwendungen auf der Grundlage von variabler Nachbarschaftssuche, Tabu-Suche und Simulated Annealing erwähnt wurden). Die Situation hat sich in den folgenden 7 Jahren deutlich geändert, wie die Übersicht von Prodhon und Prins (2014) zeigt. Diese Übersicht listet nicht nur mehr als 20 Metaheuristik-Anwendungen auf, sondern diskutiert auch Matheuristiken, die Konzepte der mathematischen Programmierung mit Heuristiken und Metaheuristiken hybridisieren.

Es zeigt sich, dass unter den metaheuristischen Ansätzen Methoden, die auf lokaler Suche oder der Verwendung von Nachbarschaftskonzepten basieren, dominieren. Dies lässt sich durch die Offensichtlichkeit der Verwendung entsprechender Ansätze erklären, die auf einfacheren Heuristiken beruhen. Darüber hinaus können Einfachheit und Geschwindigkeit der Suche ein Grund dafür sein. Sehen wir uns ein paar Beispiele näher an: So wird in Derbel et al. (2010) eine iterierte lokale Suche (ILS) auf ein Location-Routing-Problem mit Depotkapazitäten, aber ohne Fahrzeugkapazitäten angewandt. Ausgehend von einer Anfangslösung konzentriert sich die lokale Suche nur auf den Vehicle-Routing-Teil des Problems. Es werden verschiedene Nachbarschaftsstrukturen berücksichtigt, z. B. für Änderungen innerhalb der Route oder den Austausch von Kunden zwischen verschiedenen Routen. Neben dieser lokalen Suche verwendet ILS einen Perturbationsmechanismus, der in dieser Anwendung für das Schließen und Öffnen von Einrichtungen mit anschließender Neuzuweisung von Kunden verwendet wird. Dieses ILS-Konzept ist in Derbel et al. (2012) in einen genetischen Algorithmus integriert, wobei der genetische Algorith-

mus eine Lösungsrepräsentation verwendet, die aus einem Zuweisungsteil und einem Teil zur Spezifikation von Routen besteht.

In Escobar et al. (2013) finden wir ein weiteres Beispiel für eine lokale Suche auf der Grundlage einer granularen Tabu-Strategie. Der Ansatz besteht aus einer Konstruktionsphase und einer Verbesserungsphase. Nach der Konstruktion einer ersten Lösung bleiben die Standorte der Anlagen fest, so dass in der Verbesserungsphase nur verschiedene lokale Änderungen innerhalb von Touren oder zwischen Touren berücksichtigt werden.

In einer anderen Studie (Jarboui et al. 2013) wird ein Location-Routing-Problem durch eine VNS-Variante mit lokaler Suche (variable neighborhood descent) unter Verwendung von fünf Nachbarschaftsstrukturen gelöst. Die Ergebnisse werden mit denen der iterierten lokalen Suche und der Tabu-Suche verglichen, wo der vorgeschlagene Ansatz besser oder gleich gut wie die beiden anderen Metaheuristiken abschneidet.

Bei dem in Yu et al. (2010) angewandten Simulated-Annealing-Ansatz werden das Standort- und das Routingproblem gleichzeitig behandelt. Lösungen werden als Strings dargestellt, die Anlagen (Depots) mit ihren Touren sowie Nullen als Begrenzungszeichen enthalten. Geschlossene Einrichtungen werden auch mit leeren Touren (d. h. ohne zugewiesene Kunden) dargestellt. Der Simulated Annealing-Ansatz greift auf lokale Änderungen wie Swap-Moves, Insertion-Moves und 2-opt-Moves zurück. Unzulässige Lösungen werden innerhalb der Methode durch einen Strafterm bestraft.

Wie in Prodhon und Prins (2014) erörtert, sind populationsbasierte Metaheuristiken wie evolutionäre Algorithmen und Schwarmintelligenz-Ansätze für Location-Routing-Probleme noch von untergeordneter Bedeutung, möglicherweise aufgrund weniger offensichtlichen Lösungsrepräsentationen und erforderlichen Anpassungen. Eine interessante Ausnahme ist der Vorschlag eines ACO-basierten Ansatzes zur Optimierung mit mehreren Ameisenkolonien in Ting und Chen (2013). Die Grundidee der Methode besteht darin, dass das Location-Routing-Problem in drei Teilprobleme unterteilt wird: die Auswahl offener Standorte, die Zuordnung von Kunden zu Standorten und das Vehicle-Routing-Problem. Die drei Teilprobleme werden auf hierarchische Weise gelöst, wobei ein Pheromon-Austausch zwischen den verschiedenen Ebenen genutzt wird. Unzulänglichkeiten auf der Ebene der Kundenzuweisung werden durch Reparaturoperationen gelöst, die auf der Neuzuweisung von Kunden oder der Eröffnung zusätzlicher Einrichtungen basieren. Außerdem sind zwei Ansätze für die lokale Suche in den Lösungsrahmen eingebettet.

6.5 Probleme mit Hub-Standorten

Es sei auf einige Aspekte realer strategischer Planungsprobleme für Supply Chains hingewiesen, die bei den in den vorangegangenen Abschnitten erörterten Standortproblemen nicht berücksichtigt werden. Man könnte sagen, dass diese Modelle nur

eine Ebene oder Stufe in eines Liefernetzwerks berücksichtigen, z. B. Verteilzentren, die die Verbraucher beliefern. Reale Lieferketten oder -netze sind jedoch wesentlich komplexer und umfassen selbst in einfachen Fällen mehrere Ebenen oder Stufen, z. B. Lieferanten, Werke, Crossdocking-Zentren, Distributionszentren, Großhändler, Einzelhändler, Endkunden, von denen sich einige mehrfach und auf verschiedenen Stufen einer Lieferkette befinden können.

Darüber hinaus ist es interessant festzustellen, dass selbst die in den diskutierten Problemen angenommene Entscheidungsebene der Supply Chain nicht vollständig berücksichtigt wird. Geht es bei den Standortproblemen beispielsweise um den Standort von Distributionszentren, die die Endkunden beliefern, so sollte auch berücksichtigt werden, wie diese von den Produktionsbetrieben beliefert werden. Das bedeutet, dass in den Zielfunktionen nicht nur die Kosten für den Transport zu den nachfolgenden Kunden berücksichtigt werden sollten, sondern auch die Transportkosten von vorgelagerten Einrichtungen wie Werken.

Veröffentlichungen, die solche erweiterten Problemformulierungen in Bezug auf Liefer- und Zustelltransporte berücksichtigen, bezeichnen diese Probleme meist als Hub-Location-Probleme, z. B. Farahani et al. (2013). Hub-Location-Probleme gibt es in vielen verschiedenen Varianten, ähnlich wie die Varianten der in den vorherigen Abschnitten betrachteten Probleme. So gibt es beispielsweise verschiedene Möglichkeiten, Lösungen zu spezifizieren (diskreter oder kontinuierlicher Lösungsraum), verschiedene Zielfunktionen, verschiedene Beschränkungen wie Kapazitäten oder hinsichtlich der Anzahl der Hub-Knoten oder der Annahme einer einfachen oder mehrfachen Beschaffung (Single oder Multiple Sourcing).

Für die Lösung von Problemen bei der Standortwahl von Hubs wurden in den letzten Jahren recht häufig Metaheuristiken eingesetzt. In der Übersichtsstudie von Farahani et al. (2013) werden beispielsweise 37 exakte, auf mathematischen Programmiertechniken basierende Ansätze aufgeführt, die auf verschiedene Arten von Hub-Location-Problemen angewendet werden. Demgegenüber stehen 21 auf mathematischer Programmierung basierende Heuristiken, 27 einfache oder problemspezifische Heuristiken und 31 auf Metaheuristiken basierende Ansätze. Die metaheuristischen Ansätze können wie folgt unterteilt werden (zusammen mit der Anzahl der Veröffentlichungen):

- Tabu-Suche (14 Veröffentlichungen)
- genetische Algorithmen (9 Veröffentlichungen)
- Simulated Annealing (5 Veröffentlichungen)
- Ant Colony Optimization (2 Veröffentlichungen)
- andere (5 Veröffentlichungen)

Zu den anderen Ansätzen, die alle nur eine Veröffentlichung vorweisen können, gehören die variable Nachbarschaftssuche, GRASP, Path Relinking und zwei recht neue Metaheuristiken, die als Extremale Optimierung und Imperialist Competitive Algorithm bezeichnet werden. Es ist zu beachten, dass die summierte Anzahl höher ist als die obige Anzahl der Veröffentlichungen von Metaheuristiken, da einige Studien mehrere Metaheuristiken anwenden.

6.6 Multi-Echelon-Netzwerkgestaltung

Ein weiteres, allgemeineres Planungs- und Optimierungsproblem ist die Standortbestimmung von Einrichtungen auf verschiedenen aufeinanderfolgenden Stufen in einer Lieferkette, z. B. die Bestimmung von Standorten von Werken und Lagern. Solche Probleme werden häufig unter dem Begriff der Multi-Echelon-Standortprobleme (oder, wie in den meisten Fällen, Zwei-Echelon-Probleme) betrachtet.

In Alumur et al. (2015) wird ein allgemeines Supply Chain-Modell ohne eine spezifische Schichtstruktur betrachtet: Die Knoten des Netzwerks umfassen beliebige Einrichtungen aus einer Lieferkette einschließlich Kundenknoten. Einige dieser Knoten werden als nicht auswählbar betrachtet (z. B. Kundenstandorte), während andere im Hinblick auf ihre mögliche Schließung zu entscheiden sind. Darüber hinaus kann eine endliche Anzahl von zusätzlichen Standorten eröffnet werden. Weitere Entscheidungsvariablen beziehen sich auf die Transportmengen zwischen den Standorten, die Produktionsmengen und die Menge der externen Beschaffung unter der Annahme eines Mehrproduktszenarios. Die Beziehungen zwischen den Produkten während der Herstellung werden durch Stücklisten berücksichtigt. Das Modell geht von begrenzten Kapazitäten für Produktions- und Logistikaktivitäten mit Zulässigkeit von Überstunden aus. Die Zielfunktion umfasst Rüst- und Schließungskosten der Anlagen, Fixkosten während des Betriebs, Kosten für Beschaffung, Fertigung und Transport, Überstundenkosten und Opportunitätskosten für nicht befriedigte Nachfrage. Es wird vorgeschlagen, das komplexe und umfangreiche Modell mit einem Universalwerkzeug zur Optimierung zu lösen.

Ein weiteres komplexes Modell für die Gestaltung und Neugestaltung von Netzwerken wird in Cortinhal et al. (2015) betrachtet. Dieses mehrperiodige Modell berücksichtigt vier Ebenen mit Standortentscheidungen auf zwei von ihnen (Werke und Lager), begrenzte, aber erweiterbare Kapazitäten, verschiedene Transportarten, Lieferantenauswahl und Outsourcing-Optionen. Die Zielfunktion berücksichtigt Fixkosten und variable Kosten. Fixe Kosten sind die Kosten für die Eröffnung und Schließung eines Standortes, für dessen Betrieb und für die Bereitstellung von Kapazitäten. Zu den variablen Kosten gehören die Beschaffungskosten für Material und ggf. Fertigwaren, Transportkosten und Herstellungskosten. In der Produktionsplanung wird mit Hilfe von Stücklisten der Materialbedarf für die Produktion von Fertigerzeugnissen ermittelt.

Cortinhal et al. (2015) geben einen aktuellen Überblick über Modelle für die Gestaltung oder Neugestaltung einer Supply Chain mit 18 früheren Modellen. Die vorgeschlagenen Lösungsansätze für komplexe Modelle für Netzwerk-Design und Redesign basieren zumeist auf Ansätzen der mathematischen Programmierung. Es scheint, dass bisher nur in Antunes und Peeters (2001) ein CI-Ansatz, nämlich Simulated Annealing, auf das betrachtete Problem angewendet wird. Das betrachtete Problem kann als eine Erweiterung des Standortproblems für kapazitierte Anlagen

beschrieben werden, bei dem zwei Netzwerkebenen berücksichtigt werden, die Standortentscheidung aber nur auf einer von ihnen getroffen wird. Es kann jedoch davon ausgegangen werden, dass entsprechende Forschungen zu komplexeren Netzwerkmodellen einschließlich der Anwendung von Metaheuristiken in Zukunft zunehmen werden.

6.7 Schlussfolgerungen

Wie wir gesehen haben, gibt es bereits eine Fülle von Varianten grundlegender Standortprobleme, die in der Literatur beschrieben und analysiert werden. Auch der Einsatz von CI-Methoden, insbesondere Metaheuristiken, ist für diese meist NP-schweren Optimierungsprobleme weit verbreitet. Im Allgemeinen ist es schwierig zu beurteilen, welcher Ansatz für welche Art von Problem am besten geeignet ist. Da problemspezifische und meist einfache Heuristiken in der Vergangenheit eine große Rolle gespielt haben (und immer noch spielen), ist es nicht überraschend, dass solche Konzepte oft in komplexere Metaheuristiken für die lokale Suche eingebettet sind. Dies dürfte auch einer der Gründe sein, warum Metaheuristiken, die auf lokaler Suche oder der Verwendung von Nachbarschaftskonzepten basieren (insbesondere Tabusuche und variable Nachbarschaftssuche), eine führende Rolle unter den Metaheuristiken zur Lösung von Standortproblemen erlangt haben. Evolutionäre Algorithmen scheinen die zweitgrößte Gruppe der angewandten CI-Ansätze zu bilden. Ein Grund dafür könnte die Ausgereiftheit dieser Methoden sein, aber möglicherweise auch die Flexibilität, z. B. mit unterschiedlichen Variablendomänen (wie binär und reellwertig) umzugehen. Ansätze der Schwarmintelligenz wurden bisher seltener eingesetzt, was mit dem geringeren Alter vieler dieser Ansätze zu tun haben könnte. Es ist daher davon auszugehen, dass die zukünftige Forschung diese Lücke verkleinern wird. Neuronale Netze haben bisher erst bei unkapazitierten Multi-Facility-Weber-Problemen eine gewisse Bedeutung erlangt.

In Bezug auf die betrachteten Problemtypen können wir feststellen, dass einige der Standard-Problemtypen, die Anlagenstandorte auf einer Ebene einer Lieferkette berücksichtigen, immer noch einen großen Teil der Forschung ausmachen. Die meisten neueren Modelle können als Varianten der Standardtypen betrachtet werden, die sie in einem oder mehreren Aspekten erweitern. Die Literatur ist spärlich, wenn es um komplexere Modelle geht, aber es ist davon auszugehen, dass sich diese Forschungsrichtung aufgrund ihrer enormen praktischen Bedeutung in Zukunft ausweiten wird. Wie in Drexl und Schneider (2015) erwähnt, gibt es in der Logistikplanung einen „recht allgemeinen Trend zur Untersuchung ‚reichhaltigerer', umfassenderer und integrierter Modelle".

Literatur

Ahuja, R. K., Orlin, J. B., Pallottino, S., Scaparra, M. P., & Scutellà, M. G. (2004). A multi-exchange heuristic for the single-source capacitated facility location problem. *Management Science, 50*(6), 749–760.

Akyüz, M. H., Öncan, T., & Altinel, I. K. (2010). The multi-commodity capacitated multi-facility Weber problem: Heuristics and confidence intervals. *IIE Transactions, 42*(11), 825–841.

Al-Khedhairi, A. (2008). Simulated annealing metaheuristic for solving p-median problem. *International Journal of Contemporary Mathematical Sciences, 3*(28), 1357–1365.

Alp, O., Erkut, E., & Drezner, Z. (2003). An efficient genetic algorithm for the p-median problem. *Annals of Operations Research, 122*(1–4), 21–42.

Al-Sultan, K. S., & Al-Fawzan, M. A. (1999). A tabu search approach to the uncapacitated facility location problem. *Annals of Operations Research, 86*, 91–103.

Alumur, S. A., Kara, B. Y., & Melo, M. T. (2015). Location and logistics. In G. Laporte, S. Nickel & F. Saldanha da Gama (Hrsg.), *Location science* (S. 419–441). Springer International Publishing.

Antunes, A., & Peeters, D. (2001). On solving complex multi-period location models using simulated annealing. *European Journal of Operational Research, 130*(1), 190–201.

Aras, N., Özkısacık, K. C., & Altınel, İ. K. (2006). Solving the uncapacitated multi-facility Weber problem by vector quantization and self-organizing maps. *Journal of the Operational Research Society, 57*(1), 82–93.

Arostegui, M. A., Kadipasaoglu, S. N., & Khumawala, B. M. (2006). An empirical comparison of tabu search, simulated annealing, and genetic algorithms for facilities location problems. *International Journal of Production Economics, 103*(2), 742–754.

Awasthi, A., Chauhan, S. S., & Goyal, S. K. (2011). A multi-criteria decision making approach for location planning for urban distribution centers under uncertainty. *Mathematical and Computer Modelling, 53*(1), 98–109.

Berman, O., & Drezner, Z. (2008). A new formulation for the conditional p-median and p-center problems. *Operations Research Letters, 36*(4), 481–483.

Bischoff, M., & Dächert, K. (2009). Allocation search methods for a generalized class of location – allocation problems. *European Journal of Operational Research, 192*(3), 793–807.

Brimberg, J., Hansen, P., Mladenovic, N., & Taillard, E. D. (2000). Improvements and comparison of heuristics for solving the uncapacitated multisource Weber problem. *Operations Research, 48*(3), 444–460.

Brimberg, J., Mladenovic, N., & Salhi, S. (2004). The multi-source Weber problem with constant opening cost. *Journal of the Operational Research Society, 55*, 640–646.

Brimberg, J., Hansen, P., & Mladenović, N. (2006). Decomposition strategies for large-scale continuous location–allocation problems. *IMA Journal of Management Mathematics, 17*(4), 307–316.

Canós, M. J., Ivorra, C., & Liern, V. (2001). The fuzzy p-median problem: A global analysis of the solutions. *European Journal of Operational Research, 130*(2), 430–436.

Chen, C. H., & Ting, C. J. (2008). Combining Lagrangian heuristic and ant colony system to solve the single source capacitated facility location problem. *Transportation Research Part E: Logistics and Transportation Review, 44*(6), 1099–1122.

Chen, C. T. (2001). A fuzzy approach to select the location of the distribution center. *Fuzzy Sets and Systems, 118*(1), 65–73.

Cheng, E. W., Li, H., & Yu, L. (2007). A GIS approach to shopping mall location selection. *Building and Environment, 42*(2), 884–892.

Chiyoshi, F., & Galvao, R. D. (2000). A statistical analysis of simulated annealing applied to the p-median problem. *Annals of Operations Research, 96*(1–4), 61–74.

Chou, S. Y., Chang, Y. H., & Shen, C. Y. (2008a). A fuzzy simple additive weighting system under group decision-making for facility location selection with objective/subjective attributes. *European Journal of Operational Research, 189*(1), 132–145.

Chou, T. Y., Hsu, C. L., & Chen, M. C. (2008b). A fuzzy multi-criteria decision model for international tourist hotels location selection. *International Journal of Hospitality Management, 27*(2), 293–301.

Contreras, I., Fernández, E., & Reinelt, G. (2012). Minimizing the maximum travel time in a combined model of facility location and network design. *Omega, 40*(6), 847–860.

Contreras, I. A., & Díaz, J. A. (2008). Scatter search for the single source capacitated facility location problem. *Annals of Operations Research, 157*(1), 73–89.

Cooper, L. (1964). Heuristic methods for location-allocation problems. *SIAM Review, 6*(1), 37–53.

Cornuéjols, G., Nemhauser, G. L., & Wolsey, L. A. (1983). The uncapacitated facility location problem (No. MSRR-493). *Management Sciences Research Report MSRR 493*. Carnegie-Mellon University.

Correa, E. S., Steiner, M. T. A., Freitas, A. A., & Carnieri, C. (2004). A genetic algorithm for solving a capacitated p-median problem. *Numerical Algorithms, 35*(2–4), 373–388.

Cortinhal, M. J., Lopes, M. J., & Melo, M. T. (2015). Dynamic design and re-design of multi-echelon, multi-product logistics networks with outsourcing opportunities: A computational study. *Computers & Industrial Engineering, 90*, 118–131.

Cura, T. (2010). A parallel local search approach to solving the uncapacitated warehouse location problem. *Computers & Industrial Engineering, 59*(4), 1000–1009.

Davidović, T., Ramljak, D., Šelmić, M., & Teodorović, D. (2011). Bee colony optimization for the p-center problem. *Computers & Operations Research, 38*(10), 1367–1376.

Derbel, H., Jarboui, B., Hanafi, S., & Chabchoub, H. (2010). An iterated local search for solving a location-routing problem. *Electronic Notes in Discrete Mathematics, 36*, 875–882.

Derbel, H., Jarboui, B., Hanafi, S., & Chabchoub, H. (2012). Genetic algorithm with iterated local search for solving a location-routing problem. *Expert Systems with Applications, 39*(3), 2865–2871.

Díaz, J. A., & Fernandez, E. (2006). Hybrid scatter search and path relinking for the capacitated p-median problem. *European Journal of Operational Research, 169*(2), 570–585.

Doerner, K. F., Gutjahr, W. J., & Nolz, P. C. (2009). Multi-criteria location planning for public facilities in tsunami-prone coastal areas. *OR Spectrum, 31*(3), 651–678.

Doong, S. H., Lai, C. C., & Wu, C. H. (2007). Genetic subgradient method for solving location–allocation problems. *Applied Soft Computing, 7*(1), 373–386.

Drexl, M., & Schneider, M. (2015). A survey of variants and extensions of the location-routing problem. *European Journal of Operational Research, 241*(2), 283–308.

Drezner, Z., & Hamacher, H. W. (Hrsg.). (2001). *Facility location: Applications and theory*. Springer Science & Business Media.

Escobar, J. W., Linfati, R., & Toth, P. (2013). A two-phase hybrid heuristic algorithm for the capacitated location-routing problem. *Computers & Operations Research, 40*(1), 70–79.

Farahani, R. Z., SteadieSeifi, M., & Asgari, N. (2010). Multiple criteria facility location problems: A survey. *Applied Mathematical Modelling, 34*(7), 1689–1709.

Farahani, R. Z., Hekmatfar, M., Arabani, A. B., & Nikbakhsh, E. (2013). Hub location problems: A review of models, classification, solution techniques, and applications. *Computers & Industrial Engineering, 64*(4), 1096–1109.

Gamal, M. D. H., & Salhi, S. (2001). Constructive heuristics for the uncapacitated continuous location-allocation problem. *Journal of the Operational Research Society, 52*(7), 821–829.

Ghaderi, A., & Jabalameli, M. S. (2013). Modeling the budget-constrained dynamic uncapacitated facility location–network design problem and solving it via two efficient heuristics: A case study of health care. *Mathematical and Computer Modelling, 57*(3), 382–400.

Ghaderi, A., Jabalameli, M. S., Barzinpour, F., & Rahmaniani, R. (2012). An efficient hybrid particle swarm optimization algorithm for solving the uncapacitated continuous location-allocation problem. *Networks and Spatial Economics, 12*(3), 421–439.

Ghiani, G., Guerriero, F., & Musmanno, R. (2002). The capacitated plant location problem with multiple facilities in the same site. *Computers & Operations Research, 29*(13), 1903–1912.

Ghosh, D. (2003). Neighborhood search heuristics for the uncapacitated facility location problem. *European Journal of Operational Research, 150*(1), 150–162.

Gong, D., Gen, M., Xu, W., & Yamazaki, G. (1995). Hybrid evolutionary method for obstacle location-allocation. *Computers & Industrial Engineering, 29*(1), 525–530.

Guastaroba, G., & Speranza, M. G. (2014). A heuristic for BILP problems: The single source capacitated facility location problem. *European Journal of Operational Research, 238*(2), 438–450.

Guha, S., & Khuller, S. (1999). Greedy strikes back: Improved facility location algorithms. *Journal of Algorithms, 31*(1), 228–248.

Guha, S., Meyerson, A., & Munagala, K. (2003). A constant factor approximation algorithm for the fault-tolerant facility location problem. *Journal of Algorithms, 48*(2), 429–440.

Guner, A. R., & Sevkli, M. (2008). A discrete particle swarm optimization algorithm for uncapacitated facility location problem. *Journal of Artificial Evolution and Applications*. https://doi.org/10.1155/2008/861512

Hanne, T. (2012). *Intelligent strategies for meta multiple criteria decision making* (International series in operations research & management science, Bd. 33). Springer Science & Business Media.

Hansen, P., & Mladenović, N. (1997). Variable neighborhood search for the p-median. *Location Science, 5*(4), 207–226.

Harris, I., Mumford, C., & Naim, M. (2009). The multi-objective uncapacitated facility location problem for green logistics. In *IEEE Congress on Evolutionary Computation, 2009. CEC'09* (S. 2732–2739). IEEE.

Harris, I., Mumford, C. L., & Naim, M. M. (2011). An evolutionary bi-objective approach to the capacitated facility location problem with cost and CO2 emissions. In N. Krasnogor (Hrsg.), *Proceedings of the 13th annual conference on genetic and evolutionary computation* (S. 697–704). ACM.

Hassin, R., Levin, A., & Morad, D. (2003). Lexicographic local search and the p-center problem. *European Journal of Operational Research, 151*(2), 265–279.

Houck, C. R., Joines, J. A., & Kay, M. G. (1996). Comparison of genetic algorithms, random restart and two-opt switching for solving large location-allocation problems. *Computers & Operations Research, 23*(6), 587–596.

Hsieh, K. H., & Tien, F. C. (2004). Self-organizing feature maps for solving location–allocation problems with rectilinear distances. *Computers & Operations Research, 31*(7), 1017–1031.

IVI International, Inc. (2013). *IVI construction case study, 28*(10). http://ivi-intl.com/pdfs/newsletters/2013/IVI_Newsletter_OCT_2013.pdf. Zugegriffen am 12.03.2016.

Jabalameli, M. S., & Ghaderi, A. (2008). Hybrid algorithms for the uncapacitated continuous location-allocation problem. *The International Journal of Advanced Manufacturing Technology, 37*(1–2), 202–209.

Jaramillo, J. H., Bhadury, J., & Batta, R. (2002). On the use of genetic algorithms to solve location problems. *Computers & Operations Research, 29*(6), 761–779.

Jarboui, B., Derbel, H., Hanafi, S., & Mladenović, N. (2013). Variable neighborhood search for location routing. *Computers & Operations Research, 40*(1), 47–57.

Jones, P. C., Lowe, T. J., Muller, G., Xu, N., Ye, Y., & Zydiak, J. L. (1995). Specially structured uncapacitated facility location problems. *Operations Research, 43*(4), 661–669.

Kahraman, C., Ruan, D., & Doğan, I. (2003). Fuzzy group decision-making for facility location selection. *Information Sciences, 157*, 135–153.

Kariv, O., & Hakimi, S. L. (1979). An algorithmic approach to network location problems. II: The p-medians. *SIAM Journal on Applied Mathematics, 37*(3), 539–560.

Kaveh, A., & Nasr, H. (2011). Solving the conditional and unconditional p-center problem with modified harmony search: A real case study. *Scientia Iranica, 18*(4), 867–877.

Klein, C. M., & Kincaid, R. K. (1994). Technical note – The discrete anti-p-center problem. *Transportation Science, 28*(1), 77–79.

Kratica, J., Tošic, D., Filipović, V., & Ljubić, I. (2001). Solving the simple plant location problem by genetic algorithm. *RAIRO-Operations Research, 35*(1), 127–142.

Lim, A., Rodrigues, B., Wang, F., & Xu, Z. (2004). k-Center problems with minimum coverage. In K.-Y. Chwa & J. I. J. Munro (Hrsg.), *Computing and combinatorics, 10th annual international conference, COCOON 2004, Jeju Island, Korea* (S. 349–359). Springer.

Lozano, S., Guerrero, F., Onieva, L., & Larraneta, J. (1998). Kohonen maps for solving a class of location-allocation problems. *European Journal of Operational Research, 108*(1), 106–117.

Luis, M., Salhi, S., & Nagy, G. (2011). A guided reactive GRASP for the capacitated multi-source Weber problem. *Computers & Operations Research, 38*(7), 1014–1024.

Maniezzo, V., Mingozzi, A., & Baldacci, R. (1998). A bionomic approach to the capacitated p-median problem. *Journal of Heuristics, 4*(3), 263–280.

Manzour-al-Ajdad, S. M. H., Torabi, S. A., & Eshghi, K. (2012). Single-source capacitated multi-facility Weber problem – An iterative two phase heuristic algorithm. *Computers & Operations Research, 39*(7), 1465–1476.

Marić, M., Stanimirović, Z., & Milenković, N. (2012). Metaheuristic methods for solving the bilevel uncapacitated facility location problem with clients' preferences. *Electronic Notes in Discrete Mathematics, 39*, 43–50.

Megiddo, N., & Supowit, K. J. (1984). On the complexity of some common geometric location problems. *SIAM Journal on Computing, 13*(1), 182–196.

Mladenović, N., Labbé, M., & Hansen, P. (2003). Solving the p-center problem with tabu search and variable neighborhood search. *Networks, 42*(1), 48–64.

Mladenović, N., Brimberg, J., Hansen, P., & Moreno-Pérez, J. A. (2007). The p-median problem: A survey of metaheuristic approaches. *European Journal of Operational Research, 179*(3), 927–939.

Mohammadi, N., Malek, M. R., & Alesheikh, A. A. (2010). A new GA based solution for capacitated multi source Weber problem. *International Journal of Computational Intelligence Systems, 3*(5), 514–521.

Nagy, G., & Salhi, S. (2007). Location-routing: Issues, models and methods. *European Journal of Operational Research, 177*(2), 649–672.

Nickel, S., & Puerto, J. (2006). *Location theory: A unified approach*. Springer Science & Business Media.

Ohlemüller, M. (1997). Tabu search for large location-allocation problems. *Journal of the Operational Research Society, 48*(7), 745–750.

Öncan, T. (2013). Heuristics for the single source capacitated multi-facility Weber problem. *Computers & Industrial Engineering, 64*(4), 959–971.

Özsoy, F. A., & Pınar, M. Ç. (2006). An exact algorithm for the capacitated vertex p-center problem. *Computers & Operations Research, 33*(5), 1420–1436.

Prodhon, C., & Prins, C. (2014). A survey of recent research on location-routing problems. *European Journal of Operational Research, 238*(1), 1–17.

ReVelle, C. S., & Swain, R. W. (1970). Central facilities location. *Geographical Analysis, 2*(1), 30–42.

Rolland, E., Schilling, D. A., & Current, J. R. (1997). An efficient tabu search procedure for the p-median problem. *European Journal of Operational Research, 96*(2), 329–342.

Salhi, S., & Gamal, M. D. H. (2003). A genetic algorithm based approach for the uncapacitated continuous location–allocation problem. *Annals of Operations Research, 123*(1–4), 203–222.

Sankaran, J. K. (2007). On solving large instances of the capacitated facility location problem. *European Journal of Operational Research, 178*(3), 663–676.

Scaparra, M. P., Pallottino, S., & Scutellà, M. G. (2004). Large-scale local search heuristics for the capacitated vertex p-center problem. *Networks, 43*(4), 241–255.

Sherali, H. D., & Nordai, F. L. (1988). NP-hard, capacitated, balanced p-median problems on a chain graph with a continuum of link demands. *Mathematics of Operations Research, 13*(1), 32–49.

Silva, F. J. F., & De la Figuera, D. S. (2007). A capacitated facility location problem with constrained backlogging probabilities. *International Journal of Production Research, 45*(21), 5117–5134.

Sridharan, R. (1995). The capacitated plant location problem. *European Journal of Operational Research, 87*(2), 203–213.

Sun, M. (2006). Solving the uncapacitated facility location problem using tabu search. *Computers & Operations Research, 33*(9), 2563–2589.

Sun, M. (2012). A tabu search heuristic procedure for the capacitated facility location problem. *Journal of Heuristics, 18*(1), 91–118.

Tcha, D. W., & Lee, B. I. (1984). A branch-and-bound algorithm for the multi-level uncapacitated facility location problem. *European Journal of Operational Research, 18*(1), 35–43.

Ting, C. J., & Chen, C. H. (2013). A multiple ant colony optimization algorithm for the capacitated location routing problem. *International Journal of Production Economics, 141*(1), 34–44.

Tohyama, H., Ida, K., & Matsueda, J. (2011). A genetic algorithm for the uncapacitated facility location problem. *Electronics and Communications in Japan, 94*(5), 47–54.

Tragantalerngsak, S., Holt, J., & Ro, M. (1997). Lagrangian heuristics for the two-echelon, single-source, capacitated facility location problem. *European Journal of Operational Research, 102*(3), 611–625.

Wang, D., Wu, C. H., Ip, A., Wang, D., & Yan, Y. (2008). Parallel multi-population particle swarm optimization algorithm for the uncapacitated facility location problem using openMP. In *IEEE Congress on Evolutionary Computation, 2008. CEC 2008 (IEEE World Congress on Computational Intelligence)* (S. 1214–1218). IEEE.

Wen, M., & Iwamura, K. (2008). Facility location–allocation problem in random fuzzy environment: Using (α, β)-cost minimization model under the Hurewicz criterion. *Computers & Mathematics with Applications, 55*(4), 704–713.

Yu, V. F., Lin, S. W., Lee, W., & Ting, C. J. (2010). A simulated annealing heuristic for the capacitated location routing problem. *Computers & Industrial Engineering, 58*(2), 288–299.

Zhou, J., & Liu, B. (2003). New stochastic models for capacitated location-allocation problem. *Computers & Industrial Engineering, 45*(1), 111–125.

Zhou, J., & Liu, B. (2007). Modeling capacitated location–allocation problem with fuzzy demands. *Computers & Industrial Engineering, 53*(3), 454–468.

Kapitel 7
Intelligente Software für die Logistik



Es gibt es in jüngster Zeit viele akademische Beiträge zur Lösung komplexer logistischer Probleme mit Hilfe fortgeschrittener Methoden der Computational Intelligence und anderer Bereiche. Eine offensichtliche Frage ist, wie viele davon in praktische Softwarelösungen umgesetzt wurden und wie weit verbreitet ihre Nutzung durch Unternehmen ist, die sich mit diesen speziellen Problemen beschäftigen.

Bei der Betrachtung von Softwaretools gibt es verschiedene Möglichkeiten, sie zu gestalten. Eine Möglichkeit besteht darin, zwischen Software zu unterscheiden, die hauptsächlich auf Optimierungsalgorithmen ausgerichtet ist, und Software, die hauptsächlich für Logistikanwendungen konzipiert ist. Optimierungssoftware kann dann unterteilt werden in allgemeine Optimierungssoftware (Abschn. 7.1) und Werkzeuge, die sich auf spezielle Methoden oder bestimmte Optimierungsprobleme konzentrieren (Abschn. 7.2). Logistiksoftware kann unterteilt werden in allgemeine Software, die in der Regel andere Geschäftsprozesse mit Optimierungsfunktionen unterstützt (Abschn. 7.3), und Software, die sich auf die Logistik im Allgemeinen oder auf spezielle Logistikanwendungen konzentriert (Abschn. 7.4).

7.1 Allgemeine Optimierungssoftware

Die Anwendung der mathematischen Optimierung auf Unternehmensprobleme wird normalerweise auf die Entwicklung des Simplex-Algorithmus durch George Dantzig im Jahr 1947 zurückgeführt. Diese Methode ist in der Lage, lineare Optimierungsprobleme mit einer endlichen Anzahl von Schritten bis zur Optimalität zu lösen. Obwohl der Algorithmus eine exponentielle Komplexität aufweisen kann, können typische Optimierungsprobleme mit einer akzeptablen Anzahl von Iterationen (und Laufzeit) gelöst werden. Bereits in den 1950er-Jahren wurde eine auf diesem Verfahren basierende Software zur Verfügung gestellt. In den folgenden Jahrzehnten wurden verschiedene Verbesserungen dieser Methode vorgeschlagen und

© Der/die Autor(en), exklusiv lizenziert an Springer Nature Switzerland AG 2023

T. Hanne, R. Dornberger, *Computational Intelligence in Logistik und Supply Chain Management*, https://doi.org/10.1007/978-3-031-21452-3_7

umgesetzt, so dass heute auch große Probleme (mit bis zu mehreren hunderttausend Variablen und Nebenbedingungen) in der Praxis gelöst werden können. Diese Methode (mit ihren Verbesserungen) ist immer noch konkurrenzfähig und weit verbreitet, obwohl die so genannten Innere-Punkte-Methoden seit den 1980er-Jahren viel Aufmerksamkeit erregt haben, weil sie lineare Optimierungsprobleme in polynomieller Zeit lösen können.

Aufgrund seiner Wettbewerbsfähigkeit ist der Simplex-Algorithmus (mit seinen Verbesserungen) auch heute noch der Kern von Optimierungssoftwarepaketen. Der vermeintliche Marktführer in diesem Bereich ist CPLEX Optimization Studio (oder einfach CPLEX), das ursprünglich in den 1980er-Jahren entwickelt wurde und heute von IBM angeboten wird. Andere bekannte kommerzielle Optimierungspakete sind zum Beispiel XPRESS (Xpress Optimization Suite) von FICO, Gurobi (von Gurobi Optimization) und MOSEK (von MOSEK ApS).

Die lineare Optimierung kann für zahlreiche praxisrelevante Problemtypen, z. B. im Bereich der Produktionsplanung, eingesetzt werden. Bezogen auf logistische Probleme sind die Variablen jedoch oft nicht kontinuierlich, sondern ganzzahlig. Streng genommen sind auch in der Produktionsplanung die Variablen oft ganzzahlig, da Produkte meist in diskreten Mengen (d. h. Stückzahlen) produziert werden und nicht in einer Menge, die durch eine beliebige positive reelle Zahl ausgedrückt wird (wie in der linearen Optimierung angenommen).

Aufgrund der Bedeutung von diskreten oder ganzzahligen Optimierungsproblemen wurden entsprechende Algorithmen auch schon vor mehreren Jahrzehnten vorgeschlagen, z. B. Schnittebenenverfahren seit den 1950er-Jahren, Branch & Bound seit den 1960er-Jahren oder Branch & Cut in den 1980er-Jahren. Eine neuere Entwicklung ist Branch & Price, das seit den 1990er-Jahren verwendet wird.

Mehrere dieser Methoden sind Teil größerer kommerzieller Optimierungs-Toolboxen geworden. CPLEX zum Beispiel hat einen Löser für ganzzahlige und gemischt-ganzzahlige Optimierungsprobleme (MIP) integriert. Die Details dieser Methode sind nicht dokumentiert, aber sie verwendet Techniken des Verzweigens und Schneidens zusammen mit einer „Trickkiste“ (vgl. IBM (2005), Rothberg (o.J.)). Zu diesen „Tricks“ gehören z. B. Strategien zur Vorverarbeitung von Optimierungsproblemen, verschiedene Möglichkeiten zur Verwendung von Schnittebenen und verschiedene Heuristiken, die während der Suche eingesetzt werden (z. B. Knotenheuristiken) (Lima, 2010). Darüber hinaus bietet CPLEX spezielle Unterstützung für quadratische Optimierungsprobleme (kontinuierlich und ganzzahlig/gemischt-ganzzahlig) und einen speziellen Solver für Constraint Programming (CP Optimizer).

Auch andere Pakete bieten Unterstützung für ganzzahlige und gemischt-ganzzahlige Probleme. XPRESS enthält zum Beispiel einen Solver namens Xpress-MP, der auf Branch-and-Bound-Verfahren basiert. Dieser Solver wendet auch verschiedene Arten von automatisch generierten Schnittebenen an und nutzt Heuristiken während der Suche (Laundy et al., 2009). Andere Problemtypen wie die quadratische, nichtlineare und stochastische Programmierung werden ebenfalls von der Xpress-Software unterstützt. Darüber hinaus ist ein spezieller Solver verfügbar, der auf Techniken des Constraint Programming basiert und Xpress-Kalis heißt.

Wichtig ist auch die Software Gurobi, die ganzzahlige oder gemischt-ganzzahlige Probleme ebenfalls mit einem Branch & Bound-Ansatz löst, der sich durch Vorverarbeitungstechniken, die Erzeugung von Schnittebenen, die Verwendung verschiedener Heuristiken und die Unterstützung von Parallelverarbeitung auszeichnet (Gurobi Optimization, 2016).

Abgesehen von diesen Beispielen gibt es noch viele weitere kommerzielle Optimierungspakete. Wikipedia (2016c) listet zum Beispiel mehr als vierzig Pakete auf. In Fourer (2013) werden sogar mehr als fünfzig Pakete allein für Anwendungen der linearen Programmierung aufgeführt.

In einigen Fällen hat das jeweilige Softwarepaket einen weitergehenden Anwendungsbereich als nur die Optimierung. So enthalten allgemeine mathematische oder statistische Softwarepakete oft Optimierungsfunktionen, obwohl dies nicht ihr Hauptschwerpunkt ist. Bekannte Beispiele sind Mathematica (www.wolfram.com), MATLAB (www.mathworks.com) oder Maple (www.maplesoft.com). MATLAB beispielsweise enthält Optimierungsansätze in einer entsprechenden Toolbox, die eine beträchtliche Anzahl von Optimierungsroutinen bietet und eine Vielzahl unterschiedlicher Problemtypen unterstützt.

Wir können feststellen, dass moderne Universal-Optimierungs-Toolkits grundsätzlich geeignet sind, Optimierungsprobleme zu lösen, wie sie typischerweise im Bereich der Logistik auftreten. IBM (o.J.) nennt als mögliche Anwendungsbereiche des CPLEX-Optimierers für gemischt-ganzzahlige Optimierungen beispielsweise Routenplanung, Standortbestimmung, Scheduling oder Netzwerk-Design. Bei der Erörterung der Eignung dieser Art von Software für intelligente Logistikanwendungen sollten jedoch drei Punkte hervorgehoben werden: Wie kann ein geeignetes Modell für die Optimierungssoftware erstellt werden, wie kann sie in Logistikanwendungen integriert werden und wie kann die Methode an ein bestimmtes Problem angepasst werden?

7.1.1 Erstellen eines geeigneten Modells für die Optimierungssoftware

Die Erstellung eines Modells ist nicht nur für einen Anfänger in der Optimierung eine schwierige Aufgabe. Da ein Optimierungs-Toolkit in einer Programmiersprache formuliert ist, könnte ein entsprechendes Problem theoretisch direkt in den geeigneten Datenstrukturen der Software (z. B. Arrays von Variablen) definiert werden. Dies wäre eine anspruchsvolle Aufgabe, die sowohl ein gutes Verständnis der Programmierumgebung als auch ein gutes Verständnis der mathematischen Natur des Optimierungsproblems erfordert. Ein geeigneterer Ansatz wäre es, Probleme prägnant wie mit den entsprechenden mathematischen Notationen zu formulieren (was dann nur ein mathematisches Verständnis erfordern würde). Da sich mathematische Formulierungen nur schwer mit einer Tastatur schreiben lassen (und aus einigen anderen Gründen), wurde ein praktischer Ansatz gefunden, indem eine

softwareunabhängige Modellierungssprache verwendet wurde, mit der ein Problem ähnlich wie eine mathematische Spezifikation formuliert werden kann.

Ein Beispiel für eine solche Modellierungssprache ist AMPL („A Mathematical Programming Language“, vgl. Fourer et al. (2002)), die auf einer Mischung aus deklarativem und imperativem Programmierstil basiert. Die Problemformulierung erfolgt hauptsächlich durch deklarative Sprachelemente, z. B. Mengen, Entscheidungsvariablen, Parameter, Zielfunktionen und Nebenbedingungen. Prozedurale Anweisungen können z. B. für den Datenaustausch oder die Vor- und Nachbearbeitung von Daten nützlich sein.

Andere gängige Beispiele für Modellierungssprachen sind GAMS (General Algebraic Modeling System, www.gams.com), AIMMS (Advanced Interactive Multidimensional Modeling System, business.aimms.com) oder OptimJ (Java-based Modeling Language for Optimization, www.ateji.com/optimj).

Die Verwendung einer solchen Modellierungssprache macht die Problemformulierung unabhängig von einem möglichen Solver. Viele Softwarepakete, z. B. CPLEX und Xpress, unterstützen die Problemformulierung in einer solchen Sprache. Darüber hinaus verfügen einige Pakete über eine eigene Modellierungsumgebung. CPLEX zum Beispiel ist mit einer Modellierungsschicht namens Concert und einer Modellierungssprache namens OPL (Optimization Programming Language) ausgestattet.

7.1.2 Integration von Optimierungssoftware mit Logistikanwendungen

Ein zweites Problem ist die Integration der Optimierungssoftware in Logistikanwendungen. Es ist zwar möglich, die Optimierungssoftware über integrierte Benutzeroberflächen (d. h. zur manuellen Eingabe von Problemdaten) zu nutzen, aber es liegt auf der Hand, dass dies keine geeignete Option für den regelmäßigen Einsatz eines solchen Tools in der Praxis ist. Zum einen wäre dies viel zu zeitaufwändig. Andererseits führt ein einziger Fehler in den eingegebenen Daten zu falschen oder unzulässigen Lösungen oder aufgrund von Inkonsistenzen zu gar keiner Lösung.

In der Regel kann man davon ausgehen, dass es bereits eine Art von Software für die Verwaltung von Logistikaktivitäten gibt, die mit der Optimierungssoftware verbunden sein sollte, um die benötigten Daten zu liefern und eine ermittelte optimale Lösung zurückzuerhalten. Ein Transportdienstleister betreibt zum Beispiel ein Transportmanagementsystem, das die auszuführenden Transportaufträge erfasst.

Für die Verbindung mit anderen Anwendungen bieten kommerzielle Toolkits in der Regel geeignete Programmierschnittstellen. CPLEX bietet zum Beispiel Programmierschnittstellen zu C, C++, C#, Java und Python. Darüber hinaus gibt es Konnektoren zu Geschäftsanwendungen oder mathematischen Softwarepaketen wie Excel oder MATLAB. Xpress und seine Modellierungsschicht Mosel bieten Programmierschnittstellen zu C, C++, Java, .NET und Visual Basic. Gurobi umfasst

Schnittstellen zu C, C++, C#, Java, Visual Basic und Python. Darüber hinaus sind Konnektoren zu den mathematischen Softwarepaketen MATLAB und R verfügbar.

Die grundlegenden Programmierschnittstellen sind also in der Regel vorhanden, aber wenn eine entsprechende Schnittstelle in einer Logistiksoftware noch nicht vorhanden ist, bedeutet dies natürlich einen erheblichen zusätzlichen Entwicklungsaufwand für einen Anwender. Zudem muss bei der Verwendung von Standard-Logistiksoftware auch diese für die Anbindung an ein entsprechendes Optimierungstool offen sein.

7.1.3 Anpassung der Methode an das zu betrachtende Problem

Die problemspezifische Anpassungsfähigkeit ist eines der größten Probleme bei der Verwendung von allgemeiner Optimierungssoftware für eine verbesserte Logistikplanung. Selbst wenn es möglich ist, ein spezifisches Logistikproblem im Optimierungs-Toolkit zu modellieren und die erforderlichen Daten in das Toolkit zu übertragen, können die Algorithmen die Besonderheiten des Problems kaum voll ausschöpfen.

Die üblichen (mathematischen oder sonstigen) Formulierungen von linearen, ganzzahligen oder gemischt-ganzzahligen Optimierungsproblemen sind eher allgemein gehalten. Spezifische Eigenschaften eines Optimierungsproblems sind daher schwer zu erkennen und von einem Allzweckoptimierer auszunutzen. Für einige der in den vorangegangenen Kapiteln besprochenen logistischen Standardoptimierungsprobleme wurden Anstrengungen unternommen, um sie auf standardisierte Weise als ganzzahlige oder gemischt-ganzzahlige Optimierungsprobleme zu modellieren, die mit Standardsoftware lösbar sind. Darüber hinaus wurden in einigen Fällen zusätzliche redundante Randbedingungen hinzugefügt, um die Effizienz solcher Software zu erhöhen.

Viele Fortschritte bei den Optimierern in den letzten Jahrzehnten sind auf Bemühungen zurückzuführen, spezifische Eigenschaften eines aktuellen Problems zu finden und zu nutzen, z. B. in der Vorverarbeitungsphase oder während der Suche durch den Einsatz von Heuristiken. Für viele Problemtypen können solche Eigenschaften aufgezeigt werden, wenn sie im Voraus von Experten analysiert werden, und ein angepasster Algorithmus könnte dann effizienter mit ihnen umgehen.

Dies ist eine der Ideen, die bei der Entwicklung von Metaheuristiken in der Regel im Vordergrund stehen. Auch Metaheuristiken können generisch formuliert werden, d. h. ohne Berücksichtigung der Besonderheiten eines Problems, aber die Erfahrung zeigt, dass oft dramatisch bessere Ergebnisse erzielt werden, wenn spezifische Datenstrukturen (Repräsentation von Problemen und Lösungen in der Software, modifizierte Algorithmen usw.) verwendet werden.

Kommerziellen Allzwecklösern ist diese Möglichkeit aus zwei Gründen nicht gegeben: Erstens ist der Quellcode in der Regel nicht zugänglich und kann nicht verändert werden. Zweitens, selbst wenn ein modifizierbarer Quellcode verfügbar wäre, wäre es aufgrund der vorgegebenen Basisdatenstrukturen und des algorithmi-

schen Konzepts schwierig, diesen in geeigneter Weise zu modifizieren. In aktuellen Implementierungen von Universal-Solvern kann ein Benutzer einen Optimierungslauf (und damit die Leistung für das aktuell betrachtete Problem) meist nur durch die Veränderung einiger Parameter beeinflussen, die den Optimierungslauf steuern.

7.2 Software mit spezifischen Optimierungsalgorithmen oder für spezifische Optimierungsprobleme

Wie wir in Abschn. 7.1 gesehen haben, spielen spezifische Algorithmen der Computational Intelligence in bekannten kommerziellen Optimierungspaketen noch keine bedeutende Rolle. Ein Grund dafür ist die historische Entwicklung von Optimierungsmethoden und entsprechender Software. Kommerzielle Pakete werden immer noch von Algorithmen dominiert, die seit mehr als 30 Jahren im Einsatz sind. Methoden der Computational Intelligence sind meist deutlich neuer, zumindest was deren allgemeine Wahrnehmung angeht. Ein weiterer Grund ist die Konzentration auf Allzwecklöser, die die Besonderheiten bestimmter Problemtypen nicht ausnutzen.

Dennoch sind Methoden aus dem Bereich der Computational Intelligence, insbesondere Metaheuristiken, zumindest in akademischen Implementierungen weit verbreitet. Wenn eine neue Methode veröffentlicht wird, wird sie in der Regel zumindest durch einen Pseudocode beschrieben. Darüber hinaus ist für viele Methoden ein aktueller Code verfügbar, der von Websites frei heruntergeladen werden kann.

In vielen Fällen bietet der Code nur eine Implementierung der jeweiligen Optimierungsmethode, während alles andere, was nützlich sein könnte, fehlt, z. B. Routinen für die Datenein- und -ausgabe, für die Vorverarbeitung, Visualisierung oder für die Analyse der Ergebnisse. Die verwendeten Datenstrukturen und Speicherformate (falls vorhanden) sind oft proprietär und es fehlt eine Unterstützung für die Modellierung. Insbesondere kann der Code nicht direkt mit bestehenden Modellierungswerkzeugen verknüpft werden und eine Schnittstelle zu einer Geschäftsanwendung ist in der Regel nicht vorgesehen.

Eine bessere Alternative können Softwarepakete sein, die mehrere der entsprechenden Algorithmen enthalten. Sie bieten in der Regel eine bessere Unterstützung bei Aufgaben, die über die reine Optimierung hinausgehen, z. B. bei der Datenein- und -ausgabe, bei der Visualisierung von Ergebnissen oder bei der Durchführung von Rechenexperimenten. Ein weiterer Vorteil ist, dass oft mehrere Algorithmen auf die gleiche Art von Problem angewendet werden können und man so mehr Sicherheit hat, eine Methode zu finden, die für das betrachtete Problem gut geeignet ist.

Ein Beispiel aus dieser Klasse von Softwarepaketen ist ParadisEO (paradiseo.gforge.inria.fr), ein objektorientiertes Framework für den Entwurf und die Implementierung von Metaheuristiken. Das Tool umfasst mehrere Einzellösungs- und populationsbasierte Metaheuristiken, Ansätze zur Mehrzieloptimierung und

unterstützt parallele und verteilte Algorithmen. Es basiert auf der ANSI-C++-kompatiblen Bibliothek EO für evolutionäre Berechnungen und ermöglicht eine relativ einfache Erweiterung um neue Methoden. Das Paket enthält Tutorials und zugehörige Dokumente.

Ein weiteres Beispiel ist Hotframe (Heuristic OpTimization FRAMEwork, vgl. Fink und Voß (2003)). Diese Software ist ein C++-basierter Prototyp. Er besteht aus anpassbaren Komponenten einschließlich verschiedener Metaheuristiken und Beschreibungen der Zusammenarbeit zwischen diesen Komponenten und mit Anwendungen. Objektorientierte Klassen werden für die Spezifikation von Optimierungsproblemen, Methoden, Lösungen, Nachbarn und Attribute für Lösungen und deren Veränderung verwendet. Zu den implementierten Methoden gehören z. B. einfache und iterierte lokale Suche, Simulated Annealing und ähnliche Verfahren, verschiedene Varianten der Tabu-Suche, evolutionäre Verfahren, Nachbarschaftssuche mit variabler Tiefe, Kandidatenlistenansätze und einige hybride Verfahren.

Ein drittes Beispiel ist die Java-basierte Toolbox OpenOpal (= OPtimization And Learning) (www.openopal.org), die insbesondere verschiedene Metaheuristiken (z. B. evolutionäre Algorithmen) und einige Methoden zur nichtlinearen Optimierung enthält. Darüber hinaus bietet es Unterstützung für maschinelles Lernen, Datenanalyse und Visualisierung und ist mit einer grafischen Benutzeroberfläche zur Verknüpfung von Optimierungsproblemen und geeigneten Methoden ausgestattet. Es sei darauf hingewiesen, dass dieses Tool unter den Namen OpenDINO (2016) und OpenCI, das als GitLab-Projekt gehostet wird, weiterentwickelt wird.

Alle diese Werkzeuge sind als freie oder Open-Source-Software öffentlich verfügbar. Der Quellcode ist zugänglich und kann für eine bessere Anpassung an ein bestimmtes Problem geändert werden. So gibt es zum Beispiel in OpenOpal und OpenCI allgemeine Versionen für evolutionäre Algorithmen, aber auch Varianten, die an bestimmte Problemtypen angepasst sind.

Neben diesen akademischen Implementierungen sind Metaheuristiken und CI-Methoden gelegentlich auch ein wichtiger Bestandteil kommerzieller Optimierungssoftware. Ein Beispiel ist das Tool OptQuest (http://www.opttek.com/OptQuest) der Firma OptTek Systems, Inc. Der Ansatz kann als „Black Box"-Optimierer beschrieben werden, der auf Methoden wie Tabu-Suche, Scatter Search, ganzzahlige Programmierung oder neuronale Netze basiert. Er bietet verschiedene Schnittstellen zu anderen Werkzeugen, insbesondere zu Simulationssoftware wie z. B. Arena oder Taylor ED. So kann die Software zur Optimierung von Simulationsmodellen verwendet werden, die auf einer anderen Anwendung laufen. Auch Geschäftsanwendungen wie Excel können mit diesem Tool integriert werden. OptQuest verfügt über eine in C geschriebene, aufrufbare Bibliothek, auf die entweder als statische Bibliothek oder als Dynamic Link Library (DLL) oder mit einer COM-Schnittstelle für die .NET-Integration zugegriffen werden kann.

Neben reinen Optimierungssoftwarepaketen gibt es auch nicht-kommerzielle Tools mit einem breiteren Fokus, insbesondere auf allgemeine mathematische Anwendungen – ähnlich wie bei den oben genannten allgemeineren kommerziellen Softwarepaketen.

Ein Beispiel ist Scilab (www.scilab.org), ein Open-Source-Softwarepaket für numerische Berechnungen, das MATLAB ähnelt (sogar hinsichtlich der Syntax). Seine Hauptfunktionen und Anwendungsbereiche sind Rechenfunktionen, lineare Algebra und Analysis, Visualisierung (Grafiken, Animationen), zeitdiskrete und kontinuierliche Simulation und Optimierung, unterstützt durch mehrere Toolboxen. Scilab enthält einen Interpreter und eine höhere Programmiersprache mit Schnittstellenmöglichkeiten (z. B. zu C, C++, Java, etc.). Weitere MATLAB-ähnliche, aber kostenlose Tools sind Octave (http://www.gnu.org/software/octave/) und FreeMat (http://freemat.sourceforge.net/). Sie bieten ebenfalls Optimierungsmöglichkeiten, allerdings nicht so umfangreich wie MATLAB.

Ein weiteres Beispiel ist die R-Software, die vom „R Project for Statistical Computing“ bereitgestellt wird (The R Foundation, 2016). Obwohl der Schwerpunkt auf statistischen Berechnungen und Grafiken liegt, bietet es spezielle Pakete für die Optimierungsinfrastruktur, kontinuierliche Optimierung und mathematische Programmierung. Außerdem gibt es Schnittstellen zu anderen freien Optimierungswerkzeugen (z. B. Scilab) und kommerzieller Optimierungssoftware.

Im Vergleich zu kommerziellen Optimierungspaketen liegen mehrere Vor- und Nachteile auf der Hand: Bei den freien Softwarepaketen ist der Quellcode in der Regel verfügbar und damit eine spezifischere Anpassung an bestimmte schwer lösbare Optimierungsprobleme, wie sie im Bereich der Logistik auftreten, möglich. Außerdem werden in dieser Softwaregruppe häufiger Methoden der Computational Intelligence und der Metaheuristiken eingesetzt, die im Vergleich zu den typischen Algorithmen kommerzieller Pakete ihre spezifischen Vorteile (und natürlich auch einige Nachteile) haben. Wie bereits erwähnt, sind diese Methoden in der Regel für problemspezifische Anpassungen besser geeignet als klassische Optimierungsalgorithmen.

Ein offensichtlicher Nachteil dieser Art von Software ist, dass die Software weniger ausgereift ist als bei etablierten kommerziellen Paketen. Da der Code meist aus akademischen Projekten oder ehrenamtlicher Arbeit resultiert, werden diese Tools oft auf weniger professionelle Weise oder einfach mit zu geringem Entwicklungsaufwand implementiert. Ein weiterer Aspekt, der mit solchen Projekten zusammenhängt, ist, dass häufig keine professionelle Unterstützung der Nutzer oder Wartung verfügbar sind. Dennoch haben viele Open-Source-Entwicklungsprojekte bereits einen ausreichend hohen Reifegrad erreicht, und für einige von ihnen kann kommerzielle Unterstützung für ihre Nutzung in Anwendungen verfügbar sein.

Der letzte Aspekt, der diskutiert werden muss, ist die Kostenfrage. Kommerzielle Optimierungssoftware kann recht teuer sein. Lizenzkosten und damit zusammenhängende rechtliche Fragen können für ein Softwareentwicklungsunternehmen, das eine Optimierungskomponente in seine Produkte integrieren möchte, besonders wichtig sein. Bei Open-Source-Software spielt der Kostenaspekt zwar keine Rolle, aber die Lizenzbedingungen können die Nutzbarkeit für bestimmte Anwendungen beeinträchtigen. Häufig enthalten Open-Source-Lizenzsysteme wie die GNU General Public License (Wikipedia, 2016a) eine „Copyleft“-Bedingung (Wikipedia, 2016b), die die Nutzung, Änderung und Weiterverbreitung des Codes erlaubt,

jedoch mit der Einschränkung, dass der geänderte Code in seiner Nutzbarkeit nicht eingeschränkt werden darf, sondern die gleichen Lizenzbedingungen wie die ursprüngliche Software einhalten muss. Das kann es schwierig (oder sogar unmöglich) machen, einen entsprechenden Code zu verwenden, der in eine bestimmte Anwendung wie eine proprietäre und kommerzielle Logistiksoftware integriert ist.

7.3 Allgemeine Business-Software

Für ein Unternehmen, das seine Logistikaktivitäten mit fortschrittlichen Methoden planen will, wäre es natürlich ideal, wenn die Logistikoptimierung in die Unternehmenssoftware integriert wäre, die es täglich nutzt. Eine allgemeine Unternehmenssoftware, die eine Vielzahl von Prozessen innerhalb eines Unternehmens unterstützt, wird heute als Enterprise Resource Planning (ERP)-Software bezeichnet. Bei ERP-Software handelt es sich häufig um eine integrierte Suite, die aus mehreren Modulen zur Abdeckung verschiedener Geschäftsprozesse wie Beschaffung, Produktion, Vertrieb und Auftragsabwicklung besteht.

Eine ERP-Software ist ein transaktionsorientiertes System, das Objekte wie liquide Mittel, Materialien und Ressourcenkapazitäten verwaltet und den Status dieser Objekte im Auge behält.

Die Planungslogik einer solchen Software ist in der Regel begrenzt. Ein typisches Beispiel für eine Planungslogik, die in vielen Standard-ERP-Systemen vorhanden ist, ist die Materialbedarfsplanung (Material Requirement Planning, MRP). Der Zweck des MRP ist es, die Materialien (Teile, Artikel) in Bezug auf Menge und Zeit zur Deckung des Bedarfs zu bestimmen. Steht der Bedarf fest (z. B. durch eine Prognose), lassen sich die benötigten Mengen leicht berechnen.

Ausgehend vom Primärbedarfs der Produkte pro Periode verwendet die Methode die Stückliste (BOM), um die benötigten Mengen an Vorprodukten zu berechnen. Andere Informationen wie Losgrößen für produzierte Mengen oder Ausschussquoten können ebenfalls berücksichtigt werden. Auch zeitliche Aspekte wie Produktionszeiten oder Lieferzeiten werden berücksichtigt.

Eines der Hauptprobleme dieses Ansatzes ist, dass die Konsistenz der Daten zwar wichtig, aber oft nicht vollständig gegeben ist. Zum Beispiel sind die Stücklisten unvollständig, falsch oder fehlen. Dies führt natürlich zu falsch berechneten Mengen im Hinblick auf den Sekundärbedarf. Ein Hauptproblem des traditionellen MRP-Ansatzes ist jedoch, dass die Kapazitäten der Ressourcen nicht berücksichtigt werden. Insbesondere wird für die geplante Produktion nicht geprüft, ob die vorhandenen Ressourcenkapazitäten ausreichen. Außerdem werden Alternativen bei der Planung nicht berücksichtigt. Zum Beispiel kann die Produktion oft von verschiedenen Ressourcen durchgeführt werden. Auch könnten Produktion, Lagerhaltung etc. möglicherweise an verschiedenen Orten stattfinden. Schließlich werden Planungsunsicherheiten, z. B. hinsichtlich des Primärbedarfs, in diesem Planungsansatz nicht berücksichtigt.

Manufacturing Resource Planning (MRP II) ist ein Nachfolger von MRP, der die Ressourcenkapazitäten auf einfache Weise berücksichtigt. Im Grunde funktioniert es wie MRP, aber zusätzlich werden die Pläne entsprechend den verfügbaren Kapazitäten der Ressourcen angepasst. Die Planung erfolgt weiterhin sukzessive, d. h. wenn eine erste MRP-Lösung zu Ergebnissen führt, die die Kapazitäten verletzen, werden diese in beliebiger Reihenfolge nachgebessert. Dadurch werden die Ressourcen in der Regel nicht optimal genutzt und es können keine optimalen Lösungen erzielt werden.

Die ersten kommerziellen Ansätze, die versuchten, den Planungsschwerpunkt von MRP und MRP II zu erweitern und fortschrittlichere Planungsansätze zu verwenden, erschienen in den 1990er-Jahren. Diese Software wurde mit dem Akronym APS bezeichnet, das für Advanced Planning System oder Advanced Planning and Scheduling steht. Ein weiterer Grund für die Einführung der APS-Software war, dass die traditionelle Software aus dem ERP-Bereich als zu langsam und nicht benutzerfreundlich genug angesehen wurde. Eine neue, interaktivere und intelligentere Planung wurde als notwendig erachtet, auch in Anbetracht der gestiegenen Leistungsfähigkeit von normalen Computern. Außerdem sollte es möglich werden, größere Planungsprobleme zu lösen, und die Planung sollte nicht nur für einen einzelnen Standort, sondern standortübergreifend und unter Einbeziehung der damit verbundenen logistischen Aktivitäten in integrierter Form erfolgen. Die Planungsperspektive sollte sich also von einer standortbezogenen Planung hin zum Supply Chain Management verschieben.

In der Regel ist die APS-Software in verschiedene Module unterteilt, die sich an der Planungsmatrix für die Supply Chain orientieren (siehe Abb. 7.1). Beispiele für solche Module sind Netzwerkdesign, Bedarfsprognose, Distributions- und Transportplanung, Masterplanung, -Produktionsplanung und Scheduling oder die Planung der Beschaffung.

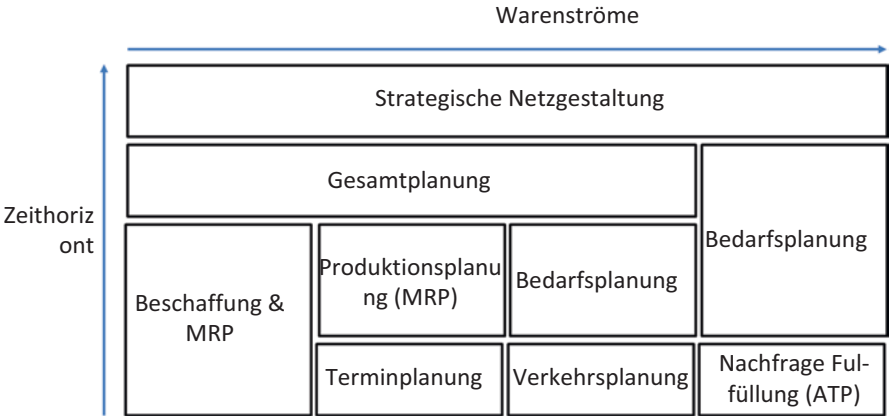


Abb. 7.1 Die Planungsmatrix der Lieferkette. SCM-bezogene Planungsaktivitäten sind durch den Planungshorizont und die betrachteten Prozesse im gesamten Warenfluss gekennzeichnet

Die Supply-Chain-Planungsmatrix visualisiert verschiedene Planungsfelder entsprechend den unterschiedlichen Managementaktivitäten von der Beschaffung bis zur Distribution und entsprechend dem Zeithorizont von der operativen bis zur strategischen Planung. Da die Logistik Querschnittstätigkeiten auf unterschiedlichen Zeitskalen umfasst, finden wir in der Supply-Chain-Planungsmatrix an verschiedenen Stellen verwandte Planungsaufgaben. Konkrete, standortübergreifende Transportprobleme treten z. B. bei der Beschaffung und Distribution auf, aber möglicherweise auch bei internen Prozessen, wenn ein Unternehmen aus mehreren Standorten besteht. Zu den internen Prozessen gehören auch innerbetriebliche Transporte. Die Lagerhaltung kann die Beschaffung (Lagerung beschaffter Materialien), die Handhabung von Zwischenprodukten und die Bevorratung von Fertigprodukten umfassen.

Daher könnte ein APS, das alle damit verbundenen Logistikprozesse unterstützt, eine ideale Lösung für ein Unternehmen sein. Leider ist in den meisten Fällen nicht hinreichend klar, ob die jeweilige APS-Software die Planung tatsächlich mit fortschrittlichen Planungsmethoden unterstützt. Wenn es eine solche Unterstützung gibt, bleibt meist unklar, wie effektiv sie ist.

Der Markt für kommerzielle APS-Lösungen ist recht heterogen und umfasst viele Produkte kleinerer Unternehmen, aber auch einige Produkte von Software-Giganten wie SAP oder Oracle. Meistens beschreiben die Software-Broschüren, White Papers oder andere Informationen auf den Websites der Unternehmen die Funktionen ihrer APS-Software auf ziemlich enthusiastische Weise. Die zugrundeliegenden Techniken bleiben jedoch meist ein Geheimnis (vgl. Bermudez, 1998; Knolmayer, 2001). Dafür gibt es vor allem zwei Erklärungen: Entweder werden die zugrunde liegenden Technologien geheim gehalten, um einen Wettbewerbsvorteil zu erhalten, oder es sind nicht wirklich viele fortschrittliche Planungsmethoden integriert.

Eine durchgeführte fragebogengestützte Studie (Akabuilo et al., 2011) versuchte, mehr Einblick in dieses Thema zu geben. Sie zeigte, dass Softwareanbieter auf dem APS-Markt meist eine Art von Optimierungstechnik oder zumindest einige Heuristiken beinhalten.

Die Art der Methodik bleibt in den meisten Fällen unklar, aber – falls überhaupt vorhanden – scheinen die traditionellen Methoden der linearen oder ganzzahligen Programmierung zu dominieren. Methoden der Computational Intelligence scheinen eine untergeordnete Rolle zu spielen. So wurden nach Angaben der an der Studie beteiligten Softwareanbieter in 11 % der APS-Lösungen genetische Algorithmen eingesetzt. CI-Methoden wie neuronale Netze oder Fuzzy-Ansätze wurden überhaupt nicht eingesetzt. Diese grundsätzlichen Erkenntnisse werden gestützt, wenn man die Unterstützung einzelner APS-Module durch Advanced Planning-Methoden betrachtet.

Eine weitere interessante Erkenntnis aus dieser Studie war, dass oft nicht alle der üblicherweise erwarteten Module in APS-Software enthalten sind. Insbesondere die stark logistikorientierten Module, z. B. für die Distributions- und Transportplanung, werden nur von einem Drittel der Softwareunternehmen angeboten.

7.4 Logistik-Software

Der Markt für Logistiksoftware ist äußerst heterogen. In der Regel gibt es keine einzige Software, die alle für ein Unternehmen relevanten Logistikaktivitäten unterstützt. Häufig erfordern unterschiedliche logistische Aktivitäten unterschiedliche Arten von Software. Darüber hinaus gibt es Softwarelösungen, die spezifisch für bestimmte Industrie- oder Dienstleistungsbereiche sind. Insbesondere einige Arten von Logistikdienstleistern benötigen ihre eigene Art von Softwarelösungen.

7.4.1 Lagerverwaltungssysteme

Beginnen wir mit allgemeinen Softwarelösungen. Das Herzstück der innerbetrieblichen Logistik ist in der Regel ein Lagerverwaltungssystem (LVS) oder ein Warehouse Execution System (WES). Der Hauptzweck dieser Art von Software ist die Verwaltung von Beständen und Lagerplätzen. Darüber hinaus sollen Einlagerungen und Auslagerungen unterstützt werden. Die jeweiligen (innerbetrieblichen) Transportprozesse sind zu planen und durchzuführen. In vielen Fällen ist das LVS mit untergeordneten Systemen zur Steuerung des Materialflusses und der zugehörigen Technik (wie z. B. Materialflussrechner oder speicherprogrammierbare Steuerungen) verbunden.

Weitere Beispiele für unterstützte Aktivitäten in der innerbetrieblichen Logistik sind der Wareneingang, die Kommissionierung, der Versand von Sendungen und die Inventur. Darüber hinaus unterstützt ein LVS die Protokollierung von Aktivitäten, kann Statistiken und Berichte erstellen und unterstützt die Kommunikation mit anderen Komponenten wie mobilen Geräten (z. B. Handscannern), Fahrzeugen (z. B. Gabelstaplern) oder mit Pick-by-Light-Systemen.

Typische logistische Planungsprobleme wie die Produktionsplanung, Materialbedarfsplanung, wirtschaftliche Bestellmengen oder externe Transportprobleme werden von der LVS/WES-Software in der Regel nicht behandelt. Es wird meist davon ausgegangen, dass solche Planungsdaten anderweitig, insbesondere auf der Ebene eines ERP-Systems, ermittelt werden. Für ein LVS/WES kann das ERP als Mastersystem betrachtet werden. Es wird davon ausgegangen, dass operative Daten wie z. B. Transportaufträge aus der ERP-Ebene kommen, so dass das LVS/WES lediglich die entsprechenden Aufgaben detaillierter abbildet und ausführt. Auch bei Stammdaten wie z. B. Artikeldaten wird angenommen, dass sie im ERP gepflegt werden.

Dennoch bieten die konkreten Aktivitäten, die vom LVS/WES ausgeführt werden sollen, ein feinschichtiges Entscheidungs- und Planungspotenzial. Wenn zum Beispiel Aufträge erfüllt werden sollen, ist das benötigte Material oft an verschiedenen Stellen im Lager verfügbar, z. B. in verschiedenen Regalgängen. Manchmal können die Aufträge durch verschiedene geeignete Ressourcen wie menschliche Kommissionierer oder automatische Regalbediengeräte (RBG) erfüllt werden. Außerdem gibt es verschiedene mögliche Transportwege (z. B. verschiedene Förderstrecken).

Alle diese Alternativen bieten ein erhebliches Optimierungspotenzial, das von einem LVS/WES oder in untergeordneten Systemen, z. B. Materialflusssteuerungen, genutzt werden kann. Beispielsweise erhält das Leitsystem eines Regalbediengerätes Aufträge, die ausgeführt werden müssen (Ein- und Auslagerungsaufträge). Deren Ausführungsreihenfolge kann variiert werden, um die Ausführungszeit zu reduzieren und die Leistung zu erhöhen.

Andere Entscheidungen sind auf der übergeordneten Ebene des LVS/WES zu treffen, z. B. die Zuweisung von freien Lagerplätzen zu neuen Behältern oder die Auswahl von Lagerplätzen für benötigte Artikel. Ein LVS/WES kann auch neue optionale Aktivitäten wie die Umlagerung von Behältern auslösen. Solche Umlagerungen können die voraussichtliche Leistung eines Lagers erhöhen, wenn z. B. Artikel, von denen angenommen wird, dass sie in Zukunft häufiger nachgefragt werden, an besseren Plätzen untergebracht werden (d. h. an Plätzen, die kürzere Transportwege verursachen).

Bei der Betrachtung spezifischer LVS/WES-Produkte können wir feststellen, dass die Optimierung von Lagertätigkeiten zwar gelegentlich erwähnt wird, aber Details der verwendeten Algorithmen (oder sogar detailliertere Beschreibungen der Optimierungsprobleme) im Allgemeinen fehlen.

Sehen wir uns einige Beispiele an: LMxt ist ein von a-SIS bereitgestelltes Lagerverwaltungssystem, das Optimierungsfunktionen für verschiedene Zwecke enthält, z. B. für die Auswahl von Kommissionierplätzen, die Volumenberechnung (Auswahl der am besten geeigneten Schachteln) oder die Ressourcenzuweisung (Asis, o.J.-a). Ein weiteres Produkt desselben Unternehmens ist Logys, mit dem sich Bestände und automatisierte Anlagen optimieren lassen (Asis, o.J.-b). Schließlich bietet das Unternehmen ein Tool namens LM Transport Order Optimizer für die Planung und Optimierung von standortübergreifenden Transporten durch Bestimmung eines Transportunternehmens, Kostenkalkulation und Simulation. Spezifische Algorithmen werden für keines dieser Produkte genannt.

Für ein anderes Produkt, die vTradEx eLOG Enterprise Suite WMS (vTradEx, o.J.) wird erwähnt, dass „der Kommissionierweg innerhalb des Lagers“ optimiert wird. Ein drittes Beispiel ist die Software Acteos. Sie soll die Ressourcen Ladung, Behälter, vorbereitende Tätigkeiten und logistische Einheiten für den Versand optimieren (Acteos, o.J.) und eine Bestandsoptimierungsstrategie sowie Echtzeit-Optimierungsmethoden enthalten. Es fehlen jedoch jegliche Details, insbesondere in Bezug auf die zugrunde liegenden Algorithmen.

Als letztes Beispiel sei hier die LVS-Software von Manhattan Associates (o.J.). In diesem Fall wird lediglich angemerkt, dass die Software „fortschrittliche Algorithmen verwendet, um Lagerabläufe mathematisch zu organisieren und zu optimieren“.

Im Vergleich zu APS-Produkten (siehe oben) ist die Abdeckung fortgeschrittener Optimierungstechniken durch LVS und verwandte Software sogar noch vager. Abgesehen von den üblichen Verdächtigen, Geschäftsgeheimnissen und dem Fehlen echter Optimierungstechniken, könnten auch andere Gründe eine Rolle spielen. Erstens sind die Optimierungsprobleme vielleicht nicht so offensichtlich oder klar definiert wie z. B. bei typischen Transportproblemen. Oder der Optimierungsnutzen

scheint geringer zu sein, da die betrachteten Prozesse oft recht klein sind. Schließlich handelt es sich bei LVS-Produkten oft um kundenindividuelle Lösungen, da innerbetriebliche Logistiksysteme oft nicht ausreichend standardisiert sind. Einerseits kann dies die Integration von Optimierungstechniken erschweren (die ebenfalls eine kundenspezifische Anpassung erfordern würden). Andererseits ist es aufgrund des teilweise individuellen Charakters der Lösungen weniger wahrscheinlich, dass Details gemeldet werden.

7.4.2 Software für die Transportplanung

Als nächsten Bereich der Logistiksoftware wollen wir uns den Bereich der Transportplanung ansehen. Da dieser Bereich in der akademischen Literatur zur Logistikoptimierung zu den am aktivsten erforschten gehört, können wir einen reiferen Markt für entsprechende kommerzielle Softwarelösungen erwarten.

Eine allgemeine Software, die die Planung, Durchführung und Kontrolle von Transportaktivitäten unterstützt, wird gewöhnlich als Transportmanagementsystem (TMS) bezeichnet. Eine solche Software kann Teil einer ERP-Lösung oder eines APS sein oder als Zusatz zu einem solchen System oder auf eigenständiger Basis arbeiten. In der Regel werden eingehende (Materialbeschaffung) und ausgehende (Versand von Aufträgen) Aktivitäten von einem solchen System unterstützt, nicht aber innerbetriebliche Transporte auf der Grundlage der Materialflusstechnik (die in der Regel von LVS/WES-Software unterstützt werden).

Eine Kernfunktionalität eines TMS ist insbesondere die Ermittlung von Routen, die dann durch eigene Ressourcen ausgeführt werden. Oder es wird ein geeigneter Dienstleister ausgewählt. Darüber hinaus gibt es in der Regel Unterstützung für das Tracing und Tracking der eigentlichen Transportvorgänge und für nachfolgende Prozesse wie Bezahlung, Auditierung und Reporting.

Die Software für die Transportplanung wird regelmäßig von Hall (2016) untersucht und in OR/MS Today in gedruckter Form und auf der Website der Zeitschrift veröffentlicht. Die Studie basiert auf Fragebögen, die von den jeweiligen Softwareanbietern ausgefüllt wurden (vgl. auch Partyka und Hall (2000)). Die Erhebung 2016 umfasst 25 Produkte von 22 Unternehmen, die auf die Fragebögen geantwortet haben. Dies ist ein deutlicher Anstieg gegenüber der Zahl der Produkte, die vier Jahre zuvor (Hall, 2012) nur 15 Produkte von 12 Unternehmen umfasste.

Von diesen 25 Produkten verwenden sieben proprietäre Algorithmen, die nicht näher spezifiziert sind. Ein Produkt verwendet Heuristiken und Constraint-Programming, sieben Produkte verwenden verschiedene Arten von Metaheuristiken und (teilweise) Konstruktionsheuristiken, sieben Produkte verwenden Heuristiken ohne nähere Spezifikation, drei Produkte geben ausdrücklich an, mehrere Heuristiken in kombinierter Form zu verwenden, und bei einem Produkt wurde der zugrunde liegende Algorithmus überhaupt nicht angegeben. Interessant ist, dass vier Jahre zuvor nur für drei Produkte die Verwendung von (verschiedenen Arten von) Metaheuristiken angegeben wurde.

Spezifische Metaheuristiken, die erwähnt wurden, sind Simulated Annealing, genetische Algorithmen und lokale Suche. In anderen Fällen wird die Verwendung von hybriden oder proprietären Metaheuristiken angegeben. Bemerkenswert ist, dass ein Produkt aus der Umfrage von 2012 (IBM ILOG Transportation Analyst) auf einem allgemeinen Optimierungsprodukt, dem IBM ILOG CPLEX Optimization Studio, basiert.

Alle Produkte unterstützen Vehicle Routing-Probleme für die Knoten eines Netzes (Node-Routing), während zwei Drittel auch das Arc-Routing unterstützen, d. h. den Besuch vordefinierter Kanten eines Graphen, wie er häufig für die Postzustellung oder die Müllabfuhr erforderlich ist. Alle Produkte unterstützen ein Re-Routing. Alle Produkte nutzen Planungsdaten wie historische Fahrzeiten oder Haltezeiten, während neun auch Echtzeit-Verkehrsinformationen nutzen können.

Einige der Tools bieten verschiedene Schnittstellenmöglichkeiten, z. B. Messaging- und Tracing-Funktionen oder die Integration in Auftragsabwicklungssysteme oder in bestimmte Supply-Chain-Management-Software.

In allen Fällen wird die Fahrerzuordnung unterstützt. In den meisten Fällen werden auch Turn-by-Turn-Navigation, Fahrzeugladepläne und Ladungsverzeichnisse unterstützt. Abhol- und Zustellprobleme werden von allen Produkten unterstützt. Spezifische Anforderungen von Linienverkehrsdienstleistern wie Bussen werden oft nicht unterstützt.

Wir möchten anmerken, dass die Umfrage bei weitem nicht vollständig ist. Andere Produkte, die in der Umfrage nicht berücksichtigt wurden, obwohl sie eine Art von Optimierung beinhalten, sind beispielsweise TransCAD, Transwide, der JDA Transportation Planner oder Voyager Transportation Planning & Management. Zum Beispiel unterstützt die Software AccellosOne Optimize (Accellos, o.J.) die Transportplanung durch Optimierung der Ladungsbildung, Scheduling und Routenplanung von Fahrzeugen. Auf der Website wird lediglich erwähnt, dass die Software „modernste Optimierungsalgorithmen nutzt, um Ladungen und Routen zu ermitteln, die Rentabilität und Kundenanforderungen ausbalancieren." Ein weiteres Beispiel ist Wide Scope Routing Logistics, eine Lösung zur Optimierung des Fahrzeugroutings, die von dem portugiesischen Unternehmen Wide Scope angeboten wird. Es unterstützt verschiedene Vehicle-Routing-Probleme wie Abhol- und Zustellprobleme, umfasst aber auch das Scheduling und verwendet verschiedene Techniken wie mathematische Programmierung, Constraint-Programming und Metaheuristiken wie lokale Suche, genetische Algorithmen, Tabu-Suche und Ant Colony Optimization (Wide Scope, o.J.).

7.4.3 Software für das Packen und Laden

Pack- und Ladeprobleme, z. B. bei der Beladung von Fahrzeugen, Containern oder Paletten, werden durch Optimierungsalgorithmen in einer Reihe allgemeinerer Logistiksoftware wie z. B. kommerziellen Transportplanungstools (siehe oben) unterstützt. Darüber hinaus gibt es mehrere spezialisierte Tools für die Lösung solcher Probleme.

MagicLogic Optimization Inc. bietet beispielsweise ein Produkt namens „Cube IQ“ an, das insbesondere auf die Anforderungen von Logistikdienstleistern eingeht. Die verwendete Methode wird als Blackbox-Optimierer beschrieben, der auf einem „proprietären, exklusiv von MagicLogic entwickelten Algorithmus“ basiert (Jonker & Smith, 2010). Auch die Konsolidierung von Aufträgen und die Beladungsreihenfolge werden von diesem Tool berücksichtigt. Das Tool kann mit LVS und ERP-Systemen integriert werden.

Ein weiteres Beispiel für Lade- und Packsoftwareprodukte sind mehrere Tools der Astrokettle-Gruppe, die die Optimierung von Cutting Stock-Problemen, Packproblemen für rechtwinklige Boxen und Scheduling-Problemen unterstützen (Astrokettle, o.J.).

Andere Softwarelösungen werden von TOPS Software (o.J.) angeboten. Ihr Tool TOPS pro bietet Unterstützung bei der Optimierung des Verpackungsgestaltung, der Versandkartongröße oder bei der Bestimmung der optimalen Menge für einen bestehenden Versandkarton. Die Software enthält einen interaktiven Editor für die Kartongröße. Ein weiteres Tool namens MaxLoad Pro ermittelt, wie viele Lkw oder Container für einen Transport benötigt werden und wie diese beladen werden sollten. Außerdem hilft dieses Tool bei der Ermittlung der optimalen Versandkartons für einen Auftrag und bei der Erstellung stabiler Mischpaletten von Sendungen. Die in der Software verwendeten Optimierungsalgorithmen werden nicht weiter spezifiziert.

Ein weiteres Instrument ist die Software für die Beladung von Lastwagen, Containern und Paletten von Logen Solutions Corp. (Logen Solutions, o.J.). Ihr Tool namens CubeMaster unterstützt viele verschiedene Arten von Beladungsproblemen, einschließlich der Beladung von Lastwagen und Anhängern, der Beladung von Seecontainern, Luftcontainern, Paletten und Kartons mit verschiedenen Ladungstypen (Einzelladungen, Mischladungen und für vordefinierte Zusammenstellungen von Artikeln). Darüber hinaus unterstützt die Software verschiedene Beschränkungen in Bezug auf die Beladung.

Der verwendete Optimierungsalgorithmus stützt sich auf Heuristiken und Metaheuristiken, die auf Tabu-Suche und einem genetischen Algorithmus basieren. Der Ansatz soll in JORS 2006 veröffentlicht worden sein, aber die genaue Referenz wurde nicht angegeben und konnte nicht gefunden werden.

7.5 Schlussfolgerungen

Der Markt für Optimierungssoftware scheint recht ausgereift zu sein, da viele der allgemeinen Tools bereits seit mehreren Jahrzehnten verfügbar sind. Die meisten dieser ausgereiften Werkzeuge beruhen auf etablierten Optimierungsalgorithmen. Diese Allzweckwerkzeuge zeigen Schwächen bei der Anwendung auf typische Logistikprobleme, die im Wesentlichen aus der Komplexität (NP-schwer) dieser Probleme resultieren.

Ansätze aus dem Bereich der Computational Intelligence scheinen geeignet zu sein, um mit der Rechenkomplexität umzugehen. Obwohl sie das Dilemma der NP-Schwere nicht lösen können, führen sie in der Regel zu guten, wenn auch nicht optimalen Lösungen für die jeweiligen Probleme.

Wenn neue Ansätze aus dem Bereich der Computational Intelligence oder der Metaheuristiken entwickelt werden, sind sie in der Regel nur in akademischen Implementierungen verfügbar. Wenn sie entwickelt werden, werden sie manchmal auf ein spezifisches Problem im Bereich der Logistik angewandt. In anderen Fällen werden sie für eine andere Art von Optimierungsproblem in Computerexperimenten untersucht und es bleibt oft unklar, ob oder wie sie auch im Bereich der Logistik erfolgreich eingesetzt werden können.

Kommerzielle Unternehmenssoftware bietet meist nur begrenzte Optimierungsfunktionen (wenn überhaupt). In der Regel sind nur eine (oder sehr wenige) etablierte Optimierungsmethoden implementiert. Diese können für die betrachteten Problemtypen gut funktionieren (da sie in der Regel gut an das spezifische Anwendungsproblem angepasst sind), aber weitere Verbesserungen, die sich aus der akademischen Forschung ergeben, wie z. B. neue Methoden, werden oft vernachlässigt.

Es besteht also ein großes Potential, die Logistikplanung durch fortgeschrittene Software weiter zu verbessern und es ist davon auszugehen, dass Methoden der Computational Intelligence in Zukunft eine stärkere Verbreitung in der Logistiksoftware finden werden.

Literatur

Accellos. (o.J.). *Transportation optimization*. http://www.accellos.com/?s=trasnport+optimization. Zugegriffen am 12.03.2016.

Acteos. (o.J.). *Acteos WMS Warehouse Management System*. http://www.acteos.com/fr/fr_EN/download/brochure/ACTEOS_WMS_EN.pdf. Zugegriffen am 06.06.2014.

Akabuilo, E., Dornberger, R., & Hanne, T. (2011). How advanced are advanced planning systems? *Proceedings of the International Symposium on Information Systems and Software Engineering: ISSE 2011*, Orlando, USA, March 27–30, 2011.

Asis. (o.J.-a). *LMxt*. http://www.a-sis.com/en/our-solutions/lmxt-warehouse-and-flow-management-system-large-companies. Zugegriffen am 12.03.2016.

Asis. (o.J.-b). *Logys*. http://www.a-sis.com/en/our-solutions/logys. Zugegriffen am 12.03.2016.

Astrokettle. (o.J.). *Astrokettle algorithms*. http://www.astrokettle.com/index.html. Zugegriffen am 12.03.2016.

Bermudez, J. (1998). *Advanced planning and scheduling: Is it as goods as it sounds*? (S. 1–24) The Report on Supply Chain Management (March).

Fink, A., & Voß, S. (2003). HotFrame: A heuristic optimization framework. In S. Voß & D. L. Woodruff (Hrsg.), *Optimization software class libraries* (S. 81–154). Springer US.

Fourer, R. (2013). 2013 Linear programming software survey. *OR/MS Today*. http://www.orms--today.org/surveys/LP/LP-survey.html. Zugegriffen am 06.06.2014.

Fourer, R., Gay, D. M., & Kernighan, B. W. (2002). *AMPL – A modeling language for mathematical programming* (2. Aufl.). AMPL Optimization LLC. http://ampl.com/resources/the-ampl-book/chapter-downloads/. Zugegriffen am 12.03.2016.

Gurobi Optimization. (2016). *Mixed integer programming basics*. http://www.gurobi.com/resources/getting-started/mip-basics. Zugegriffen am 12.03.2016.
Hall, R. W. (2012). Vehicle routing software survey. *OR/MS Today*. http://www.orms-today.org/surveys/Vehicle_Routing/vrss.html. Zugegriffen am 06.06.2014.
Hall, R. W. (2016). Vehicle routing software survey. *OR/MS Today*. http://www.orms-today.org/surveys/Vehicle_Routing/vrss.html. Zugegriffen am 12.03.2016.
IBM. (2005). *CPLEX performance tuning for mixed integer programs*. http://www-01.ibm.com/support/docview.wss?uid=swg21400023. Zugegriffen am 12.03.2016.
IBM. (o.J.). *Linear programming: An essential optimization technique*. http://www-01.ibm.com/software/commerce/optimization/linear-programming/. Zugegriffen am 12.03.2016.
Jonker, R., & Smith, T. (2010). *Load planning software: Cube-IQ BlackBox*. Magiclogic Optimization Inc. http://downloads.magiclogic.com/Documents/Cube-IQ_BlackBox_WhitePaper.pdf. Zugegriffen am 12.03.2016.
Knolmayer, G. (2001). Advanced planning and scheduling systems: Optimierungsmethoden als Entscheidungskriterium für die Beschaffung von Software-Paketen? In U. Wagner (Hrsg.), *Zum Erkenntnisstand der Betriebswirtschaftslehre am Beginn des 21. Jahrhunderts* (S. 135–155). Duncker & Humblot.
Laundy, R., Perregaard, M., Tavares, G., Tipi, H., & Vazacopoulos, A. (2009). Solving hard mixed-integer programming problems with Xpress-MP: A MIPLIB 2003 case study. *INFORMS Journal on Computing, 21*(2), 304–313.
Lima, R. (2010). *IBM ILOG CPLEX What is inside of the box? EWO Seminar*. Carnegie Mellon University.
Logen Solutions. (o.J.). *What's new at Logen Solutions*? http://www.logensolutions.com/. Zugegriffen am 06.06.2014.
Manhattan Associates. (o.J.). *Warehouse management: Industry-leading WMS software*. http://www.manh.com/solutions/distribution-management/warehouse-management. Zugegriffen am 06.06.2014.
OpenDINO. (2016). *Main page*. http://www.opendino.org/w/index.php/Main_Page. Zugegriffen am 12.03.2016.
Partyka, J. G., & Hall, R. W. (2000). On the road to service. *OR/MS Today*.
Rothberg, E. (o.J.). The CPLEX library: Mixed integer programming. *Presentation at 4th Max-Planck Advanced Course on the Foundations of Computer Science,* Saarbrücken. September 8–12, 2003.
The R Foundation. (2016). *The R project for statistical computing*. http://www.r-project.org/. Zugegriffen am 12.03.2016.
TOPS Software. (o.J.). *TOPS software solutions for packaging & distribution*. http://www.topseng.com/. Zugegriffen am 12.03.2016.
vTradEx. (o.J.). *vTradEx eLOG enterprise suite WMS*. http://www.vtradex.com/english/solution_4.html. Zugegriffen am 12.03.2016.
Wide Scope. (o.J.). *Optimization*. http://routingoptimization.com/optimization.do. Zugegriffen am 06.06.2014.
Wikipedia. (2016a). *GNU general public license*. http://en.wikipedia.org/wiki/GNU_General_Public_License. Zugegriffen am 12.03.2016.
Wikipedia. (2016b). *Copyleft*. http://en.wikipedia.org/wiki/Copyleft. Zugegriffen am 12.03.2016.
Wikipedia. (2016c). *List of optimization software*. http://en.wikipedia.org/wiki/List_of_optimization_software. Zugegriffen am 12.03.2016.

Anhang A. Kurzbiographien der Autoren

Thomas Hanne hat Diplomabschlüsse in Wirtschaftswissenschaften und Informatik und einen Doktortitel in Wirtschaftswissenschaften. Von 1999 bis 2007 arbeitete er am Fraunhofer-Institut für Techno- und Wirtschaftsmathematik (ITWM) als Wissenschaftler. Seither ist er Professor für Wirtschaftsinformatik an der Fachhochschule Nordwestschweiz und seit 2012 Leiter des Kompetenzzentrums Systems Engineering.

Thomas Hanne ist Autor von mehr als 200 Zeitschriften- und Konferenzartikeln und Herausgeber mehrerer Zeitschriften und Sonderausgaben. Seine aktuellen Forschungsinteressen umfassen multikriterielle Entscheidungsanalysen, evolutionäre Algorithmen, Metaheuristiken, Optimierung, Simulation, Logistik und Supply Chain Management.

Rolf Dornberger ist Leiter des Instituts für Wirtschaftsinformatik, Hochschule für Wirtschaft, Fachhochschule Nordwestschweiz FHNW (seit 2007) und Leiter des Kompetenzzentrums New Trends & Innovation (seit 2013) und war Leiter der Kompetenzzentren Technology, Organization & People (2014–2018) und Systems Engineering (2006–2010). Im Jahr 2002 wurde er zum Dozenten und 2003 zum ordentlichen Professor für Wirtschaftsinformatik an der Fachhochschule Nordwestschweiz FHNW bzw. an deren Vorgängerin, der Fachhochschule Solothurn, ernannt. Zusätzlich war und ist er Lehrbeauftragter an der Universität Stuttgart und der Zürcher Hochschule für Angewandte Wissenschaften sowie der Universität Basel. Bevor er an die Hochschule zurückkehrte, arbeitete er in der Industrie in verschiedenen Managementpositionen als Berater, IT-Verantwortlicher und leitender Forscher in verschiedenen Ingenieurs-, Technologie- und IT-Unternehmen im Bereich der Energieerzeugung und IT-Lösungen für die Luftfahrtbranche. Er besitzt einen Doktortitel (1998) und einen Diplomabschluss in Luft- und Raumfahrttechnik (1994). Rolf Dornberger ist Autor von mehr als 100 Zeitschriften- und Konferenzartikeln und Herausgeber mehrerer Bücher. Zu seinen derzeitigen Forschungsinteressen gehören Computational Intelligence, Optimierung, Innovations- und Technologiemanagement sowie neue Trends und Innovationen.

© Der/die Herausgeber bzw. der/die Autor(en), exklusiv lizenziert an Springer Nature Switzerland AG 2023

T. Hanne, R. Dornberger, *Computational Intelligence in Logistik und Supply Chain Management*, https://doi.org/10.1007/978-3-031-21452-3